

刚体动力学算法

罗伊·费瑟斯通

2026 年 8 月 9 日

目录

序言	5
第 1 章 引言	7
1.1 动力学算法	7
1.2 空间矢量	8
1.3 读者指南	10
1.4 进一步阅读	10
第 2 章 空间向量代数	13
2.1 数学预备知识	13
2.2 空间速度	15
2.3 空间力	18
2.4 普吕克记号法	19
2.5 线向量与自由向量	20
2.5.1 大小	21
2.6 标量积	21
2.7 使用空间向量	22
2.8 坐标变换	24
2.8.1 旋转	24
2.8.2 平移	24
2.8.3 一般变换	25
2.9 空间叉积	26
2.10 求导	28
2.10.1 在移动坐标系中的求导	29
2.10.2 点乘与环形记号	29
2.11 加速度	30
2.12 动量	33
2.13 惯性	34
2.13.1 惯性的二元表示	35
2.14 平面向量	38
2.15 进一步阅读	39

第 3 章 刚体系统动力学	41
3.1 运动方程	41
3.2 构建运动方程	43
3.2.1 求解运动方程	44
3.2.2 运动约束	44
3.2.3 应用隐式运动约束	45
3.2.4 应用显式运动约束	46
3.3 向量空间	47
3.3.1 矩阵表示	48
3.4 连接约束	52
3.5 受约束刚体的动力学	55
3.5.1 方法一	56
3.5.2 方法二	57
3.5.3 方法 3	57
3.6 多体系统的动力学	58
3.6.1 第一步：刚体运动方程	58
3.6.2 第二步：从刚体运动推导关节运动	59
3.6.3 第三步：由关节力产生的机体力	60
3.6.4 第四步：运动约束	60
3.6.5 步骤 5：约束力	61
3.6.6 步骤 6：最终方程	61
3.6.7 讨论	61
第 4 章 刚体系统建模	63
4.1 连通性	63
4.1.1 运动学树与循环	64
4.1.2 编号	66
4.1.3 关节极性	66
4.1.4 表示法	68
4.2 几何学	69
4.3 Denavit-Hartenberg 参数	71
4.4 关节模型	72
4.4.1 关节模型计算例程	73
4.4.2 极性反转	75
4.5 球面运动	75
4.5.1 欧拉角	76
4.5.2 欧拉参数	77
4.6 完整系统模型	78

第 5 章 逆动力学	79
5.1 算法复杂度	79
5.2 递推关系	80
5.3 递归牛顿-欧拉算法	81
5.3.1 算法特性	82
5.3.2 体坐标系中的方程	83
5.3.3 算法细节	84
5.4 原始版本	85
5.5 附加说明	86
第 6 章 前向动力学——惯性矩阵方法	89
6.1 关节空间惯性矩阵	89
6.2 复合刚体算法	91
6.2.1 物体坐标系中的方程	92
6.2.2 算法细节	93
6.3 物理意义	94
6.4 分支诱导稀疏性 (Branch-Induced Sparsity)	95
6.5 稀疏因子分解算法 (Sparse Factorization Algorithms)	97
6.5.1 扩展父数组 (Expanding the Parent Array)	98
6.5.2 回代 (Back-Substitution)	99
6.5.3 计算成本 (Computational Cost)	99
6.6 附加说明 (Additional Notes)	100
第 7 章 正向动力学——传播方法	103
7.1 铰接体惯性 (Articulated-Body Inertia)	103
7.1.1 基本性质	104
7.1.2 加法	104
7.1.3 逆惯性	104
7.1.4 多手柄	104
7.2 计算铰接体惯性	105
7.2.1 投影法 (The Projection Method)	106
7.2.2 多手柄	107
7.2.3 组装机 (The Assembly Method)	107
7.2.4 组装机子树 (Assembling Subtrees)	109
7.3 铰接体算法 (The Articulated-Body Algorithm)	110
7.3.1 算法	111
7.3.2 通用子表达式	112
7.3.3 体坐标方程	112
7.3.4 算法细节	112
7.4 替代组装机公式	113
7.5 多手柄	116
7.5.1 推广	118

第 8 章 闭环系统	119
8.1 运动方程	119
8.2 回路约束方程	121
8.3 约束稳定化	123
8.3.1 计算 δ	124
8.4 回路关节力	125
8.5 求解运动方程	126
8.5.1 方法 1: 直接求解式 8.36	126
8.5.2 方法 2: 先求解 λ	126
8.5.3 方法 3: 先求解式 8.35	127
8.6 计算 $C - \tau^a$ 的算法	128
8.7 计算 K 和 k 的算法	129
8.7.1 舍入误差问题	130
8.8 计算 G 和 g 的算法	130
8.9 利用 K 和 G 中的稀疏性	132
8.10 闭环系统的一些性质	132
8.11 回路闭合函数	134
8.11.1 一个完整例子	135
8.11.2 三角形公式:	135
8.12 逆动力学	136
8.12.1 致动器力	137
8.13 稀疏矩阵方法	138
第 9 章 混合动力学及其他专题	141
9.1 混合动力学 (Hybrid Dynamics)	141
9.1.1 方程 (Equations)	142
9.1.2 算法 (Algorithms)	143
9.2 铰接体混合动力学 (Articulated-Body Hybrid Dynamics)	144
9.3 浮动基座 (Floating Bases)	146
9.4 浮动基座正向动力学 (Floating-Base Forward Dynamics)	148
9.4.1 计算 C 与 p_0^c :	149
9.5 浮动基座逆向动力学 (Floating-Base Inverse Dynamics)	150
9.6 齿轮 (Gears)	152
9.7 动力学等价 (Dynamic Equivalence)	154
9.7.1 刚体系统的简化 (Simplification of Rigid-Body Systems)	155
9.7.2 增广刚体 (Augmented Bodies)	156
第 10 章 精度与效率	157
10.1 误差来源	157
10.1.1 舍入误差	158
10.1.2 动力学算法中的舍入误差	159
10.1.3 截断误差	159

10.1.4 建模误差	160
10.2 灵敏度问题	160
10.3 效率	161
10.3.1 缩写	162
10.3.2 来源	162
10.3.3 优化	162
10.3.4 分支	164
10.4 符号化简	166
第 11 章 接触与冲击	169
11.1 单点接触	169
11.2 多点接触	171
11.3 带接触的刚体系统	173
11.4 不等式约束	176
11.5 求解接触方程	177
11.5.1 对积分时间步进行插值	179
11.6 接触几何	180
11.6.1 形状	180
11.6.2 几何事件	180
11.6.3 线接触和面接触	181
11.7 冲量动力学	181
11.7.1 双物体碰撞	183
11.7.2 摩擦	184
11.8 软接触	185
11.8.1 接触法向力	186
11.8.2 扩展	186
11.8.3 库仑摩擦	187
11.9 进一步阅读	188
第 12 章 空间向量运算	189
12.0.1 A.1 简单平面运算	189
12.0.2 A.2 简单空间运算	190
12.0.3 A.3 紧凑表示	190
12.0.4 A.4 轴向螺旋变换	192
12.0.5 向量	193
12.0.6 刚体惯性	193
12.0.7 连杆刚体惯性	193
12.0.8 零值行列的连杆刚体惯性	194
12.0.9 A.5 一些效率技巧	194
12.0.10 旋转矩阵的乘积	194
12.0.11 一般矩阵的旋转	194
12.0.12 对称矩阵的旋转	195

12.0.13 绕坐标轴旋转一般矩阵	196
12.0.14 绕坐标轴旋转对称矩阵	196
12.0.15 参考文献	197
12.0.16 符号	202
12.0.17 索引	204

序言

本书旨在呈现一套极为精炼且高效的刚体动力学计算算法，并以足够详尽的方式加以阐释，使读者能够理解这些算法的工作原理，以及如何根据自身需求对其进行调整（或设计全新的算法）。本书收录了以下几类广为人知的算法：递归牛顿-欧拉算法、复合刚体算法以及关节刚体算法。此外，书中还涵盖了运动学环路和浮动基座的相关算法。每种算法均基于基本原理推导而来，并以方程组的形式呈现，同时辅以伪代码程序，后者便于轻松转换为任何适用的编程语言。

本书还详细讲解了用于构建刚体系统运动方程的一些数学技巧。特别是，本书展示了如何利用六维向量来表达动力学，并深入阐述了作为最高效算法基础的递归公式。其他主题包括：如何构建刚体系统的计算机模型；如何利用惯性矩阵中的稀疏性；关节刚体惯性的概念；动力学计算中舍入误差的来源；以及刚体之间物理接触与碰撞的动力学问题。

刚体动力学往往容易陷入代数的海洋之中。然而，这在很大程度上源于我们使用三维向量，而通过改用六维向量表示法，这一问题完全可以得到解决。本书采用基于空间向量的表示方法，将刚体运动的线性分量与角动量分量统一整合为一组统一的量和方程。其结果通常是将代数运算的复杂度降低四到六倍。这一改进同样体现在计算机代码中：程序更短、更清晰，易于阅读、编写和调试，同时其效率仍与使用标准三维向量的代码不相上下。

本书旨在面向广泛的读者群体，从高年级本科生到研究人员和专业人士皆可轻松阅读。读者通常已具备一定的刚体动力学基础知识，例如通过入门级动力学课程所获得的知识，或通过阅读入门教材的前几章而掌握的相关内容。不过，本书并不要求读者具备六维向量的基础知识，因为该主题已在第2章中从头开始进行详细讲解。此外，本书还包含一些进阶内容，这些内容或许对动力学研究者颇具吸引力。

6 维向量领域的专家与学者。本书未附带任何软件，但读者可通过作者的网站获取书中大部分算法的源代码。

本书最初计划作为一本名为《机器人动力学算法》的书籍的第二版出版，该书于1987年首次面世；然而，很快便发现其中已有足够多的新内容，足以支撑撰写一本全新的著作。与前作相比，本书最显著的新内容包括：对各类算法的明确伪代码描述；关于如何建模刚体系统的章节；用于利用分支诱导稀疏性的算法；对运动学回路及浮动基座系统更为详尽的阐述；平面向量（空间向量的平面化对应物）；数值误差与模型敏感性；以及关于如何实现空间向量运算的指导。

刚体动力学是一门历史悠久的学科，如今已因计算机技术的蓬勃发展而焕然一新、面目一新。如今，我们可以在电脑游戏中、动画与虚拟现实软件中、模拟器中、运动控制系统中，以及各种工程设计与分析工具中找到动力学计算的身影。在每一种场景下，计算机都在计算与物理系统刚体近似模型运动相关联的力、加速度等参数。

本书的主要目的，是为在计算机上执行各类动力学计算提供一套高效算法的集合。每种算法都进行了详尽的阐述，使读者能够理解其工作原理及为何如此高效；同时，书中还对一些基本概念进行了讲解，例如递归公式化、分支诱导的稀疏性以及关节肢体的惯量等。

刚体动力学通常以三维向量来表示。然而，本书所探讨的是动力学算法这一主题，而用六维向量来表述该主题更为合适。因此，我们采用基于空间向量的六维符号体系，相关内容将在第二章中详细说明。相较于三维向量，空间符号体系大大减少了代数运算的复杂度，简化了动力学算法的描述与解释过程，并且使得在计算机上实现这些算法的过程也更加便捷。

本章对本书的主题进行了简要介绍。其中既介绍了动力学算法的相关内容，又对空间向量作了简要说明，并阐述了本书的编排结构。

§ 1.1 动力学算法

刚体系统的动力学可以通过其运动方程来描述，该方程明确了作用于系统上的力与其所产生的加速度之间的关系。动力学算法是一种用于计算与动力学相关各量数值的程序。我们将主要关注两种特定计算的算法：

正向动力学：计算给定刚体系统对给定施加力的加速度响应；以及

逆向动力学：计算为了产生给定的加速度响应，必须对给定刚体系统施加的力。

正向动力学主要用于仿真领域。逆向动力学则有多种应用，例如：运动控制系统的开发、轨迹规划、机械设计，以及作为正向动力学计算中的一个组成部分。在第九章中，我们将探讨第三类计算——混合动力学，即在该类计算中，部分加速度和力变量被预先给出，而任务则是计算其余的变量。

刚体系统的运动方程可以写成如下标准形式：

$$\tau = H(q) \ddot{q} + C(q, \dot{q}).$$

在该方程中, q 、 $\dot{q}$ 和 $\ddot{q}$ 分别表示位置、速度和加速度的矢量, 而 τ 是作用力的矢量。 H 是一个惯性项矩阵, 通常写作 $H(q)$, 以表明其依赖于 q 。 C 是一个力项矢量, 用于考虑科里奥利力、离心力、重力, 以及除 τ 中所包含的力之外, 作用于系统的其他所有力。 C 通常写作 $C(q, \dot{q})$, 以表明其既依赖于 q , 也依赖于 $\dot{q}$ 。总体而言, H 和 C 构成了运动方程的系数, 而 τ 和 $\ddot{q}$ 则是方程中的变量。通常情况下, 其中一个变量已知, 而另一个变量则未知。

尽管习惯上将 $H(q)$ 和 $C(q, \dot{q})$ 写作 $H(\text{model}, q)$ 和 $C(\text{model}, q, \dot{q})$, 但更准确的说法应为 $H(\text{模型}, q)$ 和 $C(\text{模型}, q, \dot{q})$, 以表明 H 和 C 都依赖于其所作用的具体刚体系统。符号 model 指的是一个数据集合, 它以各组成部分的形式描述了某一特定的刚体系统: 包括系统的体数与关节数、它们之间的连接方式, 以及与每个部件相关联的每一个参数的值 (如惯性参数、几何参数等)。我们称这种描述为“系统模型”, 以将其与数学模型区分开来。两者的区别在于: 系统模型描述的是系统本身, 而数学模型则描述系统行为的某一方面。方程 1.1 是一个数学模型。在计算机中, “模型” 可以被定义为一种“系统模型” 类型变量, 其值则是一个数据结构, 其中包含了特定刚体系统的系统模型。

让我们将正向动力学和逆向动力学的计算封装成一对函数, 即 FD 和 ID 。这些函数满足

$$\ddot{q} = \text{FD}(\text{model}, q, \dot{q}, \tau)$$

以及

$$\tau = \text{ID}(\text{model}, q, \dot{q}, \ddot{q}).$$

将这些方程与方程 1.1 进行对比, 显而易见, FD 和 ID 必须分别计算出 $H^{-1}(\tau - C)$ 和 $H\ddot{q} + C$ 。然而, 这些

这些方程清晰地展示了每次计算的输入与输出。尤其值得注意的是, 这两者都表明系统模型在两种情况下都是输入。因此, 人们预期, 实现自由度法 (FD) 和逆向动力学 (ID) 的算法能够适用于一类刚体系统, 并且能够利用系统模型中的数据, 推导出由该模型所描述的特定刚体系统的动力学特性。我们将使用“基于模型的算法”这一名称来指代具有此类工作方式的算法。

这种方法的一大优势在于, 只需编写、测试、记录并持续完善一段计算机代码, 即可针对广泛类别的任意刚体系统计算其动力学特性。其中, 两大主要研究领域分别被称为运动树和闭环系统。大致而言, 运动树是指任何不包含运动回路的刚体系统; 而闭环系统则是指任何并非运动树的刚体系统。¹ 与闭环系统相比, 计算运动树的动力学特性要容易得多。

基于模型的算法可以根据其两大主要属性进行分类: 它们执行何种计算, 以及它们所适用的系统类别。本书中主要的算法体系包括用于运动树和闭环系统的正向动力学与逆向动力学算法。

§1.2 空间矢量

在三维空间中, 刚体拥有六自由度; 然而, 我们通常用三维矢量来表示其动力学特性。因此, 要表述刚体的运动方程, 实际上需要写出两个矢量方程:

$$f = m a_C \text{ and } n_C = I\dot{\omega} + \omega \times I\omega.$$

第一个方程表达了作用于刚体上的力与其质心线性加速度之间的关系；第二个方程则表达了作用于刚体上的力矩——以刚体质心为参考点——与其角加速度之间的关系。

在空间矢量表示法中，我们使用 6 维矢量，将刚体运动的线性分量与角动量分量相结合。因此，线性加速度与角加速度被合并成一个空间加速度矢量，力与力矩被合并成一个空间力矢量，以此类推。借助空间表示法，刚体的运动方程可以写成：

$$\mathbf{f} = I\mathbf{a} + \mathbf{v} \times^* I\mathbf{v},$$

其中， $\mathbf{f}$ 表示作用于刚体的空间力， $\mathbf{v}$ 和 $\mathbf{a}$ 分别表示刚体的空间速度与加速度，而 I 则是刚体的空间惯性张量。符号 $\times^*$ 表示空间矢量的叉积。这个方程的一个显而易见的优点是……

问题在于，它与式 1.4 存在一些名称上的冲突。我们可以通过在空间符号上方加上帽子（例如 $\hat{\mathbf{f}}$ ）来解决这一问题。

空间表示法不仅提供了运动方程这一种简化方式，还提供了更多便利。例如，若将两个刚体连接在一起形成一个整体刚体，则新刚体的空间惯性可由以下简单公式给出：

$$I_{\text{new}} = I_1 + I_2,$$

其中 I_1 和 I_2 分别为两个原始刚体的惯性。该公式在三维向量方法中取代了三个参数：一个用于计算新的质量，一个用于计算新的质心，还有一个用于计算新刚体绕新质心的旋转惯性。所有这些简化的总体效果是：与标准的三维向量表示法相比，空间表示法通常能将代数运算的复杂度至少降低 4 倍。当代数这一障碍被消除后，分析人员便可以更简洁地表述问题，用更少的步骤解决问题，并得到更加紧凑的解法。

空间向量的另一个优势在于，它们能够简化计算机代码的编写过程。生成的代码更短、更易于阅读、书写和调试，但其效率并不亚于使用三维向量的软件。例如，某些编程语言允许你用如下语句实现式 1.5：

$$\mathbf{f} = \mathbf{I} * \mathbf{a} + \mathbf{v}.\text{cross}(\mathbf{I} * \mathbf{v});$$

在此语句中，每个变量的类型都已向编译器进行声明。因此，编译器知道 $\mathbf{I}$ 和 $\mathbf{v}$ 分别代表刚体的惯性和空间运动矢量，从而能够将表达式 $\mathbf{I} * \mathbf{v}$ 编译成一段代码，这段代码会调用空间算术库中的一个例程，而该例程专为高效执行这种特定的乘法操作而设计。

1.3 单位与符号

角度均以弧度为单位进行测量。除此之外，本书中的各种公式与所选用的单位无关，只需确保所选的单位系统保持一致即可。在少数明确提及单位的地方，我们采用国际单位制（SI）单位。

数学符号一般遵循 ISO 规范。因此，变量用斜体表示；常量和函数用正体表示；向量和矩阵则用粗体斜体表示。向量用小写字母表示，²；矩阵则用大写字母表示。符号的简要列表可在第 265 页找到，部分符号在索引中也有对应的条目。

这里有点细节值得特别提及。符号 $\mathbf{0}$ 和 $\mathbf{1}$ 分别表示零矩阵和单位矩阵，而 $\mathbf{0}$ 也指代零向量。上标 $^{-T}$ 表示逆矩阵的转置，因此 A^{-T} 意味着 $(A^{-1})^T$ 。一个

表达式 “ $\mathbf{a} \times$ ” 表示一种将 $\mathbf{b}$ 映射为 $\mathbf{a} \times \mathbf{b}$ 的运算符。若某一量被描述为一个数组，则它实际上是一个由多个元素组成的有序列表。若 λ 是一个数组，则 λ 的第 i 个元素可写作 $\lambda(i)$ 。若某个符号前带有上标（例如 ${}^A\mathbf{v}$ ），则该上标用于标识坐标系。坐标系也可以以下标的形式出现。在空间向量中，有时会用帽子标记，而坐标向量则用下划线标注，如第二章所详述。

§1.3 读者指南

本书的内容可以分为三个部分：准备篇、主要算法篇以及附加主题篇。预备篇分别为第2章、第3章和第4章；主要算法则在第5章至第8章中进行详细阐述。具备足够背景知识的读者可以跳过这些预备章节，直接进入第5章。第9章及之后的内容假定读者已对前期知识有基本了解，但其余部分则自成体系，无需依赖先前的知识。

第2章从最基本的原则出发，介绍了空间向量代数；第3章则讲解了如何建立并分析刚体系统的运动方程。这两章最为注重数学性，且相较于后续算法所需的最低要求，它们对相关主题的阐述更为深入。第4章则详细描述了系统模型中的各个组成部分。许多在后续章节中使用的物理量，均在此处予以定义。

第5章、第6章和第7章分别介绍了三种最著名的运动学树算法：逆动力学的递归牛顿-欧拉算法，以及正动力学的复合刚体算法和关节刚体算法，按此顺序依次展开。第5章还阐释了递归公式化这一概念，而正是这一概念奠定了这些算法高效运行的基础。第8章随后介绍了多种用于计算闭环系统动力学的技术。这四章的编排大致遵循复杂度逐渐升高的顺序（即，算法的复杂程度会逐步提升）。

第9章、第10章和第11章则探讨了多种额外的主题。第9章进一步探讨了多种算法，包括正动力学与逆动力学的混合算法，以及适用于浮动基座系统的算法。第10章重点讨论了数值精度与计算效率的问题；第11章则研究了受接触与碰撞影响的刚体系统的动力学。

读者会在全书各处偶见一些示例。在某些情况下，这些示例用来阐释主文本中已涉及的概念；而在另一些情况下，它们则以实例的形式，更直观地说明了某些较为抽象的理论思想。此外，读者还会发现，每一种主要算法都以方程组的形式呈现，同时也以伪代码程序的形式加以说明。在许多情况下，这两种形式并置呈现。伪代码的设计初衷是自明且易于转化为实际的计算机代码。

包括作者在内，许多人都觉得，在学习新算法时，手边备着一些可用的源代码会很有帮助。不过，本书并未附带任何源代码，因为这类软件往往过快地被淘汰。

读者不妨前往网络上查找这些代码。截至撰写本文时，作者的网站³已收录了书中大部分所介绍算法的源代码，但读者也可能在其他地方找到有用的软件。

§1.4 进一步阅读

近年来出版了多部关于刚体动力学的著作，不过其中许多如今已绝版。以下几本是较为适合的入门教材：Amirouche（2006年）、Moon（1998年）、Shabana（2001年）以及Huston（1990年）。前三本均为正式的教学用书，配有大量示例和课后练习；而Huston的这本书则更偏向于教程性质。这些书的设计初衷就是让本科生能够轻松理解并掌握相关内容。更为高级的教材包括Stejskal和Valášek（1996年）、Roberson和Schwertassek（1988年）以及Wittenburg（1977年）。若想从不同视角进行学习，可以尝试Coutinho（2001年）一书，其内容更多聚焦于计算机游戏与虚拟现实领域。

本书中的大部分内容都源自机器人学界。其他关于机器人动力学的书籍包括Balafoutis和

Patel (1991 年)、Lilly (1993 年)、Yamane (2004 年), 以及作者早先的作品——Featherstone (1987 年), 该书现已为本卷所取代。关于动力学的各节与各章, 也可在 Angles (2003 年)、Khalil 和 Dombre (2002 年)、Siciliano 和 Khatib (2008 年) 以及其他多部机器人学相关书籍中找到。

对各类 6 维向量感兴趣的研究者, 或许会对以下书籍和文章感兴趣: Ball (1900 年)、Brand (1953 年)、von Mises (1924a、b 年)、Murray 等人 (1994 年) 以及 Selig (1996 年)。此外, Featherstone (1987 年) 中对空间向量的处理方式, 与本书所介绍的略有不同。

空间向量代数

空间向量是将刚体运动与力的线性成分与角动量成分相结合而成的 6 维向量。它们为研究刚体动力学提供了一种简洁的表示方式：一个空间向量即可完成两个 3 维向量的工作，而一个空间方程则可替代两个（或有时更多）3 维向量方程。通过使用空间向量符号，我们能够快速推导出运动方程，并以简洁的符号形式将其表达出来。同时，这种表示方式也使我们能够迅速推导出动力学算法，并以紧凑的形式将其表述，便于高效地转化为计算机代码。

本章从基本原理出发，系统地介绍了空间向量代数，并推导出了单个刚体的运动方程。此外，本章还介绍了平面向量代数——该代数将空间向量代数拓展至仅在二维平面上运动的刚体。空间向量运算的实现方法已在附录 A 中予以详述；而利用空间向量分析一般刚体系统的相关内容，则在第三章中有所介绍。

§2.1

数学预备知识

本节简要回顾了空间向量代数中所用的一些数学概念与符号。更多详细内容可参阅格林伍德（1988）、塞利格（1996）等多部著作。

向量与向量空间：线性代数将向量定义为向量空间中的一个元素；不同类型的向量则属于不同的向量空间。本书中经常出现四种向量空间，并分别赋予了专用名称：

$\mathbf{R}^n$ —坐标向量

$\mathbf{E}^n$ —欧几里得向量

$\mathbf{M}^n$ —空间运动向量

$\mathbf{F}^n$ —空间力向量

在每种情况下，上标均表示维度。另一个经常出现的空间是 $m \times n$ 矩阵空间，记作 $\mathbf{R}^{m \times n}$ 。

坐标向量是由 n 个实数构成的 n 元组，或以矩阵形式表示为 $n \times 1$ 的实数矩阵（即列向量）。坐标向量通常用于表示其他向量；我们使用“抽象向量”这一术语来指代所要表示的向量。欧几里得向量具有一个特殊性质：它们上可以定义欧几里得内积。这种乘积赋予了它们熟悉的模长与方向属性。用于描述刚体动力学的三维向量正是欧几里得向量。空间向量并非欧几里得向量，而是由两组向量空间的元素组成：一组用于运动向量，另一组用于力向量。空间运动向量描述了刚体运动的诸多属性，例如速度和加速度；而空间力向量则描述了力、冲量和动量。 $\mathbf{M}^6$ 和 $\mathbf{F}^6$ 这两个空间是本章的主要研究对象。

帽子与下划线：当空间向量与三维向量同时出现时，我们会用帽子标记某些空间向量（如 $\hat{\mathbf{v}}$ 、 $\hat{\mathbf{f}}$ ），以避免与同名的三维向量发生命名冲突。同样地，我们通常不区分坐标向量与它们所代表的抽象向量，但如果确实需要加以区分，则会用下划线标注坐标向量（如 $\underline{\mathbf{v}}$ 表示 $\mathbf{v}$ ）。我们只在必要时使用这些符号化手段，而在本章之外几乎从不使用。

向量空间的对偶空间：设 V 为一个向量空间。其对偶空间，记作 V^* ，是一个与 V 同维数的向量空间，并且具备以下性质：在该空间中定义了与 V 之间的标量积（即，标量积分别取自两个空间中的一个元素）。若 $\mathbf{u} \in V^*$ ， $\mathbf{v} \in V$ ，则该标量积可以写作 $\mathbf{u} \cdot \mathbf{v}$ 或 $\mathbf{v} \cdot \mathbf{u}$ ，这两种表达方式含义相同。对偶性是一种对称的关系：若 $U = V^*$ ，则 $V = U^*$ 。

对偶性的概念与空间向量代数密切相关，因为 $\mathbf{M}^n$ 和 $\mathbf{F}^n$ 这两个空间是互为对偶的（即，它们彼此互为对偶）。特别地，运动矢量与力矢量之间定义了标量积，使得若 $\mathbf{m} \in \mathbf{M}^6$ 描述刚体的速度， $\mathbf{f} \in \mathbf{F}^6$ 描述作用于该刚体的力，则 $\mathbf{m} \cdot \mathbf{f}$ 即为该力所传递的功率。

向量空间与其对偶空间之间的标量积必须是非退化（亦称非奇异）。若一个标量积具有如下性质，则称其为非退化：对于任意 $\mathbf{v} \in V$ ，若 $\mathbf{v} \neq \mathbf{0}$ ，则至少存在一个 $\mathbf{u} \in V^*$ ，使得 $\mathbf{v} \cdot \mathbf{u} \neq 0$ 。这一性质是保证 V 和 V^* 上存在对偶基的充分条件。

对偶基：假设我们有两个向量空间 U 和 V ，其中 $U = V^*$ 。设 $\mathcal{D} = \{d_1, \dots, d_n\}$ 是 U 的一个基，设 $\mathcal{E} = \{e_1, \dots, e_n\}$ 是 V 的一个基。如果这两个基满足以下条件，则它们构成一对对偶基。在 U 和 V 上：

$$d_i \cdot e_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{otherwise.} \end{cases}$$

这被称为互反性条件。若 $\mathcal{D}$ 和 $\mathcal{E}$ 满足此条件，则二者互为对方的倒数。只要给出其中一个基，另一个基便可通过互反性条件唯一确定。我们可以用 $\mathcal{D}^*$ 来表示 $\mathcal{D}$ 的倒数。需要注意的是，对偶基由两组基向量组成，且覆盖了两个向量空间。

若 $U = \mathbf{E}^n$ ，则可以将 V 设为 U ，从而使 $\mathcal{D}$ 和 $\mathcal{E}$ 都成为同一向量空间上的基。同时，我们也可以找到一些特殊情形的 $\mathcal{D}$ ，其性质为 $\mathcal{D} = \mathcal{E}$ 。这类基称为正交基。因此，正交基是双基的一种特例，在这种情况下， $U = V = \mathbf{E}^n$ ，且 $\mathcal{D} = \mathcal{E}$ 。

双坐标系：双基底定义了双坐标系。在表示空间向量时，我们始终采用双坐标；而我们大多会使用普吕克坐标，这是一种尤为便捷的双坐标系选择。双坐标的特殊性质在于：

$$\mathbf{u} \cdot \mathbf{v} = \underline{\mathbf{u}}^T \underline{\mathbf{v}},$$

其中 $\underline{\mathbf{u}}$ 和 $\underline{\mathbf{v}}$ 分别是用双坐标系中表示 $\mathbf{u} \in U$ 和 $\mathbf{v} \in V$ 的坐标向量，该双坐标系由 $\mathcal{D}$ 和 $\mathcal{E}$ 定义。一个直接的推论是， $\underline{\mathbf{u}}$ 和 $\underline{\mathbf{v}}$ 中的各个元素可由如下方式给出：

$$u_i = e_i \cdot \mathbf{u} \text{ and } v_i = d_i \cdot \mathbf{v}.$$

要将一个双坐标系中的坐标变换到另一个双坐标系中，我们需要两个变换矩阵：一个用于作用于 $\underline{\mathbf{u}}$ ，另一个用于作用于 $\underline{\mathbf{v}}$ 。若 $\mathbf{X}$ 是对 $\underline{\mathbf{u}}$ 进行所需坐标变换的矩阵，则对 $\underline{\mathbf{v}}$ 执行相同操作的矩阵记作 $\mathbf{X}^*$ ，这两个矩阵之间的关系由下式给出：

$$\mathbf{X}^* = \mathbf{X}^{-T}.$$

这一关系源于这样一个要求：对于所有 $\underline{\mathbf{u}}$ 和 $\underline{\mathbf{v}}$ ，有 $\underline{\mathbf{u}}^T \underline{\mathbf{v}} = (\mathbf{X}\underline{\mathbf{u}})^T (\mathbf{X}^*\underline{\mathbf{v}})$ 。

运算符 $a \cdot$ 和 $a \times$ ：我们可以将诸如 $a \cdot b$ 和 $a \times b$ 这样的表达式理解为：将运算符 $a \cdot$ 或 $a \times$ 应用于运算对象 b 得到的结果。因此， $a \cdot$ 是将 b 映射为标量 $a \cdot b$ 的运算符，而 $a \times$ 则将 b 映射为 $a \times b$ 。若 a 是一个坐标向量，则 $a \cdot = a^T$ ，而 $a \times$ 是一个方阵。 $a \times$ 的一些性质列于表 2.1 和表 2.3 中。表 2.3 还列出了运算符 $a \times^*$ 的性质，该运算符是 $a \times$ 的对偶运算符。

二元组与二元数：形式为 ab 的表达式被称为二元组。它是一种线性运算符，能够将向量映射为向量。特别地，二元组 ab 会将向量 c 映射为向量 $a(b \cdot c)$ ，后者是 a 的倍数。若 $\underline{a}$ 和 $\underline{b}$ 分别是表示 a 和 b 的坐标向量，则二元组 ab 可以用矩阵 $\underline{a} \underline{b}^T$ 来表示，该矩阵的秩为 1。在最一般的情况下，我们可以让 $a \in U$ 、 $b \in V$ ，而 $c \in W$ ，且这些向量空间的选择并无任何限制，只是 V 与 W 之间必须定义一个数量积。这样的二元组会将 W 映射到 U 。

二元（或二元张量）是一种对向量进行的线性映射。任何二元都可以表示为 r 个线性无关的二元之和：

$$\mathcal{L} = \sum_{i=1}^r a_i b_i.$$

其中 $a_i \in U$ ， $b_i \in V$ ，且 $r \leq \min(m, n)$ ，其中 $m = \dim(U)$ ， $n = \dim(V)$ 。（一组二元是线性独立的，如果向量 a_i 在 U 中是线性独立的，而向量 b_i 在 V 中也是线性独立的。）若 a_i 和 b_i 分别是坐标向量，则 L 可以写成

$$\mathcal{L} = \sum_{i=1}^r a_i b_i^T,$$

这是一个秩为 r 的 $m \times n$ 矩阵。若 $m = n = r$ ，则 L 是一个一一对应映射，并且可逆。二元在与空间惯性张量相关联时被广泛应用。

§2.2 空间速度

我们来推导一个用于计算刚体空间速度的公式。首先，我们选取一个刚体 B ，并选择一个固定点 O ，该点可以位于空间中的任意位置。（参见图 2.1(a)。）在给定 O 的条件下，可以通过一对三维向量来确定 B 的速度：即刚体固定点的线性速度 $\mathbf{v}_O$ ——该点当前与 O 重合——以及角速度向量 $\boldsymbol{\omega}$ 。随后，我们可以将整个刚体视为以线性速度 $\mathbf{v}_O$ 进行平移运动，同时又以角速度 $\boldsymbol{\omega}$ 沿着通过 O 的轴旋转。

根据这一描述，我们可以利用以下公式计算刚体中任意其他点的速度：

$$\mathbf{v}_P = \mathbf{v}_O + \boldsymbol{\omega} \times \overrightarrow{OP},$$

其中 P 是所关心的点， $\mathbf{v}_P$ 是当前位于 P 的刚体固定点的速度，而 $\overrightarrow{OP}$ 则给出了 P 相对于 O 的位置。该方程清晰地表明， $\mathbf{v}_P$ 包含两部分贡献：一部分是由于整个刚体沿 $\mathbf{v}_O$ 进行平移运动所致，另一部分则是由于刚体绕着通过 O 的轴旋转而产生的。

通过 O 。尽管每一项贡献都依赖于 O 的位置，但它们的总和却不会依赖于 O 的位置（即各因素之间的相互作用相互抵消），因此 $\mathbf{v}_P$ 仅取决于 B 的运动状态以及 P 的位置。需要注意的是， P 、 $\mathbf{v}_P$ 和 $\boldsymbol{\omega}$ 共同描述了与 O 、 $\mathbf{v}_O$ 和 $\boldsymbol{\omega}$ 相同的刚体运动。

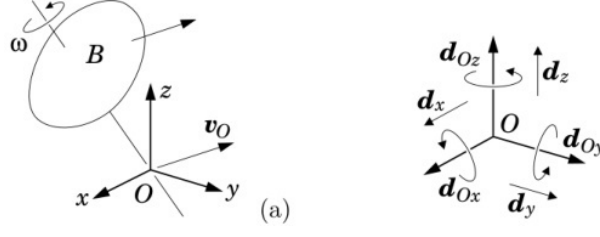


图 2.1: 图 2.1: 刚体的速度用 ω 和 v_O 表示 (a), 以及普吕克运动坐标系的基矢量 (b)

我们现在引入一个直角坐标系 O_{xyz} , 其原点位于 O 。该坐标系定义了三个互相垂直的方向—— x 、 y 和 z , 以及三条有向直线—— Ox 、 Oy 和 Oz , 每条直线都经过点 O 。此外, 该坐标系还定义了一个正交规范基底 $\{i, j, k\} \subset \mathbb{E}^3$, 我们可以利用这一基底, 将 ω 和 v_O 用其笛卡尔坐标表示出来:

$$\omega = \omega_x i + \omega_y j + \omega_z k$$

并且

$$v_O = v_{Ox} i + v_{Oy} j + v_{Oz} k.$$

在将 ω 和 v_O 分解为各自的坐标后, 我们可以将 B 的速度描述为六种基本运动的总和: 绕 Ox 直线旋转的角速度 ω_x , 绕 Oy 直线旋转的角速度 ω_y , 绕 Oz 直线旋转的角速度 ω_z , 以及分别沿 x 、 y 和 z 方向进行的位移, 位移幅度分别为 v_{Ox} 、 v_{Oy} 和 v_{Oz} 。

我们的目标是得到一个空间速度矢量 $\hat{v} \in \mathbb{M}^6$, 它能够描述与 ω 和 v_O 同样的运动。我们首先可以在 $\mathbb{M}^6$ 上定义一个基底, 然后在该基底中求出 $\hat{v}$ 的坐标。我们将使用的基底是:

$$\mathcal{D}_O = \{d_{Ox}, d_{Oy}, d_{Oz}, d_x, d_y, d_z\} \subset \mathbb{M}^6,$$

如图 2.1(b) 所示。该基底被称为普吕克基底, 并且在 $\mathbb{M}^6$ 上定义了一个普吕克坐标系。前三个基矢量 d_{Ox} 、 d_{Oy} 和 d_{Oz} 分别是对 Ox 、 Oy 和 Oz 直线的单位旋转; 其余基矢量则分别是在 x 、 y 和 z 方向上的单位平移。将这些基矢量的定义与刚才所描述的六种基本运动进行对比, 可以立即得出结论:

$$\hat{v} = \omega_x d_{Ox} + \omega_y d_{Oy} + \omega_z d_{Oz} + v_{Ox} d_x + v_{Oy} d_y + v_{Oz} d_z.$$

因此, 在 $\mathcal{D}_O$ 中, $\hat{v}$ 的普吕克坐标就是 ω 和 v_O 在 $\{i, j, k\}$ 基底中的笛卡尔坐标。等式右侧的每一项都依赖于 O_{xyz} 的位置和方向, 但可以证明, 整个表达式是不变的。(参见例 2.2。)在 $\mathcal{D}_O$ 坐标系中表示 $\hat{v}$ 的坐标向量可以写成:

$$\hat{\underline{v}}_O = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \\ v_{Ox} \\ v_{Oy} \\ v_{Oz} \end{bmatrix} = \begin{bmatrix} \underline{\omega} \\ \underline{v}_O \end{bmatrix},$$

其中，右端的表达式是一种便捷的紧凑形式，将 6 维坐标向量表示为两个 3 维坐标向量的串联： $\underline{\omega} = [\omega_x \ \omega_y \ \omega_z]^T$ 和 $\underline{v}_O = [v_{Ox} \ v_{Oy} \ v_{Oz}]^T$ 。

例 2.1 中的方程 2.1 将每个空间点 P 与一个向量 v_P 相关联。因此，它定义了一个向量场——即物体 B 的速度场。我们也可以将该方程写成如下形式：

$$\mathbf{V}(P) = \mathbf{v}_O + \omega \times \overrightarrow{OP},$$

其中， $\mathbf{V}$ 表示矢量场，而 $\mathbf{V}(P)$ 是由 $\mathbf{V}$ 与点 P 相关联的矢量。尽管 O 出现在 $\mathbf{V}$ 的定义中，但等式右侧对 O 的位置是不变的，因此该矢量场本身与 O 无关。这种空间矢量与矢量场之间的联系，有助于我们更直观地理解空间矢量究竟为何物，以及空间矢量表达式的真实含义。例如，假设 $\hat{\mathbf{v}}_1$ 和 $\hat{\mathbf{v}}_2$ 是两种空间速度，而 $\mathbf{V}_1$ 和 $\mathbf{V}_2$ 分别是对应的矢量场。若 $\hat{\mathbf{v}}_{\text{sum}} = \hat{\mathbf{v}}_1 + \hat{\mathbf{v}}_2$ ，那么与 $\hat{\mathbf{v}}_{\text{sum}}$ 对应的矢量场具有如下性质：对于所有 P ，有 $\mathbf{V}_{\text{sum}}(P) = \mathbf{V}_1(P) + \mathbf{V}_2(P)$ 。

例 2.2 让我们来探究式 2.3 对原点选择的不变性。我们可以通过将 $\hat{\mathbf{v}}'$ 定义为以点 P 为原点所得到的空间速度，并证明 $\hat{\mathbf{v}}' = \hat{\mathbf{v}}$ 来完成这一验证。因此， $\hat{\mathbf{v}}'$ 的表达式为

$$\hat{\mathbf{v}}' = \omega_x d_{Px} + \omega_y d_{Py} + \omega_z d_{Pz} + v_{Px} d_x + v_{Py} d_y + v_{Pz} d_z,$$

任务是将此表达式与式 2.3 进行等价推导。为了简化问题，我们选择 P 的坐标为 $(r, 0, 0)$ ，相对于 O_{xyz} 来确定，如图 2.2 所示。在这种情况下，线性速度的坐标可由式 2.1 给出为

$$\begin{bmatrix} v_{Px} \\ v_{Py} \\ v_{Pz} \end{bmatrix} = \underline{v}_O + \underline{\omega} \times \begin{bmatrix} r \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} v_{Ox} \\ v_{Oy} + \omega_z r \\ v_{Oz} - \omega_y r \end{bmatrix};$$

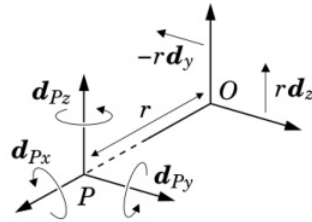


图 2.2: 在 O 处表示 d_{Py} 和 d_{Pz}

并且，依赖于 P 的三个基矢量分别为

$$\begin{aligned} d_{Px} &= d_{Ox}, \\ d_{Py} &= d_{Oy} + rd_z, \\ d_{Pz} &= d_{Oz} - rd_y. \end{aligned}$$

将这些方程综合起来, 可得

$$\begin{aligned} \dot{\vartheta}' &= \omega_x d_{Ox} + \omega_y (d_{Oy} + rd_z) + \omega_z (d_{Oz} - rd_y) \\ &+ v_{Ox} d_x + (v_{Oy} + \omega_z r) d_y + (v_{Oz} - \omega_y r) d_z, \end{aligned}$$

可以看出, 该结果与式 2.3 完全相同。

回到这三个基矢量, 我们有 $\mathbf{d}_{Px} = \mathbf{d}_{Ox}$, 因为直线 Px 和 Ox 实际上是同一条直线。要理解 $\mathbf{d}_{Py}$ 的表达式, 不妨想象一个刚体, 它以单位角速度绕 Py 旋转, 从而其空间速度为 $\mathbf{d}_{Py}$ 。此时, 位于 O 的刚体固定点将拥有大小为 r 的 z 方向线性速度, 如图 2.2 所示; 因此, 刚体的速度可以表示为: 以单位角速度绕 Oy 直线旋转, 再加上大小为 r 的 z 方向线性速度 (即 $\mathbf{d}_{Oy} + r\mathbf{d}_z$)。¹ 通过同样的推导, 再用 Pz 代入, 即可得出 $\mathbf{d}_{Pz}$ 的给定表达式。

本文所作的分析很容易推广到任意位置 P 的情形, 并且同样可以运用类似的论证, 证明式 2.3 对 O_{xyz} 的方向是保持不变的。

§2.3 空间力

既然已经得到了空间速度的表达式, 现在我们来对空间力进行同样的处理。给定一个任意点 O ——该点可以位于空间中的任何位置——作用在刚体 B 上的最一般力由以下两部分组成: 沿通过 O 的直线作用的线性力 $\mathbf{f}$, 以及一个力偶 $\mathbf{n}_O$ 。

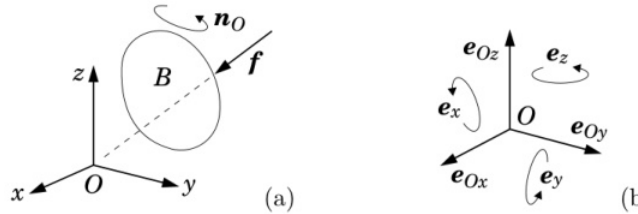


图 2.3: 以 $\mathbf{f}$ 和 $\mathbf{n}_O$ 表示的刚体受力 (a), 以及普吕克利坐标系的基矢量 (b)

其大小等于以 O 为轴心的总力矩。(参见图 2.3(a)) 再次强调, 这一表示方式仅需一个任意点和两个三维向量。 ω 被 $\mathbf{f}$ 取代, 而 $\mathbf{n}_O$ 则被 $\mathbf{v}_O$ 取代。

在已知 O 、 $\mathbf{n}_O$ 和 $\mathbf{f}$ 的情况下, 我们可以利用公式计算任意其他点 P 的总力矩:

$$\mathbf{n}_P = \mathbf{n}_O + \mathbf{f} \times \overrightarrow{OP}.$$

该方程是方程 2.1 的力矢量版本, 并且与该方程具有相同的性质: 等式右侧与 O 的位置无关。量 P 、 $\mathbf{n}_P$ 和 $\mathbf{f}$ 共同描述了与 O 、 $\mathbf{n}_O$ 和 $\mathbf{f}$ 相同的施加力。

我们现在引入一个笛卡尔坐标系 O_{xyz} , 其原点位于 O , 并利用该坐标系来定义正交基 $\{i, j, k\} \in \mathbb{E}^3$, 从而能够将 n_O 和 f 用其笛卡尔坐标表示出来:

$$n_O = n_{Ox}i + n_{Oy}j + n_{Oz}k$$

并且

$$f = f_x i + f_y j + f_z k.$$

在将 n_O 和 f 分解为各自的分量后, 我们可以将作用于 B 的力描述为六个基本力之和: 分别在 x 、 y 和 z 方向上的力偶, 大小分别为 n_{Ox} 、 n_{Oy} 和 n_{Oz} ; 以及沿着 Ox 、 Oy 和 Oz 直线作用的线性力, 大小分别为 f_x 、 f_y 和 f_z 。

这个坐标系还定义了 $\mathbf{F}^6$ 上的以下普吕克基底:

$$\mathcal{E}_O = \{e_x, e_y, e_z, e_{Ox}, e_{Oy}, e_{Oz}\} \subset \mathbf{F}^6,$$

如图 2.3(b) 所示。前三个元素分别是 x 、 y 和 z 方向上的单位力偶, 其余元素则是沿着 Ox 、 Oy 和 Oz 直线作用的单位线性力。将这些基矢量的定义与刚刚描述的六个基本力进行对比, 可以立即得出结论: 若 $\hat{f} \in \mathbf{F}^6$ 是代表与 O 、 n_O 和 f 同一力的空间力矢量, 则

$$\hat{f} = n_{Ox}e_x + n_{Oy}e_y + n_{Oz}e_z + f_x e_{Ox} + f_y e_{Oy} + f_z e_{Oz}.$$

该等式右侧的每一个单项式都依赖于 O_{xyz} 的位置和方向, 但整个表达式却是不变的。因此, $\hat{f}$ 的值完全不依赖于 O_{xyz} , 而仅取决于作用在 B 上的力。在 $\mathcal{E}_O$ 坐标系中, 表示 $\hat{f}$ 的坐标向量为

$$\underline{\hat{f}}_O = \begin{bmatrix} n_{Ox} \\ n_{Oy} \\ n_{Oz} \\ f_x \\ f_y \\ f_z \end{bmatrix} = \begin{bmatrix} \underline{n}_O \\ \underline{f} \end{bmatrix},$$

其中 $\underline{n}_O = [n_{Ox} \ n_{Oy} \ n_{Oz}]^T$, $\underline{f} = [f_x \ f_y \ f_z]^T$ 。

§2.4 普吕克记号法

普吕克坐标可追溯至 19 世纪, 但其基矢却已焕然一新。它们是 6 维向量的理想坐标系, 一方面是因为这些坐标易于实现高效的计算机运算, 另一方面则是因为它们最为方便、使用起来也最为轻松。

公式 2.4 和 2.9 中出现的记号, 将普吕克坐标与专用于速度和力的记号相结合。在速度方面, 我们采用了两种不同的符号 ($\boldsymbol{v}$ 和 $\boldsymbol{\omega}$) 来表示 3D 向量中的线性速度和角速度, 并且在普吕克坐

标列表中沿用这些符号。对于力而言，情况亦然：我们为 3D 线性力和力矩向量分别使用了不同的符号，并将这些符号保留在普吕克坐标列表中。

在一般情况下，这些坐标与向量同名。因此，如果 $\hat{m}$ 和 $\hat{f}$ 分别表示 $\mathbf{M}^6$ 和 $\mathbf{F}^6$ 中的通用元素，则它们的普吕克坐标将为

$$\hat{m}_O = \begin{bmatrix} m_x \\ m_y \\ m_z \\ m_{Ox} \\ m_{Oy} \\ m_{Oz} \end{bmatrix} = \begin{bmatrix} \underline{m} \\ \underline{m}_O \end{bmatrix} \text{ and } \hat{f}_O = \begin{bmatrix} f_{Ox} \\ f_{Oy} \\ f_{Oz} \\ f_x \\ f_y \\ f_z \end{bmatrix} = \begin{bmatrix} \underline{f} \\ \underline{f}_O \end{bmatrix}.$$

我们始终会按照“角先线后”的顺序列出普吕克坐标；也就是说，三个角坐标将始终排在首位。不过，读者应知晓，完全可以先列出三个线性坐标，而且在文献中也会遇到这种替代性的排列方式。从……

从数学的角度来看，坐标排列的顺序并无差别。然而，从计算的角度来看，这却至关重要——因为软件通常都是按照某一特定的排列顺序进行编写和运行的。

§2.5 线向量与自由向量

线向量是一种以有向线段和大小为特征的物理量。刚体的纯旋转即为线向量，同样，作用于刚体的线性力也属于线向量。自由向量则是指一种既能确定大小又能明确方向的物理量。刚体的纯平移属于自由向量，纯力偶亦然。线向量可用五个数值来描述，而自由向量则只需三个数值即可。此外，线向量还可以由一个自由向量以及线上的任意一点共同确定。

设 $\hat{s}$ 为任意空间向量或运动矢量、力矢量，且 $\mathbf{s}$ 和 $\mathbf{s}_O$ 分别是用于提供 $\hat{s}$ Plücker 坐标值的两个三维坐标向量。以下是关于直线向量和自由向量的一些基本事实。

若 $s = 0$ ，则 $\hat{s}$ 是一个自由向量。

若 $s \cdot s_O = 0$ ，则 $\hat{s}$ 是一条直线向量。该直线的方向由 s 确定，而这条直线本身是由所有满足 $\overrightarrow{OP} \times s = s_O$ 的点 P 组成的集合。

任何空间向量都可以表示为一条直线向量与一个自由向量之和。如果该直线向量必须经过某一给定的点，则这一表示式是唯一的。

除自由向量外，任何空间向量都可以唯一地表示为一条直线向量与一个平行的自由向量之和。对于运动向量而言，其表示式为：

$$[s_O - hs] + \begin{bmatrix} \mathbf{0} \\ hs \end{bmatrix} \text{ where } h = \frac{s \cdot s_O}{s \cdot s}.$$

这一最后的结果表明，除自由向量外，任何空间向量都可以通过一条有向直线、一个线性大小以及一个角向量来唯一地加以描述。自由向量同样可以采用这种形式进行描述，但这种描述并非唯一性的，因为只有直线的方向才是关键。

根据螺旋理论，刚体最一般的运动包括在空间中绕某条直线旋转，同时沿该直线作平移运动（即螺旋运动或螺纹运动）。这样的运动被称为扭转。同样地，作用于刚体的最一般的力，包括沿空间中某条直线作用的线性力，以及沿该直线作用的力偶（因此力和力偶是平行的）。这样的力被称为扳手力。在两种情况下，该直线本身被称为螺旋轴，而自由向量大小与直线向量大小之比被称为螺距。纯自由向量被视为无限螺距的扭转和扳手力，它们并不具有确定的螺旋轴。 $\mathbf{M}^6$ 和 $\mathbf{F}^6$ 的各元素，分别可被视为扭转和扳手力。

2.5.1 大小

空间向量并非欧几里得空间中的向量，因此不具有欧几里得意义上的大小。不过，有四类空间向量可以定义出有意义的大小：旋转、平移、线性力以及力偶（即 $\mathbf{M}^6$ 和 $\mathbf{F}^6$ 中的所有直线向量和自由向量）。大小仅在各分类内才可进行比较；例如，一个旋转的大小可以与其他旋转的大小相比较，但不能与平移或力的大小相比较。

一个模长为 1 的向量可以被称为单位向量。然而，“模长为 1”的含义取决于所选的单位制。例如，单位旋转速度是指每选定的时间单位内旋转 1 弧度；而单位平移速度则是每选定的时间单位内移动 1 个长度单位。因此，所有均为单位向量的普吕克基底向量，不仅依赖于坐标系的位置和速度，还依赖于所选的单位制。

§2.6 标量积

标量积定义在空间向量之上，其中其中一个参数必须是运动向量，另一个参数必须是力向量，其结果是一个标量，用于表示能量、功率或类似量。给定 $m \in \mathbf{M}^6$ 和 $f \in \mathbf{F}^6$ ，标量积可以写作 $m \cdot f$ 或 $f \cdot m$ ，二者含义相同；但表达式 $m \cdot m$ 和 $f \cdot f$ 并未被定义（即， $\mathbf{M}^6$ 和 $\mathbf{F}^6$ 都不存在内积）。若 f 是作用于刚体的力，而 m 是该刚体的速度，则 $m \cdot f$ 即为 f 所传递的功率。

标量积在空间向量之间建立了联系，这种联系在形式上被称为对偶关系：每个空间都是另一个空间的^{对偶}。这一性质对于空间向量代数的实际意义如下：

我们在 $\mathbf{M}^6$ 和 $\mathbf{F}^6$ 上使用对偶坐标：

力向量与运动向量遵循不同的坐标变换规则；以及

存在两种叉积运算符：一种用于运动向量，另一种用于力（参见 §2.9）。

$\mathbf{M}^6$ 和 $\mathbf{F}^6$ 上的对偶坐标系，是指由两个基底构成的坐标系，即 $\{d_1, \dots, d_6\} \subset \mathbf{M}^6$ 和 $\{e_1, \dots, e_6\} \subset \mathbf{F}^6$ ，其中基矢满足以下条件：

$$d_i \cdot e_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{otherwise.} \end{cases}$$

从公式 2.2 和 2.7 中对 $\mathcal{D}_O$ 和 $\mathcal{E}_O$ 的定义可以看出，这两个基底构成了一个对偶坐标系。（将 d_{Ox} 视作 d_1 ， d_{Oy} 视作 d_2 ，以此类推。）

对偶坐标系最重要的性质在于：如果 $\underline{m}$ 和 $\underline{f}$ 在对偶坐标系中分别代表 $m \in \mathbf{M}^6$ 和 $f \in \mathbf{F}^6$ ，那么

$$\underline{m} \cdot \underline{f} = \underline{m}^T \underline{f}.$$

这一性质的一个直接后果是：我们需为运动和力分别使用不同的坐标变换矩阵。若矩阵 $\mathbf{X}$ 对运动向量执行坐标变换，而 $\mathbf{X}^*$ 对力向量执行同样的变换，那么两者之间的关系可表示为

$$\mathbf{X}^* = \mathbf{X}^{-T}.$$

该等式源于这样一个要求：对于所有 $\underline{m}$ 和 $\underline{f}$ ，有 $\underline{m}^T \underline{f} = (\mathbf{X}\underline{m})^T (\mathbf{X}^* \underline{f})$ 。

§2.7 使用空间向量

空间向量是用于表达和分析刚体及刚体系统的动力学行为的工具。本节列出了使用空间向量的一系列规则，这些规则阐述了空间向量与其所代表的物理现象之间的联系。最后几项规则为一些我们尚未接触过的量提供了预览：加速度、动量和惯性。不过，这一列表尚不完整。

用法： $\mathbf{M}^6$ 的各元素描述了刚体运动的诸多方面，例如：速度、加速度、无穷小位移，以及运动自由度和约束方向。 $\mathbf{F}^6$ 的各元素则描述了作用于刚体上的力，以及与力相关的量，如动量、冲量，以及力的自由度和约束方向。

唯一性： $\mathbf{M}^6$ 的各元素与刚体所有可能运动的集合之间存在一一对应的关系。同样地， $\mathbf{F}^6$ 的各元素与作用于一刚体的所有可能力的集合之间也存在一一对应的关系（即，所有力矩的集合）。

相对速度：若物体 B_1 和 B_2 分别具有速度 $\underline{v}_1$ 和 $\underline{v}_2$ ，则 B_2 相对于 B_1 的相对速度为 $\underline{v}_{rel} = \underline{v}_2 - \underline{v}_1$ 。

刚性连接：若两个物体以刚性方式连接在一起，则它们的速度相同。

力的合成：若力 $\underline{f}_1$ 和 $\underline{f}_2$ 同时作用于同一个刚体，则它们可等效为一个单一的力 $\underline{f}_{tot}$ ，其大小为 $\underline{f}_{tot} = \underline{f}_1 + \underline{f}_2$ 。

作用力与反作用力：若物体 B_1 对物体 B_2 作用一个力 $\underline{f}$ ，则 B_2 也会对 B_1 作用一个大小为 $-\underline{f}$ 的力（牛顿第三定律）。

标量积：若一个力 $\underline{f}$ 作用于一个具有速度 $\underline{v}$ 的刚体，那么该力所传递的功率为 $\underline{f} \cdot \underline{v}$ 。

尺度变换：当一个力 $\alpha \underline{f}$ 作用于一个具有速度 $\beta \underline{v}$ 的物体时，其传递的功率是作用于一个具有速度 $\underline{v}$ 的物体上、由 $\underline{f}$ 所产生的力的 $\alpha\beta$ 倍。以恒定速度 $\underline{v}$ 运动的物体，在 α 个时间单位内完成的运动，与以速度 $\alpha \underline{v}$ 运动的物体在 1 个时间单位内完成的运动，其实质上是相同的。

加速度：空间加速度是指空间速度的变化率。空间加速度是真矢量，并且遵循与速度相同的求和法则。例如，若 $\underline{v}_{rel} = \underline{v}_2 - \underline{v}_1$ ，则 $\underline{a}_{rel} = \underline{a}_2 - \underline{a}_1$ 。

惯量的叠加：若两个物体 B_1 和 B_2 的惯量分别为 I_1 和 I_2 ，且它们被连接在一起形成一个单一的复合刚体，则该复合体的惯量为 $I_1 + I_2$ 。

动量：若一个刚体具有空间惯量 I ，并具有速度 $\underline{v}$ ，则其动量为 $I\underline{v}$ 。

运动方程：物体动量的变化率等于作用于该物体的力；即 $\underline{f} = \frac{d}{dt}(I\underline{v})$ 。对上述表达式进行求导，可得公式 $\underline{f} = I\underline{a} + \underline{v} \times^* I\underline{v}$ 。

例 2.3 图 2.4 展示了一组刚体以链状连接在一起，其中一端与固定底座相连。这种系统是工业机器人机械臂的典型结构，被称为运动链。该系统共有 N 个运动体和 N 个关节，各运动体之

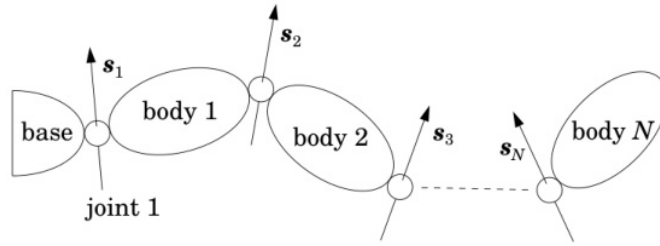


图 2.4: 运动链

间通过连接方式相互连接，其中关节 i 从运动体 $i-1$ 连接到运动体 i 。（底座为运动体 0。）我们说，关节将“从”一个运动体“连接”到另一个运动体，因此我们可以定义关节处的速度为：相对于“从”运动体而言，“到”运动体的速度。因此，若 v_{ji} 是关节 i 处的速度，而 v_i 是运动体 i 的速度，则有

$$v_{ji} = v_i - v_{i-1}.$$

我们将假设每个关节仅允许单个运动自由度，因此 v_{ji} 可以表示为单个运动矢量的标量倍数。具体而言，

$$v_{ji} = s_i \dot{q}_i,$$

其中 s_i 和 $\dot{q}_i$ 分别为关节 i 的轴向矢量和速度变量。若关节 i 为回转关节，则 s_i 将是一个单位旋转矢量，因此当 $\dot{q}_i = 1$ 时， v_{ji} 将是一个单位旋转速度。同样地，若关节 i 为直动关节，则 s_i 将是一个单位平移矢量。

我们来计算系统中各运动体的速度。根据式 2.14 和式 2.15，我们有

$$v_i = v_{i-1} + s_i \dot{q}_i. (v_0 = 0)$$

此方程使我们能够依次计算各运动体的速度，从底座开始，逐步向链的自由端推进。速度的另一种表示形式为

$$v_i = \sum_{j=1}^i s_j \dot{q}_j.$$

从计算的角度来看，式 2.16 比式 2.17 更加高效，因为它可以重复利用先前计算的结果。此外， v_i 的另一种表示形式为

$$v_i = [s_1 s_2 \cdots s_i 0 \cdots 0] \begin{bmatrix} \dot{q}_1 \\ \vdots \\ \dot{q}_N \end{bmatrix} = J_i \dot{q}.$$

在该方程中， J_i 是一个 $6 \times N$ 的矩阵，称为运动体 i 的身体雅可比矩阵，而 $\dot{q}$ 则是关节空间速度矢量。

§2.8 坐标变换

对于运动矢量，从 A 坐标系到 B 坐标系的坐标变换通常表示为 ${}^B\mathbf{X}_A$ ；而对于力矢量，同样的变换则为 ${}^B\mathbf{X}_A^*$ 。两者之间的关系由下式给出： ${}^B\mathbf{X}_A^* = \mathbf{B}\mathbf{X}_A^{-T}$ （参见式 2.13）。在本节中，我们将推导出用于在普吕克坐标系之间进行坐标变换的公式。

设 A 和 B 分别为两个普吕克坐标系。每个坐标系均由一个笛卡尔坐标系的位置与方向定义，且每个坐标系同时也定义了一个笛卡尔坐标系。为方便起见，我们对这三者均使用相同的名称；因此，坐标系 A 同时定义了普吕克坐标系 A 和笛卡尔坐标系 A ，坐标系 B 亦然。 ${}^B\mathbf{X}_A$ 的计算公式仅依赖于坐标系 B 相对于坐标系 A 的位置与方向，因此它可以表示为两个更简单的变换的乘积：一个取决于坐标系 B 的相对位置，另一个则取决于其相对方向。

2.8.1 旋转

设 A 和 B 是两个具有共同原点 O 的笛卡尔坐标系。在已知 O 的情况下，任意运动矢量 $\hat{\mathbf{m}} \in \mathbb{M}^6$ 都可以由两个三维向量 $\mathbf{m}$ 和 $\mathbf{m}_O$ 来表示，而 $\hat{\mathbf{m}}$ 的普吕克坐标将分别是 $\mathbf{m}$ 和 $\mathbf{m}_O$ 的笛卡尔坐标。因此，令 ${}^A\hat{\mathbf{m}}$ 、 ${}^A\mathbf{m}$ 、 ${}^A\mathbf{m}_O$ 、 ${}^B\hat{\mathbf{m}}$ 、 ${}^B\mathbf{m}$ 和 ${}^B\mathbf{m}_O$ 分别代表 $\hat{\mathbf{m}}$ 、 $\mathbf{m}$ 和 $\mathbf{m}_O$ 在 A 和 B 坐标系中的坐标向量，并令 $\mathbf{E}$ 为将三维向量从 A 坐标系转换至 B 坐标系的 3×3 旋转矩阵（即 $\mathbf{E} = {}^B\mathbf{E}_A$ ）。至此，我们已获得：

$${}^B\hat{\mathbf{m}} = \begin{bmatrix} {}^B\mathbf{m} \\ {}^B\mathbf{m}_O \end{bmatrix} = \begin{bmatrix} \mathbf{E}^A\mathbf{m} \\ \mathbf{E}^A\mathbf{m}_O \end{bmatrix} = \begin{bmatrix} \mathbf{E} & 0 \\ 0 & \mathbf{E} \end{bmatrix} {}^A\hat{\mathbf{m}},$$

SO

$${}^B\mathbf{X}_A = \begin{bmatrix} \mathbf{E} & 0 \\ 0 & \mathbf{E} \end{bmatrix}.$$

力矢量的等效变换完全相同：

$${}^B\mathbf{X}_A^* = {}^B\mathbf{X}_A^{-T} = \begin{bmatrix} \mathbf{E} & 0 \\ 0 & \mathbf{E} \end{bmatrix}.$$

2.8.2 平移

设 O 和 P 为空间中的两个点，且每个点都对应一个笛卡尔坐标系，这两个坐标系具有相同的方向。任意运动矢量 $\hat{\mathbf{m}} \in \mathbb{M}^6$ 可以在 O 点由三维向量 $\mathbf{m}$ 和 $\mathbf{m}_O$ 表示，在 P 点则由向量 $\mathbf{m}$ 和 $\mathbf{m}_P$ 表示，其中 $\mathbf{m}_P = \mathbf{m}_O - \overrightarrow{OP} \times \mathbf{m}$ （依据式 2.1）。在两种情况下， $\hat{\mathbf{m}}$ 的普吕克坐标都是相应一对三维向量的笛卡尔坐标，因此我们有

$$\hat{\mathbf{m}}_P = \begin{bmatrix} \mathbf{m} \\ \mathbf{m}_P \end{bmatrix} = \begin{bmatrix} \mathbf{m} & \mathbf{m} \\ \mathbf{m}_O - \mathbf{r} \times \mathbf{m} & \mathbf{m} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -\mathbf{r} \times & 1 \end{bmatrix} \hat{\mathbf{m}}_O,$$

其中 $\mathbf{r} = \overrightarrow{OP}$ 。因此

$$P_{XO} = \begin{bmatrix} 1 & 0 \\ -r \times & 1 \end{bmatrix}.$$

力矢量的等效变换是

$$P_{XO}^* = P_{XO}^{-T} = \begin{bmatrix} 1 & -r \times \\ 0 & 1 \end{bmatrix}.$$

形式为 $a \times$ 的表达式（读作“a-cross”）表示一种运算符，它将 b 映射为 $a \times b$ 。若 a 是一个三维坐标向量，则 $a \times$ 由以下公式给出的 3×3 矩阵表示：

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} \times = \begin{bmatrix} 0 & -z & y \\ z & 0 & -x \\ -y & x & 0 \end{bmatrix}.$$

表 2.1：欧几里得交叉运算符的性质

该运算符具有多项数学性质，这些性质列于表 2.1 中。我们将形式为 $a \times b \times c$ 的表达式解释为 $(a \times)(b \times)c$ ，这与 $a \times (b \times c)$ 是同一概念。任何其他解释方式都必须使用括号。同样地，表达式 $a + b \times c$ 和 $Ea \times$ 分别表示 $a + (b \times c)$ 和 $E(a \times)$ ，而任何其他解释方式也都需要使用括号。

2.8.3 一般变换

设 A 和 B 分别为以 O 和 P 为原点的直角坐标系；设 r 为用 A 坐标表示 $\overrightarrow{OP}$ 的坐标向量；设 E 为将三维向量从 A 坐标转换为 B 坐标的旋转矩阵。则从 A 坐标到 B 坐标的普吕克变换，即通过向 r 平移并结合 E 进行旋转得到的结果。运动矢量变换的公式为

$$B_{XA} = \begin{bmatrix} E & 0 \\ 0 & E \end{bmatrix} \begin{bmatrix} 1 & 0 \\ -r \times & 1 \end{bmatrix} = \begin{bmatrix} E & 0 \\ -Er \times & E \end{bmatrix};$$

力矢量的等效变换是

$$B_{XA}^* = \begin{bmatrix} E & 0 \\ 0 & E \end{bmatrix} \begin{bmatrix} 1 & -r \times \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} E & -Er \times \\ 0 & E \end{bmatrix};$$

而这两种变换的逆变换分别是

$$A_{XB} = \begin{bmatrix} 1 & 0 \\ r \times & 1 \end{bmatrix} \begin{bmatrix} E^T & 0 \\ 0 & E^T \end{bmatrix} = \begin{bmatrix} E^T & 0 \\ r \times E^T & E^T \end{bmatrix}$$

以及

$$A_{XB}^* = \begin{bmatrix} \mathbf{1} & r \times \\ \mathbf{0} & \mathbf{1} \end{bmatrix} \begin{bmatrix} E^T & \mathbf{0} \\ \mathbf{0} & E^T \end{bmatrix} = \begin{bmatrix} E^T & r \times E^T \\ \mathbf{0} & E^T \end{bmatrix}.$$

作为对比，从 A 坐标到 B 坐标（齐次）的 4×4 齐次坐标变换矩阵的公式为

$$B_{T_A} = \begin{bmatrix} E & -Er \\ 0 & 1 \end{bmatrix}.$$

$$\text{rx}(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c & s \\ 0 & -s & c \end{bmatrix} \quad \text{ry}(\theta) = \begin{bmatrix} c & 0 & -s \\ 0 & 1 & 0 \\ s & 0 & c \end{bmatrix} \quad \text{rz}(\theta) = \begin{bmatrix} c & s & 0 \\ -s & c & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$(c = \cos(\theta), s = \sin(\theta))$$

$$\text{rot}(E) = \begin{bmatrix} E & 0 \\ 0 & E \end{bmatrix} \text{xlt}(\mathbf{r}) = \begin{bmatrix} \mathbf{1} & 0 \\ -\mathbf{r} \times & \mathbf{1} \end{bmatrix} \text{rotx}(\theta) = \text{rot}(\text{rx}(\theta))$$

表 2.2: 旋转与普吕克变换的函数

为了便于描述普吕克变换，我们偶尔会使用表 2.2 中列出的这些函数。从这些函数的角度来看，方程 2.24 可以写成 ${}^B\mathbf{X}_A = \text{rot}(\mathbf{E}) \text{xlt}(\mathbf{r})$ ，依此类推。

最后，请注意，我们在描述坐标变换时采用了以下约定： ${}^B\mathbf{X}_A$ （或 ${}^B\mathbf{E}_A$ ）的公式，对应于这样一种描述：你需要如何移动坐标系 A ，才能使它与坐标系 B 合一。例如，“ $\text{rotx}(\theta)$ ”这一表达式，可以理解为“绕 x 轴旋转 θ 度”的简写形式，其值即为当坐标系 B 相对于坐标系 A 以 θ 度的角度绕其共同的 x 轴旋转时，对应的坐标变换 ${}^B\mathbf{X}_A$ 。读者应牢记，有些作者采用不同的约定。

§2.9 空间叉积

设 $\mathbf{r} \in \mathbb{E}^3$ 是一个随角速度 $\boldsymbol{\omega}$ 旋转的欧几里得向量，但其本身并未发生任何变化。该向量的导数可由著名的公式 $\dot{\mathbf{r}} = \boldsymbol{\omega} \times \mathbf{r}$ 表示。在这一背景下， $\boldsymbol{\omega} \times$ 像一个求导算子一样运作：它将 $\mathbf{r}$ 映射为 $\dot{\mathbf{r}}$ 。

事实证明，这一思想可以推广到空间向量中，但我们需要引入两个运算符：一个用于描述运动，另一个用于描述力。因此，设 $\hat{\mathbf{m}} \in \mathbb{M}^6$ 和 $\hat{\mathbf{f}} \in \mathbb{F}^6$ 为两个以速度 $\hat{\mathbf{v}} \in \mathbb{M}^6$ 运动的空间向量（例如，它们固定在具有该速度的刚体中），但其本身并未发生任何变化。现在我们定义了两个新运算符 $\times$ 和 $\times^*$ ，它们需满足以下等式：

$$\dot{\hat{\mathbf{m}}} = \hat{\mathbf{v}} \times \hat{\mathbf{m}}$$

以及

$$\dot{\hat{\mathbf{f}}} = \hat{\mathbf{v}} \times^* \hat{\mathbf{f}}.$$

运算符 $\times^*$ 可被视为 $\times$ 的对偶运算。这两个运算符之间的关系，与坐标变换矩阵 X 和 X^* 之间的关系相似。

这些运算符可以通过图 2.5 所示的乘法表明确地定义。表中每一项都是上排所列基矢的时导数，前提是该基矢的速度等于左列所列的基矢速度。例如，图 2.5(a) 展示了一个以速度 d_{Ox} 运动的坐标系，这意味着该坐标系正以单位角速度绕 Ox 直线旋转。在 δt 个时间单位内，这种运动

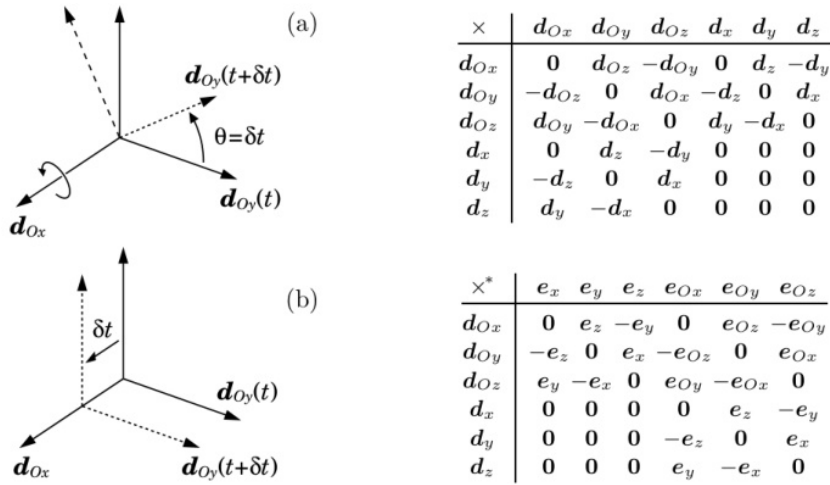


图 2.5: 空间叉积表

对 Ox 直线并无影响, 却使 Oy 在 $y-z$ 平面上旋转了 δt 弧度。因此, 我们有 $d_{Ox}(t+\delta t) = d_{Ox}(t)$, 但 $d_{Oy}(t+\delta t) = d_{Oy}(t) + \delta t d_{Oz}$ 。由此可知, 时间导数为

$$\begin{aligned} \frac{d}{dt} d_{Ox} &= \lim_{\delta t \rightarrow 0} \frac{d_{Ox}(t+\delta t) - d_{Ox}(t)}{\delta t} \\ &= \lim_{\delta t \rightarrow 0} \frac{d_{Ox}(t) - d_{Ox}(t)}{\delta t} = 0 \end{aligned}$$

以及

$$\begin{aligned} \frac{d}{dt} d_{Oy} &= \lim_{\delta t \rightarrow 0} \frac{d_{Oy}(t+\delta t) - d_{Oy}(t)}{\delta t} \\ &= \lim_{\delta t \rightarrow 0} \frac{d_{Oy}(t) + \delta t d_{Oz} - d_{Oy}(t)}{\delta t} = d_{Oz}. \end{aligned}$$

这解释了第一张表第一行的前两项。同样地, 图 2.5(b) 展示了一个以速度 d_x 运动的坐标系, 这意味着该坐标系以单位线性速度沿 x 方向进行平移。在 δt 个时间单位内, 这种运动对 Ox 直线并无影响, 却使 Oy 沿 x 方向移动了 δt 个长度单位。因此, 我们有

表 2.3: 空间叉积运算符的性质

$d_{Ox}(t+\delta t) = d_{Ox}(t)$, 这意味着 $\dot{d}_{Ox} = \mathbf{0}$, 但 $d_{Oy}(t+\delta t) = d_{Oy}(t) + \delta t d_z$, 意味着 $\dot{d}_{Oy} = \mathbf{d}_z$ 。这解释了第一张表中第四行前两项的含义。其余各项则以类似的方式进行计算。

利用图 2.5 中的表格, 我们可以推导出在普吕克坐标系中表示算子 $\hat{\mathbf{v}} \times$ 和 $\hat{\mathbf{v}} \times^*$ 的 6×6 矩阵分别为

$$\hat{\mathbf{v}}_{O \times} = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v}_O \end{bmatrix} \times = \begin{bmatrix} \boldsymbol{\omega} \times & \mathbf{0} \\ \mathbf{v}_O \times & ! \times \end{bmatrix}$$

以及

$$\hat{\mathbf{v}}_{O \times^*} = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v}_O \end{bmatrix} \times^* = \begin{bmatrix} \boldsymbol{\omega} \times & \mathbf{v}_O \times \\ \mathbf{0} & \boldsymbol{\omega} \times \end{bmatrix} = -(\hat{\mathbf{v}}_{O \times})^T,$$

并且两个坐标向量的空间叉积为

$$\begin{bmatrix} \omega \\ v_O \end{bmatrix} \times \begin{bmatrix} m \\ m_O \end{bmatrix} = \begin{bmatrix} \omega \times m \\ \omega \times m_O + v_O \times m \end{bmatrix}$$

以及

$$\begin{bmatrix} \omega \\ v_O \end{bmatrix} \times^* \begin{bmatrix} f_O \\ f \end{bmatrix} = \begin{bmatrix} \omega \times f_O + v_O \times f \\ \omega \times f \end{bmatrix}.$$

这些算子的各种性质列于表 2.3 中。

$\boldsymbol{v} \times^* = -\boldsymbol{v} \times^T$		
$(\lambda \boldsymbol{v}) \times = \lambda(\boldsymbol{v} \times)$	$(\lambda \boldsymbol{v}) \times^* = \lambda(\boldsymbol{v} \times^*)$	(λ 是一个标量)
$(\boldsymbol{u} + \boldsymbol{v}) \times = \boldsymbol{u} \times + \boldsymbol{v} \times$	$(\boldsymbol{u} + \boldsymbol{v}) \times^* = \boldsymbol{u} \times^* + \boldsymbol{v} \times^*$	
$(\boldsymbol{X}\boldsymbol{v}) \times = \boldsymbol{X}\boldsymbol{v} \times \boldsymbol{X}^{-1}$	$(\boldsymbol{X}\boldsymbol{v}) \times^* = \boldsymbol{X}^* \boldsymbol{v} \times^* (\boldsymbol{X}^*)^{-1}$	(普吕克变换)
$\boldsymbol{u} \times \boldsymbol{v} = -\boldsymbol{v} \times \boldsymbol{u}$	$(\boldsymbol{u} \times^* \boldsymbol{f}) \cdot = -\boldsymbol{f} \cdot \boldsymbol{u} \times$	
$(\boldsymbol{u} \times \boldsymbol{v}) \cdot = -\boldsymbol{v} \cdot \boldsymbol{u} \times^*$	$(\boldsymbol{u} \times \boldsymbol{v}) \times^* = \boldsymbol{u} \times^* \boldsymbol{v} \times^* - \boldsymbol{v} \times^* \boldsymbol{u} \times^*$	

§ 2.10

求导

向量的求导方式与标量相同。如果 $\boldsymbol{v}(x)$ 是关于标量变量 x 可微的向量函数，则其导数可由以下公式给出：

$$\frac{d}{dx} \boldsymbol{v}(x) = \lim_{\delta x \rightarrow 0} \frac{\boldsymbol{v}(x + \delta x) - \boldsymbol{v}(x)}{\delta x}.$$

该公式适用于所有向量。特别是，它既适用于抽象向量，也适用于坐标向量。因此，坐标向量的导数为

表 2.4: 向量表达式的求导公式

如以下公式所示，其逐元素求导结果为：

$$\frac{d}{dx} \begin{bmatrix} v_1(x) \\ \vdots \\ v_n(x) \end{bmatrix} = \lim_{\delta x \rightarrow 0} \frac{1}{\delta x} \begin{bmatrix} v_1(x + \delta x) - v_1(x) \\ \vdots \\ v_n(x + \delta x) - v_n(x) \end{bmatrix} = \begin{bmatrix} \frac{dv_1}{dx} \\ \vdots \\ \frac{dv_n}{dx} \end{bmatrix}.$$

该公式同样适用于矩阵和二阶张量。我们主要关注时间导数，通常用点号表示，即 $d\boldsymbol{v}/dt = \dot{\boldsymbol{v}}$ ， $d\dot{\boldsymbol{v}}/dt = \ddot{\boldsymbol{v}}$ ，以此类推。向量的导数仍然是同类型向量；因此，运动向量的导数是运动向量，力向量的导数是力向量，以此类推。各类向量表达式对应的求导公式详见表 2.4。

2d (n=20)	2d (n=20)	2d (n=20)	2d (n=20)	2d (n=20)	2d (n=20)	2d (n=20)	2d (n=20)
3 (n=20)	3 (n=20)	3 (n=20)	3 (n=20)	3 (n=20)	3 (n=20)	3 (n=20)	3 (n=20)
4 (n=20)	4 (n=20)	4 (n=20)	4 (n=20)	4 (n=20)	4 (n=20)	4 (n=20)	4 (n=20)
5 (n=20)	5 (n=20)	5 (n=20)	5 (n=20)	5 (n=20)	5 (n=20)	5 (n=20)	5 (n=20)

2.10.1 在移动坐标系中的求导

在移动坐标系中进行求导的问题可以表述如下：已知表示向量 $\boldsymbol{v}$ 的坐标向量，求表示 $\dot{\boldsymbol{v}}$ 的坐标向量。若基矢量为常量，则解法只需对坐标向量进行逐元素求导；否则，需要使用一个考虑基矢量运动的公式。

设 A 为与空间速度为 $\boldsymbol{v}_A$ 的笛卡尔坐标系相关联的普吕克坐标系。假设我们有两条坐标向量 ${}^A\boldsymbol{m}$ 和 ${}^A\boldsymbol{f}$ ，分别表示空间向量 $\boldsymbol{m} \in \mathbb{M}^6$ 和 $\boldsymbol{f} \in \mathbb{F}^6$ ，并希望求出在 A 坐标系中表示 $\dot{\boldsymbol{m}}$ 和 $\dot{\boldsymbol{f}}$ 的坐标向量。这些向量由以下公式给出：

$${}^A\left(\frac{d\boldsymbol{m}}{dt}\right) = \frac{d^A\boldsymbol{m}}{dt} + {}^A\boldsymbol{v}_A \times {}^A$$

以及

$${}^A\left(\frac{d\boldsymbol{f}}{dt}\right) = \frac{d^A\boldsymbol{f}}{dt} + {}^A\boldsymbol{v}_A \times {}^A\boldsymbol{f}.$$

左侧的表达式表示在 A 坐标系中表示 $\dot{\boldsymbol{m}}$ 和 $\dot{\boldsymbol{f}}$ 的坐标向量，而 $d^A\boldsymbol{m}/dt$ 和 $d^A\boldsymbol{f}/dt$ 则表示分别表示 ${}^A\boldsymbol{m}$ 和 ${}^A\boldsymbol{f}$ 的逐元素导数。 ${}^A\boldsymbol{v}_A$ 是表示 $\boldsymbol{v}_A$ 在 A 坐标系中的坐标向量。作为对比，以相同符号表示的三维欧几里得向量在旋转坐标系中的导数的著名公式为：

$${}^A\left(\frac{d\boldsymbol{u}}{dt}\right) = \frac{d^A\boldsymbol{u}}{dt} + {}^A\boldsymbol{\omega}_A \times {}^A\boldsymbol{u}.$$

在该方程中， $\boldsymbol{u} \in \mathbb{E}^3$ 是待求导的向量，而 $\boldsymbol{\omega}_A$ 则是坐标系的角速度。

注意：在构建后续章节中所涉及的动力学算法时，我们通常会避免在运动坐标系中进行求导操作，而是采用静止坐标系来编写算法——这些静止坐标系恰好在当前时刻与运动坐标系重合。

2.10.2 点乘与环形记号

将点乘记号与坐标向量混合使用，稍显风险，因为很难判断诸如 ${}^A\boldsymbol{m}$ 这样的符号究竟表示 ${}^A(d\boldsymbol{m}/dt)$ ，还是 $d(d^A\boldsymbol{m})/dt$ 。尽管如此，我们仍需承担这一风险，并作出如下定义：对于任意抽象向量 $\boldsymbol{v}$ ，以及任意表示 $\boldsymbol{v}$ 在 A 坐标系中的坐标向量 ${}^A\boldsymbol{v}$ ，

$$A_{\dot{\boldsymbol{v}}} = {}^A\left(\frac{d\boldsymbol{v}}{dt}\right)$$

且

$$A_{\dot{\boldsymbol{v}}}^{\circ} = \frac{d^A\boldsymbol{v}}{dt}.$$

借助这一记号, 可将式 2.36 和 2.37 写成

$$A\dot{m} = A\dot{m} + A v_A \times A m$$

且

$$A\dot{f} = A\dot{f} + A v_A \times^* A f.$$

此外, 若坐标系的身份在上下文中已十分明确, 则无需再予以明确标注。此时, 方程便可写成

$$\dot{m} = \dot{m} + v_A \times m$$

且

$$\dot{f} = \dot{f} + v_A \times^* f.$$

符号 $\dot{m}$ 和 $\dot{f}$ 可被视为由以速度 v_A 运动的观察者所看到的 m 和 f 的“表观变化率”。

示例 2.4 设 A 和 B 分别为两个具有速度 v_A 和 v_B 的直角坐标系, 且 ${}^B\mathbf{X}_A$ 是从 A 坐标系到 B 坐标系的普吕克坐标变换矩阵。我们可以利用这四组方程——源自式 2.39 至 2.41——推导出 ${}^B\mathbf{X}_A$ 的时间导数:

$$\begin{aligned} {}^B\dot{m} &= {}^B\mathbf{X}_A A\dot{m} \\ {}^B\dot{m} &= \frac{d}{dt} ({}^B\mathbf{X}_A A m) = \left(\frac{d}{dt} {}^B\mathbf{X}_A \right) A m + {}^B\mathbf{X}_A A\dot{m} \\ A\dot{m} &= A\dot{m} + A v_A \times A m \\ {}^B\dot{m} &= {}^B\dot{m} + {}^B v_B \times {}^B m. \end{aligned}$$

推导过程如下:

$$\begin{aligned} \left(\frac{d}{dt} {}^B\mathbf{X}_A \right)^A m &= {}^B \overset{\circ}{m} - {}^B\mathbf{X}_A A \overset{\circ}{m} \\ &= {}^B \overset{\circ}{m} - {}^B v_B \times {}^B m - {}^B\mathbf{X}_A (A \overset{\circ}{m} - A v_A \times A m) \\ &= {}^B \overset{\circ}{m} - {}^B\mathbf{X}_A A \overset{\circ}{m} - {}^B v_B \times {}^B\mathbf{X}_A A \overset{\circ}{m} + {}^B\mathbf{X}_A A v_A \times A m \\ &= ({}^B\mathbf{X}_A A v_A \times - {}^B v_B \times {}^B\mathbf{X}_A) A m. \end{aligned}$$

由于这一关系对所有 $A m$ 都必须成立, 因此我们有

$$\frac{d}{dt} {}^B\mathbf{X}_A = {}^B(v_A - v_B) \times {}^B\mathbf{X}_A.$$

§ 2.11 加速度

我们把刚体的空间加速度定义为其空间速度的时间导数。因此, 若某刚体在原点 O 处的空间速度表达为 $[\omega^T v_O^T]^T$, 则其空间加速度为

$$\hat{a}_O = \frac{d}{dt} \left[\frac{\omega}{v_O} \right] = \lim_{\delta t \rightarrow 0} \frac{1}{\delta t} \left[\frac{\omega(t + \delta t) - \omega(t)}{v_O(t + \delta t) - v_O(t)} \right] = \left[\frac{\dot{\omega}}{v_O} \right].$$

这个定义听起来或许显而易见，但对于那些已经精通传统刚体加速度描述方法的人来说，其中却暗藏玄机。以图 2.6 所示的物体为例，该物体以恒定角速度绕空间中的某固定直线旋转。由于该物体具有恒定的空间速度，因此其空间加速度必然为零；然而，物体中几乎每一个点都在沿圆周运动，因而必然存在加速度。这一悖论很容易得到解决。首先，我们必须认识到，空间加速度是整个物体的属性，而非单个固连于物体的各点所特有的性质。其次，我们还必须明白： v_O 并不是某个特定固连于物体的点的速度，而是衡量各点在 O 点处的流动速率。因此， $\dot{v}_O$ 并不是单个点的加速度，而是对流动速率变化率的度量。如果流动速率保持恒定，则即使每个点都在加速， $\dot{v}_O$ 也必然为零。

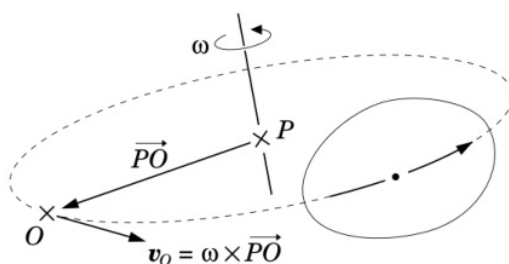


图 2.6: 图 2.6: 匀速旋转物体的加速度

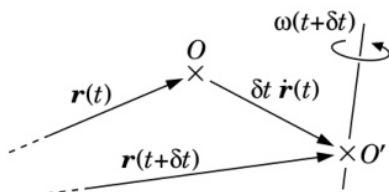


图 2.7: 图 2.7: 从 $\dot{r}$ 计算 $\dot{v}_O$

让我们将空间加速度与传统的加速度描述方法进行对比。为此，我们引入另外两个量：第二个点 O' ，它是在时间 t 时与 O 重合的固连于物体的点；以及一个向量 r ，用于表示 O' 的位置随时间的变化。因此， $\dot{r}$ 是 O' 的速度， $\ddot{r}$ 是 O' 的加速度。在经典力学的标准教材中，刚体的加速度通常由两个三维向量 $\dot{\omega}$ 和 $\ddot{r}$ 来描述；那么，让我们来考察 v_O 与 $\dot{r}$ 有何不同。根据式 2.46，我们有

$$\dot{v}_O = \lim_{\delta t \rightarrow 0} \frac{v_O(t + \delta t) - v_O(t)}{\delta t}.$$

现在， $v_O(t) = \dot{r}(t)$ ，因为 O' 在时间 t 时与 O 重合；但 $v_O(t + \delta t)$ 并不等于 $\dot{r}(t + \delta t)$ ，因为 O' 已经远离 O ，移动了 $\delta t \cdot \dot{r}(t)$ 的距离（参见图 2.7）。因此， $v_O(t + \delta t)$ 的值实际上由以下公式给出：

$$v_O(t + \delta t) = \dot{r}(t + \delta t) - \omega \times \delta t \dot{r}(t).$$

将这些方程结合在一起，我们得到

$$\begin{aligned}\dot{v}_O &= \lim_{\delta t \rightarrow 0} \frac{\dot{r}(t+\delta t) - \dot{r}(t) - \omega \times \delta t \dot{r}(t)}{\delta t} \\ &= \ddot{r} - \omega \times \dot{r},\end{aligned}$$

这表明

$$\hat{a}_O = [\ddot{r} - \omega \times \dot{r}].$$

为了便于比较，我们现在定义了一个新的 6 维向量，

$$\hat{a}'_O = \begin{bmatrix} \dot{\omega} \\ \ddot{r} \end{bmatrix},$$

我们称之为刚体的经典加速度，其在点 O 处的表示形式。这一名称源于这样一个事实：该矢量由用于刚体加速度经典描述的两个三维矢量构成。从物理意义上讲，经典加速度是位于 O' 、而非 O 的普吕克坐标系中，空间速度的表观导数——也就是说，该坐标系具有纯线性速度 $\dot{r}$ 。因此，根据式 2.43， $\hat{a}_O$ 与 $\hat{a}'_O$ 之间的关系为

$$\hat{a}_O = \hat{a}'_O + \begin{bmatrix} 0 \\ \dot{r} \end{bmatrix} \times \hat{v}_O,$$

这一点可由式 2.48 和 2.49 轻松验证。

空间加速度的大优势在于，它是一个真正的矢量，并且其遵循与速度相同的叠加规则和坐标变换规则。例如，若 $\hat{v}_1$ 和 $\hat{v}_2$ 分别表示物体 B_1 和 B_2 的空间速度，而 $\hat{v}_{rel}$ 是 B_2 相对于 B_1 的相对速度，那么我们已经知道

$$\hat{v}_{rel} = \hat{v}_2 - \hat{v}_1.$$

空间加速度的等效公式可直接通过对速度公式求导得到：

$$\frac{d}{dt} \hat{v}_{rel} = \frac{d}{dt} \hat{v}_2 - \frac{d}{dt} \hat{v}_1 \Rightarrow \hat{a}_{rel} = \hat{a}_2 - \hat{a}_1.$$

因此，空间加速度只需简单地将它们相加，正如速度一样。这与传统的加速度合成公式相比，具有明显优势——传统公式往往需要借助科里奥利项。

例 2.5 中，例 2.3 介绍了若干用于描述运动链中各物体速度的公式。现在，我们来推导这些物体加速度的相应公式。首先，我们定义 $\mathbf{a}_{J_i}$ 为第 i 个关节的加速度， $\mathbf{a}_i$ 为第 i 个物体的加速度；因此， $\mathbf{a}_{J_i} = \mathbf{v}_{J_i}$ ， $\mathbf{a}_i = \mathbf{v}_i$ 。由于空间加速度具有可叠加性，我们立刻得到

$$a_{J_i} = a_i - a_{i-1},$$

这正是式 2.14 的时间导数。接下来，对式 2.15 求导得

$$a_{J_i} = s_i \ddot{q}_i + \dot{s}_i \dot{q}_i,$$

其中 $\ddot{q}_i$ 是第 i 个关节的加速度变量。鉴于 $\mathbf{s}_i$ 表示在第 i 个物体（以及第 $i-1$ 个物体）中固定的关节运动轴，我们有

$$\dot{\mathbf{s}}_i = \mathbf{v}_i \times \mathbf{s}_i.$$

将这三个方程结合在一起，即可得到

$$\mathbf{a}_i = \mathbf{a}_{i-1} + \mathbf{s}_i \ddot{\mathbf{q}}_i + \mathbf{v}_i \times \mathbf{s}_i \dot{\mathbf{q}}_i,$$

这正是式 2.16 的导数。接下来，对式 2.17 求导得

$$\begin{aligned} \mathbf{a}_i &= \sum_{j=1}^i \mathbf{s}_j \ddot{\mathbf{q}}_j + \mathbf{v}_j \times \mathbf{s}_j \dot{\mathbf{q}}_j \\ &= \sum_{j=1}^i \mathbf{s}_j \ddot{\mathbf{q}}_j + \sum_{j=1}^i \sum_{k=1}^{j-1} \mathbf{s}_k \times \mathbf{s}_j \dot{\mathbf{q}}_j \dot{\mathbf{q}}_k. \end{aligned}$$

最后，对式 2.18 求导得

$$\mathbf{a}_i = \mathbf{J}_i \ddot{\mathbf{q}} + \dot{\mathbf{J}}_i \dot{\mathbf{q}},$$

其中 $\ddot{\mathbf{q}}$ 是关节空间加速度矢量， $\dot{\mathbf{J}}_i = [\dot{\mathbf{s}}_1 \mathbf{s}_2 \cdots \mathbf{s}_i \mathbf{0} \cdots \mathbf{0}]$ 。

§ 2.12 动量

图 2.8 所示为一运动刚体，其质心与当前时刻的固定点 C 重合。该刚体具有空间速度 $\hat{\mathbf{v}}$ ，在 C 处由两个三维向量 $\boldsymbol{\omega}$ 和 $\mathbf{v}_C$ 表示。若该刚体的质量为 m ，且在其质心周围具有旋转惯量 $\bar{I}_C$ ，则其动量可由两个三维向量 $\mathbf{h} = m\mathbf{v}_C$ 和 $\mathbf{h}_C = \bar{I}_C \boldsymbol{\omega}$ 来描述。其中， $\mathbf{h}$ 表示刚体线性动量的大小与方向，而 $\mathbf{h}_C$ 则表示刚体的固有角动量。线性动量实际上是一种线矢量（如同线性力），其作用线通过刚体的质心。刚体关于任意给定点 O 的角动量，等于其固有角动量与该点处的线性动量之和：

$$\mathbf{h}_O = \mathbf{h}_C + \overrightarrow{OC} \times \mathbf{h},$$

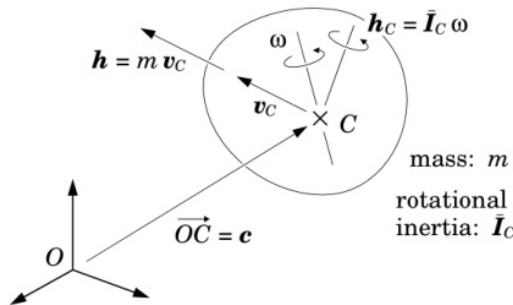


图 2.8: 空间动量

这本质上与式 2.6 所用的公式完全相同。

设 $\hat{\mathbf{h}}$ 表示该刚体的空间动量，并设 $\hat{\mathbf{h}}_C$ 和 $\hat{\mathbf{h}}_O$ 分别为在以 C 和 O 为原点的普吕克坐标系中表示 $\hat{\mathbf{h}}$ 的坐标向量。这些坐标向量的定义如下：

$$\hat{h}_C = \begin{bmatrix} h_C \\ h \end{bmatrix} = \begin{bmatrix} \bar{I}_C \omega \\ m v_C \end{bmatrix}$$

以及

$$\hat{h}_O = \begin{bmatrix} h_O \\ h \end{bmatrix} = \begin{bmatrix} \bar{I}_C \omega + \overrightarrow{OC} \times m v_C \\ m v_C \end{bmatrix} = \begin{bmatrix} 1 & \overrightarrow{OC} \times \\ 0 & 1 \end{bmatrix} \hat{h}_C.$$

由于空间动量向量是 $\mathbf{F}^6$ 中的元素，它们与其他空间力向量具有相同的代数性质。特别是，它们遵循以下变换公式： ${}^B \hat{\mathbf{h}} = {}^B \mathbf{X}_A^* {}^A \hat{\mathbf{h}}$ ，并且动量与 $\mathbf{M}^6$ 中的元素之间还定义了标量积。图 2.8 中刚体的动能为 $\frac{1}{2} \hat{\mathbf{h}} \cdot \hat{\mathbf{v}}$ 。

§ 2.13 惯性

刚体的空间惯性张量定义了其速度与动量之间的关系。因此，空间惯性是将 $\mathbf{M}^6$ 映射到 $\mathbf{F}^6$ 的一个映射。若一个刚体具有空间速度 $\mathbf{v}$ ，且其空间惯性为 $\mathbf{I}$ ，则其动量可由以下公式给出：

$$h = I v.$$

若 h 和 v 为坐标向量，则 I 是一个 6×6 的矩阵。

设 I_O 和 I_C 分别为在以 O 和 C 为原点的普吕克坐标系中表示某一给定刚体的空间惯性的矩阵，其中 O 为任意一点，而 C 与刚体的质心重合。根据式 2.59，我们有

$$h_C = \begin{bmatrix} I_C & 0 \\ 0 & m \mathbf{1} \end{bmatrix} v_C,$$

这表明

$$I_C = \begin{bmatrix} \bar{I}_C & 0 \\ 0 & m \mathbf{1} \end{bmatrix}.$$

当原点与质心重合时，惯性矩阵将呈现出这种特殊形式。根据式 2.60，我们有

$$\begin{aligned} h_O &= \begin{bmatrix} 1 & c \times \\ 0 & 1 \end{bmatrix} I_C v_C \\ &= \begin{bmatrix} 1 & c \times \\ 0 & 1 \end{bmatrix} I_C \begin{bmatrix} 1 & 0 \\ c \times^T & 1 \end{bmatrix} v_O, \end{aligned}$$

其中 $c = \overrightarrow{OC}$ ，这表明

$$I_O = \begin{bmatrix} \bar{I}_C + m c \times c \times^T & m c \times \\ m c \times^T & m \mathbf{1} \end{bmatrix}.$$

这是普吕克坐标系中惯性矩阵的一般形式。量 $\bar{I}_C + m c \times c \times^T$ 是刚体绕 O 的转动惯量。由于 $\bar{I}_C$ 是一个对称矩阵，因此可以推知 I_C 和 I_O 都是对称的。此外，若 $m > 0$ 且 $\bar{I}_C$ 为正定矩阵，则 I_C 和 I_O 均为正定。

2.13.1 惯性的二元表示

形如 $f g \cdot$ 的表达式，其中 $f, g \in \mathbf{F}^6$ ，是一种二元运算，可将运动矢量映射为力矢量。具体而言，它将运动矢量 $m \in \mathbf{M}^6$ 映射为力矢量 $f(g \cdot m)$ ，而 $f(g \cdot m)$ 是 f 的标量倍数。该二元运算在对偶坐标系中可由一个 6×6 矩阵来表示，其形式为 f, g^T ，其中 f 和 g 分别是所选坐标系中表示 f 和 g 的坐标向量。若 $f = g$ ，则该二元运算被称为对称的，其矩阵表示同样是对称的。

空间惯性是一个对称的二元张量，它从 M^6 映射到 F^6 。因此，它可以表示为六个对称二元张量之和，如下所示：

$$I = \sum_{i=1}^6 g_i g_i \cdot \left(g_i \in \mathbf{F}^6 \right).$$

此外，物体的空间惯性是一个在物体内部固有的量：它不会随物体的运动而变化，仅会随物体的运动状态而改变。因此，我们可以将 I 表示为若干二元张量之和，其中各个向量 g_i 都固定在它们所指向的物体内部。基于这一表达式，我们可以按如下方式计算空间惯性的时间导数：

$$\begin{aligned} \frac{d}{dt} I &= \sum_{i=1}^6 (\dot{g}_i g_i \cdot + g_i \dot{g}_i \cdot) \\ &= \sum_{i=1}^6 (v \times^* g_i) g_i \cdot + \sum_{i=1}^6 g_i (v \times^* g_i) \cdot \\ &= v \times^* \sum_{i=1}^6 g_i g_i \cdot - \left(\sum_{i=1}^6 g_i g_i \cdot \right) v \times \\ &= v \times^* I - I v \times, \end{aligned}$$

其中 v 是物体的速度。在得出这一结果时，我们使用了式 2.30 以及表 2.3 和表 2.4 中的相关公式。

表 2.5：二元张量的性质

从式 2.64 中还可以推导出另一条性质，即空间惯性的坐标变换规则。设 $^A I$ 和 $^B I$ 分别表示在 A 坐标系和 B 坐标系中所对应的某一给定的空间惯性矩阵。因此，我们有

$$^A I = \sum_{i=1}^6 {}^A g_i {}^A g_i^T \text{ and } ^B I = \sum_{i=1}^6 {}^B g_i {}^B g_i^T,$$

其中 ${}^A g_i$ 和 ${}^B g_i$ 分别是 g_i 在 A 坐标系和 B 坐标系中的坐标向量。现在， ${}^B g_i = {}^B X_A^* {}^A g_i$ ，因此我们得到

$$\begin{aligned} ^B I &= \sum_{i=1}^6 {}^B X_A^* {}^A g_i {}^A g_i^T ({}^B X_A^*)^T \\ &= {}^B X_A^* {}^A I {}^A X_B. \end{aligned}$$

空间二重数

空间惯性是从 $\mathbf{M}^6$ 到 $\mathbf{F}^6$ 的映射；然而，我们还可以定义另外三种与这两个空间相关的映射，从而总共形成四种类型的空间二重数张量。其部分基本性质列于表 2.5 中。示例列中的符号 Φ 表示逆惯性（即 $\Phi = I^{-1}$ ）。“二重数形式”这一列则展示了每种张量对应的二重数中运动矢量与力矢量的排列规律。若将二重数视为从“源”空间到“目标”空间的映射，则二重数中的第一个矢

量必须属于“目标”空间，而第二个矢量则必须属于与“源”空间对偶的空间。各坐标变换公式均被标注为相似变换或全等变换。相似变换能够保持诸如秩和特征值等性质，但无法保持对称性或正定性；而全等变换则能保持诸如秩、对称性和正定性等性质，却无法保持特征值。

空间惯性的性质

我们已经介绍了坐标变换（式 2.66）、时间导数（式 2.65）以及普吕克坐标系中空间惯性的通用形式（式 2.63）。

以及惯性将速度映射为动量这一事实（式 2.61）。这里再列举几条性质。

要定义一个空间刚体的惯性，需要十个参数。更为通用的惯性类型，例如第七章中介绍的关节式刚体惯性，所需的参数多达 21 个，而这正是对称的 6×6 矩阵中独立变量的最大数量。

由多个部件组成的刚体的惯性，等于其各个部件惯性的总和。

具有空间速度 v 、空间惯性 I 的刚体的动能为

$$T = \frac{1}{2} v \cdot I v.$$

2.14 运动方程

刚体的空间运动方程表明，作用于该刚体的净力等于其动量变化率：

$$\begin{aligned} f = \frac{d}{dt}(Iv) &= Ia + (v \times^* I - Iv \times) v \\ &= Ia + v \times^* Iv. \end{aligned}$$

我们常常将运动方程写成非齐次线性方程的形式。

$$f = Ia + p,$$

其中， I 和 p 分别为系数， f 和 a 为变量。在典型应用中，通常假设系数已知，而其中一个或两个变量则未知。 p 被称为偏置力，它只是当 f 必须取值以产生零加速度时所对应的数值。若 f 是作用于物体的合外力，则 $p = v \times^* Iv$ 。

式 2.68 中的 f 始终是作用于物体的合外力，但式 2.69 提供了将合外力中某些已知分量从 f 转移到 p 的可能性。例如，假设合外力 f_{net} 是一个未知力 f_u 与一个已知的重力 f_g 的总和。此时，式 2.68 可以写成：

$$f_{net} = Ia + v \times^* Iv;$$

不过，我们也可以选择将 f_u 作为式 2.69 中的左侧项，此时方程将变为：

$$f_u = Ia + p,$$

而 p 将被定义为：

$$p = v \times^* Iv - f_g.$$

例 2.6：传统的三维运动方程可以通过将空间量展开为其三维分量，从式 2.68 中恢复出来。为了保持方程的简洁性，我们把原点置于质心处。于是，式 2.68 展开后变为：

$$\begin{aligned}
\begin{bmatrix} n_C \\ f \end{bmatrix} &= \begin{bmatrix} \bar{I}_C & 0 \\ 0 & m\mathbf{1} \end{bmatrix} \begin{bmatrix} \omega & v_C \times \\ \ddot{c} - \omega \times v_C & \end{bmatrix} + \begin{bmatrix} \omega \times & v_C \times \\ 0 & \omega \times \end{bmatrix} \begin{bmatrix} \bar{I}_C & 0 \\ 0 & m\mathbf{1} \end{bmatrix} \begin{bmatrix} \omega \\ v_C \end{bmatrix} \\
&= \begin{bmatrix} \bar{I}_C \dot{\omega} & \omega \\ m \ddot{c} - m \omega \times v_C & \end{bmatrix} + \begin{bmatrix} \omega \times \bar{I}_C \omega + m v_C \times v_C \\ m \omega \times v_C \end{bmatrix} \\
&= \begin{bmatrix} \bar{I}_C \dot{\omega} + \omega \times \bar{I}_C \omega \\ m \ddot{c} \end{bmatrix}.
\end{aligned}$$

（我们已使用式 2.48，将物体的空间加速度表示为 $\dot{\omega}$ 、 $\ddot{c}$ 和 $\mathbf{v}_C$ （即 $\dot{c}$ ）的函数。）因此，我们成功地恢复了牛顿关于质心运动的方程，

$$f = m \ddot{c},$$

同时，我们也恢复了欧拉关于物体绕其质心旋转运动的方程，

$$n_C = \bar{I}_C \dot{\omega} + \omega \times \bar{I}_C \omega.$$

第 2.15 节：反向惯性

有时，将运动方程写成如下形式会很有用：

$$a = \Phi f + b,$$

其中， Φ 为物体的反向惯性， b 为其偏置加速度——即如果 $f = 0$ ，物体本应具有的加速度。若该方程是式 2.69 的逆运算，则 $\Phi = I^{-1}$ ，而 $b = -\Phi p$ 。与式 2.69 相比，式 2.72 的优势在于，它可以应用于受约束的刚体。

逆惯性是一个对称的二元张量，它从 $\mathbf{F}^6$ 映射到 $\mathbf{M}^6$ 。其部分性质已在表 2.5 中列出。另一个有用性质是：逆惯性的值域是 $\mathbf{M}^6$ 中的一个子空间，该子空间内，其所作用的物体可以自由运动。由此又可推知，逆惯性的秩等于其作用对象的运动自由度。因此，如果物体的运动自由度少于六个，则 Φ 将为奇异矩阵，但 I 并不存在。在 C （质心）和 O （任意一点）处，以普吕克坐标表示的无约束刚体逆惯性公式为：

$$\Phi_C = \begin{bmatrix} \bar{I}_C^{-1} & 0 \\ 0 & \frac{1}{m}\mathbf{1} \end{bmatrix}$$

以及

$$\Phi_O = \begin{bmatrix} \bar{I}_C^{-1} & \bar{I}_C^{-1} c \times^T \\ c \times \bar{I}_C^{-1} & \frac{1}{m}\mathbf{1} + c \times \bar{I}_C^{-1} c \times^T \end{bmatrix}.$$

这些仅仅是式 2.62 和 2.63 的逆运算结果。

映射	例如	二元张量	坐标变换	
映射	例如	形式	公式	
$\mathbf{M}^n \mapsto \mathbf{M}^n$	$\mathbf{v} \times$	$m\mathbf{f} \cdot$	${}^B\mathbf{X}_A (\cdots)^A \mathbf{X}_B$	相似性
$\mathbf{F}^n \mapsto \mathbf{F}^n$	$\mathbf{v} \times$	$f\mathbf{m} \cdot$	${}^B\mathbf{X}_A^* (\cdots)^A \mathbf{X}_B^*$	相似性
$\mathbf{M}^n \mapsto \mathbf{F}^n$	$\mathbf{I}$	$f\mathbf{f} \cdot$	${}^B\mathbf{X}_A^* (\cdots)^A \mathbf{X}_B$	同构性
$\mathbf{F}^n \mapsto \mathbf{M}^n$	Φ	$m\mathbf{m} \cdot$	${}^B\mathbf{X}_A (\cdots)^A \mathbf{X}_B^*$	同构性

§2.14 平面向量

若一个刚体被限制沿某一给定平面作平行运动，则该刚体实际上就成为“平面”刚体。若刚体系统中的每一个物体都被限制沿同一平面作平行运动，则该系统即为平面刚体系统。许多实际的刚体系统都是平面刚体。

空间向量能够轻松描述平面刚体的运动，因为平面运动是空间运动的一个子集。然而，为平面刚体专门定义一种精简版的空间向量代数——平面向量代数，往往更为有利。平面刚体具有三个运动自由度，因此平面向量是向量空间 $\mathbf{M}^3$ 和 $\mathbf{F}^3$ 的元素。

推导平面向量代数最简单的方法，是从空间向量代数出发，并将其限定在 x - y 平面上。本质上，这意味着去除第一个、第二个和第六个基向量及其对应的坐标。因此：

$$\left. \begin{matrix} d_{Ox} & e_x & \omega_x & n_{Ox} \\ d_{Oy} & e_y & \omega_y & n_{Oy} \\ d_{Oz} & e_z & e & n_{Oz} \\ d_x & e_{Ox} & & \\ d_y & e_{Oy} & & \\ d_z & e_{Oz} & & \end{matrix} \right\} \Rightarrow \left. \begin{matrix} d_O & e_{Ox} & v_{Ox} & f_x \\ d_O & e_{Oy} & v_{Oy} & f_y \\ d_O & e_{Oz} & v_{Oz} & f_z \end{matrix} \right\} \Rightarrow \left. \begin{matrix} d_{Ox} & e_{Ox} & f_x & f_{xy} \\ d_{Ox} & e_{Ox} & f_{xy} & f_{xz} \\ d_{Ox} & e_{Ox} & f_{xz} & f_{xz} \end{matrix} \right\}$$

请注意，我们已从角基向量及其坐标中去掉了 z 下标。

空间叉积算子、坐标变换以及惯性矩阵的平面版本，均可通过从 6×6 矩阵中提取相应的 3×3 子矩阵来获得。例如，平面叉积算子矩阵的获取方式如下：

$$\begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v}_O \end{bmatrix} \times = \begin{bmatrix} \boldsymbol{\omega} \times & \mathbf{0} \\ \mathbf{v}_O \times & \boldsymbol{\omega} \times \end{bmatrix} = \begin{bmatrix} 0 & -\omega_z & \omega_y & 0 & 0 & 0 \\ \omega_z & 0 & -\omega_x & 0 & 0 & 0 \\ -\omega_y & \omega_x & -v_{Oz} & & & \\ 0 & -v_{Oz} & & & & \\ v_{Oz} & 0 & 0 & & & \\ -v_{Oy} & v_{Ox} & 0 & -\omega_y & \omega_x & 0 \end{bmatrix},$$

SO

$$\begin{bmatrix} \omega \\ v_{Ox} \\ v_{Oy} \end{bmatrix} \times = \begin{bmatrix} 0 & 0 & 0 \\ v_{Oy} & 0 & -\omega \\ -v_{Ox} & \omega & 0 \end{bmatrix}$$

以及

$$\begin{bmatrix} \omega \\ v_{Ox} \\ v_{Oy} \end{bmatrix} \times \begin{bmatrix} m \\ m_{Ox} \\ m_{Oy} \end{bmatrix} = \begin{bmatrix} 0 \\ v_{Oy} m - \omega m_{Oy} \\ \omega m_{Ox} - v_{Ox} m \end{bmatrix}.$$

同样地，我们还可以推导出：当原点从 $(0,0)$ 移动到 (x,y) ，随后将坐标系绕其新原点旋转一个角度 θ 时，对应于平面坐标变换矩阵的公式为

$$\mathbf{X} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c & s \\ 0 & -s & c \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ -y & 1 & 0 \\ x & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ sx - cy & c & s \\ cx + sy & -s & c \end{bmatrix},$$

其中 $c = \cos(\theta)$, $s = \sin(\theta)$ 。需要注意的是，平面向量继承了其空间向量对应物所具有的性质： $\mathbf{v}^{\times*} = -(\mathbf{v} \times)^T$ ，以及 $\mathbf{X}^* = \mathbf{X}^{-T}$ 。最后，平面刚体的惯性矩阵公式为：

$$\mathbf{I} = \begin{bmatrix} I_C + m(c_x^2 + c_y^2) & -m c_y & m c_x \\ -m c_y & m & 0 \\ m c_x & 0 & m \end{bmatrix} = \begin{bmatrix} I_O & -m c_y & m c_x \\ -m c_y & m & 0 \\ m c_x & 0 & m \end{bmatrix},$$

其中 m 为质量， c_x 和 c_y 分别为质心的坐标，而 I_C 和 I_O 分别为该刚体绕其质心和坐标原点的转动惯量。要完全定义一个平面刚体的惯性，需要四个参数。

§2.15 进一步阅读

空间向量与螺线理论中的扭转与力矩、实数电机以及李代数 $\mathfrak{se}(3)$ 及其对偶空间 $\mathfrak{se}^*(3)$ 的各个元素有着相似之处。螺线理论诞生于 19 世纪，早于现代意义上的向量概念。关于螺线理论的主要著作有 Ball (1900 年) 的作品，但更为现代的论述则见于 Angeles (2003 年)、Hunt (1978 年) 和 Selig (1996 年)。电机有两种类型：一种是实数域上的 6 维向量，另一种是双数环上的 3 维向量。前者在 von Mises (1924a, b) 中有所描述，后者则见于 Brand (1953 年)。空间向量与电机之间的主要区别在于：所有电机都位于同一个向量空间中。从这一角度来看，Featherstone (1987 年) 中的空间向量与电机代数极为相似。从形式上讲，李代数是李群在单位元处的切空间。空间向量 $\mathbf{M}^6$ 和 $\mathbf{M}^3$ 分别对应李代数 $\mathfrak{se}(3)$ 和 $\mathfrak{se}(2)$ ，而 $\mathbf{F}^6$ 和 $\mathbf{F}^3$ 则分别对应它们的对偶空间 $\mathfrak{se}^*(3)$ 和 $\mathfrak{se}^*(2)$ 。 $\mathfrak{se}(3)$ 和 $\mathfrak{se}(2)$ 是与特殊欧几里得群 $SE(3)$ 和 $SE(2)$ 相关的代数——这两类群分别代表了刚体在三维或二维欧几里得空间中所有可能位移的集合。李代数的相关内容详见 Murray 等人 (1994 年) 和 Selig (1996 年) 的著作。此外，还有许多其他值得关注的文章，例如 Jain (1991 年)；Park 等人 (1995 年)；Plücker (1866 年)；Rodriguez 等人 (1991 年)；Woo 和 Freudenstein (1970 年)。当然，相关文献的数量远不止于此——各类 6 维向量的文献浩如烟海。

刚体系统动力学

刚体系统动力学

动力学算法的用途在于对运动方程进行数值求解；但在我们能够对其求解之前，首先必须明确该方程究竟是什么。本章将介绍通用刚体系统的运动方程，并提供多种方法，用更简单的方程来构建这些方程。这些方法构成了后续各章中出现的各种算法的基础。不过，本章的重点主要在于刚体动力学的数学原理，而非特定的算法本身。

刚体系统是由若干刚体组成的集合，这些刚体可通过关节相互连接，并且可能受到多种力的作用。关节的作用是为与其相连的两个刚体施加运动约束：在某些方向上允许相对运动，而在其他方向上则不允许。更准确地说，关节的作用是在系统中引入一个约束力。该力具有这样一种特殊性质：我们无法得知其具体大小，但我们可以清楚地知道它对系统产生的影响——只要系统所需的运动状态能够满足运动约束，该力就会按照需要的大小施加。本章的大部分内容都致力于介绍如何描述运动约束，并将这些方法应用于刚体运动方程中。

第 3.1 节和第 3.2 节对本课题进行了高层次的介绍：第 3.1 节介绍了各种标准形式的运动方程；第 3.2 节则讲解了这些方程是如何由描述子系统、单个刚体以及运动约束的方程构建而成的。接下来的三个章节则更深入地探讨运动约束这一主题：第 3.3 节展示了如何将运动约束表示为向量空间；第 3.4 节对刚体系统中可能出现的各种约束类型进行了标准化分类；第 3.5 节则提供了一种详细的关约束模型。（我们把关节定义为任何用于描述两个刚体相对运动的运动学约束。）最后，第 3.6 节和第 3.7 节展示了如何为单个

刚体，以及由关节相互连接的刚体集合。后两个章节阐述了在建立运动方程时运用空间向量的方法。

§3.1
运动方程

对于一般的刚体系统，其运动方程通常可以写成如下规范形式：

$$H(q)\ddot{q} + C(q, \dot{q}) = \tau.$$

在该方程中， q 、 $\dot{q}$ 和 $\ddot{q}$ 分别表示广义位置、速度和加速度的向量，而 τ 是广义力的向量。 H 为广义惯性矩阵，其写法为 $H(q)$ ，以表明它依赖于 q 。同样地， C 为广义偏置力，其写法为 $C(q, \dot{q})$ ，以表明它既依赖于 q 又依赖于 $\dot{q}$ 。一旦理解了这些依赖关系，便采用更简短的符号 H 和 C 。偏置力仅是能使加速度为零的 τ 的值。它包含了科里奥利力、离心力、重力，以及作用于系统的、除

τ 中所包含的力之外的其他所有力。总体而言, H 和 C 构成了运动方程的系数, 而 τ 和 $\ddot{q}$ 则是变量。除了在运动方程中发挥作用外, H 还具有另一项重要性质: 系统的动能

$$T = \frac{1}{2} \dot{q}^T H \dot{q}.$$

方程 3.1 中的每一个量要么是坐标向量, 要么是矩阵。不妨简要地思考一下, 这些量分别代表何种类型的对象。为此, 我们用符号 M^n 和 F^n 分别表示 n 维运动空间和力矢量空间, 其中 n 是由方程 3.1 所描述的系统自由度。现在我们可以断言, $\dot{q}$ 和 $\ddot{q}$ 代表 M^n 的元素, 而 τ 和 C 代表 F^n 的元素, H 则是从 M^n 映射到 F^n 的映射。然而, q 并不表示一个向量, 而是代表系统构型空间中的一个点, 而该空间是一个 n 维流形。

根据定义, 广义力变量集合不能随意选取, 而必须选择使得 $\tau^T \dot{q}$ 成为 τ 向系统传递的功率。这意味着, 在 M^n 和 F^n 上的广义坐标系实际上是一个对偶坐标系。反过来, 这也意味着广义运动矢量和力矢量共享了许多空间向量的代数特性, 正如第二章所描述的那样。唯一的重大例外是: 在广义运动和力上, 并未定义叉积运算符。

严格来说, H 和 C 并不只依赖于 q 和 $\dot{q}$, 还依赖于刚体系统本身。因此, 更准确的写法是 $H(\cdot, q)$ 和 $C(\cdot, q, \dot{q})$, 其中“模型”一词指代一组数据, 这些数据以各组成部分为依据, 对某一特定刚体系统进行描述: 该

身体和关节的数量、它们彼此连接的方式、各身体的惯性参数值等等。我们把这种描述称为系统模型, 这也是第四章的研究内容。目前, 我们只需知道这样的模型确实存在, 并且诸如 H 和 C 这样的量都依赖于它。

我们的最终目标是能够计算出式 3.1 中所出现的一个或多个量的数值。特别是, 我们对以下两项计算抱有浓厚兴趣: 正向动力学——即根据力矩 τ 计算 $\ddot{q}$; 以及逆向动力学——即根据 $\ddot{q}$ 计算力矩 τ 。我们可以将这两项计算简洁地表示如下:

$$\ddot{q} = \text{FD}(\text{model}, q, \dot{q}, \tau)$$

并且

$$\tau = \text{ID}(\text{model}, q, \dot{q}, \ddot{q}),$$

其中 FD 和 ID 分别表示正向动力学和逆向动力学的计算函数。将这些公式与式 3.1 进行对比, 可以清楚地看出, FD 和 ID 应分别计算出 $H^{-1}(\tau - C)$ 和 $H\ddot{q} + C$ 。然而, 实现这些算法的算法并不一定需要通过计算 C 、 H 或 H^{-1} 来完成。因此, 式 3.3 和 3.4 的实用之处在于: 它们清晰地展示了每项计算的输入与输出, 却并未限定任何特定的计算方法。

式 3.1 假设位置变量是速度变量的积分。但实际情况并非总是如此。例如, 如果系统中存在非完整约束 (参见 §3.4), 那么其位置变量的数量就会多于速度变量。另一方面, 某些类型的运动用冗余的位置变量来描述会更加合适, 这也导致位置变量的数量多于速度变量。即便位置变量和速度变量的数量相同, 有时让后者不再是前者的导数, 反而会更有助于解决问题。无论出于何种原因, 我们只需简单地将 $\dot{q}$ 替换为另一组速度变量 α , 即可对式 3.1 进行调整, 得到:

$$H(q) \dot{\alpha} + C(q, \alpha) = \tau.$$

随后, 对式 3.3 和式 3.4 进行相应的修正。出于仿真需要, 我们仍需知道 $\dot{q}$ 的值, 以便将其积分以得到 q 。因此, 式 3.5 必须补充一个方程, 用于根据 q 和 α 计算 $\dot{q}$ 。该方程可写为

$$\dot{q} = Q(q)\alpha,$$

其中, Q 是一个依赖于 q 和系统模型的矩阵。或者, 也可以写成

$$\dot{q} = \text{qdfn}(\text{model}, q, \alpha),$$

其中, qdfn 表示根据 q 和 α 计算 $\dot{q}$ 的过程。式 3.6 揭示了 q 、 $\dot{q}$ 与 α 之间关系的数学结构——它表明 $\dot{q}$ 与 α 呈线性关系; 而式 3.7 则展示了该计算过程的输入与输出, 且并未暗示任何特定的计算方法。

在仿真方面, 大多数仿真器都期望能够获得一组一阶微分方程。我们可以通过定义状态变量 $x = [q^T \alpha^T]^T$ 来满足这一需求, 并将正向动力学以状态空间形式表达:

$$\dot{x} = \begin{bmatrix} \dot{q} \\ \dot{\alpha} \end{bmatrix} = \begin{bmatrix} H^{-1}(\tau - C) \end{bmatrix}.$$

若 FD_x 表示状态空间正向动力学计算函数, 则

$$\dot{x} = FD_x(\text{model}, x, \tau) = \begin{bmatrix} \text{qdfn}(\text{model}, q, \alpha) \\ FD(\text{model}, q, \alpha, \tau) \end{bmatrix}.$$

在后续的章节和段落中, 我们将主要处理那些可以用式 3.1 来描述的系统, 而非用式 3.5 所描述的更通用的系统。这是因为, 我们针对较简单系统所得出的大部分结果, 都可以直接且简便地迁移到更通用的系统中。大多数动力学算法对式 3.1 与式 3.5 之间的区别并不敏感, 除非在编程细节层面。

最后, 还有一种可能需要提及。有时, 识别并使用一组明确的独立速度变量既不切实际, 也未必理想。在这种情况下, 我们仍然可以沿用式 3.1 或式 3.5 作为运动方程; 不过, 此时 $\dot{q}$ 和 $\ddot{q}$ (或 α 和 $\dot{\alpha}$) 会受到一系列运动约束的影响, 因此, 运动方程必须辅以另一条方程, 以描述这些约束条件。本主题将在下一节中详细探讨。

§3.2 构建运动方程

刚体系统的运动方程是经过一系列数学运算后得出的最终结果。其起点是各个刚体和/或子系统各自的运动方程; 而我们对它们执行的两大主要操作是:

将各个方程汇总在一起, 形成更大子系统的方程; 以及
应用额外的运动约束。
施加更多的运动约束。

通过按照特定的顺序执行这些操作, 我们便可以定义出一种特定的求解方法, 以获得所需的运动方程, 从而也为计算机计算该方程的数值解提供了一套特定的算法 (或一类算法)。以下是一些最常见的求解方法。

将所有体 (即把它们的运动方程汇总成一个单一的方程), 然后施加所有的约束条件。

.....

首先求取树状结构的运动方程，随后再施加剩余的所有约束条件。

.....

.....

先求取树状结构的运动方程，然后再施加所有剩余的约束条件。

.....

从单个体或单个子系统开始，依次执行以下步骤：增加一个体或一个子系统；施加与该体或该子系统相关的约束条件；重复此过程。

.....

将子系统两两组合；在每个子系统内部分别施加约束条件；如此反复进行。

.....

方法 1 是许多简单动力学算法的典型做法。这种方法易于理解，但会生成较大且稀疏的矩阵。采用这种方法的算法必须运用稀疏矩阵技术，否则相较于其他算法，其运行速度会明显较慢。本方法的一个实例见第 3.7 节。方法 2 是闭环系统的标准方法，在第 8 章中有详细阐述。闭环系统的树状结构在第 4 章中被定义为一种运动学树，它不仅包含了原始系统中的所有体，还仅考虑了部分运动约束。方法 2 的要点在于：目前已有非常高效的算法可用于计算树状结构的动力学特性。方法 3 大致与关节体算法的工作原理相同，只不过它采用的是关节体方程，而非完整的运动方程。该算法在第 7 章中有详细说明。最后，方法 4 则由分治算法所使用（Featherstone, 1999a、b）。这类算法非常适合在并行计算机上实现。

.....

3.2.1 求解运动方程

.....

假设我们有一组包含 N 个刚体子系统的集合，其中系统 i 的运动方程为 $\mathbf{H}_i \ddot{\mathbf{q}}_i + \mathbf{C}_i = \boldsymbol{\tau}_i$ 。若我们希望将这些子系统视为一个整体系统，则该系统的运动方程为

.....

$$\begin{bmatrix} H_1 & 0 & \cdots & 0 \\ 0 & H_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & H_N \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{q}}_1 \\ \ddot{\mathbf{q}}_2 \\ \vdots \\ \ddot{\mathbf{q}}_N \end{bmatrix} + \begin{bmatrix} \mathbf{C}_1 \\ \mathbf{C}_2 \\ \vdots \\ \mathbf{C}_N \end{bmatrix} = \begin{bmatrix} \boldsymbol{\tau}_1 \\ \boldsymbol{\tau}_2 \\ \vdots \\ \boldsymbol{\tau}_N \end{bmatrix}.$$

.....

请注意，单个刚体本身也属于一个刚体子系统。因此，我们完全可以从一组 N 个刚体入手，甚至可以任意组合单个刚体与其他系统。如果子系统 i 正好是一个单独的刚体，那么我们可以使用其空间运动方程 $\mathbf{I}_i \mathbf{a}_i + \mathbf{p}_i = \mathbf{f}_i$ ，来代替广义坐标运动方程（即用 $\mathbf{I}_i$ 替代 $\mathbf{H}_i$ 等）。之所以如此操作，是因为空间运动与力矢量是广义运动与力矢量的特例；也就是说， $\mathbf{M}^6$ 和 $\mathbf{F}^6$ 是 $\mathbf{M}^n$ 和 $\mathbf{F}^n$ 的特例，而 $\mathbf{M}^6$ 和 $\mathbf{F}^6$ 上的普吕克坐标，也是 $\mathbf{M}^n$ 和 $\mathbf{F}^n$ 上广义坐标的特例。因此，我们可以自由地将空间速度与广义速度相结合，同样地，对于加速度和力也是如此。

3.2.2 运动约束

运动约束是对系统运动变量的代数性约束。它们可以通过以下两种不同的方式来表达：

	position	velocity	acceleration
implicit :	$\phi(q) = 0$	$K\dot{q} = 0$	$K\ddot{q} = k$
explicit :	$q = \gamma(y)$	$\dot{q} = G\dot{y}$	$\ddot{q} = G\ddot{y} + g$

其中

$$K = \frac{\partial \phi}{\partial q}, k = -\dot{K}\dot{q}, G = \frac{\partial \gamma}{\partial y} \text{ and } g = \dot{G}y.$$

(这些并非最通用的方程。一般情况将在 §3.4 中予以阐述。) 如果 ϕ 和 γ 都描述了相同的约束, 则

$$\phi \circ \gamma = 0, KG = 0 \text{ and } Kg = k.$$

运动约束是由约束力引起的。若一个无约束 (或约束较松的) 刚体系统具有运动方程

$$H\ddot{q} + C = \tau,$$

当该系统受到运动约束时, 受约束系统的运动方程为

$$H\ddot{q} + C = \tau + \tau_c,$$

其中 τ_c 为约束力。该力尚不可知, 但其具有一个重要的特性, 使我们能够根据需要计算其值, 或将其从方程中消去:

在与运动约束相容的每一方向上, 约束力在速度自由度上均不产生功。(朱尔丹的虚拟功原理。)

实际上, 这一表述表明, τ_c 必须满足 $\tau_c \cdot \dot{q} = 0$, 不仅对某一特定的 $\dot{q}$ 值成立, 而且对所有符合约束条件的值都成立。若系统受到隐式运动约束, 则可证明 τ_c 的形式为

$$\tau_c = K^T \lambda,$$

其中 λ 是一列未知力变量向量; 若系统受到显式运动约束, 则 τ_c 具有如下性质

$$G^T \tau_c = 0.$$

这些方程很容易验证。若 $\tau_c = K^T \lambda$, 则 $\tau_c \cdot \dot{q} = \lambda^T K \dot{q}$, 且对于所有满足 $K \dot{q} = 0$ 的 $\dot{q}$, 该表达式均等于零。同样地, 若 $\dot{q} = G \dot{y}$, 则 $\dot{q} \cdot \tau_c = \dot{y}^T G^T \tau_c$, 且当 $G^T \tau_c = 0$ 时, 该表达式对所有 $\dot{y}$ 都为零。 λ 的各个元素可以被视为一组拉格朗日乘子。

3.2.3 应用隐式运动约束

要将隐式运动约束应用于刚体系统, 我们只需将式 3.15 代入式 3.14, 并将所得结果与式 3.11 中的隐式加速度约束方程相结合。其结果为:

$$\begin{bmatrix} H & K^T \\ K & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ -\lambda \end{bmatrix} = \begin{bmatrix} \tau - C \\ k \end{bmatrix}.$$

这是一组包含 $n + n_c$ 个未知量的方程，其中 $n = \dim(\dot{\mathbf{q}})$ 是无约束系统的自由度，而 $n_c = \dim(\boldsymbol{\lambda}) = \dim(\boldsymbol{\phi})$ 则是对该系统施加的约束数量。可以证明，系数矩阵的秩为 $n + n_{ic}$ ，其中 $n_{ic} = \text{rank}(\mathbf{K})$ 是系统所施加的独立约束数量。因此，如果 $n_{ic} = n_c$ ，则系数矩阵是非奇异的，式 3.17 可同时求解 $\ddot{\mathbf{q}}$ 和 $\boldsymbol{\lambda}$ 。然而，若 $n_{ic} < n_c$ ，尽管仍可求解式 3.17 以求得 $\ddot{\mathbf{q}}$ ，但 $\boldsymbol{\lambda}$ 中将存在 $n_c - n_{ic}$ 个不定性。在这样的系统中，被称为过约束系统。理想情况下，我们应始终使 $n_{ic} = n_c$ ，但在实际操作中，这一条件并不总是能够得到保证。

3.2.4 应用显式运动约束

要将显式运动约束应用于刚体系统，我们将式 3.14 与式 3.16 以及式 3.11 中的显式加速度约束方程结合在一起。所得方程为：

$$\begin{bmatrix} H & -1 & 0 \\ -1 & 0 & G \\ 0 & G^T & 0 \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{q}} \\ \tau_c \\ \ddot{\mathbf{y}} \end{bmatrix} = \begin{bmatrix} \tau - C \\ -g \\ 0 \end{bmatrix}.$$

对这个方程进行一步高斯消元法运算，可得

$$\begin{bmatrix} H & -1 & 0 \\ 0 & H^{-1} & G \\ 0 & G^T & 0 \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{q}} \\ \tau_c \\ \ddot{\mathbf{y}} \end{bmatrix} = \begin{bmatrix} H^{-1}(\tau - C) - g \\ 0 \end{bmatrix},$$

由此我们可以提取

$$\begin{bmatrix} H^{-1} & G \\ G^T & 0 \end{bmatrix} \begin{bmatrix} \tau_c \\ \ddot{\mathbf{y}} \end{bmatrix} = \begin{bmatrix} H^{-1}(\tau - C) - g \\ 0 \end{bmatrix};$$

对这个方程进行高斯消元法运算，可得

$$\begin{bmatrix} H^{-1} & G \\ \mathbf{0} & -G^T H G \end{bmatrix} \begin{bmatrix} \tau_c \\ \ddot{\mathbf{y}} \end{bmatrix} = \begin{bmatrix} H^{-1}(\tau - C) - g \\ -G^T(\tau - C - Hg) \end{bmatrix},$$

由此我们得到

$$G^T H G \ddot{\mathbf{y}} = G^T(\tau - C - Hg).$$

最后，如果我们对 $u = G^T \tau$ 进行替换，其中 u 代表一组新的广义力，那么我们便得到

$$H_G \ddot{\mathbf{y}} + C_G = u,$$

何处

$$H_G = G^T H G \text{ and } C_G = G^T(C + Hg).$$

因此，我们得到了一个适用于受约束系统的新的运动方程，其代数形式与无约束系统在式 3.13 中的运动方程完全相同。

在此有必要稍作说明。首先，式 3.19 可以被视为式 3.17 的对偶形式，因为运动变量和力变量的位置已互换。然而，在实际应用中，式 3.17 比式 3.19 更为实用。其次，式 3.20 可直接由式 3.14 推导而来：首先将式 3.14 乘以 G^T ，从而消去 τ_c ，然后利用式 3.11 中明确给出的加速度约束，代入 $\ddot{q}$ 。这种方法有时被称为“投影法”，因为第一步实际上就是将运动方程投影到 G^T 的秩空间上。第三， H_G 继承了 H 的对称性和正定性等性质。此外，受约束系统的动能还具有以下特性：

$$T = \frac{1}{2} \dot{y}^T H_G \dot{y}.$$

最后，我们可以证明， $u \cdot \dot{y}$ 是传递给受约束系统的功率，具体如下：

$$u \cdot \dot{y} = (G^T \tau)^T \dot{y} = \tau^T G \dot{y} = \tau \cdot \dot{q}.$$

因此， u 确实满足广义力向量的定义。

§ 3.3 向量子空间

向量子空间是描述和分析运动学约束的天然数学工具，因为它们能够捕捉约束的本质特征。例如，式 3.13 中的系统可以在 $\mathbf{M}^n$ 中自由移动，因为 $\dot{\mathbf{q}}$ 属于 $\mathbf{M}^n$ ，且该系统不受任何约束。然而，式 3.14 中的系统则被约束，只能在 $\mathbf{M}^n$ 的子空间 S 中移动。如果约束是以隐式方式描述的，则 $S = \text{null}(\mathbf{K})$ ；如果约束是以显式方式描述的，则 $S = \text{range}(\mathbf{G})$ 。若 $\mathbf{K}_1$ 和 $\mathbf{K}_2$ 都描述了相同的约束，那么它们共同具备的唯一属性就是： $\text{null}(\mathbf{K}_1) = \text{null}(\mathbf{K}_2) = S$ 。同样地，若 $\mathbf{G}_1$ 和 $\mathbf{G}_2$ 都描述了相同的约束，那么它们共同具备的唯一属性就是： $\text{range}(\mathbf{G}_1) = \text{range}(\mathbf{G}_2) = S$ 。因此，真正捕捉约束本质的，并非某一个特定的矩阵，而是子空间 S 。本节将对向量子空间的基本知识以及其应用方法进行回顾。

基础知识

向量空间的一个子空间是指同时也是一个向量空间的子集。因此，子空间是指在加法和标量乘法运算下封闭的任意子集。设 V 是一个向量空间，且 $Y = \{\mathbf{y}_1, \mathbf{y}_2, \dots\}$ 是 V 的任意子集。集合 $\text{span}(Y)$ 是 Y 中各元素的所有线性组合构成的集合，因此它本身也是一个子空间。由此可知，任何满足 $Y = \text{span}(Y)$ 的子集都属于子空间。若 S 是 V 的一个子空间，且 $Y = \{\mathbf{y}_1, \mathbf{y}_2, \dots\}$ 是 S 的一个基，则 $S = \text{span}(Y)$ ，且 $\dim(S) = |Y|$ （即 S 的维数等于 Y 中元素的数量）。零维子空间是指仅包含零向量的集合。由于子空间本身就是向量空间，因此子空间可以拥有子空间，如此反复下去。

“是……的子空间”这一表述并没有专用符号，而是采用“子集”的符号来表示。因此，我们用 $Y \subseteq V$ 来表明 Y 是 V 的子集，用 $S \subseteq V$ 来表明 S 是一个子空间。要理解其确切含义，必须结合上下文来判断。表达式 $S \subset V$ 表示 S 是 V 的真子空间，这意味着 $\dim(S) < \dim(V)$ 。

假设 $S \subseteq V$ ，且 $\dim(S) = m$ ， $\dim(V) = n$ 。由此可知， m 必须位于 $0 \leq m \leq n$ 的范围内。若 $m = 0$ ，则 $S = \{\mathbf{0}\}$ ；若 $m = n$ ，则 $S = V$ ；但若 m 取其他值，则存在无数个具有该维度的子空间。要唯一确定一个 n 维向量空间中 m 维子空间所需的参数数量为 $m(n - m)$ 。

3.3.1 矩阵表示

表示子空间最简单的方式, 就是将其表示为某个矩阵的列空间或零空间。我们大多会使用前者。设 S 是 V 的一个 m 维子空间, 且 $\mathcal{S} = \{\mathbf{s}_1, \dots, \mathbf{s}_m\}$ 是 S 的一个基。那么, 任一向量 $\mathbf{v} \in S$ 都可以表示为 $\mathbf{v} = \sum_{i=1}^m \mathbf{s}_i \alpha_i$, 其中 α_i 是 $\mathbf{v}$ 在 $\mathcal{S}$ 中的坐标。如果 $\mathbf{s}_i$ 本身是按照 V 的基表示的坐标向量, 那么它们可以被拼成一个 $n \times m$ 的矩阵 $\mathbf{S} = [\mathbf{s}_1 \cdots \mathbf{s}_m]$, 该矩阵具有如下性质: $\text{range}(\mathbf{S}) = S$ 。此时, $\mathbf{v}$ 可以表示为 $\mathbf{v} = \mathbf{S}\boldsymbol{\alpha}$, 其中 $\boldsymbol{\alpha} = [\alpha_1 \cdots \alpha_m]^T$ 。

矩阵 $\mathbf{S}$ 同时定义了一个子空间和一个基。事实上, 在 $\mathbf{S}$ 中所包含的 mn 个数中, 我们可以指出 $m(n-m)$ 用于定义该子空间, 而 m^2 则用于定义基。如果 $\mathcal{S}' = \{\mathbf{s}'_1, \dots, \mathbf{s}'_m\}$ 是 S 的另一个基, 则矩阵 $\mathbf{S}' = [\mathbf{s}'_1 \cdots \mathbf{s}'_m]$ 定义了与 $\mathbf{S}$ 相同的子空间, 但使用的是不同的基。在这种情况下, $\mathbf{S}'$ 可以表示为 $\mathbf{S}' = \mathbf{S}\mathbf{A}$, 其中 $\mathbf{A}$ 是从 $\mathcal{S}'$ 转换到 $\mathcal{S}$ 坐标系的坐标变换矩阵。($\mathbf{A}$ 的各元素满足 $s'_{ij} = \sum_{k=1}^m \mathbf{s}_k A_{ik}$ 。) 因此, 只要 $\mathbf{A}$ 可逆, 任何矩阵 $\mathbf{S}\mathbf{A}$ 都能描述与 $\mathbf{S}$ 相同的子空间。

向量的分解

对向量可以执行的最基本的操作之一, 就是将其分解为在两个子空间中的分量。给定一个向量 $\mathbf{v} \in V$ 和

子空间 $S_1, S_2 \subseteq V$, 其核心思想是将 $\mathbf{v}$ 表示为 $\mathbf{v} = \mathbf{v}_1 + \mathbf{v}_2$ 的形式, 其中 $\mathbf{v}_1 \in S_1, \mathbf{v}_2 \in S_2$ 。这一分解成立的条件是 $\mathbf{v} \in \text{span}(S_1 \cup S_2)$; 若 $S_1 \cap S_2 = \{0\}$ (即 S_1 和 S_2 没有共同的非零元素), 则该分解是唯一的。若我们希望对于任意 $\mathbf{v}$, 分解既可行又唯一, 那么 S_1 和 S_2 必须满足

$$S_1 \cap S_2 = \{0\} \text{ and } \dim(S_1) + \dim(S_2) = \dim(V).$$

(这些条件表明, $\text{span}(S_1 \cup S_2) = V$ 。) 若 S_1 和 S_2 满足式 3.24, 我们可以说 V 是 S_1 和 S_2 的直和, 记作

$$V = S_1 \oplus S_2.$$

该等式只是用一种简化的表达方式, 说明 V 中的任意元素都可以唯一地分解为 S_1 中的一个元素与 S_2 中的一个元素之和。本质上, 式 3.24 和式 3.25 所述的内容是一致的。

现在我们来执行分解。我们已知一个向量 $\mathbf{v} \in V$, 以及两个矩阵 $\mathbf{S}_1$ 和 $\mathbf{S}_2$, 它们分别代表满足式 3.25 的子空间; 任务是求出 α_1 和 α_2 , 使得

$$\mathbf{v} = \mathbf{S}_1 \alpha_1 + \mathbf{S}_2 \alpha_2 = [\mathbf{S}_1 \ \mathbf{S}_2] \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix}.$$

这个问题可以通过以下结果来解决: 若 $S_1 \oplus S_2 = V$, 则 $[\mathbf{S}_1 \ \mathbf{S}_2]$ 是 V 上的一个基, 因此也是一个非奇异矩阵。由于 $[\mathbf{S}_1 \ \mathbf{S}_2]$ 存在逆矩阵, 式 3.26 的解只需简单地

$$\begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} = [\mathbf{S}_1 \ \mathbf{S}_2]^{-1} \mathbf{v}.$$

此公式以显而易见的方式延伸至将向量分解为多个子空间中的分量这一任务。

正交补

若两个欧几里得向量 $\mathbf{u}$ 和 $\mathbf{v}$ 满足 $\mathbf{u} \cdot \mathbf{v} = 0$, 则称它们为正交的。若 Y_1 和 Y_2 是 $\mathbb{E}^n$ 的子集, 且 Y_1 中的每一个元素都与 Y_2 中的每一个元素正交, 则称这两个子集为正交的, 记作 $Y_1 \perp Y_2$ 。

与给定子集 $Y \subseteq \mathbb{E}^n$ 正交的所有向量构成的集合，被称为 Y 的正交补，记作 $Y^\perp$ 。正交补始终是一个子空间。表 3.1 列出了正交子集的一些基本性质。

这种正交性形式基于欧几里得内积，因此仅适用于欧几里得向量。不过，我们也可以定义其他基于不同标量积的正交性形式。特别是，若 $u \in U$ 且 $v \in V$ ，其中 $U = V^*$ ，则可以定义 u 与 v 之间的标量积，并且如果 $u \cdot v = 0$ ，则可以断言：

表 3.1: 正交子集的性质。在一般情况下， $S, S_1, Y, Y_1 \subseteq U$ ，以及 $S_2, Y_2 \subseteq V$ ，其中 $U = V^*$ 。在欧几里得情形下， $S, \dots, Y_2 \subseteq \mathbb{E}^n$ 。

是正交的。进一步说，若 Y_1 和 Y_2 分别是 U 和 V 的子集，且 Y_1 中的每一个元素都与 Y_2 中的每一个元素正交，则称 $Y_1 \perp Y_2$ 。同样地，与 $Y \subseteq U$ 正交的所有 V 中的元素构成的集合，就是 Y 的正交补。

请注意，欧几里得形式与对偶形式的正交性在数学性质上既不同又彼此不兼容。尤其是，当且仅当 Y_1 和 Y_2 属于同一向量空间时，“ $Y_1 \perp Y_2$ ”这一关系才在欧几里得意义上成立；而若它们属于不同的向量空间，则这一关系才在对偶意义上成立。¹ 在机器人学文献中，正交补曾一度因一种错误的做法而广为人知——即把 6 维向量当作欧几里得向量来对待，并使用欧几里得形式的正交补，而非对偶形式的正交补。这一问题在 Duffy (1990) 一书中已有详细讨论。一些作者试图通过使用诸如“自然正交补”或“互反补”等替代名称来避免这种错误。

在刚体动力学中，正交补发挥着如下作用：如果一个系统最初可以在 $\mathbf{M}^n$ 中自由运动，而运动约束将运动限制在子空间 $S \subseteq \mathbf{M}^n$ 中，则施加该约束给系统的约束力必然属于 $S^\perp \subseteq \mathbf{F}^n$ 。

例 3.1 假设我们已知一个运动子空间 $S \subseteq \mathbf{M}^6$ ，以及一个空间惯性 $\mathbf{I} : \mathbf{M}^6 \rightarrow \mathbf{F}^6$ ，且该惯性是正定的。借助这些参数，我们可以定义两个子空间 $T_a = \mathbf{I}S$ 和 $T_c = S^\perp$ ，它们具有如下性质： $T_a \oplus T_c = \mathbf{F}^6$ 。（表达式 $\mathbf{I}S$ 是 S 在映射 $\mathbf{I}$ 下的像，即集合 $\{\mathbf{I}s \mid s \in S\}$ 。）我们可以通过以下方式证明这一性质：根据式 3.24，我们必须证明 $\dim(T_a) + \dim(T_c) = 6$ ，并且 $T_a \cap T_c$ 不包含任何非零元素。第一部分很容易： $\dim(T_a) = \dim(S)$ ， $\dim(T_c) = 6 - \dim(S)$ 。要证明第二部分，我们假设 $\mathbf{t} \neq 0 \in (T_a \cap T_c)$ ，并由此推导出矛盾：若 $\mathbf{t} \in T_a$ ，则存在一个非零 $\mathbf{s} \in S$ ，使得 $\mathbf{t} = \mathbf{I}s$ ；但若 $\mathbf{t} \in T_c$ ，则 $\mathbf{s}^\top \mathbf{t} = 0$ ，从而推出 $\mathbf{s}^\top \mathbf{I}s = 0$ ；然而，这

这是不可能的，因为 $\mathbf{I}$ 是正定的。

既然已经证明了 $T_a \oplus T_c = \mathbf{F}^6$ ，我们就可以将任意 $f \in \mathbf{F}^6$ 独立地分解为 $f_a \in T_a$ 和 $f_c \in T_c$ 两个分量。对于任意一个生成 S 的矩阵 S ，我们可以通过如下方法得到 f_a 和 f_c ：

$$S^\top f = S^\top f_a + S^\top f_c = S^\top f_a,$$

不过， $f_a = \mathbf{I}S\alpha$ ，其中 α 为某个常数，因此

$$S^\top f = S^\top \mathbf{I}S\alpha,$$

这表明

$$\alpha = (S^\top \mathbf{I}S)^{-1} S^\top f,$$

因此

$$f_a = \mathbf{I}S(S^\top \mathbf{I}S)^{-1} S^\top f.$$

f_c 由此可直接得出, 因为 $f_c = f - f_a$ 。式 3.28 中的这一表达式显然应与 S 的具体取值无关。我们可以通过将该等式中每个 S 的实例替换为 SA 来证明这一点, 其中 A 是任意一个大小合适的可逆矩阵, 并且可以证明这些 A 会相互抵消。

这一分解的物理意义如下: 若 $\mathbf{I}$ 表示一个刚体在约束条件下沿子空间 S 运动时的惯性, 而 $\mathbf{f}$ 是作用于该刚体的力, 则 $\mathbf{f}_a$ 是导致刚体加速度的力分量, $\mathbf{f}_c$ 则是被约束力所抵消的分量 (即, 约束力为 $-\mathbf{f}_c$)。如果刚体处于静止状态, 且仅受到 $\mathbf{f}$ 和运动约束力这两股力的作用, 则其加速度为

$$a = I^{-1}f_a = S(S^T I S)^{-1} S^T f.$$

表达式 $S(S^T I S)^{-1} S^T$ 是约束刚体的表观逆惯性 (参见式 3.54)。

第 3.4 节 约束的分类

运动学约束可以根据其对刚体系统施加的约束性质进行分类。在每一种情况下, 约束都由系统的运动变量中的代数方程来描述, 但方程的形式却各不相同。标准的分类层次结构如图 3.1 所示。

首先区分了等式约束和不等式约束。前者源于两刚体之间永久性的物理接触; 后者则出现在刚体既可接触又可分离的情况下。不等式约束可用于描述诸如碰撞、弹跳以及接触丧失等现象。这些约束将在第 11 章中详细讨论, 此处不再赘述。

接下来的区分在于全域性约束与非全域性约束。前者是对位置变量的约束, 通常由……产生。

子集	子空间
$Y_1 \perp Y_2 \Rightarrow Y_2 \perp Y_1$	$\dim(S) + \dim(S^\perp) = \dim(U)$
$Y_1 \perp Y_2 \Rightarrow \text{span}(Y_1) \perp Y_2$	$S_1 \perp S_2 \Leftrightarrow S_1^T S_2 = 0$
$Y_1 \perp Y_2 \wedge Z \subseteq Y_1 \Rightarrow Z \perp Y_2$	$S_1 \oplus S_2^\perp = U \Leftrightarrow (S_1^T S_2)^{-1}$ 存在
$Y_1 \perp Y_2 \Rightarrow Y_1 \subseteq Y_2^\perp$	$S_1 \oplus S_2^\perp = U \Leftrightarrow S_1^\perp \oplus S_2 = V$
$Y^\perp = (\text{span}(Y))^\perp$	欧几里得子空间
$(Y^\perp)^\perp = \text{span}(Y)$	$S \oplus S^\perp = E^n$

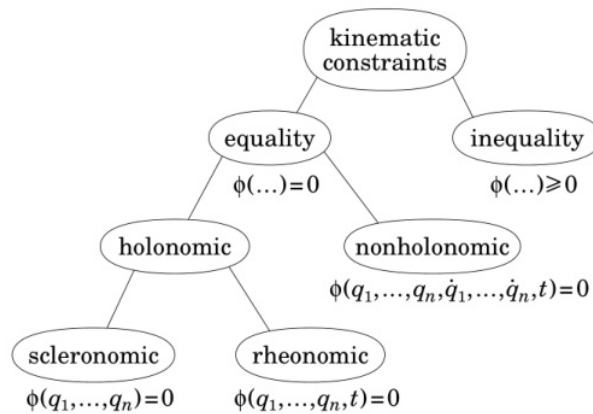


图 3.1: 运动学约束的分类

滑动接触。后者是对速度变量的约束，通常由滚动接触引起。非全域性约束函数 ϕ_{nh} 在速度变量中呈线性形式，因此可以表示为：

$$\phi_{nh}(\mathbf{q}, \dot{\mathbf{q}}, t) = \phi_{nh}^0 + \sum_{i=1}^n \phi_{nh}^i \dot{q}_i,$$

其中，系数 ϕ_{nh}^i 仅依赖于位置变量和时间。作为对比，全域性约束函数 ϕ_h 的时间导数为

$$\frac{d}{dt}\phi_h(\mathbf{q}, t) = \frac{\partial \phi_h}{\partial t} + \sum_{i=1}^n \frac{\partial \phi_h}{\partial q_i} \dot{q}_i,$$

其代数形式完全相同。因此，在速度和加速度的层次上，无约束与有约束之间并无差别。这两组方程之间的重大区别在于：方程 3.29 无法进行积分；也就是说， ϕ_{nh} 并不是任何函数的导数——否则，它就只是以导数形式表达的无约束条件。² 这一性质的后果是：包含非无约束约束的系统，其位置自由度多于速度自由度，因此需要的位姿变量也多于速度变量。若刚体系统中至少存在一个非无约束约束，则该系统被称为非无约束系统；反之，则称为无约束系统。因此，方程 3.1 只能描述无约束系统，而方程 3.5 则既可描述无约束系统，也可描述有约束系统。

最终的区别在于：一种是“硬化约束”，其仅依赖于位置变量；另一种是“弹性约束”，其依赖于

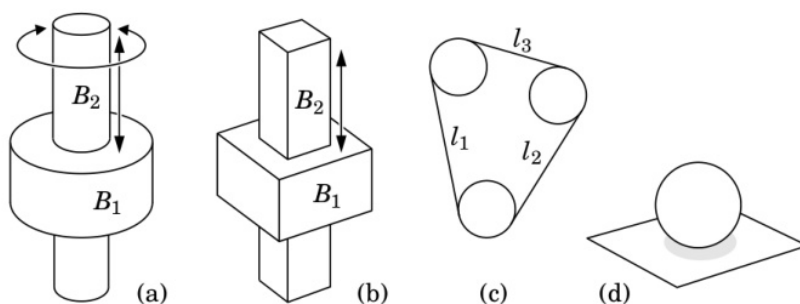


图 3.2: 运动学约束示例

同样也与时间有关。硬化约束源于一个固定在某一个物体上的表面与另一个固定在另一个物体上的表面之间的滑动接触。在典型的机械系统中，硬化约束既是结构最简单、也是最常见的约束类型。弹性约束可以被视为硬化约束的一种，只不过其中的一个或多个滑动自由度被强制按照预设的时间函数进行变化。弹性约束的作用在于向系统中引入运动学激励，即施加预定的运动。

图 3.2 中展示了若干运动学约束的实例。图 (a) 和图 (b) 分别展示了圆柱关节和棱柱关节。两者均为硬化约束。圆柱关节允许物体 B_2 相对于 B_1 进行滑动和旋转，而棱柱关节则仅允许相对滑动。因此，圆柱关节具有两个运动自由度，而棱柱关节仅有一个。如果我们把圆柱关节的一个自由度改为随时间的预设函数，那么它就会变成一个具有一个自由度的弹性约束。如果我们将同样的操作应用于棱柱关节，那么它就会变成一个零自由度的弹性约束（实际上，这在很多情况下非常有用）。

运动学约束几乎总是两个物体之间的相互关系。然而，也可以设计出涉及多个物体的约束，而这些约束无法归结为一组双体约束。图 3.2(c) 中展示了一个例子。该图显示了一个二维刚体系统，其中三个完全相同的圆形物体被约束在一条无质量、直径为零、非弹性绳索的周长之内。假

设绳索始终保持张紧状态，这三颗物体可以自由地以任何满足方程 $l_1 + l_2 + l_3 + 2\pi r = p$ 的方式运动，其中 p 和 r 分别表示绳索的周长和物体的半径。

图 3.2(d) 展示了一个球体，其 **resting** 在一个平面表面上。如果这个球体被约束成无滑动滚动，则它具有三种瞬时运动自由度：它可以沿两个方向滚动，也可以绕接触点旋转。然而，通过适当的操纵序列，球体完全可以到达平面任意一点，并且可以以任意姿态存在。因此，

具有五种位置自由度。这是非诺洛姆约束的典型特征。如果球体可以自由滑动，那么它将属于斯克诺诺米约束。

从图 3.2(a) 和 (b) 中各物体的形状可以看出，它们不可能被分开；但没有任何物理因素会阻止图 3.2(c) 中的绳索松弛，或图 3.2(d) 中的球体与平面失去接触。如果我们对作用于这些系统的力有充分的了解，能够确保此类事件不会发生，那么就可以将这些约束建模为等式约束；否则，就必须将其建模为不等式约束。

§3.4 连接约束

我们把连接约束定义为两个物体之间的运动学约束。因此，连接约束可以对一对刚体的相对速度施加从零到六种不同的约束。我们将这两个物体分别称为前驱体和后继体，并说连接约束是从前驱体连接到后继体的。随后，连接速度 v_J 被定义为后继体相对于前驱体的速度：

$$v_J = v_s - v_p,$$

其中 v_p 和 v_s 分别是前驱体和后继体的速度。连接约束的一般形式为

$$v_J = S(q, t) \dot{q} + \sigma(q, t),$$

其中， S 为关节的运动子空间矩阵， σ 为偏置速度， q 和 $\dot{q}$ 分别为关节的位置和速度变量， t 为时间。若位置变量并非速度变量的积分，则必须使用不同的符号来表示 $\dot{q}$ 或 q 。偏置速度是指当 $\dot{q} = 0$ 时， v_J 所取的值；只有当关节明确依赖于时间时（即该关节属于可动性约束或随时间变化的非完整约束），偏置速度才具有非零值。若关节不依赖于时间，则式 3.32 可简化为

$$v_J = S(q) \dot{q}.$$

式 3.32 的作用是将速度 $v_J - \sigma$ 限制在子空间 $S \subseteq \mathbb{M}^6$ 内。若关节允许 n_f 个相对运动自由度，则 $\dim(S) = n_f$ ，且 S 是一个 $6 \times n_f$ 的矩阵。³ 关节所施加的约束数量为 $n_c = 6 - n_f$ ，而约束力必须位于 n_c 维的子空间 $S^\perp \subseteq \mathbb{F}^6$ 中。

设 f_J 为通过关节传递的力。我们将其定义为从前一关节传递至后一关节的力；因此，作用于后一关节的力为 f_J ，而作用于前一关节的力则为 $-f_J$ 。这一力可以被视为主动力与约束力之和。前者来自执行器、弹簧、阻尼器，或任何其他作用于关节（或两体之间）的力元件；后者则是施加运动约束的力。此时，我们面临一种选择：约束力必须属于子空间 $S^\perp$ ，但主动力的处理方式有两种：既可以将其视为已知的空间力，也可以将其映射为作用于 $\mathbb{F}^6$ 子空间的广义力。在此，我们选择后者。

设 T 和 T_a 分别为约束子空间和主动力子空间。根据定义, $T = S^\perp$, 但 T_a 可以是任意满足 $T \oplus T_a = \mathbf{F}^6$ 的子空间。现在, 我们可以将 f_j 表示为:

$$f_j = T_a \tau + T \lambda,$$

其中 $T_a \tau$ 为主动力, $T \lambda$ 为约束力, T_a 和 T 满足

$$T_a^T S = 1$$

以及

$$T^T S = 0,$$

τ 是对应于 $\dot{q}$ 的广义力向量, 而 λ 是约束力变量向量。鉴于 T_a 和 T 的性质, 我们可以立即推导出:

$$S^T f_j = \tau$$

以及

$$T^T v_j = T^T \sigma.$$

方程 3.37 是通过计算在关节处传递的空间力来获得该关节广义力的公式; 而方程 3.38 则是速度约束方程的隐式形式。

方程 3.35 的推导依据如下。关节所输出的总功率为 $f_j \cdot v_j$, 但这一功率源自两个独立的功率源: σ 和 τ 。其中, 归属于 σ 的功率为 $f_j \cdot \sigma$, 因此归属于 τ 的功率必然是 $f_j \cdot S \dot{q}$ 。然而, 这股第二类功率也必须等于 $\tau \cdot \dot{q}$, 因为它们分别是关节处的广义力向量与速度向量, 因此我们得到了以下功率平衡方程:

$$\tau \cdot \dot{q} = f_j \cdot S \dot{q} = \tau^T T_a^T S \dot{q};$$

但该方程对于所有 τ 和 $\dot{q}$ 都必须成立, 因此我们必须有 $T_a^T S = 1$ 。

关节的加速度约束方程直接由方程 3.32 和 3.38 经过求导得到。方程 3.32 的导数为

$$\begin{aligned} a_j &= S \ddot{q} + \dot{S} \dot{q} + \dot{\sigma} \\ &= S \ddot{q} + (\dot{S} + v_s \times S) \dot{q} + (\dot{\sigma} + v_s \times \sigma) \\ &= S \ddot{q} + \dot{S} \dot{q} + \dot{\sigma} + v_s \times (S \dot{q} + \sigma) \\ &= S \ddot{q} + c_j + v_s \times v_j, \end{aligned}$$

其中, $\dot{S}$ 和 $\dot{\sigma}$ 分别是在随继任体运动的坐标系中, S 和 σ 的表观导数, 且

$$c_j = \overset{\circ}{S} \dot{q} + \overset{\circ}{\sigma}.$$

(实际上, $c_j = \dot{v}_j$ 。) 如果 S 和 σ 是 q 和 t 的函数, 正如方程 3.32 所示, 则 $\dot{S}$ 和 $\dot{\sigma}$ 的一般公式为

$$\dot{\overset{\circ}{S}} = \frac{\partial S}{\partial t} + \sum_{i=1}^{n_p} \frac{\partial S}{\partial q_i} \dot{q}_i$$

以及

$$\dot{\sigma} = \frac{\partial \sigma}{\partial t} + \sum_{i=1}^{n_p} \frac{\partial \sigma}{\partial q_i} \dot{q}_i,$$

其中, n_p 为关节位置变量的数量。

在继续下一步之前, 读者应牢记, 上述相对复杂的方程适用于一般情况——而在实际应用中, 这种情况几乎从不出现。一方面, 大多数常见的关节类型具有这样的特性: $\dot{S} = \mathbf{0}$, $\sigma = \mathbf{0}$ (从而导致 $c_J = \mathbf{0}$), 另一方面, 在极少数情况下, 若关节存在明确的时间依赖性 (即, 对于那些 $\sigma \neq \mathbf{0}$ 的关节而言), 几乎总是可以通过第 9 章中的混合动力学算法来实现时间依赖项, 而无需使用 σ 。

最后, 隐式加速度约束方程既可以由对方程 3.38 求导得到, 也可以通过将方程 3.40 乘以 T^T 得到。两者结果相同, 但后者更为简便:

$$\begin{aligned} T^T a_J &= T^T (S\ddot{q} + c_J + v_s \times v_J) \\ &= T^T (c_J + v_s \times v_J). \end{aligned}$$

示例 3.2 我们来探讨如何定义一个主动力子空间。图 3.3 展示了一个系统: 一个运动体 B 通过旋转关节与固定基座相连, 而一台电机 M 则通过小齿轮和齿圈驱动该关节。图中还标出了坐标系的 x 轴和 y 轴, 以及 z 轴。

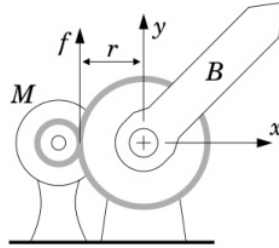


图 3.3: 图 3.3: 定义主动力子空间

与关节的旋转轴对齐。在这些坐标系中, 关节的运动和约束力子空间矩阵分别为

$$S = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad \text{and} \quad T = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

严格来说, 这并不是 S 和 T 的唯一可能取值, 因为只要任意两个能够覆盖正确子空间的矩阵即可。

主动力在两齿轮接触的处传递至 B ，且该力为沿 y 方向、大小为 f 的线性力。接触点的坐标为 $(-r, 0, 0)$ ，其中 r 为齿轮轮的半径，因此主动空间力的普吕克坐标为 $f = [0 \ 0 \ -rf \ 0 \ f \ 0]^T$ 。在此例中， τ 仅仅是一个标量，因此若 $f = T_a \tau$ ，则 T_a 只是 f 的标量倍数。根据式 3.35， T_a 必须满足 $T_a^T S = \mathbf{1}$ ，因此其解简单地

$$T_a = f / (f^T S) = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ -1/r \\ 0 \end{bmatrix}.$$

请注意， T_a 和 $\text{range}(T_a)$ 都依赖于 r 。这种依赖关系不会影响运动方程，但会改变约束力的数值。从物理意义上讲，约束力源自关节的轴承。因此，如果任务是模拟系统的运动，则 r 无关；但如果任务是为关节选择合适的轴承，则必须考虑 r 的取值。

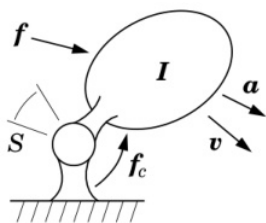


图 3.4: 受约束的刚体

§3.5 受约束刚体的动力学

考虑图 3.4 所示的刚体。该刚体通过一个具有子空间 $S \subseteq \mathbb{M}^6$ 的关节与固定基座相连。该关节对刚体的运动施加了 n_c 个约束条件，使刚体剩余 n_f 个运动自由度，其中 $n_f = \dim(S)$ ，而 $n_c = 6 - n_f = \dim(S^\perp)$ 。作用于该刚体的有三股力：作用力 f 、约束力 f_c ，以及未示出的重力 f_g 。因此，该刚体的运动方程为

$$f + f_c + f_g = I a + v \times^* I v_\gamma,$$

其中 I 、 a 和 v 分别代表刚体的惯性、加速度和速度。我们并不特别关注单独的项 f_g 和 $v \times^* I v$ ，因此我们将它们合并为一个统一的偏置力项 $p = v \times^* I v - f_g$ 。于是，受约束刚体的运动方程为

$$f + f_c = I a + p,$$

在约束条件下

$$v \in S \text{ and } f_c \in S^\perp.$$

我们寻求一种解决此方程组的方法，其中加速度被表示为施加力的函数。请注意， f_c 是一个未知量，必须在求解过程中予以求值和/或消去。

3.5.1 方法一

我们引入一个 $6 \times n_f$ 的矩阵 S ，该矩阵用于张成子空间 S 。借助该矩阵，方程 3.46 中的约束条件可以写成如下形式：

$$v = S \dot{q}$$

并且

$$S^T f_c = 0;$$

同时，加速度约束为

$$a = S \ddot{q} + \dot{S} \dot{q},$$

其中 $\dot{q}$ 和 $\ddot{q}$ 分别是关节速度和加速度变量的向量。方程 3.48 表明，我们可以通过将等式两边乘以 S^T ，从而从方程 3.45 中消去 f_c ，得到

$$S^T f = S^T (I a + p).$$

将方程 3.49 代入该式子后，可得

$$S^T f = S^T (I (S \ddot{q} + \dot{S} \dot{q}) + p),$$

该式子可按如下方式求解 $\ddot{q}$ ：

$$\ddot{q} = (S^T I S)^{-1} S^T (f - I \dot{S} \dot{q} - p).$$

表达式 $S^T I S$ 是一个 $n_f \times n_f$ 的对称正定矩阵，因此它必定可逆。在找到了 $\ddot{q}$ 的表达式之后，最后一步是将方程 3.51 重新代入方程 3.49，从而得到

$$a = S \left(S^T I S \right)^{-1} S^T (f - I \dot{S} \dot{q} - p) + \dot{S} \dot{q}.$$

该方程可以写成如下形式

$$a = \Phi f + b,$$

其中

$$\Phi = S \left(S^T I S \right)^{-1} S^T$$

并且

$$b = \dot{S}\dot{q} - \Phi(I\dot{S}\dot{q} + p).$$

Φ 是受约束物体的视在逆惯性矩阵，而 b 是其偏置加速度，即若 $f = 0$ ，该物体本应具有的加速度。视在逆惯性描述了当施加力发生变化时，加速度将如何变化。它是一个对称且半正定的矩阵，具有以下性质：范围 $(\Phi) = S$ ，零空间 $(\Phi) = S^\perp$ ，秩 $(\Phi) = n_f$ 。

3.5.2 方法二

解决方程组的方法之一，即通过引入一个 $6 \times n_c$ 的矩阵 T ，该矩阵用于张成 $S^\perp$ 。借助该矩阵，约束条件可以写成如下形式

$$T^T v = 0$$

并且

$$f_c = T\lambda,$$

其中 λ 是约束力变量的向量，而加速度约束为

$$T^T a + \dot{T}^T v = 0.$$

将方程 3.57 代入方程 3.45 后，可得

$$f + T\lambda = I a + p.$$

随后，可将方程 3.58 和方程 3.59 合并为一个单一的方程。

$$\begin{bmatrix} I & T \\ T^T & 0 \end{bmatrix} \begin{bmatrix} a \\ -\lambda \end{bmatrix} = \begin{bmatrix} f - p \\ -\dot{T}^T v \end{bmatrix},$$

该方程可以用来求解 a 和 λ 。此方程中的系数矩阵是对称且非奇异，但并非正定。

3.5.3 方法 3

第三种可能性是推导出用广义坐标表示的运动方程。向量 $\dot{q}$ 和 $\ddot{q}$ 可作为受约束刚体的广义速度和加速度变量，因此方程 3.51 已将广义加速度以作用于刚体的外力 f 的函数形式给出。设 τ 为作用在刚体上的广义力。要使 τ 与 f 等价，这两种力必须在 S 中的任意速度下都输出相同的功率。 τ 所输出的功率为 $\tau^T \dot{q}$ ，而 f 所输出的功率为 $f^T v$ ，因此我们有

$$\tau^T \dot{q} = f^T v = f^T S \dot{q};$$

但该方程必须对所有 $\dot{q}$ 均成立，因此

$$\tau = S^T f.$$

将此方程代入方程 3.51 后, 得到

$$\ddot{q} = (S^T I S)^{-1} (\tau - S^T (I \dot{S} \dot{q} + p)),$$

这便是用广义坐标表示的受约束刚体的运动方程。它可按标准形式书写:

$$H \ddot{q} + C = \tau,$$

其中

$$H = S^T I S$$

是广义惯性矩阵, 而

$$C = S^T (I \dot{S} \dot{q} + p)$$

是广义偏置力。

方程 3.61 表明, 从 f 到 τ 的转换会带来少量的信息损失。无穷多组不同的 f 值对应于相同的 τ 值, 因而会产生相同的加速度, 但每组值都会产生不同的 f_c 值。如果仅知道 τ , 则可以计算出总和 $f + f_c$ 的值, 却无法精确确定各向量的具体数值。若目标是进行运动仿真, 则丢失的信息无关紧要; 但如果目标是设计机械系统, 则这些丢失的信息对于诸如计算关节轴承的动态载荷力等任务而言至关重要。

§3.6

多体系统的动力学

我们通过推导包含多个刚体和关节的系统的运动方程来结束本章。这一分析揭示了第 3.5 节和第 3.6 节中与关节相关的材料, 以及第 3.2 节中广义约束之间的联系——通过展示如何从前者构建后者。

假设我们有一个刚体系统, 其中包含以下元素: 一个固定基座, 我们将其视为零号刚体; N_B 个可移动的刚体, 编号为 1 至 N_B ; 以及 N_J 个关节, 编号为 1 至 N_J 。对于每个关节 j , 我们定义 $p(j)$ 和 $s(j)$ 分别为该关节的前驱体和后继体的刚体编号。这组 $2N_J$ 个数字共同构成了系统的连通性。在后续分析中, 我们不对系统的连通性作出任何限制。

在推导运动方程时, 我们将沿用第 3.6 节中的方法 2, 但在当前系统复杂度较高的情况下, 必要时还需增加额外步骤。

3.6.1 第一步: 刚体运动方程

我们假定, 作用于刚体 i 的唯一力是重力 f_{gi} , 以及与其相连的各关节施加的力。因此, 刚体 i 的运动方程可以写成:

$$f_i = I_i a_i + p_i,$$

其中 f_i 为作用于刚体 i 的各关节力之和, 而 $p_i = v_i \times^* I_i v_i - f_{gi}$ 为偏置力。所有 N_B 个刚体的运动方程可以整合为一个单一的方程:

$$\begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_{N_B} \end{bmatrix} = \begin{bmatrix} I_1 & 0 & \cdots & 0 \\ 0 & I_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & I_{N_B} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_{N_B} \end{bmatrix} + \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_{N_B} \end{bmatrix}.$$

我们可以将此方程以更简洁的形式表示为:

$$f = I a + p,$$

其中 f 、 a 和 p 均为 $6N_B$ 维向量, I 则是一个 $6N_B \times 6N_B$ 的分块对角矩阵。为完整性起见, 我们还定义 $v = [v_1^T v_2^T \cdots v_{N_B}^T]^T$ ——其中 v_i 为刚体 i 的速度, 因为该向量将在关约束方程中被用到。

3.6.2 第二步: 从刚体运动推导关节运动

式 3.66 中的向量 a 代表刚体加速度的向量; 然而, 关节所施加的约束条件却是以关节速度和加速度来表达的。因此, 我们需要一种公式, 将关节运动用刚体运动来表示。设 v_{Jj} 和 a_{Jj} 分别表示关节 j 处的速度和加速度; 而 v_J 和 a_J 则是由 $v_{J1} \cdots v_{JN_J}$ 和 $a_{J1} \cdots a_{JN_J}$ 分别拼接而成的 $6N_J$ 维向量。根据定义, v_{Jj} 和 a_{Jj} 分别是关节后继体相对于其前驱体的速度和加速度; 因此,

$$v_{Jj} = v_{s(j)} - v_{p(j)}$$

并且

$$a_{Jj} = a_{s(j)} - a_{p(j)}.$$

这些方程可以汇总起来, 并表述如下:

$$v_J = \begin{bmatrix} v_{J1} \\ \vdots \\ v_{JN_J} \end{bmatrix} = \begin{bmatrix} v_{s(1)} - v_{p(1)} \\ \vdots \\ v_{s(N_J)} - v_{p(N_J)} \end{bmatrix} = P^T v$$

并且

$$a_J = \begin{bmatrix} a_{J1} \\ \vdots \\ a_{JN_J} \end{bmatrix} = \begin{bmatrix} a_{s(1)} - a_{p(1)} \\ \vdots \\ a_{s(N_J)} - a_{p(N_J)} \end{bmatrix} = P^T a,$$

其中

$$P_{ij} = \begin{cases} \mathbf{1}_{6 \times 6} & \text{if } i = s(j) \\ -\mathbf{1}_{6 \times 6} & \text{if } i = p(j) \\ \mathbf{0}_{6 \times 6} & \text{otherwise.} \end{cases}$$

因此, P 是一个 $6N_B \times 6N_J$ 的矩阵, 其结构为一个 $N_B \times N_J$ 的分块矩阵, 由 6×6 个块组成。该矩阵将采集到的机体速度和加速度向量 ($\mathbf{v}$ 和 $\mathbf{a}$) 映射至采集到的关节速度和加速度向量 ($\mathbf{v}_J$ 和 $\mathbf{a}_J$)。 P 中的每一行⁴ 都对应一个机体, 每一列则对应一个关节。(请注意, 我们使用 i 作为机体编号索引, 使用 j 作为关节编号索引。) 每列最多包含两个非零块: 一行中对应关节的后继机体处为 $\mathbf{1}$, 另一行中对应关节的前驱机体处为 $-\mathbf{1}$ 。如果某个关节连接着两个运动体, 则其列中将恰好有两个非零块, 正如刚才所描述的那样。

然而, 若该关节与固定基座 (机体 0) 相连或从其出发, 则其列中将只包含一个非零块: 根据具体情况, 在其与所连接的运动体对应的行中, 出现 $\mathbf{1}$ 或 $-\mathbf{1}$ 。对于每一行 i , 当关节的机体编号为 i 时, 该行将包含一个 $\mathbf{1}$; 而当关节的机体编号为 i 时, 该行将包含一个 $-\mathbf{1}$ 。

3.6.3 第三步: 由关节力产生的机体力

设 f_j 为通过关节 j 传递的力, 而 f_J 则是通过将向量 $f_{J_1} \cdots f_{J_{N_J}}$ 拼接而成的 $6N_J$ 维向量。根据定义, f_{J_j} 是从机体 $p(j)$ 通过关节 j 传递至机体 $s(j)$ 的力。如果我们把集合 $s^{-1}(i)$ 和 $p^{-1}(i)$ 分别定义为所有具有机体 i 作为其后继体或前驱体的关节的集合, 那么向量 f_i (此前被定义为作用于机体 i 上的所有关节力之和) 可以表示为如下形式:

$$f_i = \sum_{j \in s^{-1}(i)} f_{Jj} - \sum_{j \in p^{-1}(i)} f_{Jj}.$$

然而, 将这一表达式与 P 行 i 的描述进行对比, 我们可以清楚地看到, 同样也可以将 f_i 表示为如下形式:

$$f_i = \sum_{j=1}^{N_J} P_{ij} f_{Jj}.$$

因此

$$\mathbf{f} = P \mathbf{f}_J.$$

3.6.4 第四步: 运动约束

设 T_j 为跨度子空间 $S_j^\perp$ 的 $6 \times n_{cj}$ 矩阵, 其中 S_j 是关节 j 的运动子空间, n_{cj} 是其施加的约束数量。于是, 关节 j 的速度约束方程可写为

$$T_j^T \mathbf{v}_{Jj} = 0,$$

而加速度约束则为

$$T_j^T \mathbf{a}_{Jj} + \dot{T}_j^T \mathbf{v}_{Jj} = 0.$$

将加速度约束综合起来, 可得

$$T^T \mathbf{a}_J + \dot{T}^T \mathbf{v}_J = 0,$$

其中, T 是一个 $6N_j \times n_c$ 的分块对角矩阵, 其第 j 条对角线块中包含 T_j 。 n_c 是由各关节施加的约束总数, 其值为 $n_c = \sum_{j=1}^{N_j} n_{cj}$ 。 将式 3.71 与式 3.67 和式 3.68 结合, 可得关于机体加速度的如下约束:

$$T^T P^T a + \dot{T}^T P^T v = 0.$$

3.6.5 步骤 5: 约束力

f_{Jj} 是主动力与约束力之和, 其可表示为:

$$f_{Jj} = T_{aj}\tau_j + T_j\lambda_j$$

(参见式 3.34)。在该式中, T_{aj} 是一个 $6 \times n_{fj}$ 的矩阵, 用于表征关节 j 的主动力子空间; τ_j 是关节 j 处的广义力向量, 其维数为 n_{fj} ; λ_j 是约束力变量向量, 其维数为 n_{cj} ; $n_{fj} = 6 - n_{cj}$ 表示关节 j 允许的运动自由度数量。将所有关节的方程汇总起来, 我们得到:

$$f_J = T_a\tau + T\lambda,$$

其中, τ 和 λ 分别由 $\tau_1 \cdots \tau_{N_j}$ 和 $\lambda_1 \cdots \lambda_{N_j}$ 拼接而成的向量构成, 而 T_a 则是分块对角矩阵, 其第 j 条对角线块中包含 T_{aj} 。

3.6.6 步骤 6: 最终方程

将式 3.70 和式 3.73 代入式 3.66, 可得以下运动方程:

$$P(T_a\tau + T\lambda) = I a + p;$$

并将此与式 3.72 结合, 可得

$$\begin{bmatrix} I & PT \\ T^T P^T & 0 \end{bmatrix} \begin{bmatrix} a \\ -\lambda \end{bmatrix} = \begin{bmatrix} PT_a\tau - p \\ -\dot{T}^T P^T v \end{bmatrix}.$$

这便是原始刚体系统的运动方程。其中, 两个未知量分别为 a 和 λ , 且该方程可唯一求解出 a 。若系数矩阵非奇异, 则 λ 亦为唯一解。通过将此处的方程与第 3.2 节中的方程进行对比, 可以发现: $T^T P^T$ 和 $-T^T P^T v$ 在此处分别等价于式 3.11 中的 K 和 k ; I 、 a 和 p 在此处分别等价于式 3.14 中的 H 、 $\ddot{q}$ 和 C ; $PT_a\tau$ 和 $PT\lambda$ 在此处分别等价于式 3.14 中的 τ 和 τ_c ; 而式 3.74 整体则等价于式 3.17。

3.6.7 讨论

方程 3.74 是构建动力学算法的可行基础, 但在算法设计过程中仍需解决若干细节问题。例如, 速度和加速度变量显然应为各独立体的速度与加速度向量的普吕克坐标, 但究竟应采用何种坐标系? 对于位置变量, 以及如何计算 I 、 T 、 $T_a\tau$ 或 $\dot{T}^T v$ 等问题, 均未作任何提及。另一个问题是: 如何求解方程 3.74? 该系数矩阵规模庞大, 但同时也具有稀疏性, 采用稀疏分解方法将大幅提高计算效率。然而, 如果方程 3.72 中的约束并非全部线性独立, 则系数

矩阵将变为奇异，此时便需要采用适用于奇异矩阵的求解方法。最后，还存在约束稳定化的问题：正如方程 3.72 所规定，运动约束是在加速度层面进行的；然而，在仿真过程中，由于数值舍入和截断误差的影响，位置约束和速度约束中极有可能累积误差。这一问题将在第 8.3 节中予以详细讨论。

刚体系统建模

刚体系统建模

刚体系统是由多个组成部件构成的集合。这些部件包括刚体、关节以及各类力元。关节负责实现系统的运动学约束，因此我们广义地将“关节”一词理解为任意两刚体之间可能存在的运动学关系。在本章中，我们将探讨以各组成部件来描述刚体系统的方法。特别是，我们致力于描述仅由刚体和关节组成的系统。这种描述可被称为系统模型，因为其旨在描述系统本身，而非其行为的某一方面。

系统模型的作用可以这样理解：假设我们希望在计算机上计算某一特定刚体系统的动力学特性。原则上，有两种可行的处理方式：

推导出给定系统的符号运动方程，并将这些方程转化为计算机代码；或者
构建一个描述给定系统的系统模型，并将其作为参数提供给以下任一工具：

(a) 基于模型的动力学计算程序，或

(b) 基于模型的自动方程（及代码）生成器。

第二种方法更为可取，一方面是因为构建和修改系统模型比手动推导运动方程要容易得多；另一方面，基于模型的算法可以在充分调试并进行优化后，为后续大量使用做好准备。本书中的动力学算法均基于模型，并且在设计时预期会获得与本章所述内容相一致的系统模型。第 10.4 节将对第二种方法（b）进行详细讨论。

§4.1
连通性

一个系统模型必须包含以下数据：组件的描述，以及它们之间如何相互连接的描述。后者被称为系统的拓扑结构或连通性，可以用具有以下特性的连通图来表示：

节点代表各体。

弧线代表关节。

恰好有一个节点代表固定体，该节点被称为基体。其余所有节点则代表运动体。

该图是无向图。

该图是连通的（即，任意两个节点之间都存在路径）。

术语“运动体”指任何能够发生运动的物体，无论其是否真正实现运动。因此，即使某物因运动学约束而被固定不动，仍被视为运动体。从现在起，“运动体”这一术语及“图”应理解为“无向图”，除非另有说明。

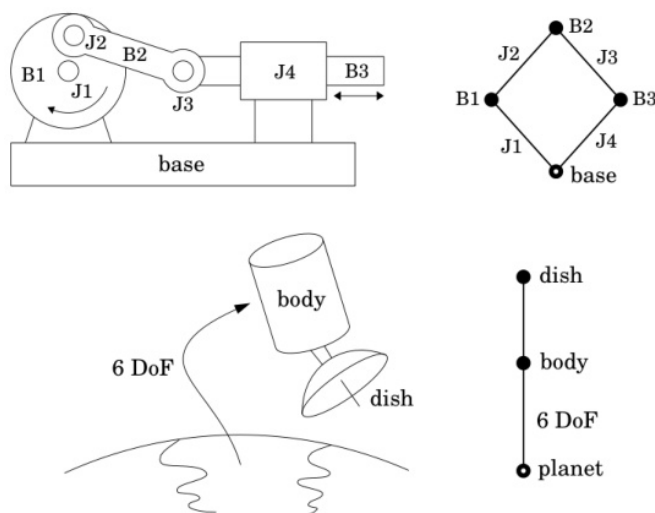


图 4.1: 两个刚体系统的连通图

连通图的一些示例见图 4.1。第一个例子是一个简单的平面机构，名为曲柄滑块机构。它由三个运动体（B1 至 B3）、一个固定基体、三个转动关节（J1 至 J3）以及一个平移关节（J4）组成。当曲柄 B1 旋转时，B3 会相对于基体前后滑动。在连通图中，每个节点代表

一个体，每条弧线则代表一个关节。通常有必要为至少部分节点和弧线标注标签，以便清晰地呈现图与原始机构之间的对应关系。此外，有时将基体节点与其他节点区分开来，以方便直观识别，这可以通过在基体节点上标记一个白色圆点来实现。需要注意的是，机构中的各个体通常被称为“连杆”。

第二个例子是一个在轨道上的卫星。卫星本身由两个运动体和一个关节组成。然而，我们无法直接从这个双体系统中构建出一个连通图，因为规则 3 要求其中一个节点必须代表一个固定基准。因此，我们添加了第三个体——在本例中为下方的行星——作为固定基准。在增加了第三个体之后，规则 5 要求我们再添加一个第二关节，以便将行星与另外两个体中的任意一个连接起来。由于卫星与行星之间不存在物理连接，我们使用了一个特殊的关节，该关节可提供六自由度的运动自由度——一个 6 自由度关节。该关节不施加任何运动学约束，因此对系统本身并无实际影响。它的作用是定义额外的关节变量，从而使得整个系统的位姿和速度变量可以由其所包含的各个关节的位姿和速度变量来推导得出。在本例中，卫星总共有 $6 + n_s$ 个自由度，其中 n_s 为卫星所用关节允许的自由度。因此，我们总共需要 $6 + n_s$ 个速度变量，而这正是由这两个关节所界定的数量。

通过 6 自由度关节与固定基准相连的体被称为浮动基准。在本例中，选择了卫星的主体作为浮动基准，但我们也可以选择天线盘作为浮动基准。

4.1.1 运动学树与循环

如果图中任意两个节点之间都存在且仅存在一条路径，则该图即为拓扑树。若刚体系统的连通图是拓扑树，则我们将该系统本身称为运动学树。图 4.1 中的卫星是运动学树，但曲柄滑块机构则不是。运动学树具有特殊性质，使得其动力学分析能够以高效的方式进行。

设 G 为任意一个连通图。 G 的生成树，记作 G_t ，是 G 的一个子图，它包含 G 中的所有节点，并且包含 G 中任意一个弧的子集，使得 G_t 成为一棵拓扑树。每一个连通图至少有一个生成树。如果 G 本身就是一颗树，则 $G_t = G$ ；否则， G_t 是 G 的真子图，且并非唯一。图 4.2 展示了若干生成树的示例。

拥有 n 个节点的拓扑树有 $n - 1$ 条弧。因此，如果 G 包含 n_n 个节点和 n_a 条弧，则 G_t 将包含 n_n 个节点和 $n_n - 1$ 条弧，这意味着 G 中存在 $n_a - n_n + 1$ 条弧，这些弧既存在于 G 中，又不在 G_t 中。这些非树状弧被称为弦。尽管 G 中弦的数量是固定的，但恰好构成弦的弧集合会随着 G_t 的选择而变化。这种变化在图 4.2 中得到了生动的展示。（我们通过用虚线绘制弦来区分弦与树状弧。）

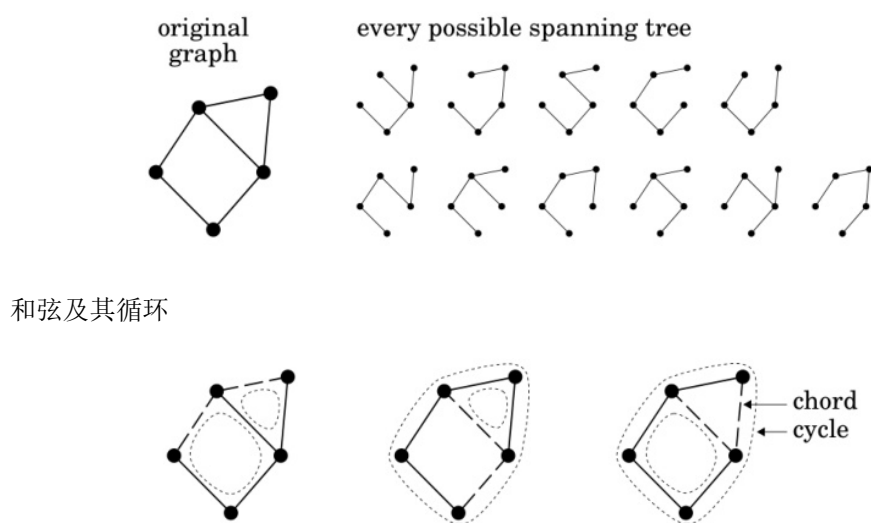


图 4.2: 生成树、和弦与循环

循环是指图中的一条闭合路径。树中不存在循环，因此图中的每一条循环至少会经过一个和弦。我们尤其关注那些恰好经过一个和弦的循环。对于图中的每一个和弦，总有一条这样的循环；不过，其走行路线取决于所选的生成树。例如，图 4.2 所示的图包含两个和弦和三个循环。无论选择何种生成树，其中两个循环都是单和弦循环，而第三个循环（未在图中展示）则同时经过这两个和弦。单和弦循环的路径由其所经过的那条和弦，以及生成树中连接该和弦两端节点之间的唯一路径组成。

连通性图中的每一个循环都定义了一个运动学回路。然而，我们并不关心所有的循环，而只对独立的运动学回路感兴趣。在建立运动方程时，这正是必须纳入考量的最小循环集合。单和弦循环的重要性在于，它们能够确定独立的运动学回路集合。如果我们把一个刚体系统描述为具有 n 个运动学回路，那么实际上我们所指的，是该系统拥有 n 个独立的运动学回路。

考虑一个刚体系统，该系统由固定基座、 N_B 个运动体以及 N_J 个关节组成，其中后者的范围包括所有新增的 6 自由度关节，以确保连通性图保持连通。设 G 为该系统的连通性图，且设 G_t 为 G 的生成树。这两个图都将包含 $N_B + 1$ 个节点。然而， G 将包含 N_J 条弧，而 G_t 仅包含 N_B 条弧，因此系统中和弦的数量为 $N_J - N_B$ 。系统中独立的运动学回路数量 N_L 等于 G 中的和弦数量，因此可由下式给出：

$$N_L = N_J - N_B.$$

将此公式应用于图 4.1 中的曲柄滑块机构，可以发现该机构只有一个运动学回路。

如果一个系统存在运动学回路，则称其为闭环系统；否则，它就是运动学树。与和弦相关的关节被称为回路关节或回路闭合关节，而属于生成树的关节则称为树状关节。因此，刚体系统拥有 N_B 个树状关节和 N_L 个回路关节。

4.1.2 编号

身体和关节可以通过多种方式加以识别，但最便捷的方式是进行编号。我们将采用一种非常适合描述和实现动力学算法的编号方案。根据这一方案，身体的编号从 0 开始至 N_B ，关节的编号从 1 开始至 N_J ，而编号的分配规则如下。

选择一棵生成树，即 G_t 。

将编号为 0 的节点指定为固定基点，并将该节点定义为 G_t 的根节点。

按照任意顺序为剩余的节点从 1 到 N_B 编号，使得每个节点在 G_t 中的编号都高于其父节点。

按照从 1 到 N_B 的顺序为 G_t 中的各条弧编号，其中弧 i 连接节点 i 与其父节点。

按照任意顺序为 $N_B + 1$ 到 N_J 的所有剩余弧编号。

每个构件的编号与其节点的编号相同，每个关节的编号与其弧的编号相同。

这种方案被称为规则编号法。如果将这些规则应用于无分支的运动学树（即树中没有任何一个节点拥有超过一个子节点的树），那么构件和关节将从基点开始按顺序依次编号，如图 4.3 所示。在其他情况下，编号并不唯一。例如，图 4.4 展示了图 4.1 中曲柄滑块机构的每一种可能的编号方式。这八种可能性源自四种可能的生成树。其中两种树是无分支的，因此仅有一种编号方式。这两种树分别出现在图中的左上角和右下角图形中。另外两种树在根节点处存在分支，每种树都有三种不同的编号方式。

4.1.3 关节极性

每个关节连接两个构件。我们把其中一个构件确定为关节的前驱节点，另一个构件则确定为关节的后继节点。关节本身被定义为从其前驱节点连接到其后继节点。这一术语的用途在于……

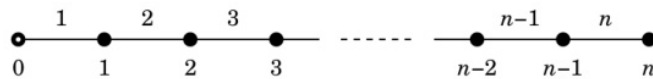


图 4.3: 图 4.3: 无分支运动学树的唯一编号方式

具有双重含义：一是界定关节与构件变量之间的关系，二是明确关节在系统中连接的方向。例如，关节处的速度被定义为后继节点相对于前驱节点的速度；关节处传递的力则被定义为前驱节点向后继节点传递的力。

在两个构件之间确定关节连接方向的过程，被称为关节的极性。我们可以通过在连接图中的一条弧上放置箭头，使箭头从前驱节点指向后继节点，来识别关节的极性。图 4.5 和图 4.6 中均展示了此类实例。出于方便起见，我们对树状关节的极性采用以下约定：树状关节的正常极性是：从父节点连接到子节点。除非另有说明，所有树状关节都被假定具有此极性。

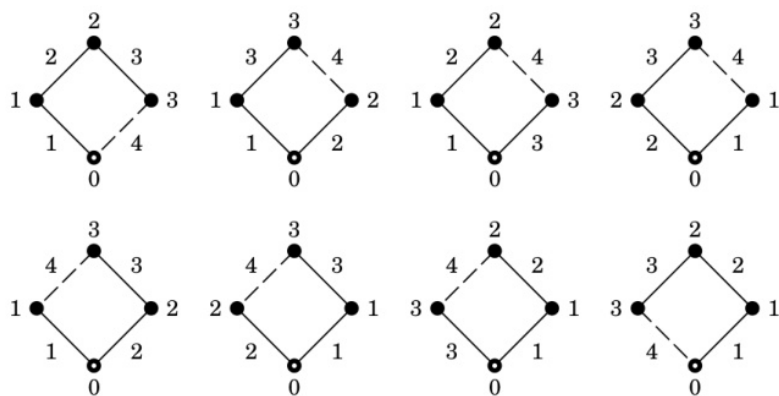


图 4.4: 图 4.1 中曲柄滑块机构的每一种可能的编号方式

有些关节在物理上是关于对称的，这意味着继任关节相对于前任关节的运动，在质上与前任关节相对于继任关节的运动完全相同。不具备这一性质的关节则属于非对称关节。将对称关节的极性进行切换，对刚体系统并不会产生任何实际影响；然而，将非对称关节的极性进行切换则会带来显著变化。例如，如果两个物体通过一个球窝关节相连，而该关节是关于对称的，那么究竟哪个物体拥有球头、哪个物体拥有碗座其实并不重要——如果我们把它们互换位置，运动状态依然保持不变。相反，如果这两个物体通过一个齿条-齿轮关节相连，而该关节是非对称的，那么究竟哪个物体拥有齿条、哪个物体拥有小齿轮，就变得至关重要了。因此，非对称关节的极性

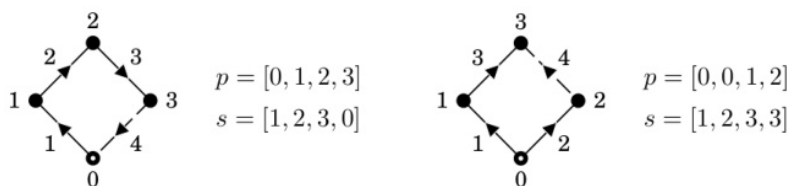


图 4.5: 图 4.5: 关节前驱体与后继体数组

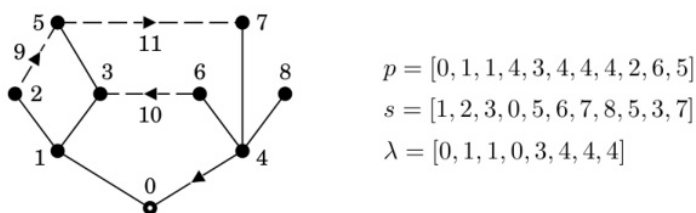


图 4.6: 图 4.6: 前驱体、后继体与父节点数组

关节由物理系统决定，但对称关节的极性却并非如此，这使我们能够在系统模型中自由选择其极性。

4.1.4 表示法

在计算机中，表示连通图的方式有很多。其中最简单的一种方法是将关节的前驱体和后继体的节点编号分别记录在两个数组 p 和 s 中，其中 $p(i)$ 和 $s(i)$ 分别代表关节 i 的前驱体和后继体编号。这一方法如图 4.5 所示，它复现了图 4.4 中的两个连通图，并在图中添加了箭头，以展示每个关节的极性。

通过 p 和 s ，我们可以完整地描述一个连通图，并由此计算出其他有用的相关量。其中第一个量是父节点数组 λ ，它用于标识树状结构中每个节点的父节点： $\lambda(i)$ 是节点 i 的父节点。根据编号方案的规则 3 和 4，树状结构中的关节 i 连接着节点 i 和 $\lambda(i)$ ，且 $\lambda(i) < i$ ，因此

$$\lambda(i) = \min(p(i), s(i)). (1 \leq i \leq N_B)$$

图 4.5 中两个图的父节点数组分别为 $\lambda = [0, 1, 2]$ 和 $\lambda = [0, 0, 1]$ ；更大型的示例则见图 4.6。

若某树状关节满足 $s(i) = i$ ，则该关节从父节点连接到子节点，被称为树状结构中的正向关节；反之，若该关节从子节点连接到父节点，则被称为树状结构中的反向关节。图 4.5 中的所有树状关节均处于正向方向（箭头指向远离根节点的方向）。图 4.6 中的树状关节同样全部处于正向方向，仅第 4 号关节例外。树状关节的编号以及指向正向的箭头已

本图中未提及的两项——前者是因为它们可由编号方案推导得出，后者则是因为我们把“向前”视为树节点连接的标准化方向。

还有三种量能够为运动学树的连通性提供有用的信息：对于除基体之外的任意节点 i ， $\kappa(i)$ 是位于节点 i 与基体之间路径上的所有树节点集合；而对于包括基体在内的任意节点 i ， $\mu(i)$ 是节点 i 的子节点集合，而 $\nu(i)$ 则是以节点 i 为起点的子树中的所有节点集合。这些集合分别被称为支撑集、子节点集和子树集。（当一个树节点支持某个节点时，我们称其为“支撑节点”。） $\nu(i)$ 也可以被视为由节点 i 所支撑的所有节点的集合。对于图 4.6 所示的生成树而言，支撑集、子节点集和子树集分别为：

$$\begin{array}{lll} \kappa(1) = \{1\} & \mu(0) = \{1, 4\} & \nu(0) = \{0, 1, 2, \dots, 8\} \\ \kappa(2) = \{1, 2\} & \mu(1) = \{2, 3\} & \nu(1) = \{1, 2, 3, 5\} \\ \kappa(3) = \{1, 3\} & \mu(2) = \{\} & \nu(2) = \{2\} \\ \vdots & \vdots & \vdots \\ \kappa(8) = \{4, 8\} & \mu(8) = \{\} & \nu(8) = \{8\} \end{array}$$

κ 、 λ 、 μ 和 ν 显然具有许多简单的性质，这些性质均源于它们的定义。例如， $\mu(i) \subseteq \nu(i)$ ， $j \in \kappa(i) \Rightarrow i \in \nu(j)$ ，等等。其中一项并不显而易见却十分实用的性质是恒等式：

$$\sum_{i=1}^{N_B} \sum_{j \in \kappa(i)} (\dots) = \sum_{j=1}^{N_B} \sum_{i \in \nu(j)} (\dots).$$

左侧的求和表达式涵盖了所有 i, j 对，其中节点 j 支持节点 i ；而右侧的求和表达式同样涵盖了所有 i, j 对，其中节点 j 也支持节点 i 。因此，这两者实际上是一致的。

例 4.1 通过以下简单示例来说明 λ 在动力学算法中的作用。假设我们希望根据树节点之间的速度信息，计算刚体系统中每个节点的速度。设 $\mathbf{v}_i$ 为节点 i 的速度， $\mathbf{v}_{ji}$ 为节点 i 与节点 j 之间的速度，其中后者是指子节点相对于父节点的速度（即节点间的连接方向为“向前”）。现在，节点

i 的速度可以递归地定义为：它自身父节点的速度加上与其父节点相连的节点间速度之和；因此，计算节点速度的公式为：

$$\mathbf{v}_i = \mathbf{v}_{\lambda(i)} + \mathbf{v}_{J_i} \cdot (\mathbf{v}_0 = \mathbf{0})$$

用于计算这些速度的算法因此为：

对于 $i = 1$ 到 N_B ，令 $\mathbf{v}_0 = \mathbf{0}$ ；然后， $\mathbf{v}_i = \mathbf{v}_{\lambda(i)} + \mathbf{v}_{J_i}$ 。

性质 $\lambda(i) < i$ 确保了 $\mathbf{v}_{\lambda(i)}$ 总是在被使用之前被计算出来。另一种表示节点速度的方式是：

$$\mathbf{v}_i = \sum_{j \in \kappa(i)} \mathbf{v}_{J_j}.$$

例 4.2 例 2.3 中引入了一个名为“节点雅可比矩阵”的矩阵，该矩阵将单个节点的空间速度作为关节速度变量的函数给出。若 $\mathbf{J}_i$ 为节点 i 的节点雅可比矩阵，则有：

$$\mathbf{v}_i = \mathbf{J}_i \dot{\mathbf{q}}.$$

方程 2.18 给出了一个用于 $\mathbf{J}_i$ 的公式，该公式仅适用于无分支的运动链这一特殊情形。接下来，我们来推导出通用公式。设关节 j 上的位移速度为 $S_j \dot{\mathbf{q}}_j$ ，而 ϵ_{ij} 的定义如下：

$$\epsilon_{ij} = \begin{cases} 1 & \text{if } j \in \kappa(i) \\ 0 & \text{otherwise.} \end{cases}$$

因此，当关节 j 支撑着物体 i 时， ϵ_{ij} 才会不为零。现在，物体 i 的速度可以表示为：

$$\mathbf{v}_i = \sum_{j \in \kappa(i)} S_j \dot{\mathbf{q}}_j = \sum_{j=1}^{N_B} \epsilon_{ij} S_j \dot{\mathbf{q}}_j = [\epsilon_{i1} S_1 \quad \cdots \quad \epsilon_{iN_B} S_{N_B}] \begin{bmatrix} \dot{\mathbf{q}}_1 \\ \vdots \\ \dot{\mathbf{q}}_{N_B} \end{bmatrix}.$$

因此，物体雅可比矩阵的通用公式为

$$\mathbf{J}_i = [\epsilon_{i1} S_1 \epsilon_{i2} S_2 \cdots \epsilon_{iN_B} S_{N_B}].$$

§ 4.2 几何学

若两个物体通过一个关节相连，则它们之间的相对运动受到约束。要完整描述这种约束，必须了解两件事：关节允许的运动方式，以及关节相对于每个物体的位置。前者由关节模型加以描述；后者则由系统的几何结构来确定。

刚体系统的几何模型，是对连接位置的描述。具体而言，它描述了每个刚体上各关节的相对位置。图 4.7 展示了一个刚体系统的几何模型：系统由一个固定基座 B_0 、三个运动体 B_1 至 B_3 、三个树状关节 J_1 至 J_3 ，以及一个环形关节 J_4 组成。图中虚线仅表示各关节的连接关系，并未精确标示其物理位置。

每个运动体都内置了一个坐标系，用以定义该体的局部坐标系。这些坐标系分别标记为 F_1 至 F_3 ，

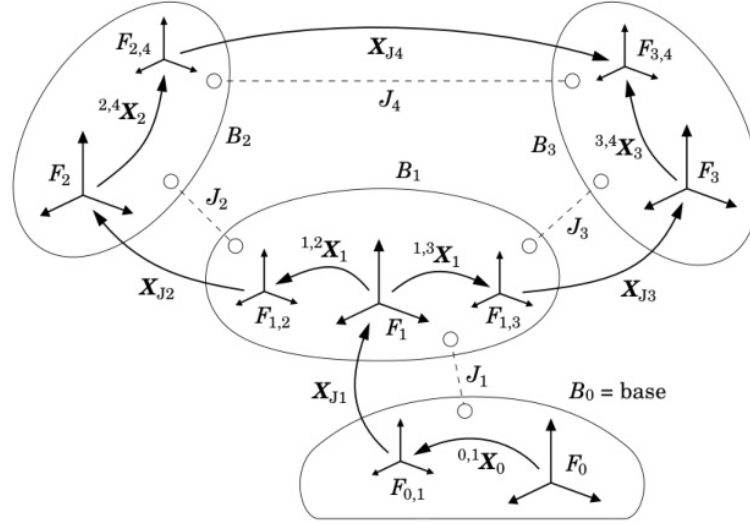


图 4.7: 图 4.7: 刚体系统的几何模型

并且，它们所定义的坐标系被称为体坐标系或连杆坐标系。此外，还有一个嵌入在基座中的坐标系 F_0 ，用于定义整个刚体系统的绝对坐标系、参考坐标系或基准坐标系。运动体 B_i 的位置，被定义为 F_i 相对于 F_0 的位置，并可通过坐标变换 iX_0 来表示。

图中还展示了多个坐标系，其名称形式为 $F_{i,j}$ ，其中 i 表示某个运动体， j 表示某个关节。这些坐标系标识了各关节在各个运动体上的位置。每个树状关节 i 从坐标系 $F_{\lambda(i),i}$ 连接到坐标系 F_i （反之亦然，如果关节处于反向连接），而每个环形关节 i 则从 $F_{p(i),i}$ 连接到 $F_{s(i),i}$ 。因此，总共有 $N_B + 2N_L$ 个定位坐标系：每个树状关节对应一个，每个环形关节对应两个。坐标系 $F_{i,j}$ 相对于 F_i 的位置，由变换 ${}^{i,j}X_i$ 确定，且该变换是常量。这些变换——或一组可据此计算得到的参数——共同构成了刚体系统的几何模型。

为方便起见，我们定义了一个树变换数组 $\mathbf{X}_T$ ，使得 $\mathbf{X}_T(i) = {}^{\lambda(i),i}X_{\lambda(i)}$ 。这些变换用于表示树节点的定位变换，而该数组则完整地描述了支撑树的几何结构。我们还定义了数组 $\mathbf{X}_P$ 和 $\mathbf{X}_S$ ，它们的索引分别从 1 到 N_L ，其中 $\mathbf{X}_P(i - N_B) = {}^{p(i),i}X_{p(i)}$ ， $\mathbf{X}_S(i - N_B) = {}^{s(i),i}X_{s(i)}$ ，适用于每个循环节点 i 。这些变换用于表示循环节点的定位变换。

最后，每个节点都定义了一个节点变换 $\mathbf{X}_{J_i}$ ，该变换是节点位置变量的函数。这些变换将在第 4.4 节中进行详细讨论。如图所示，当 i 是树节点时， $\mathbf{X}_{J_i}$ 等于 ${}^iX_{\lambda(i),i}$ ；而当 i 是循环节点时， $\mathbf{X}_{J_i}$ 等于 ${}^{s(i),i}X_{p(i),i}$ 。

示例 4.3 假设我们希望计算运动学树中每个体的位置；也就是说，我们希望计算每个体的 iX_0 。实现这一目标的算法如下：

对于 $i = 1$ 到 N_B ，令 $X_J = \text{jcalc}(\text{jtype}(i), q_i)$ ； ${}^iX_{\lambda(i)} = X_J X_T(i)$ ；若 $\lambda(i) \neq 0$ ，则 ${}^iX_0 = {}^iX_{\lambda(i)} \lambda(i) X_0$ 。

在这段代码片段中， $\mathbf{X}_J$ 是一个局部变量， $\text{jtype}(i)$ 是描述关节 i 的数据结构，而 jcalc 是一个函数，它根据关节描述和关节位置变量向量来计算关节变换。 $\text{jtype}(i)$ 和 jcalc 将在第 4.4 节中介

绍。

§ 4.3 Denavit-Hartenberg 参数

一般来说，确定一个坐标系相对于另一个坐标系的位置需要六个参数。然而，对于某些类型的刚体系统 (rigid-body system)，可以采用这样的方式布置物体坐标系，使得所需的参数更少。其中一种方法最初由 Denavit 和 Hartenberg 提出 (Denavit and Hartenberg, 1955)，每个关节只需要四个参数。该方法利用了如下事实：一些最常见的关节类型可以用空间中的一条直线来刻画，而确定一条直线只需四个参数。

图 4.8 展示了 Denavit-Hartenberg (DH) 方法如何应用于表示机械臂或类似装置的无分支运动学树 (unbranched kinematic tree)。该系统的重要几何特征有：

一个基座坐标系 (base coordinate frame)，

n 个关节轴 (joint axes)，以及

一个嵌入在末端连杆中的末端执行器坐标系 (end-effector coordinate frame)。

末端执行器坐标系用于机器人正确执行定位命令，但对于动力学计算则并非必需。DH 坐标系按照以下规则放置。

轴 z_0 和 z_{n+1} 分别与基座坐标系和末端执行器坐标系的 z 轴对齐。

轴 z_1 到 z_n 与 n 个关节轴对齐，使得 z_i 与关节 i 的轴线对齐。(如果关节 i 恰好是移动关节 (prismatic)，那么 z_i 可以是任何方向正确的轴；但合理的选择是与 z_{i-1} 相交的轴。)

图 4.8: Denavit-Hartenberg 坐标系与参数

轴 x_i 是 z_i 与 z_{i+1} 之间的公垂线 (common perpendicular)，方向由 z_i 指向 z_{i+1} 。如果 z_i 与 z_{i+1} 相交，则 x_i 的方向存在 180° 的歧义。如果 z_i 与 z_{i+1} 平行，则公垂线的位置不确定，此时合理的做法是选择一条与 x_{i-1} 相交的公垂线。

物体坐标系 F_i 由轴 x_i 、 y_i 和 z_i 定义，其中 y_i 由 x_i 和 z_i 导出，以构成右手坐标系 (right-handed coordinate frame)。

在布置好各坐标系之后，它们的相对位置由以下 DH 参数描述：

d_i ：沿 z_i 从 x_{i-1} 到 x_i 的距离。

θ_i ：绕 z_i 从 x_{i-1} 到 x_i 的角度。

a_i ：沿 x_{i-1} 从 z_{i-1} 到 z_i 的距离。由 x_{i-1} 的定义可知 $a_i \geq 0$ 。

α_i ：绕 x_{i-1} 从 z_{i-1} 到 z_i 的角度。

所有参数 a_i 和 α_i 都是常数，但参数 d_i 和 θ_i 中的一部分充当关节变量 (joint variable)。具体来说，如果关节 i 是旋转关节 (revolute)，则 θ_i 是其关节变量；如果关节 i 是移动关节，则 d_i 是其关节变量；如果关节 i 是圆柱关节 (cylindrical)，则 θ_i 和 d_i 都是关节变量；如果关节 i 是导程为 h_i 的螺旋关节 (screw joint)，则 θ_i 是关节变量，而 x_{i-1} 到 x_i 的距离定义为 $d_i + h_i\theta_i$ 。如果所有关节都是旋转关节，那么 DH 参数为图 4.8 中的无分支树定义了以下几何模型：

$$\begin{aligned} i = 1: & \quad \mathbf{X}_T(1) = \text{xlt}([a_1 \ 0 \ d_1]^T) \text{rotx}(\alpha_1) \text{rotz}(\theta_0) \text{xlt}([0 \ 0 \ d_0]^T) \\ i > 1: & \quad \mathbf{X}_T(i) = \text{xlt}([a_i \ 0 \ d_i]^T) \text{rotx}(\alpha_i) \end{aligned}$$

(平移函数和旋转函数的定义参见表 2.2。另外，螺旋变换 (screw transform) 参见附录 §A.4。)

描述一个具有 n 个关节的系统总共需要 $4n + 6$ 个 DH 参数，其中至少有 n 个是关节变量而非参数。不过，最后四个参数仅用于确定末端执行器坐标系的位置，如果不需要这些信息则可以省略。如果基座坐标系可以选择为与 z_1 对齐，则还可以再省略最多五个参数。

在其基本形式中，Denavit-Hartenberg 方法适用于任何每个关节都能用一条关节轴线刻画、且拓扑为无分支运动学树或单一闭合运动学环路的刚体系统。不过，该方法可以扩展到更广泛的系统类别，如 Khalil and Dombre (2002) 所述。DH 参数在许多教科书中都有介绍，例如 Angeles (2003) 和 Craig (2005)。不同作者采用的记法细节往往各不相同，尤其是 x_i 、 a_i 和 α_i 的编号。本书采用的记法遵循《Springer Handbook of Robotics》(Siciliano and Khatib, 2008)。

图 4.9: 若干关节位置变量的几何解释

§4.4 关节模型

关节 (joint) 可以看作两个笛卡尔坐标系之间的运动约束——其中一个嵌入在关节的后继体 (successor body) 中，另一个嵌入在其前驱体 (predecessor) 中。我们将这两个坐标系分别称为 F_s 和 F_p 。关节模型 (joint model) 的目的是提供将 F_s 相对于 F_p 的运动表示为关节变量的函数所需的信息。

关节允许的运动类型取决于其关节类型。例如，旋转关节允许绕单一轴线的纯转动，而球关节 (spherical joint) 允许绕特定点的任意转动。因此，每种类型的关节都需要一个关节模型。关节模型的内容是用于计算特定量的一组数据和公式，例如：关节变换 (joint transform)(sX_p)、关节速度 (joint velocity)($\mathbf{v}_j$)、关节的运动子空间 (motion subspace)($\mathbf{S}$) 等。表 4.1 给出了几种常见关节类型的公式，其中一些关节的几何形状如图 4.9 所示。

让我们更详细地考察一个模型。表 4.1 中的第一项描述了一个旋转关节，它允许 F_s 绕其与 F_p 共同的 z 轴相对于 F_p 转动，并将关节变量定义为转动角度。标记为 $\mathbf{E}$ 和 $\mathbf{r}$ 的两列描述了关节变换的转动分量和平移分量，变换本身由方程 ${}^sX_p = \text{rot}(\mathbf{E}) \text{xlt}(\mathbf{r})$ 给出。(rot、xlt 和 rz 的定义参见表 2.2。) 注意，当关节变量 (q_1) 为零时， F_s 与 F_p 重合。我们尽可能地将这一约定应用于所有关节模型。我们应用于所有关节模型的另一项约定是， $\mathbf{S}$ 、 $\mathbf{T}$ 及相关量都在 F_s 坐标系中表示。对于旋转关节，这一点其实无关紧要，因为 ${}^s\mathbf{S} = {}^p\mathbf{S}$ ，但对于其他关节则很重要。

在设计良好的动力学仿真器中，通常会有一个关节模型库，包含软件所支持的每种关节类型的一个模型。然后，系统模型会为系统中的每个关节包含一个关节类型描述符 (joint type descriptor)。我们将使用表达式 $\text{jtype}(i)$ 来表示关节 i 的关节类型描述符。在最简单的情况下，关节类型描述符只是一个类型代码，用于标识库中的特定模型。例如，类型代码 'R'

表 4.1: 关节模型公式

可以标识旋转关节，'P' 标识移动关节。然而，有些关节既由类型刻画，也由一组参数刻画。例如，螺旋关节既由其为螺旋这一事实刻画，也由指定螺旋导程 (pitch) 的参数刻画。刚体系统中的每个螺旋关节都可以具有不同的导程；因此，诸如此类的参数是单个关节的属性，而不是关节模型的属性。为了容纳关节参数，我们将关节类型描述符定义为一种数据结构，既包含类型代码，也包含该类型关节所需的一组参数值 (如果有的话)。

4.4.1 关节模型计算例程

后续章节将介绍几种动力学算法，它们都要使用关节模型数据。为了简化算法描述，我们假设存在一个名为 `jcalc` 的单一函数，在一次调用中计算出所有请求的关节相关数据。该函数的一次示例调用如下：

$$\begin{aligned} [X_J, S, v_J, c_J] &= \text{jcalc}(\text{type}, q, \alpha) && \text{(explicit joint constraints)} \\ [\delta, T] &= \text{jcalc}(\text{type}, {}^s X_p) && \text{(implicit joint constraints)} \\ [\dot{q}] &= \text{jcalc}(\text{type}, q, \alpha) && \text{(for qdfn (Eq. 3.7))} \end{aligned}$$

在上述每种情况下，左边的表达式都是所请求的返回值列表，参数 `type` 指关节类型描述符（例如 `jtype(i)`）。使用 `jcalc` 计算隐式关节约束的内容将在第 8 章介绍，而使用它计算 $\dot{q}$ 的内容将在本章后面介绍。在算法清单中，我们通常假设 $\alpha = \dot{q}$ ，并在参数列表中用 $\dot{q}$ 代替 α 。如果关节显式地依赖于时间，则必须将时间作为列表中的最后一个参数加入。注意， v_J 、 c_J 、 S 和 T 都用 F_s 坐标系表示。 v_J 和 c_J 的表达式见第 3.5 节。

示例 4.4 示例 4.3 展示了如何计算运动学树中每个物体的坐标变换 ${}^i X_0$ 。我们现在扩展这个示例，同时计算 ${}^i X_0$ 和 ${}^0 v_i$ （物体 i 的速度，以基座坐标表示）。完成这一计算的算法如下：

```

 ${}^0 v_0 = 0$ 
for  $i = 1$  to  $N_B$  do
   $[X_J, v_J] = \text{jcalc}(\text{jtype}(i), q_i, \dot{q}_i)$ 
   ${}^i X_{\lambda(i)} = X_J X_T(i)$ 
  if  $\lambda(i) \neq 0$  then
     ${}^i X_0 = {}^i X_{\lambda(i)} {}^{\lambda(i)} X_0$ 
  end
   ${}^0 v_i = {}^0 v_{\lambda(i)} + {}^i X_i v_J$ 
end

```

这段代码包含一个新特性：它计算 ${}^i X_0$ ，但随后使用 ${}^0 X_i$ 来变换 v_J 。这种做法的解释见附录 A，其中展示了如何直接从 X 和 v 计算 $X^{-1}v$ ，而无需先计算 X^{-1} 。同样，也可以直接从 X 和 f 计算 X^*f 和 $(X^*)^{-1}f$ ，而无需先计算 X^* 或 $(X^*)^{-1}$ 。我们在几个动力学算法清单中利用了这一点。

示例 4.5 平面关节 (planar joint) 和表 4.1 中的六自由度关节 (6-DoF joint) 的一个特殊之处在于，平移关节变量以 F_s 坐标系而不是 F_p 坐标系来表示平移。这样做的原因是为了使运动子空间矩阵成为简单的常数，从而有 $\dot{S} = 0$ 、 $c_J = 0$ ，并且 S 的简单形式允许对动力学算法应用各种优化。然而，这种安排可能存在一个很大的缺点：如果后继体距前驱体很远，并且开始旋转，那么这种旋转将导致位置变量发生很大的变化率。（想一想翻滚的卫星或飞行器。）一种两全其美的方法是使用一组以 F_p 坐标表示平移的位置变量，同时使用一组以 F_s 坐标表示线速度的速度变量。

设 p_{q1} 、 p_{q2} 和 p_{q3} 是平面关节的一组位置变量，其中 p_{q2} 和 p_{q3} 是沿 F_p 的 x 和 y 方向的平移；再设 ${}^s q_1$ 、 ${}^s q_2$ 和 ${}^s q_3$ 是同一平面关节的第二组位置变量，其中 ${}^s q_2$ 和 ${}^s q_3$ 是沿 F_s 的 x 和 y 方向的平移。第二组变量就是表 4.1 中所用、图 4.9 所示的那组变量。这两组变量之间的关系为

$$\begin{bmatrix} pq_1 \\ pq_2 \\ pq_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_1 & -s_1 \\ 0 & s_1 & c_1 \end{bmatrix} \begin{bmatrix} sq_1 \\ sq_2 \\ sq_3 \end{bmatrix}$$

我们希望以 pq_i 作为位置变量，而以 sq_i 作为速度变量。为满足这一需要，我们需要一个公式，从位置变量和速度变量计算位置变量的导数。对上述方程求导，即可得到这样的公式：

$$\begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ -q_3 & c_1 & -s_1 \\ q_2 & s_1 & c_1 \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{bmatrix},$$

其中我们去掉了 pq_i 和 pq_i 上的上标 p ，并用 α_i 代替了 sq_i 。如果 jcalc 被要求针对这种特定的平面关节模型从 $\mathbf{q}$ 和 α 计算 $\dot{\mathbf{q}}$ ，那么这正是它必须执行的运算。

示例 4.6 让我们为图 4.10 所示的齿轮齿条关节 (rack-and-pinion joint) 构造一个关节模型。该关节允许属于后继体一部分的小齿轮 (pinion) 沿着属于前驱体一部分的齿条 (rack) 滚动。齿条

图 4.10: 齿轮齿条关节

与 F_p 的 x 轴平行，小齿轮以 F_s 的 z 轴为中心。小齿轮的半径为 r ，这是关节模型的一个参数。该关节只允许一个运动自由度，因此只有一个关节变量，记为 q_1 ，它被选为小齿轮的转动角度。

如果当 $q_1 = 0$ 时 F_p 与 F_s 重合，那么从 F_p 到 F_s 的变换由沿 x 方向平移 rq_1 、随后绕 z 轴转动 q_1 组成。因此关节变换为

$$X_J = {}^sX_p = \begin{bmatrix} E & 0 \\ 0 & E \end{bmatrix} \begin{bmatrix} 1 & 0 \\ -p \times & 1 \end{bmatrix},$$

其中

$$E = rz(q_1) = \begin{bmatrix} c_1 & s_1 & 0 \\ -s_1 & c_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ and } p = \begin{bmatrix} rq_1 \\ 0 \\ 0 \end{bmatrix}.$$

下一步是求出 v_J 和 S 的公式。我们首先将关节速度表示为关节位置变量和速度变量的函数。相对于 F_p ，小齿轮以角速度 $\omega = [00\dot{q}_1]^T$ 绕通过 p 的轴线转动，并以线速度 $\dot{p} = [r\dot{q}_1 00]^T$ 平移。这等价于如下以 F_p 坐标表示的空间速度 (spatial velocity)：

$${}^p v_J = \begin{bmatrix} \varepsilon \\ p \times \omega \end{bmatrix} + \begin{bmatrix} 0 \\ \dot{p} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \dot{q}_1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ \dot{q}_1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ r\dot{q}_1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ r\dot{q}_1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

由于只有一个关节变量，运动子空间矩阵由下式给出

即公式 $S = v_J / \dot{q}_1$ 。因此

$$\mathbf{v}_J = {}^s\mathbf{X}_p{}^p\mathbf{v} = \begin{bmatrix} 0 \\ 0 \\ \dot{q}_1 \\ rc_1\dot{q}_1 \\ -rs_1\dot{q}_1 \\ 0 \end{bmatrix} \quad \text{and} \quad \mathbf{S} = \frac{\mathbf{v}_J}{\dot{q}_1} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ rc_1 \\ -rs_1 \\ 0 \end{bmatrix}$$

最后，由于 S 不是常数，我们需要速度积项 (velocity-product term) c_J 的公式。它由下式给出

$$c_J = \dot{S} \dot{q}_1 = \frac{\partial S}{\partial q_1} \dot{q}_1^2 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -rs_1\dot{q}_1^2 \\ -rc_1\dot{q}_1^2 \\ 0 \end{bmatrix}$$

($\dot{S}$ 参见式 3.42。) 注意，如果 $r = 0$ ，齿轮齿条关节就退化为旋转关节。

4.4.2 极性反转

如第 4.1.4 节所述，运动学树中的关节，如果前驱体是后继体的父体，则称该关节处于正向 (forward direction)；否则称其处于反向 (reverse direction)。如果关节是对称的，那么在定义系统模型时就可以选择其极性，使其处于正向；但如果关节是不对称的，则其极性由所建模的刚体系统决定。因此，无法保证每个树关节都处于正向。

处理反向关节最简单的方法是把它转换成正向关节。可以按如下方式进行。首先，在关节描述符数据结构中添加一个布尔变量，当关节是处于反向的树关节时其值为 **true**，否则为 **false**。有了这一额外信息，就可以修改 **jcalc**，使其在该布尔变量为 **true** 时模拟极性相反的关节。实际上，**jcalc** 交换了前驱体和后继体的角色。为完成这一任务，**jcalc** 只需在返回之前修改部分计算值。例如，它不返回按给定关节类型的公式计算得到的矩阵 ${}^s\mathbf{X}_p$ ，而是返回该矩阵的逆；并且，它不返回 $\mathbf{v}_J$ 的计算值，而是返回 $-{}^p\mathbf{X}_s\mathbf{v}_J$ 。完整的修改后返回值集合见表 4.2。这种方法的好处是，它使动力学算法可以假设所有树关节都处于正向。这反过来又使算法更简单、更容易实现。

表 4.2: **jcalc** 如何模拟反极性关节

§ 4.5 球面运动

表示一般三维旋转的方法有很多 (例如见 Rooney, 1977)，但主要的选择是欧拉角 (Euler angles) 和欧拉参数 (Euler parameters)。后者也称为单位四元数 (unit quaternions)。欧拉角的优点是位置变量是一组三个独立的角度，速度变量是它们的导数。这使得它们相对容易实现。其缺点是它们存在奇异性 (singularity)：当姿态接近奇异点时，表示会变得越来越病态 (ill-conditioned)，

最终完全失效。欧拉参数的主要优点是它们没有奇异性；但其缺点是它们是一组四个非独立的位置变量，这意味着速度变量不能是位置变量的导数。使用欧拉参数的软件必须编写成允许位置变量的数目不同于速度变量的数目。

4.5.1 欧拉角

我们使用“欧拉角”这一术语时既取广义也取狭义。在广义上，它指任何通过绕指定坐标轴的三次转动序列来表示一般旋转的方法。在狭义上，它特指第一次和第三次转动绕 z 轴、第二次转动绕 x 轴或 y 轴的情况。

每种版本的欧拉角都存在奇异性。每当第一轴和第三轴对齐时就会发生奇异性。如果第一次和第三次转动使用相同的坐标轴，那么当第二个角度为 0° 或 180° 时会发生奇异性；否则，当第二个角度为 $\pm 90^\circ$ 时会发生奇异性。在奇异点附近，所表示旋转的微小变化可以引起第一和第三个角度的大幅变化。基于欧拉角的关节模型在奇异点处是无效的。

让我们使用偏航角 (yaw)、俯仰角 (pitch) 和滚转角 (roll) 来构造一个球关节模型。它们依次是绕 z 、 y 和 x 轴的三次转动。

因此， F_s 相对于 F_p 的正确姿态是这样得到的：先把 F_s 与 F_p 重合，然后绕其 z 轴转动 q_1 (偏航)，再绕其 y 轴转动 q_2 (俯仰)，最后绕其 x 轴转动 q_3 (滚转)。因此 E 的公式为

$$\begin{aligned} E &= \text{rx}(q_3) \text{ry}(q_2) \text{rz}(q_1) \\ &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_3 & s_3 \\ 0 & -s_3 & c_3 \end{bmatrix} \begin{bmatrix} c_2 & 0 & -s_2 \\ 0 & 1 & 0 \\ s_2 & 0 & c_2 \end{bmatrix} \begin{bmatrix} c_1 & s_1 & 0 \\ -s_1 & c_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} c_1 c_2 & s_1 c_2 & -s_2 \\ c_1 s_2 s_3 - s_1 c_3 & s_1 s_2 s_3 + c_1 c_3 & c_2 s_3 \\ c_1 s_2 c_3 + s_1 s_3 & s_1 s_2 c_3 - c_1 s_3 & c_2 c_3 \end{bmatrix}. \end{aligned}$$

S 的公式可以通过把关节速度表示为 $\dot{q}$ 的函数，并利用 $S = \partial v_J / \partial \dot{q}$ 得到。我们将跳过细节，直接给出最终结果：

$$S = \begin{bmatrix} -s_2 & 0 & 1 \\ c_2 s_3 & c_3 & 0 \\ c_2 c_3 & -s_3 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

S 的各列包含各次转动所绕轴在 F_s 中的坐标：第一列是 F_p 的 z 轴；第二列是第一次转动后 y 轴所在的位置；第三列是前两次转动后 x 轴所在的位置。由于 S 不是常数，我们还需要 c_j 的公式。它由下式给出

$$\begin{aligned}
c_J = \dot{S} \dot{q} &= \begin{bmatrix} -c_2 \dot{q}_2 & 0 & 0 \\ -s_2 s_3 \dot{q}_2 + c_2 c_3 \dot{q}_3 & -s_3 \dot{q}_3 & 0 \\ -s_2 c_3 \dot{q}_2 - c_2 c_3 \dot{q}_3 & -c_3 \dot{q}_3 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \end{bmatrix} \\
&= \begin{bmatrix} -c_2 \dot{q}_1 \dot{q}_2 \\ -s_2 s_3 \dot{q}_1 \dot{q}_2 + c_2 c_3 \dot{q}_1 \dot{q}_3 - s_3 \dot{q}_2 \dot{q}_3 \\ -s_2 c_3 \dot{q}_1 \dot{q}_2 - c_2 s_3 \dot{q}_1 \dot{q}_3 - c_3 \dot{q}_2 \dot{q}_3 \\ 0 \\ 0 \\ 0 \end{bmatrix}.
\end{aligned}$$

式 4.7 和式 4.8 取代了表 4.1 中 E 和 S 的公式，但 T 的公式保持不变。因此，这是一个 S 变化而 T 保持不变的关节模型的例子。这种情况之所以可能，是因为 $\text{range}(S)$ 是常数，而 T 只需要满足 $\text{range}(T) = \text{range}(S)^\perp$ 。

4.5.2 欧拉参数

一般旋转也可以用单位向量和角度来描述。单位向量指定旋转轴，角度指定旋转量。欧拉参数与这种表示密切相关。如果 u 和 θ 分别是单位向量和角度，那么描述同一旋转的欧拉参数为

$$\begin{aligned}
p_0 &= \cos(\theta/2) \\
p_1 &= \sin(\theta/2)u_x \\
p_2 &= \sin(\theta/2)u_y \\
p_3 &= \sin(\theta/2)u_z.
\end{aligned}$$

它们不是独立的，但必须满足方程

$$p_0^2 + p_1^2 + p_2^2 + p_3^2 = 1.$$

球关节必须有三个独立的速度变量。因此，如果我们希望用四个非独立的欧拉参数作为位置变量，那么速度变量就不能是位置变量的导数。速度变量的最佳选择是使用关节角速度向量 ω 在 F_S 中的笛卡尔坐标。这一选择使 S 取表 4.1 所示的简单常数值。因此，我们只需要一个 E 的公式和一个位置变量导数的公式。它们为

$$E = 2 \begin{bmatrix} p_0^2 + p_1^2 - 1/2 & p_1 p_2 + p_0 p_3 & p_1 p_3 - p_0 p_2 \\ p_1 p_2 - p_0 p_3 & p_0^2 + p_2^2 - 1/2 & p_2 p_3 + p_0 p_1 \\ p_1 p_3 + p_0 p_2 & p_2 p_3 - p_0 p_1 & p_0^2 + p_3^2 - 1/2 \end{bmatrix}$$

以及

$$\begin{bmatrix} \dot{p}_0 \\ \dot{p}_1 \\ \dot{p}_2 \\ \dot{p}_3 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} -p_1 & -p_2 & -p_3 \\ p_0 & -p_3 & p_2 \\ p_3 & p_0 & -p_1 \\ -p_2 & p_1 & p_0 \end{bmatrix} \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}.$$

基于这些方程的球关节模型在数值上优于基于式 4.7、4.8 和 4.9 的模型。不过，它确实给外围软件带来了负担，即必须处理位置变量数目与速度变量数目不同的关节模型。还有三个额外的复杂之处：位置变量的值必须通过对式 4.13 求积分得到，而不是直接对速度变量求积分；零旋转由一组非零的欧拉参数定义；以及在积分过程中必须频繁地对欧拉参数重新归一化。（积分过程中的截断误差会导致参数不能精确满足式 4.11，从而引起影响关节所支撑的整个子树的线性缩放误差。理想情况下，应在每个积分步之后对它们重新归一化。）

表 4.3: 完整系统模型

§ 4.6 完整系统模型

表 4.3 给出了刚体系统的完整系统模型 (system model)。对于运动学树，所需的数据有： N_B 、父数组 (parent array)、关节类型描述符数组、树变换数组 (tree transform array) 以及刚体惯量 (rigid-body inertia) 数组。惯量以物体坐标表示，在物体坐标中它们是常数。对于闭环 (closed-loop) 系统，额外所需的数据有： N_L ，以及每个环路关节 (loop joint) 的前驱体和后继体的物体编号、关节类型描述符，以及前驱体变换和后继体变换。

在最简单的情况下，软件包可能只支持一组有限的关节类型，例如只有旋转关节和移动关节。在这种情况下， $jtype(i)$ 可以只是一个数字类型代码，将 $\mathbf{X}_j$ 、 v_j 和 $\mathcal{S}$ 的适当取值直接硬编码进 jcalc 的源代码，甚至直接硬编码进动力学算法的源代码，都是可行的。另一个简化涉及对关节变量向量的访问，例如 q 和 $\dot{q}$ 。在一般情况下， q_i 是 q 的一个子向量，它在 q 中的位置取决于所有前导子向量的大小。因此，访问和更新 q_i 的代码应该有一个偏移量数组，每个关节一个，指明 q 的哪个元素是 q_i 的第一个元素。不过，如果每个关节恰好只有一个自由度，例如旋转关节和移动关节的情况，那么 q_i 就是 q 的第 i 个元素。

如果目标是开发通用的软件包，那么更合理的做法是创建一个关节模型库，并让 jcalc 访问这个库。一种特别有用的关节是用户自定义关节 (user-defined joint)。这一功能可以通过使库可扩展来提供。或者，也可以简单地要求用户自定义关节的“参数”之一是用户为该关节提供的 jcalc 函数的地址，并使内置 jcalc 编程为将控制权移交给用户提供的函数。

逆动力学

逆动力学 (inverse dynamics) 是这样一类问题：求在刚体系统中产生给定加速度所需的力。逆动力学计算被用于运动控制系统、轨迹设计与优化（例如用于机器人和动画形象）、机械设计，并作为某些正动力学 (forward dynamics) 算法的组成部分。最后这一应用将在第 6 章中讨论。逆动力学计算可用如下方程概括：

$$\tau = \text{ID}(\text{model}, q, \dot{q}, \ddot{q}),$$

其中 q 、 $\dot{q}$ 、 $\ddot{q}$ 和 τ 分别是广义位置、速度、加速度和力的变量向量，而 model 是某一特定刚体系统的系统模型。

本章给出一个计算运动链 (kinematic tree) 逆动力学的算法。用于闭环系统的算法将在第 8 章出现。运动链的逆动力学特别容易计算，这正是我们从这一问题入手的原因。该算法称为递归牛顿-欧拉算法 (recursive Newton-Euler algorithm)，它是完成此任务最简单、最高效的算法。它的计算复杂度为 $O(n)$ ，这意味着计算量随链中物体（或关节变量）的数目线性增长。算法先以其基本形式给出；然后补充实现细节；再将空间向量版本与原始的 3D 向量版本进行比较。本章还涵盖算法复杂度这一主题，并解释为什么递归算法如此高效。

§5.1 算法复杂度

我们将算法的计算代价 (computational cost) 定义为其运算次数——即在执行算法的过程中所执行的算术运算的数目。如果算法涉及实数运算，如动力学算法那样，那么习惯上只计浮点运算的次数。

因此，诸如计算数组下标或递增循环变量所需的整数运算将被忽略。

算法的计算复杂度 (computational complexity) 是衡量代价如何随问题规模而增长的量。我们用 **big O** 记号 (big O notation) 来刻画这个量。如果某个算法被描述为具有 $O(n^p)$ 复杂度，其中 n 是问题的规模，那么当 $n \rightarrow \infty$ 时代价与 n^p 成正比增长。更精确地说，如果存在正常数 $C_{\min}$ 、 $C_{\max}$ 和 $n_{\min}$ ，使得对所有 $n > n_{\min}$ ，计算代价至少为 $C_{\min}f(n)$ 但至多为 $C_{\max}f(n)$ ，则称该算法具有 $O(f(n))$ 复杂度。

表达式 $O(f(n))$ 可以解释为所有当 $n \rightarrow \infty$ 时与 $f(n)$ 成正比增长的函数所组成的集合。因此，诸如“算法 X 是 $O(f(n))$ ”这样的说法意味着 X 的代价——表示为 n 的函数——是集合 $O(f(n))$ 中的一个元素。而像 $O(f(n)) = O(g(n))$ 这样的表达式则是集合相等，它蕴含 $g(n) \in O(f(n))$ ，

反之亦然。在引用复杂度时，我们选取集合中最简单的函数作为 O 的参数。例如，如果某个算法的精确代价由多项式 $a_0 + a_1n + \cdots + a_p n^p$ 给出，那么与该多项式成正比增长的最简单函数是 n^p ，因此我们将其复杂度引用为 $O(n^p)$ 。一般而言，只有 f 中增长最快的部分才重要。如果 f 收敛到一个常数，那么我们说 f 是 $O(1)$ 。

一个算法的复杂度指标取决于我们如何度量问题的规模。例如，矩阵求逆的复杂度被认为是 $O(n^3)$ ，因为对一个 $n \times n$ 矩阵求逆的代价当 $n \rightarrow \infty$ 时与 n^3 成正比增长。这个指标依赖于 n 是矩阵的维数这一事实。如果 n 改为矩阵中元素的数目，那么复杂度指标将是 $O(n^{1.5})$ 。更一般地，如果 m 和 n 是问题规模的两种不同度量，且 $m = g(n)$ ，那么 $O(f(m)) = O(f(g(n)))$ 。

对于动力学算法，问题的规模就是刚体系统的规模。如果系统是运动链，那么有两种合理的系统规模度量：物体的数目 N_B ，或自由度的数目 n 。如果链中每个关节至少有一个自由度，那么这两个数满足 $N_B \leq n \leq 6N_B$ ，由此可知

$$O(N_B^p) = O(n^p).$$

因此，在引用复杂度时无需区分 N_B 与 n 。

§5.2 递推关系

现代动力学算法是递归的，它们比非递归的前身快若干个数量级。它们之所以被称为递归，是因为它们利用递推关系（recurrence relation）来计算一系列中间结果。正是递推关系的使用带来了高效性；因此让我们花点时间考察这些递推关系是如何运作的。

递推关系是一个用序列中前面的元素来定义下一个元素的公式。这个公式连同初值集合一起，构成了对序列的递归定义。例如，斐波那契数列（Fibonacci numbers）0, 1, 1, 2, 3, 5, 8, 13, ... 由递推关系 $F_i = F_{i-1} + F_{i-2}$ 及初值 $F_0 = 0$ 、 $F_1 = 1$ 递归定义。在刚体系统中，如果物体按照 4.1.2 节的规则编号，那么物体速度序列 $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2, \dots$ 由递推关系 $\mathbf{v}_i = \mathbf{v}_{\lambda(i)} + \mathbf{S}_i \dot{\mathbf{q}}_i$ 及 $\mathbf{v}_0$ 的一个给定初值递归定义。

让我们考察计算物体速度这一任务，分别看使用和不使用递推关系两种情况。为简单起见，假设系统是一个具有 1 自由度关节的无分支运动链。这意味着 $\lambda(i) = i - 1$ ，且关节 i 上的速度是关节轴向量 $\mathbf{s}_i$ 与标量关节速度变量 $\dot{\mathbf{q}}_i$ 的乘积。于是物体速度的递推关系为

$$\mathbf{v}_i = \mathbf{v}_{i-1} + \mathbf{s}_i \dot{\mathbf{q}}_i,$$

初值条件为 $\mathbf{v}_0 = 0$ ；而物体速度的非递归公式为

$$\mathbf{v}_i = \sum_{j=1}^i \mathbf{s}_j \dot{\mathbf{q}}_j.$$

设 $\mathbf{m}$ 和 $\mathbf{a}$ 分别表示一次标量-向量乘法和一次向量加法的代价。那么式 5.2 的一次应用代价为 $\mathbf{m} + \mathbf{a}$ ，式 5.3 的代价为 $i\mathbf{m} + (i-1)\mathbf{a}$ 。因此，通过式 5.2 计算前 n 个物体速度的代价为 $n(\mathbf{m} + \mathbf{a})$ ，而通过式 5.3 计算的代价为 $\frac{1}{2}(n(n+1)\mathbf{m} + n(n-1)\mathbf{a})$ 。于是，使用式 5.2 得到高效的 $O(n)$ 计算，而式 5.3 则导致低效的 $O(n^2)$ 计算。

低效的来源不难看出。如果用式 5.3 完成该任务，那么我们最终要计算

$$\begin{aligned} v_1 &= s_1 \dot{q}_1 \\ v_2 &= s_1 \dot{q}_1 + s_2 \dot{q}_2 \\ v_n &= s_1 \dot{q}_1 + s_2 \dot{q}_2 + \cdots + s_n \dot{q}_n. \end{aligned}$$

乘积 $s_1 \dot{q}_1$ 被重复计算了 n 次，而它本只需计算一次。同样，乘积 $s_2 \dot{q}_2$ 被计算了 $n-1$ 次，和 $s_1 \dot{q}_1 + s_2 \dot{q}_2$ 被计算了 $n-1$ 次，依此类推。因此，递推关系为计算相关量的序列提供了一种系统性的方法来避免不必要的重复，而这正是它们高效的秘诀。

当计算更加复杂时，递推关系的计算优势甚至更大。例如，无分支链中物体加速度的递推关系为

$$a_i = a_{i-1} + s_i \ddot{q}_i + v_i \times s_i \dot{q}_i,$$

它正是式 5.2 的时间导数，而相应的非递归公式为

$$a_i = \sum_{j=1}^i s_j \ddot{q}_j + \sum_{j=1}^i \sum_{k=1}^{j-1} (s_k \dot{q}_k) \times (s_j \dot{q}_j).$$

（在这两种情形下，我们都假设了 $\dot{s}_i = v_i \times s_i$ 。）通过式 5.4 计算加速度是 $O(n)$ 的计算；但通过式 5.5 计算则是 $O(n^3)$ 的计算。

式 5.3 和式 5.5 直接用它们的基本构成要素——关节轴向量以及关节速度和加速度变量——来表达物体速度和加速度。这类方程称为闭式（closed form）方程。刚体系统以闭式形式表达的运动方程为

$$\tau_i = \sum_{j=1}^n H_{ij} \ddot{q}_j + \sum_{j=1}^n \sum_{k=1}^n C_{ijk} \dot{q}_j \dot{q}_k + g_i,$$

其中 H_{ij} 是广义惯性矩阵的元素， C_{ijk} 是离心力和科里奥利力的系数， g_i 是重力项。早期的非递归算法正是通过这类方程来计算动力学，其计算复杂度通常为 $O(n^4)$ 。递归算法则高效得多，其计算复杂度可低至 $O(n)$ 。当递归牛顿-欧拉算法首次出现时，人们发现对于一台 6 自由度的机械臂，它比非递归的前身 Uicker/Kahn 算法快大约一百倍（Hollerbach, 1980）。在高效的递归算法发明之前，闭式方程是表达和求值运动方程的标准方式。

§5.3 递归牛顿-欧拉算法

一般运动链的逆动力学可以通过如下三个步骤来计算：

计算链中每个物体的速度和加速度。

计算产生这些加速度所需的力。

根据作用在物体上的力，计算通过关节传递的力。

这些就是递归牛顿-欧拉算法的步骤。现在，假设该运动链由量 $\lambda(i)$ 、 $\mu(i)$ 、 $\mathcal{S}_i$ 和 $\mathcal{I}_i$ 建模（如第 4 章所述），我们来推导每一步的方程。

图 5.1: 作用在物体 i 上的力

第 1 步: 设 v_i 和 a_i 分别为物体 i 的速度和加速度。运动链中任一物体的速度都可以递归地定义为其父体的速度与连接它与父体的关节上的速度之和。于是,

$$v_i = v_{\lambda(i)} + S_i \dot{q}_i. (v_0 = 0)$$

(初值以括号标出。)加速度的递推关系通过对该方程求微分得到, 结果为

$$a_i = a_{\lambda(i)} + S_i \ddot{q}_i + \dot{S}_i \dot{q}_i. (a_0 = 0)$$

将这两个公式反复迭代, i 从 1 取到 N_B , 就可以计算出链中每个物体的速度和加速度。

第 2 步: 设 f_i^B 为作用在物体 i 上的合力。这个力通过运动方程与物体 i 的加速度相关联:

$$f_i^B = I_i a_i + v_i \times^* I_i v_i.$$

第 3 步: 参照图 5.1, 设 f_i 为通过关节 i 从物体 $\lambda(i)$ 传递给物体 i 的力, 并设 f_i^x 为作用在物体 i 上的外力 (如果有的话)。外力可以用来建模各种环境影响: 力场、物理接触, 等等。它们被视为算法的输入; 也就是说, 假设它们的值是已知的。于是物体 i 上的合力为

$$f_i^B = f_i + f_i^x - \sum_{j \in \mu(i)} f_j,$$

重新整理该式可得到关节力的递推关系:

$$f_i = f_i^B - f_i^x + \sum_{j \in \mu(i)} f_j.$$

将这一公式反复迭代, i 从 N_B 递减到 1, 就可以计算出链中每个关节上的空间力。该公式不需要初值, 因为对任何没有子物体的物体, $\mu(i)$ 都是空集。

计算出每个关节上的空间力之后, 剩下的只是计算关节处的广义力, 它由下式给出:

$$\tau_i = S_i^T f_i$$

(参见式 3.37)。式 5.7 至式 5.11 以最简单的形式描述了递归牛顿-欧拉算法。

外力可以用来建模各种环境影响, 包括引力。然而, 一个均匀引力场可以更高效地建模为基体的一个虚拟加速度。为此, 只需将式 5.8 的初值替换为 $a_0 = -a_g$, 其中 a_g 是重力加速度向量。如果采用这一技巧, 那么计算得到的 a_i 和 f_i^B 就不再是物体 i 的真实加速度和合力, 而是分别偏移了重力加速度向量和重力向量。(物体 i 上的重力为 $I_i a_g$ 。)

5.3.1 算法特性

式 5.7 和式 5.8 涉及数据从根到叶的传播; 式 5.10 涉及数据从叶到根的传播; 而式 5.9 和式 5.11 描述的则是每个物体局部的计算。因此, 递归牛顿-欧拉算法是一个两遍 (two-pass) 算法: 它先对运动链做一遍向外 (outward) 遍历, 在此过程中计算速度和加速度; 然后再做一遍向内 (inward) 遍历, 在此过程中计算关节力。物体力的计算既可以安排在第二遍, 也可以安排在第一遍。前者通常是更好的选择。

这个算法计算的不只是 τ_i 。其他量，如 v_i 和 f_i ，有时也很有用。例如，机械设计者的任务之一是选用强度足够的关节轴承，以承受运行过程中的动态反作用力。这些力可以由 f_i 计算出来。

通过将某些输入设为零，可以计算动力学的特定分量。例如，如果外力以及关节速度和加速度项都设为零，但保留虚拟的基体加速度，那么该算法就只计算重力补偿项（gravity compensation term）——即静态平衡重力的各项。这类项用于机器人控制系统。或者，如果我们把重力、加速度和外力各项都设为零，但保留关节速度，那么该算法就只计算科里奥利项和离心项。如果效率是关切点，那么可以使用该算法的一个精简版本来完成这些专门计算。例如，如果目标只是计算重力补偿项

那么算法就简化为

$$f_i = -I_i a_g + \sum_{j \in \mu(i)} f_j$$

$$\tau_i = S_i^T f_i.$$

要计算真实的重力项，需将 $-I_i a_g$ 替换为 $I_i a_g$ 。

5.3.2 体坐标系中的方程

式 5.7 至式 5.11 在表达算法时不涉及任何特定的坐标系。在实践中，如果与物体 i 有关的计算在物体 i 坐标系中完成（如 4.2 节所定义），算法的效果最佳。这就是算法的体坐标系（body coordinates）（或称连杆坐标系，link coordinates）版本。

让我们把递归牛顿-欧拉算法的各方程变换到体坐标系。量 v_i 、 a_i 、 S_i 、 I_i 、 f_i 和 f_i^B 现在都认为是物体 i 坐标系中表达的。注意，系统模型已经以这些坐标提供了 S_i 和 I_i 。在体坐标系中表达时，式 5.7 和式 5.8 变成

$$v_i = {}^i X_{\lambda(i)} v_{\lambda(i)} + S_i \dot{q}_i \quad (v_0 = 0)$$

以及

$$a_i = {}^i X_{\lambda(i)} a_{\lambda(i)} + S_i \ddot{q}_i + \dot{S}_i \dot{q}_i + v_i \times S_i \dot{q}_i \quad (a_0 = -a_g)$$

与原方程相比，作了如下改动：插入了一个坐标变换，把 $v_{\lambda(i)}$ 和 $a_{\lambda(i)}$ 从物体 $\lambda(i)$ 坐标系变换到物体 i 坐标系；矩阵 $\dot{S}_i$ 被替换为 $\dot{S}_i + v_i \times S_i$ ，其中 $\dot{S}_i$ 是 S_i 在体坐标系中的视在导数。我们还借机将原来的初始加速度值替换为 $a_0 = -a_g$ ，以模拟重力的效果。

关节计算例程 jcalc 能够提供量 S_i 、 $v_{ji} = S_i \dot{q}_i$ 和 $c_{ji} = \dot{S}_i \dot{q}_i$ ，且全部在物体 i 坐标系中。因此，为了利用这些可得数据，我们把式 5.14 和式 5.15 修改如下：

$$v_{ji} = S_i \dot{q}_i$$

$$c_{ji} = \dot{S}_i \dot{q}_i$$

$$v_i = {}^i X_{\lambda(i)} v_{\lambda(i)} + v_{ji} \quad (v_0 = 0)$$

$$\mathbf{a}_i = {}^i\mathbf{X}_{\lambda(i)} \mathbf{a}_{\lambda(i)} + \mathbf{S}_i \ddot{\mathbf{q}}_i + \mathbf{c}_{Ji} + \mathbf{v}_i \times \mathbf{v}_{Ji}. (\mathbf{a}_0 = -\mathbf{a}_g)$$

在体坐标系版本中，式 5.9 和式 5.11 保持不变，但式 5.10 变成

$$\mathbf{f}_i = \mathbf{f}_i^B - {}^i\mathbf{X}_0^* \mathbf{f}_i^x + \sum_{j \in \mu(i)} {}^i\mathbf{X}_j^* \mathbf{f}_j.$$

表 5.1: 递归牛顿-欧拉方程与算法

在该方程中，我们假设外力来自系统外部，并因此假设它们是在绝对坐标系（即物体 0 坐标系）中表达的。式 5.16 至式 5.20 连同式 5.9 和式 5.11，就是递归牛顿-欧拉算法体坐标系版本的各个方程。

5.3.3 算法细节

表 5.1 展示了算法体坐标系版本的伪代码，连同基本版本和体坐标系版本两者的方程。请记住，体坐标系版本中的符号 $\mathbf{v}_i$ 、 $\mathbf{a}_i$ 等是在体坐标系中表达的，而基本版本中的同一符号则表示抽象向量，或未指定的公共坐标系中的坐标向量。

算法的两遍结构在伪代码的两个 for 循环中显而易见。符号 $\mathbf{X}_j$ 、 $\mathbf{v}_j$ 和 $\mathbf{c}_j$ 是局部变量，它们在第一个循环的每次迭代中取新值。如果运动链中包含这样的关节：其速度变量不是位置变量的导数，那么

只需分别将符号 $\dot{\mathbf{q}}_i$ 和 $\ddot{\mathbf{q}}_i$ 替换为 α_i 和 $\dot{\alpha}_i$ 。同样，如果运动链中包含任何显式依赖于时间的关节，那么 jcalc 将自动用式 3.32 和式 3.41 而非式 5.16 和式 5.17 来计算 $\mathbf{v}_j$ 和 $\mathbf{c}_j$ ；伪代码中唯一的变化是把当前时间作为第四个实参传给 jcalc。 ${}^i\mathbf{X}_0$ 唯一被使用的地方是把外力从基坐标系变换到体坐标系。如果没有外力，那么计算这些变换的 if 语句可以省略。

在实现式 5.20 时，我们把它从使用 $\mu(i)$ 的计算改造成了使用 $\lambda(i)$ 的计算。把式 5.20 逐字翻译成伪代码是这样的：

```
for $i = N_B$ to $1$ do
  $f_i = f_i^B - {}^iX_0^* f_i^x$
  for each $j$ in $\mu(i)$ do
    $f_i = f_i + {}^iX_j^* f_j$
  end
end
```

然而，同样的计算可以用另一种次序完成，但得到相同的结果，即下面的代码片段：

```
for $i = 1$ to $N_B$ do
  $f_i = f_i^B - {}^i\mathbf{X}_0^* f_i^x$
end
for $i = N_B$ to $1$ do
  if $\lambda(i) \neq 0$ then
    $f_{\lambda(i)} = f_{\lambda(i)} + {}^{\lambda(i)}\mathbf{X}_i^* f_i$
  end
end
```

在把这个代码片段与算法的其余部分合并时，第一个循环的循环体就成为算法中第一个循环循环体的最后一行。

§5.4 原始版本

递归牛顿-欧拉算法最早在 Luh 等人 (1980a) 的论文中描述，该论文后被收录于 Brady 等人 (1982)。这个原始算法是用 3D 向量表述的，表面上看起来与 5.3 节描述的算法大不相同。然而，这两个算法实际上几乎相同，唯一显著的差别在于一个使用空间加速度，而另一个使用经典加速度（参见 2.11 节）。

表 5.2 展示了原始算法的一个特殊情形，其中所有关节都是转动关节。用 3D 向量表达刚体动力学的缺点之一是方程对关节类型敏感。该算法

原始方程：

$$\begin{aligned}
 \omega_i &= {}^i E_{i-1} \omega_{i-1} + z \dot{q}_i \\
 v_i &= {}^i E_{i-1} \left(v_{i-1} + \omega_{i-1} \times {}^{i-1} \mathbf{1} r_i \right) \\
 \dot{\omega}_i &= {}^i E_{i-1} \dot{\omega}_{i-1} + z \ddot{q}_i + \left({}^i E_{i-1} \omega_{i-1} \right) \times z \dot{q}_i \\
 \dot{v}_i &= {}^i E_{i-1} \left(\dot{v}_{i-1} + \dot{\omega}_{i-1} \times {}^{i-1} \mathbf{1} r_i + \omega_{i-1} \times {}^{i-1} \mathbf{1} \dot{r}_i \right) \\
 F_i &= m_i (v_i + \omega_i \times c_i + \omega_i \times \omega_i \times c_i) \\
 N_i &= I_i^{cm} \dot{\omega}_i + \omega_i \times I_i^{cm} \omega_i \\
 f_i &= F_i + {}^i E_{i+1} f_{i+1} \\
 n_i &= N_i + c_i \times F_i + {}^i E_{i+1} n_{i+1} + {}^i r_{i+1} \times {}^i E_{i+1} f_{i+1} \\
 \mathcal{T}_i &= z^T n_i
 \end{aligned}$$

符号说明：

$\omega_i, \dot{\omega}_i$ 物体 i 的角速度和角加速度； F_i （物体 i 坐标系）原点的线速度和线加速度

F_i, N_i 作用在物体 i 上、在其质心处表达的合力和合力矩

f_i, n_i 通过关节 i 作用在物体 i 上的力和力矩

m_i, c_i, I_i^{cm} 物体 i 在 F_i 坐标系中的质量、质心位置和绕质心的转动惯量

${}^i E_{i-1}$ 从 F_{i-1} 坐标变换到 F_i 坐标的 3×3 正交旋转矩阵

${}^{i-1} r_i$ 原点 F_i 相对于 F_{i-1} 的位置，在 F_{i-1} 坐标中表达

$\dot{q}_i, \ddot{q}_i, \tau_i$ 关节 i 的速度、加速度和力变量

z 关节 i 轴在 F_i 坐标中的方向

与表 5.1 中版本的对应关系：

$$\begin{aligned}
\hat{v}_i &= \begin{bmatrix} \omega_i \\ v_i \end{bmatrix} \quad \hat{a}_i = \begin{bmatrix} \dot{\omega}_i \\ \dot{v}_i - \omega_i \times v_i \end{bmatrix} \quad \hat{f}_i^B = \begin{bmatrix} N_i + c_i \times F_i \\ F_i \end{bmatrix} \quad \hat{f}_i = \begin{bmatrix} n_i \\ f_i \end{bmatrix} \\
{}^i X_{i-1} &= \begin{bmatrix} {}^i E_{i-1} & 0 \\ -E_{i-1}^{i-1} r_i \times & E_{i-1} \end{bmatrix} \quad \hat{I}_i = \begin{bmatrix} I_i^{cm} - m_i c_i \times c_i \times & m_i c_i \times \\ -m_i c_i \times & m_i \mathbf{1} \end{bmatrix} \\
S_i &= \begin{bmatrix} z \\ 0 \end{bmatrix} \quad z = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad \lambda(i) = i - 1, \mu(i) = \{i + 1\}
\end{aligned}$$

在 Luh 等人 (1980a) 中描述的该算法是为转动关节和棱柱关节两者制定的，它充满了如下形式的方程：

$$\text{variable} = \begin{cases} \text{one expression} & \text{if joint is revolute} \\ \text{another expression} & \text{if joint is prismatic.} \end{cases}$$

本表中 3D 向量符号的选择，是在本书所用记号与列示该算法的其他书籍所用记号之间的一种折衷。本表中的方程是为无分支运动链而写的，与其他地方描述该算法的方式一致，但把它们修改为适用于有分支运动链是件很简单的事。

表 5.2 还展示了原始算法中的 3D 向量与表 5.1 所示算法中的空间向量之间的对应关系。只有两处 3D 向量与空间向量的相应 3D 分量不匹配：线加速度和物体净力矩。后者是一个微不足道的差别，在用 N_i 计算 n_i 时立刻就被修正了。它是由 N_i 在质心而非体坐标系原点计算引起的。前者则是一个非平凡的差别，因为它会影响从一个物体传播到下一个物体的量。¹

原始算法的一个有趣特性是线速度从未在任何后续表达式中被使用，因此无需计算。这是使用经典加速度所产生的一个特殊性质。如果省略速度计算，那么这个算法比空间版本略高效一些。然而，差别只有几个百分点，很容易被其他与效率相关的事项所淹没，例如编程语言的选择、计算机硬件，等等。

§5.5 附加说明

Luh 等人 (1980a) 中给出的算法，就是我们通常所认为的递归牛顿-欧拉算法。然而，把牛顿-欧拉运动方程与递归求值策略相结合的想法比这个算法早出现几年。特别是，可以在 Stepanenko 和 Vukobratovic (1976) 以及 Orin 等人 (1979) 中看到递归牛顿-欧拉算法的早期形式。此后该算法不断被精炼和优化，其中一些优化将在第 10 章讨论。两个最快的公开发表版本见 He 和 Goldenberg (1989) 以及 Balafoutis 等人 (1988)，后者也出现在 Balafoutis 和 Patel (1991) 中。在它被发明之时，

即当时拉格朗日动力学比牛顿-欧拉动力学更流行；这促使 Hollerbach 设计了递归的拉格朗日算法，并比较了各种递归和非递归算法的计算复杂度 (Hollerbach, 1980)。正是这项工作把以 Uicker/Kahn 算法 (Kahn 和 Roth, 1971) 为代表的非递归方法的 $O(n^4)$ 复杂度，与递归算法的 $O(n)$ 复杂度进行了对比。

当时机器人学界进行这类研究的推动力，是开发一种足够快、可用于机器人运动实时控制的逆动力学算法。这方面的一个早期例子见 Luh 等人 (1980b)，尽管操作空间 (operational-space)

表述最终变得更为流行 (Khatib, 1987)。当时计算机相对较慢的速度使一些研究者转而研究并行计算机实现 (Lathrop, 1985)，另一些研究者则研究符号优化技术 (Murray 和 Neuman, 1984; Neuman 和 Murray, 1987)。一个有趣的新方向是开发高效算法来计算逆动力学对某个设计参数的偏导数。这方面的一个例子见 Fang 和 Pollard (2003)。

前向动力学——惯性矩阵方法

前向动力学 (forward dynamics) 是在给定施加力的情况下求刚体系统加速度的问题。它主要用于仿真, 有时也被称为“直接动力学 (direct dynamics)”, 或简称“动力学”。在本章和下一章中, 我们考察运动树 (kinematic tree) 的前向动力学。闭环系统的动力学将在第 8 章中介绍。

求解运动树前向动力学问题主要有两种方法:

对整个系统建立运动方程, 并对其加速度变量求解; 或者

以这样的方式从一个物体到下一个物体传播约束, 使得加速度可以一次一个关节地计算出来。

本章考察第一种方法, 第二种方法在第 7 章中考察。第一种方法涉及计算一个 $n \times n$ 关节空间惯性矩阵 (joint-space inertia matrix) 的元素, 然后对该矩阵进行分解, 以求解关于加速度变量的一组 n 个线性方程。这类算法在最坏情况下的复杂度为 $O(n^3)$, 有时被统称为 $O(n^3)$ 算法。然而, 将它们描述为 $O(nd^2)$ 算法更为准确, 其中 d 是树的深度。第二种方法涉及对树进行固定次数的遍历, 每次遍历对每个物体执行固定次数的计算。采用这种方法的算法具有 $O(n)$ 复杂度, 因此是包含大量物体的系统的最佳选择。然而, $O(nd^2)$ 算法在物体数量很少、或者分支足够多而深度很小的树上, 可以与 $O(n)$ 算法的速度相当, 甚至略胜一筹。 $O(nd^2)$ 算法也是闭环动力学算法的重要组成部分。这两种方法之间的效率权衡将在第 10 章讨论。

本章首先介绍一些关于如何使用关节空间惯性矩阵求解前向动力学问题的基础内容。然后, 第 6.2 节介绍复合刚体算法 (composite-rigid-body algorithm), 这是计算该矩阵最快的算法; 第 6.3 节给出第二种推导, 反映了该算法最初被发现和描述的方式。接下来的两节描述分支诱导稀疏性 (branch-induced sparsity) 这一现象, 并提出两种利用该稀疏性的分解算法。这种稀疏性是计算关节空间惯性矩阵的成本仅为 $O(nd)$ 、分解成本仅为 $O(nd^2)$ 的原因。最后, 第 6.6 节介绍一些关于前向动力学问题的背景材料。

§ 6.1

关节空间惯性矩阵

运动树的运动方程可以表示为矩阵形式

$$\tau = H(q)\ddot{q} + C(q, \dot{q}, f^x),$$

其中 q 、 $\dot{q}$ 、 $\ddot{q}$ 和 τ 分别是关节位置、速度、加速度和力变量组成的向量, f^x 是外力向量 (如果有的话), H 是关节空间惯性矩阵, C 是关节空间偏置力 (joint-space bias force)。 H 和 C 是运

动方程的系数，而 τ 和 $\ddot{q}$ 是变量。该方程表明 H 是 q 的函数， C 是 q 、 $\dot{q}$ 和 f^x 的函数。一旦理解了这些函数依赖关系，通常就把它省略掉。它们所传达的最有用的信息是： H 只依赖于位置变量。如果系统处于静止状态，并且除 τ 之外没有力作用于其上，那么 $C = 0$ ，运动方程简化为 $\tau = H\ddot{q}$ 。

在前向动力学问题中， τ 是给定的，任务是计算 $\ddot{q}$ 。本章所描述的方法通过以下步骤来求解该问题：

计算 C ，

计算 H ，

求解 $H\ddot{q} = \tau - C$ ，得到 $\ddot{q}$ 。

这三个步骤的计算复杂度分别为 $O(n)$ 、 $O(nd)$ 和 $O(nd^2)$ ，其中 d 是树的深度。由于 d 满足 $d \leq n$ ，步骤 2 和步骤 3 的最坏情况复杂度为 $O(n^2)$ 和 $O(n^3)$ 。在另一个极端，如果 d 受到一个固定上限的约束（在实践中可能发生），那么三个步骤的复杂度都是 $O(n)$ 。如下文所述，步骤 1 可以很容易地通过逆动力学算法完成，因此本章主要关注步骤 2 和步骤 3。

H 是一个对称、正定的 $n \times n$ 矩阵，被组织成一个 $N_B \times N_B$ 的子矩阵阵列。（回想一下，对于树， $N_B = N_J$ 。）如果 n_i 是关节 i 的自由度，那么 H_{ij} 是 H 中占据块行 (block-row) i 和块列 (block-column) j 的 $n_i \times n_j$ 子矩阵。如果 $p_i = \sum_{k=1}^i n_k$ ，那么块行 i 由行 $p_{i-1} + 1$ 至 p_i （含）组成，块列 j 由列 $p_{j-1} + 1$ 至 p_j （含）组成。物理上， H_{ij} 将关节 j 处的加速度与关节 i 处的力联系起来。如果树中的每个关节都只有一个自由度，那么 $n = N_B$ ， H_{ij} 是 H 第 i 行第 j 列的 1×1 矩阵。

假设我们有一个像第 5 章描述的那样的逆动力学计算函数 ID 。该函数执行如下计算：

$$\tau = ID(model, q, \dot{q}, \ddot{q}, f^x).$$

将此表达式与式 6.1 比较，可以立即得出

$$C = ID(model, q, \dot{q}, 0, f^x)$$

以及

$$H\ddot{q} = ID(model, q, \dot{q}, \ddot{q}, f^x) - C.$$

因此， C 的值可以通过一次对 ID 的调用获得， H 与任意向量的乘积则是两次对 ID 调用的差。让我们定义一个微分逆动力学函数 (differential inverse dynamics function) ID_δ ，它计算对应于期望加速度变化的关节力变化。该函数由下式给出

$$ID_\delta(model, q, \ddot{q}) = ID(model, q, \dot{q}, \ddot{q}, f^x) - ID(model, q, \dot{q}, 0, f^x).$$

利用这个函数，我们可以如下计算 H 的第 α 列：

$$H\delta_\alpha = ID_\delta(model, \dot{q}, \delta_\alpha),$$

其中 δ_α 是一个 n 维坐标向量，其第 α^{th} 个元素为 1，其余元素为 0。通过让 α 取 1 到 n 的每一个值重复这一计算，我们可以一次一列地计算出整个 H 。这个想法是 Walker 和 Orin (1982) 所描述的方法 1 和方法 2 的基础。

式 6.4 的右侧有大量的项发生抵消。特别是，每一个依赖于 $\dot{\mathbf{q}}$ 或 $\mathbf{f}^x$ 的项都会被抵消，这就是为什么这些向量被从 $\mathbf{ID}_\delta$ 的参数列表中省略掉。因此，可以制定一个大大简化的递归牛顿-欧拉算法 (recursive Newton-Euler algorithm) 版本来专门计算 $\mathbf{ID}_\delta$ 。该算法的方程和伪代码如表 6.1 所示。伪代码省略了 $\mathbf{S}_i$ 和 ${}^i\mathbf{X}_{\lambda(i)}$ 的计算，因为它们被假定为计算 $\mathbf{C}$ 时附带产生的可用结果。

表 6.1 中的算法具有简单直接的优点。然而，它并不是计算 H 的最有效方法。最快的算法是复合刚体算法，它是下一节的主题。

表 6.1: 计算 $\tau = H\dot{\mathbf{q}}$ 的方程和算法

§ 6.2 复合刚体算法

运动树的动能是其各个物体动能之和。因此，

$$T = \frac{1}{2} \sum_{k=1}^{N_B} \mathbf{v}_k^T \mathbf{I}_k \mathbf{v}_k.$$

物体 k 的速度由下式给出

$$\mathbf{v}_k = \sum_{i \in \kappa(k)} \mathbf{S}_i \dot{\mathbf{q}}_i,$$

其中 $\kappa(k)$ 是支撑物体 k 的关节集合，即物体 k 与基座之间路径上的关节集合。（关于 $\kappa(i)$ 、 $\nu(i)$ 和 $\mu(i)$ 的描述见 §4.1.4。）将式 6.6 代入式 6.5，得到

$$T = \frac{1}{2} \sum_{k=1}^{N_B} \sum_{i \in \kappa(k)} \sum_{j \in \kappa(k)} \dot{\mathbf{q}}_i^T \mathbf{S}_i^T \mathbf{I}_k \mathbf{S}_j \dot{\mathbf{q}}_j.$$

现在，这个方程是对所有这样的 i, j, k 三元组求和，其中关节 i 和关节 j 都支撑物体 k 。因此，它可以重排为

$$T = \frac{1}{2} \sum_{i=1}^{N_B} \sum_{j=1}^{N_B} \sum_{k \in \nu(i) \cap \nu(j)} \dot{\mathbf{q}}_i^T \mathbf{S}_i^T \mathbf{I}_k \mathbf{S}_j \dot{\mathbf{q}}_j,$$

其中 $\nu(i) \cap \nu(j)$ 是关节 i 所支撑的物体集合与关节 j 所支撑的物体集合的交集。因此，式 6.8 是对所有这样的 i, j, k 三元组求和，其中物体 k 同时被关节 i 和关节 j 支撑，这与式 6.7 中的求和相同。 $\nu(i) \cap \nu(j)$ 的值由下式给出

$$\nu(i) \cap \nu(j) = \begin{cases} \nu(i) & \text{如果 } i \in \nu(j) \\ \nu(j) & \text{如果 } j \in \nu(i) \\ \emptyset & \text{其他情况} \end{cases}$$

其中 $\emptyset$ 表示空集。

现在，关节空间惯性矩阵的一个性质是：树的动能由以下表达式给出

$$T = \frac{1}{2} \dot{q}^T H \dot{q} = \frac{1}{2} \sum_{i=1}^{N_B} \sum_{j=1}^{N_B} \dot{q}_i^T H_{ij} \dot{q}_j.$$

将该方程与式 6.8 比较, 可以看出

$$H_{ij} = \sum_{k \in v(i) \cap v(j)} S_i^T I_k S_j.$$

现在让我们引入一个新的量 I_i^c , 它是根在物体 i 处的子树作为单个复合刚体 (composite rigid body) 处理时的惯性。(这就是该算法名字的由来。) I_i^c 由下式给出

$$I_i^c = \sum_{j \in v(i)} I_j;$$

并且可以使用如下公式递归计算

$$\mathcal{I}_i^c = \mathcal{I}_i + \sum_{j \in \mu(i)} \mathcal{I}_j^c.$$

将式 6.11、6.12 和 6.9 结合起来, 得到关节空间惯性矩阵的如下公式:

$$H_{ij} = \begin{cases} S_i^T I_i^c S_j & \text{如果 } i \in v(j) \\ S_i^T I_j^c S_j & \text{如果 } j \in v(i) \\ 0 & \text{其他情况.} \end{cases}$$

式 6.13 和 6.14 是通用运动树复合刚体算法的基本方程。对于无分支树 (unbranched tree) 的特殊情况, 这些方程简化为

$$I_i^c = I_i + I_{i+1}^c$$

以及

$$H_{ij} = S_i^T I_{\max(i,j)}^c S_j.$$

6.2.1 物体坐标系中的方程

式 6.13 可以表示为物体坐标形式

$$I_i^c = I_i + \sum_{j \in \mu(i)} X_j^* I_j^c X_i.$$

原则上, 式 6.14 可以表示为物体坐标形式

$$H_{ij} = \begin{cases} S_i^T I_i^c X_j S_j & \text{如果 } i \in v(j) \\ S_i^T X_j^* I_j^c S_j & \text{如果 } j \in v(i) \\ 0 & \text{其他情况.} \end{cases}$$

然而，要为此方程设计一个高效的计算方案，有必要定义一个新的量 jF_i ，它是用物体 j 的坐标系表示的 $I_i^c S_i$ 的值，即 ${}^jF_i = {}^jX_i^* I_i^c S_i$ 。要计算 H ，我们需要对每一对满足 $j \in \kappa(i)$ 的 i, j 都有一个 jF_i 的实例。它们可以使用如下公式递归计算

$$\lambda^{(j)} F_i = \lambda^{(j)} X_j^* {}^jF_i, \left({}^iF_i = I_i^c S_i \right)$$

其中 i 取 1 到 N_B 范围内的每一个值，而 j （对每个 i 分别地）依次经过 i 、 $\lambda(i)$ 、 $\lambda(\lambda(i))$ 等值，一直回溯到基座。关节空间惯性矩阵则由下式给出

$$H_{ij} = \begin{cases} {}^jF_i^T S_j & \text{如果 } i \in \nu(j) \\ H_{ji}^T & \text{如果 } j \in \nu(i) \\ 0 & \text{其他情况.} \end{cases}$$

式 6.17、6.19 和 6.20 是物体坐标系形式的复合刚体算法的方程。

6.2.2 算法细节

表 6.2 展示了该算法物体坐标系版本的伪代码，同时包括基本方程和物体坐标系算法的方程。请记住，基本版本中的符号 I_i 、 I_i^c 和 S_i 表示抽象的张量，或者用未指定的公共坐标系表示的张量，而物体坐标系方程中相同的符号则表示用物体 i 的坐标系表示的量。伪代码不包括计算 S_i 或 ${}^iX_{\lambda(i)}$ 的代码，因为这些量被假定为在应用递归牛顿-欧拉算法计算式 6.1 中的 C 时附带产生的可用结果。

在实现式 6.17 时，我们已将其从使用 $\mu(i)$ 的计算变换为使用 $\lambda(i)$ 的计算。将式 6.17 逐字翻译成伪代码看起来像这样：

表 6.2：复合刚体方程和算法

```
for $i = N_B$ to $1$ do
  $I_i^c = I_i$
  for each $j$ in $\mu(i)$ do
    $I_i^c = I_i^c + \{^iX_j^* I_j^c \}^j X_i$
  end
end
```

然而，同样的计算可以按不同的顺序进行，但结果相同，使用如下代码片段（参见 §5.3 中式 5.20 的实现）：

```
for $i = 1$ to $N_B$ do
  $I_i^c = I_i$
end
for $i = N_B$ to $1$ do
  if $\lambda(i) \neq 0$ then
    $I_{\lambda(i)}^c = I_{\lambda(i)}^c + \{^{\lambda(i)}X_i^* I_i^c \}^i X_{\lambda(i)}$
  end
end
```

该算法从两个准备步骤开始：将 H 置零，并将每个 I_i^c 设置为 I_i 。包含第一步是为了提醒读者

图 6.1: 经历 $\ddot{q} = \delta_\alpha$ 加速度的运动树

其余代码将只初始化 H 中的非零子矩阵。这一步是否必要取决于 H 的用途。然后，主循环实现式 6.17、6.19 和 6.20 的计算。 F 是一个局部变量，初始设置为 $I_i^c S_i$ （因此等于方程中的 ${}^i F_i$ ）。while 循环在计算子矩阵 H_{ij} 和 H_{ji} 的同时，依次把这个变量变换到从物体 i 到基座路径上每个物体 j 的坐标系（即每个 $j \in \kappa(i)$ ）中。请注意，只有需要访问上三角矩阵时，才需要设置 H_{ji} 。

如果 d 是运动树的深度，那么对于任何 i 值，while 循环的执行次数都不会超过 $d - 1$ 次。因此可以得出，复合刚体算法的计算成本不可能超过 $O(nd)$ ， H 中非零元素的数目最多为 $O(nd)$ 。详细的成本分析见第 10.3.2 节。

§ 6.3 物理意义

本节给出复合刚体算法的另一种推导，更接近原始版本，目的是展示在该算法发现过程中起作用的一些物理直觉。

如前所述，如果运动树处于静止状态，并且除 τ 之外没有力作用于其上，那么运动方程简化为 $\tau = H\ddot{q}$ 。如果在这种条件下令 $\ddot{q} = \delta_\alpha$ ，那么 τ 将是 H 的第 α^{th} 列。图 6.1 显示了当 $\ddot{q} = \delta_\alpha$ 时树中发生的情况。 $\ddot{q}$ 中的第 α^{th} 个变量必然属于某个关节。因此，我们确定拥有这个变量的关节，称之为关节 i 。同样，我们确定 S_i 中与第 α^{th} 个变量相对应的列，称之为 s_α 。（ s_α 是一个空间运动向量 (spatial motion vector)。）树中运动现在可以这样描述：跨越关节 i 的加速度为 s_α ，跨越每个其他关节的加速度为零。因此，关节 i 所支撑的子树中每个物体的加速度都是 s_α ，每个其他物体的加速度都为零。换句话说，

$$a_j = \begin{cases} s_\alpha & \text{如果 } j \in \nu(i) \\ \mathbf{0} & \text{其他情况.} \end{cases}$$

设 f_j^B 和 f_j 分别是物体 j 上的净力和跨越关节 j 传递的总力。由于没有速度项， f_j^B 由下式给出

$$f_j^B = I_j a_j = \begin{cases} I_j s_\alpha & \text{如果 } j \in \nu(i) \\ \mathbf{0} & \text{其他情况.} \end{cases}$$

现在，关节 j 是子树 $\nu(j)$ 与系统其余部分之间的唯一连接。由于除通过关节传递的力之外，没有任何力作用于这些刚体，因此 f_j 必定是作用于子树 $\nu(j)$ 上的净力。然而，子树上的净力也等于该子树上各刚体净力之和，因此我们有

$$f_j = \sum_{k \in \nu(j)} f_k^B.$$

将这两个方程联立可得

$$f_j = \begin{cases} \sum_{k \in \nu(j)} I_k s_\alpha & \text{if } j \in \nu(i) \\ \sum_{k \in \nu(i)} I_k s_\alpha & \text{if } j \in \kappa(i) \\ 0 & \text{otherwise,} \end{cases}$$

利用式 (6.12), 该式可化简为

$$f_j = \begin{cases} I_j^c s_\alpha & \text{if } j \in \nu(i) \\ I_i^c s_\alpha & \text{if } j \in \kappa(i) \\ 0 & \text{otherwise} \end{cases}$$

最后, 关节力变量 (joint force variable) 为

$$\tau_j = S_j^T f_j = \begin{cases} S_j^T I_j^c s_\alpha & \text{if } j \in \nu(i) \\ S_j^T I_i^c s_\alpha & \text{if } j \in \kappa(i) \\ 0 & \text{otherwise.} \end{cases}$$

这些正是 $\mathbf{H}$ 第 α 列的子向量。

式 (6.24) 中的第二种情形揭示了促成复合刚体算法 (composite-rigid-body algorithm) 发现的两个关键事实: 其一是 f_i 等于关节运动向量与复合刚体惯量的乘积; 其二是对 $\kappa(i)$ 中的所有 j (即图 6.1 中的深灰色关节) 都有 $f_j = f_i$ 。因此, 该算法由以下步骤组成:

通过式 (6.17) 计算复合刚体惯量;

在刚体 i 的坐标系中计算 $f_i = I_i^c s_\alpha$;

图 6.2: 分支诱导稀疏性 (branch-induced sparsity) 示例

将 f_i 变换到 $\kappa(i)$ 中每个刚体的坐标系; 以及

计算 H 中第 α 列主对角线以上部分的元素, 并将其复制到第 α 行主对角线以下部分, 由于对称性, 这两部分是相同的。

该算法与 6.2 节中算法的唯一区别在于, 后者按块 (block) 而非逐个地处理各列, 每个块对应单个关节的所有列。若关节 i 有 n_i 个自由度, 则式 (6.19) 中的量 ${}^i F_i$ 就是一个 $6 \times n_i$ 矩阵, 其各列即为属于关节 i 的每个 α 所对应的力 $f_i = I_i^c s_\alpha$ 。

§ 6.4

分支诱导稀疏性 (Branch-Induced Sparsity)

式 (6.14) 表明, 仅仅由于连通关系, H 的某些元素就会自动为零。根据该式, 只要 i 既不是 j 的祖先, 也不是 j 的后代, 也不等于 j , 则 H_{ij} 即为零。这种情况只会在 i 和 j 位于树的不同分支上时发生, 因此我们称之为分支诱导稀疏性。

图 6.2 展示了几个简单的连通图 (connectivity graph) 及其所产生的稀疏模式。例 (a) 是一个极端情形: 每个刚体都直接连接到基座 (base) 上, 从而产生最大可能的分支诱导稀疏性。此时, 所有非对角线子矩阵均为零。更一般地, 如果基座有多个子节点, 则 H 总是块对角矩阵 (或其对称置换)。例 (b) 展示了相反的极端情形: 树没有分支, 因此不存在分支诱导稀疏性。例

(c) 展示了一种较为典型的稀疏模式；例 (d) 和例 (e) 则说明了编号方案的影响：同一张图采用两种不同的编号方式，得到的矩阵互为对称置换。

图 6.3：稀疏性在因子分解中得到保持！

原则上，这两个矩阵具有相同的底层稀疏模式，编号方式的选择不会影响我们利用这一稀疏性的能力。

分支诱导稀疏性常常能影响到 $\mathbf{H}$ 中相当大比例的元素。我们可以用下面的经验法则粗略估计其收益：如果 $\mathbf{H}$ 的密度为 ρ ，那么计算它的成本约为计算同尺寸稠密 $\mathbf{H}$ 的成本的 ρ 倍，而对其作因子分解的成本约为对同尺寸稠密矩阵作因子分解的成本的 ρ^2 倍。这里 ρ 定义为 $\mathbf{H}$ 中非分支诱导零元素所占的比例，因此 $0 < \rho \leq 1$ 。密度约为 0.5 的情形并不少见，密度接近于零的情形也可能出现。总体而言，分支诱导稀疏性的作用是使惯量矩阵方法（inertia-matrix method）在分支运动学树上比在同尺寸无分支树上更为高效。

复合刚体算法只需计算 $\mathbf{H}$ 的非零子矩阵，便能自动利用分支诱导稀疏性。然而，如果我们试图用标准的因子分解算法对所得矩阵进行分解，那么该算法会把矩阵当作稠密矩阵处理，执行 $O(n^3)$ 次算术运算。因此，为了充分利用稀疏性，我们需要一种适用于含分支诱导稀疏性矩阵的因子分解算法。这个问题解决起来其实很容易，解决办法是将 $\mathbf{H}$ 分解为 $\mathbf{L}^T\mathbf{L}$ 或 $\mathbf{L}^T\mathbf{DL}$ ，并设计因子分解算法使其跳过分支诱导零元素。

在稀疏矩阵文献中，分解 $\mathbf{H} = \mathbf{L}^T\mathbf{L}$ 被称为重排 Cholesky 分解（reordered Cholesky factorization），意即它等价于对原矩阵的某个置换执行标准 Cholesky 分解（George and Liu, 1981）。类似地，分解 $\mathbf{H} = \mathbf{L}^T\mathbf{DL}$ 被称为重排 LDL^T 分解。这意味着 $\mathbf{L}^T\mathbf{L}$ 与 $\mathbf{L}^T\mathbf{DL}$ 分解具有与标准 Cholesky 和 LDL^T 分解相同的数值性质。

当 $\mathbf{L}^T\mathbf{L}$ 或 $\mathbf{L}^T\mathbf{DL}$ 分解应用于含分支诱导稀疏性的矩阵时，其特殊性质在于分解过程不会产生填充（fill-in）。换言之，矩阵中的每个分支诱导零元素在整个分解过程中始终保持为零。（这一点已在 Featherstone（2005）中得到证明。）具有这种性质的分解被称为最优分解（optimal factorization, Duff et al., 1986）。

** 表 6.3: $\mathbf{L}^T\mathbf{L}$ 与 $\mathbf{L}^T\mathbf{DL}$ 因子分解算法 **

$\mathbf{L}^T\mathbf{L}$ 分解:

```
for $k=n$ to $1$ do
  $H_{kk}=\sqrt{H_{kk}}$
  $i=\lambda(k)$
  while $i\neq 0$ do
    $H_{ki}=H_{ki}/H_{kk}$
    $i=\lambda(i)$
  end
  $i=\lambda(k)$
  while $i\neq 0$ do
    $j=\lambda(i)$
    $a=H_{ki}$
    while $j\neq 0$ do
      $H_{ij}=H_{ij}-aH_{kj}$
      $j=\lambda(j)$
    end
  end
end
```

```

        end
        $i=\lambda(i)$
    end
end

```

L^TDL 分解:

```

for $k=n$ to $1$ do
    $i=\lambda(k)$
    while $i\neq 0$ do
        $j=\lambda(i)$
        $a=H_{\{ki\}}/H_{\{kk\}}$
        while $j\neq 0$ do
            $H_{\{ij\}}=H_{\{ij\}}-aH_{\{kj\}}$
            $j=\lambda(j)$
        end
        $H_{\{ki\}}=a$
        $i=\lambda(i)$
    end
end
end

```

由此直接得到的一个结果是, H 的稀疏模式在所得因子中得到保持, 因此这些因子本身也是稀疏的。图 6.3 展示了这一效果, 同时给出了原始模式与所得三角因子中的模式。这种稀疏性可以在回代 (back-substitution) 过程中加以利用。无填充性质的第二个结果是, 在分解过程中可以完全忽略分支诱导零元素。

§ 6.5 — 稀疏因子分解算法 (Sparse Factorization Algorithms)

表 6.3 给出了稀疏 L^TL 与 L^TDL 因子分解算法的伪代码。这两个算法需要两个输入: 一个 $n \times n$ 矩阵 H 和一个包含 n 个元素的整数数组 λ 。 H 应为对称正定矩阵, 且 λ 的元素应满足 $0 \leq \lambda(i) < i$ 。此外, H 应具有如下稀疏模式, 该模式可被算法最优地利用: 在 H 的第 k 行, 主对角线以下的非零元素只出现在第 $\lambda(k)$ 、 $\lambda(\lambda(k))$ 等列上。这条规则划定了算法视为非零的元素集合, 其余所有元素均假定为零并被忽略。若对所有 k 均有 $\lambda(k) = k - 1$, 则算法会将每个元素都视为非零, 此时它执行的便是稠密矩阵的 L^TL 或 L^TDL 分解。

图 6.4 展示了这两个算法的工作方式。它们都在给定矩阵上就地 (in situ) 操作, 且从不访问主对角线以上的任何元素。 L^TL

```

 $L^TL$ 
 $H_{kk} = \sqrt{H_{kk}}$ 
不进行任何操作
 $H_{ki} = H_{ki}/H_{kk}$ 

```

$$\begin{aligned}
2a. \quad & H'_{ki} = H_{ki} / H_{kk} \\
& H_{ij} = H_{ij} - H_{ki} H_{kj} \\
& H_{ij} = H_{ij} - H_{ki} H_{kj} \\
2b. \quad & H_{ki} = H'_{ki}
\end{aligned}$$

图 6.4: 分解过程的示意图

算法计算 L 并将其返回到 H 的下三角中。 L^TDL 算法计算 D 和 L ，将 D 返回到主对角线上，并将其以下的 L 的非对角线元素返回到主对角线下方。之所以可行，是因为该算法计算的是一个单位下三角矩阵，其对角线元素已知为 1，因此无需返回。

每个算法都有一个外层循环，它从 n 开始依次逐行处理，直至第 1 行。在分解过程中的任一阶段，第 $k+1$ 行到第 n 行已经完成，其中存放着所得因子的第 $k+1$ 行到第 n 行。第 1 行到第 k 行可分为三个区域，如图所示。区域 1 仅包含元素 $H_{k,k}$ ；区域 2 包含第 k 行的第 1 到第 $k-1$ 个元素；区域 3 包含第 1 到第 $k-1$ 行的三角区域。每个区域中进行不同的计算，如图所示。 L^TL 分解的伪代码在两个独立的循环中执行区域 2 和区域 3 的计算；而 L^TDL 分解的伪代码则以交错的方式执行这些计算，使其在任一点都只需记住至多一个 H'_{ki} 值。当前记住的值存放在局部变量 a 中。

内层循环被设计为只对 $\lambda(k)$ 、 $\lambda(\lambda(k))$ 等值进行迭代，这正是利用稀疏性的关键所在。实际上，算法知道零元素的位置，直接将其跳过。图 6.4 通过展示对图 6.3 中矩阵 H 作分解的前三个步骤来说明成本的降低。在 $k=7$ 时，算法执行两次区域 2 的计算和三次区域 3 的计算；在 $k=6$ 和 $k=5$ 时情况类似。这远少于若无零元素时所必需的计算次数。

表 6.4: 计算扩展父数组 (expanded parent array) 的算法，以及扩展连通图的图示

6.5.1 扩展父数组 (Expanding the Parent Array)

运动学树的父数组 (parent array) 包含 N_B 个元素，但因子分解算法需要一个具有 n 个元素的数组。若 $n = N_B$ ，则无需任何操作；否则，必须构造一个扩展父数组供分解过程使用。

表 6.4 给出了完成这一工作的算法，以及该算法所作所为的图示。本质上，我们从原始连通图出发，将每个多自由度关节替换为一条单自由度关节链。这样就得到了一个扩展图 (expanded graph)，其中每条弧代表一个关节变量。随后对该新图重新编号，使弧 i 代表 $\bar{q}$ 中的第 i 个变量；扩展父数组即由此图计算得到。图示给出了某个运动学树的原始连通图与扩展连通图，在该树中，关节 2 具有三个自由度，而其余关节都只有一个自由度。因此，弧 2 被替换为一条由三条弧组成的链，并重新编号新树，使每条弧的编号等于其变量在 $\bar{q}$ 中的索引。由于关节 2 的变量占据 $\bar{q}$ 中的第二、三、四位，新弧必须编号为 2、3 和 4。原始图的父数组为 (0,1,1,2,2,3)；扩展图的父数组为 (0,1,2,3,1,4,4,5)。

表 6.4 中的算法直接由原始父数组计算出扩展父数组 λ' 。第一步是将 λ' 初始化为数组 (0,1,2,...,n-1)。这是一个巧妙的技巧，可以正确初始化 λ' 中的 $n - N_B$ 个元素，只剩下 N_B 个元素需要由 λ 计算。第二个循环构造一个映射数组，使得 $map(i) = \sum_{j=1}^i n_j$ ，其中 n_j 是关节 j 的自由度。(关节 i 的变量出现在 $\bar{q}$ 中从 $map(i-1)+1$ 到 $map(i)$ (含) 的位置上。) 第三个循环将两次映射合而为一：把 $\lambda(i)$ 中存储的值转换为 $map(\lambda(i))$ ，这是应插入 λ' 的正确值；并将该值插入 λ' 中 $map(i-1)+1$ 的位置，这也是插入它的正确位置。对于表中所示例子，映射数组为 (0,1,4,5,6,7,8) (从零开始索引)； λ' 中

校对标注：下方修正文本原以公式形式给出 (LaTeX 已损坏)，现以文本形式保留其内容。

- $y = Lx$ 算法: for $i = 1$ to n : $y_i = L_{ii} \cdot x_i$; $j = \lambda(i)$; while $j \neq 0$: $y_i = y_i + L_{ij} \cdot x_j$; $j = \lambda(j)$ 。
- $y = L^T x$ 算法: $y_j = y_j + L_{ji} \cdot x_i$; $j = \lambda(j)$ 。
- $y = Hx$ 算法: $y_i = y_i + H_{ij} \cdot x_j$; $j = \lambda(j)$; $y_j = y_j + H_{ji} \cdot x_i$ 。

在第一个循环中被正确初始化的两个元素是 $\lambda'(3)$ 和 $\lambda'(4)$ ；元素 $\lambda(1) \cdots \lambda(3)$ 未经改动地映射到 $\lambda'(1)$ 、 $\lambda'(2)$ 和 $\lambda'(5)$ ；而 $\lambda(4) \cdots \lambda(6)$ 各加 2 后映射到 $\lambda'(6) \cdots \lambda'(8)$ 。

一旦计算出扩展父数组，就应将其存储在系统模型中，供因子分解算法日后使用。

6.5.2 回代 (Back-Substitution)

H 中的稀疏模式同样出现在 L 中，因此我们可以用与 H 相同的方法来利用 L 中的稀疏性。表 6.5 给出了计算量 Lx 、 $L^T x$ 、 $L^{-1}x$ 和 $L^{-T}x$ 的算法，它们针对一般向量 x ，仅需给定 λ 、 L 和 x 。回代要用到 $L^{-1}x$ 和 $L^{-T}x$ 。表中还给出了计算 Hx 的算法。所有算法都通过让内层循环只对 L 或 H 的非零元素进行迭代来利用稀疏性。这些算法假定 L 是一般的下三角因子，例如由 $L^T L$ 分解得到的因子。若要将其修改为适用于单位下三角因子，只需将 L_{ii} 替换为 1 即可。

计算 Lx 、 $L^T x$ 和 Hx 的算法将结果放入单独的向量 y 中，并保持 x 不变。 Lx 算法允许 y 与 x 是同一个向量，此时它就地工作；但另外两个算法要求 $y \neq x$ 。计算 $L^{-1}x$ 和 $L^{-T}x$ 的算法均就地工作。

表 6.6: 稀疏算法的计算成本

要实现 $y = L^{-1}x$ 或 $y = L^{-T}x$ ，只需将 x 复制到 y ，然后对 y 应用相应算法即可。

6.5.3 计算成本 (Computational Cost)

表 6.6 给出了各算法的计算成本数据。符号 m 、 a 、 d 和 $\sqrt{}$ 分别表示一次浮点乘法、加法、除法和平方根运算的成本。量 D_1 和 D_2 取决于 H 中的稀疏模式，其定义为

$$D_1 = \sum_{k=1}^n (d_k - 1)$$

以及

$$D_2 = \sum_{k=1}^n \frac{d_k(d_k - 1)}{2},$$

其中 d_k 是刚体 k 与基座之间路径上的关节数目 (即 $d_k = |\kappa(k)|$)，可视为刚体 k 在树中的深度。表达式 $d_k - 1$ 和 $d_k(d_k - 1)/2$ 分别统计处理第 k 行时执行区域 2 和区域 3 计算的次数；因此 D_1 和 D_2 分别统计区域 2 和区域 3 计算的总次数。 D_1 同时也是主对角线以下非零元素的总数，因此 H 中非零元素的数目为 $n + 2D_1$ 。若不存在分支诱导稀疏性，则 D_1 和 D_2 取到其最大可能值，即

$$D_1 = \frac{n^2 - n}{2} \quad \text{且} \quad D_2 = \frac{n^3 - n}{6}.$$

将这些值代入表 6.6 中的公式，得到的成本数据与标准 Cholesky 和 LDL^T 因子分解算法完全相同。若记 d 为树的深度，定义为最深刚体的深度，则 D_1 和 D_2 受限于

$$D_1 \leq n(d-1) \quad \text{且} \quad D_2 \leq \frac{nd(d-1)}{2}.$$

这些界限表明，分解过程最坏情况下为 $O(nd^2)$ 。

§ 6.6

附加说明 (Additional Notes)

动力学算法最早的实例之一归功于 Uicker (1965, 1967)。它将 Denavit 与 Hartenberg (1955) 提出的 4×4 矩阵运动学表示与一个 4×4 伪惯量矩阵 (pseudoinertia matrix) 相结合，并运用拉格朗日 (Lagrangian) 方法求得了闭链机构 (closed-loop mechanism) 的运动方程。大约与此同时，Hooker 与 Margulies (1965) 利用向量式 (牛顿—欧拉) 动力学和增广刚体 (augmented body) 求得了代表卫星的自由漂浮运动学树的运动方程。到 20 世纪 70 年代中期，已有多套计算机程序可用来计算各类刚体系统 (rigid-body system) 的正向动力学；Paul (1975) 对这批早期工作作了出色的综述。其中一个被证明特别成功的早期思想，是程序 ADAMS (Orlande et al., 1977) 中所用的稀疏矩阵公式 (sparse-matrix formulation)。这一时期出现了第一本计算刚体动力学的教科书 (Wittenburg, 1977)，它详细阐述了构建通用刚体系统面向计算机的模型并计算其动力学的方法。

复合刚体算法首次以 Walker 与 Orin (1982) 中的方法 3 出现。它为计算运动学树的关节空间惯量矩阵 (joint-space inertia matrix) 提供了一种新方法，比此前的各种方法快得多。随后，该算法更高效的版本以及对这一基本思想的一些变体出现在 Featherstone (1984, 1987)；Balfoutis 与 Patel (1989, 1991)；Lilly 与 Orin (1991)；Lilly (1993)；McMillan 与 Orin (1998) 等文献中。不过，分支诱导稀疏性这一现象直到 Featherstone (2005) 才得到认识。

正向动力学传播方法 (propagation method) 的第一个实例出现在 Vereshchagin (1974) 中。在该文中，动力学利用高斯最小约束原理 (Gauss' principle of least constraint) 来表述，所得的极小化问题通过动态规划求解。该文中的算法实际上与铰接刚体算法 (articulated-body algorithm) 几乎相同，但直到铰接刚体算法发表之后，其意义才得到认识。传播算法的下一个实例出现在 Armstrong (1979) 中，随后是 Featherstone (1983) 中铰接刚体算法的初版，以及 Featherstone (1984, 1987) 中的完整版。其后，更高效的版本出现在 Brandl et al. (1988)；McMillan 与 Orin (1995)；McMillan et al. (1995) 中。还有一些其他算法被证明实际上与铰接刚体算法相同；例如 Rodriguez 等人的正向动力学算法 (Rodriguez, 1987；Rodriguez et al., 1991) 以及 Rosenthal (1990) 中的 $O(n)$ 算法。

表面上看，惯量矩阵方法与传播方法似乎差别很大。然而，人们已经发现了两条有趣的联系。借鉴卡尔曼滤波 (Kalman filter) 理论，Rodriguez 等人给出了关节空间惯量矩阵的两种分解：一种蕴含递归牛顿—欧拉算法 (recursive Newton-Euler algorithm)，另一种的逆蕴含铰接刚体算法 (Rodriguez, 1987；Rodriguez et al., 1991)。另一条联系是由

Ascher et al. (1997) 发现的。他们从与式 (3.18) 形式相近的一个方程出发, 证明对系数矩阵的某一置换应用高斯消元 (Gaussian elimination) 即可得到关节空间惯量矩阵, 而对这一矩阵的另一置换应用高斯消元则得到铰接刚体算法。

严格来说, 惯量矩阵方法与传播方法只是计算运动学树正向动力学的三种可能策略中的两种。第三种可能性是将各刚体的运动方程连同约束方程 (equations of constraint) 组装成一个大矩阵方程 (例如第 3 章所示)。这种策略对闭链系统非常有用, 但在应用于运动学树时, 它无法与惯量矩阵方法和传播方法竞争。此类算法的一个实例见 Baraff (1996)。

正向动力学——传播方法

正向动力学问题向我们呈现了两组未知量：关节加速度和关约束力。通常不可能在任何一个物体上局部求解这些未知量；但有可能列出它们必须满足的方程。传播方法的工作原理是在局部计算这些方程的系数，并把它们传播给相邻物体，直到我们最终到达一个可以在某个特定物体上局部求解动力学问题的位置。一旦在该物体上获得解，就能在相邻物体上求解动力学，依此类推，直到整个问题求解完毕。原则上，这个过程类似于先对系数矩阵进行三角化、再通过回代来求解线性方程。事实上，有些传播算法正是通过这种类比提出的（例如见 Ascher et al., 1997; Rosenthal, 1990; Saha, 1997）。传播算法比惯性矩阵算法更复杂，但其计算复杂度为 $O(N_B)$ ，这是求解正向动力学问题的理论最小值。

铰接体算法 (articulated-body algorithm) 是传播算法的主要代表，也是计算运动学树 (kinematic tree) 正向动力学最快的算法。本章大部分内容用于描述该算法及其相关概念，如铰接体惯性 (articulated-body inertia)。最后两节将对这一主题进行更广泛的考察，介绍构成其他算法基础的一些技巧和方程。

§7.1

铰接体惯性 (Articulated-Body Inertia)

铰接体惯性是一个物体在成为刚体系统一部分时所表现出的惯性。考虑图 7.1 的图 (a) 中所示的刚体 B 。施加力 f 与由此产生的

图 7.1: 刚体 (a) 与铰接体 (b) 的比较

加速度 a 之间的关系由该物体的运动方程给出：

$$f = Ia + p.$$

该方程中的系数 I 和 p 分别是物体 B 的刚体惯性和偏置力 (bias force)。现在考虑图 (b) 中所示的两物体系统，其中第二个物体 B' 通过一个关节与 B 相连。这第二个物体的作用是改变 B 的加速度响应，使得 f 与 a 之间的关系现在由下式给出

$$f = I^A a + p^A.$$

这是一个铰接体运动方程。它具有与式 (7.1) 相同的代数形式，但系数不同。该方程中的量 I^A 和 p^A 是物体 B 在由 B 、 B' 及其连接关节组成的两物体系统中的铰接体惯性和偏置力。

像式 (7.2) 这样的方程描述了单个刚体的加速度响应，同时考虑了特定一组其他物体和关节的动力学效应。我们称该物体为一个手柄 (handle)，并称包含它的系统为一个铰接体 (articulated body)。因此，手柄是 I^A 和 p^A 所对应的物体；而铰接体是刚体系统或子系统，其动力学效应被包含在 I^A 和 p^A 之中。在图 7.1(b) 中，手柄是物体 B，铰接体则是整个两物体系统。

7.1.1 基本性质

铰接体惯性与刚体惯性共享以下性质：它们是对称、正定的矩阵；它们是从 $\mathbf{M}^6$ 到 $\mathbf{F}^6$ 的映射；并且它们遵循相同的坐标变换规则。然而，铰接体惯性是从加速度到力的映射，而不是从速度到动量的映射，因此像 $I^A v$ 和 $\frac{1}{2} v^T I^A v$ （其中 v 表示速度）这样的表达式没有意义。铰接体可能只由单个刚体组成，此时铰接体惯性和刚体惯性以及偏置力是相同的。这为递归计算方法提供了一个起点。

图 7.2：铰接体惯性和偏置力的加法

铰接体惯性只取决于各组成物体的惯性和各组成关节所施加的约束。因此，它们是关节位置变量的函数，而不是速度变量或各种力项的函数。在这方面，它们类似于广义惯性矩阵。速度项、外力项（除 f 外）等只出现在偏置力中。在三维空间中，定义一个一般的铰接体惯性需要 21 个参数，而刚体惯性只需要 10 个。在二维空间中，定义一个一般的铰接体惯性需要 6 个参数，而刚体惯性只需要 4 个。注意，21 和 6 分别是对称 6×6 矩阵和 3×3 矩阵中独立值的个数。一般来说，铰接体惯性的视在质量 (apparent mass) 随施加力的方向而变化，并且它没有视在质量中心 (centre of apparent mass)。(见示例 7.1。)

7.1.2 加法

加法定义在铰接体惯性上，这种运算的物理含义如图 7.2 所示。该图显示了两个铰接体 A_1 和 A_2 ，它们将被连接成单个铰接体 A 。连接方式是把物体 B_1 和 B_2 粘合在一起，形成一个新的物体 B 。若符号 I_i^A 、 p_i^A 、 I^A 和 p^A 分别表示 B_i 在 A_i 中以及 B 在 A 中的铰接体惯性和偏置力，则 $I^A = I_1^A + I_2^A$ 且 $p^A = p_1^A + p_2^A$ 。

7.1.3 逆惯性

式 (7.2) 假设手柄具有六个自由度。如果不是这种情况，则必须改用更一般的方程

$$a = \Phi^A f + b^A$$

在该方程中， Φ^A 和 b^A 分别是手柄的铰接体逆惯性 (inverse inertia) 和偏置加速度 (bias acceleration) (参见式 (2.72))。这种形式的方程总是存在的。如果手柄具有六个自由度，则 $\Phi^A = (I^A)^{-1}$ 且 $b^A = -\Phi^A p^A$ 。否则， Φ^A 是奇异的，其秩等于手柄的运动自由度。铰接体逆惯性是对称、半正定的矩阵，并且它们像刚体逆惯性一样进行变换。

7.1.4 多手柄

一个铰接体可以有多个手柄。如果把手柄从 1 到 h 编号，并定义向量 f_i 和 a_i 为与手柄 i 关联的力和加速度，则式 (7.3) 推广为

$$\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_h \end{bmatrix} = \begin{bmatrix} \Phi_{11}^A & \Phi_{12}^A & \cdots & \Phi_{1h}^A \\ \Phi_{21}^A & \Phi_{22}^A & \cdots & \Phi_{2h}^A \\ \vdots & \vdots & \ddots & \vdots \\ \Phi_{h1}^A & \Phi_{h2}^A & \cdots & \Phi_{hh}^A \end{bmatrix} \begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_h \end{bmatrix} + \begin{bmatrix} b_1^A \\ b_2^A \\ \vdots \\ b_h^A \end{bmatrix}$$

在该方程中， Φ_i^A 和 b_i^A 是手柄 i 的铰接体逆惯性和偏置加速度， Φ_{ij}^A 是描述对手柄 j 施加力在手柄 i 处引起的加速度的交叉耦合项（cross-coupling term）。系数矩阵是对称、半正定的，其秩等于这组手柄所拥有的独立运动自由度的总数。如果秩为 $6h$ ，则可以对式 (7.4) 求逆，得到式 (7.2) 的多手柄等价形式。

图 7.3：一个铰接体

示例 7.1 图 7.3 显示了一个铰接体，它由一个带槽的箱体和一个恰好装进槽内的圆柱体组成。圆柱体相对于箱体具有两个自由度：它可以绕 x 轴旋转，也可以沿 y 方向平移。假定两个物体之间的接触

是无摩擦的。两个物体的质心都在原点。箱体质量为 m_1 ，绕其质心的转动惯量为 I_1 （即其转动惯量矩阵为 I_1 乘以单位矩阵）；圆柱体质量为 m_2 ，转动惯量为 I_2 。

让我们把箱体当作手柄，并计算它的铰接体惯性。如果沿着 x 轴或 z 轴对箱体施加一个力，那么两个物体将作为一个整体运动，所产生的加速度等于施加力大小的 $1/(m_1 + m_2)$ 倍；但如果沿着 y 轴施加力，那么只有箱体运动，所产生的加速度等于施加力大小的 $1/m_1$ 倍。同样，如果对箱体施加一个绕 y 轴或 z 轴的纯力偶，那么两个物体将作为一个整体旋转，视在惯性将是 $I_1 + I_2$ ；但绕 x 轴的纯力偶只会使箱体旋转，视在惯性将是 I_1 。因此，完整的铰接体惯性矩阵为

$$I^A = \begin{bmatrix} I_1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & I_1 + I_2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & I_1 + I_2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & m_1 + m_2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & m_1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & m_1 + m_2 \end{bmatrix}$$

注意，视在质量随方向变化：箱体在 x 方向和 z 方向上看起来比在 y 方向上具有更大的质量。

在这个特定示例中，手柄表现出一个位于原点的视在质量中心。任何沿一条穿过该点的直线施加在手柄上的纯力都会引起手柄的纯线加速度。这样的点一般来说并不存在。

§7.2 计算铰接体惯性

计算铰接体惯性有两种基本策略：投影法（projection）和组装法（assembly）。在投影法中，整个铰接体的运动方程被投影到手柄（或多个手柄）的运动空间上。在组装法中，一个铰接体由其组成部件一步步组装而成，沿途计算出一系列铰接体惯性。投影法通常用于计算机器人控制系

统中使用的操作空间惯性 (operational-space inertia) (Khatib, 1987, 1995); 而组装法用于 $O(n)$ 动力学算法。

7.2.1 投影法 (The Projection Method)

铰接体是一个刚体系统。因此, 它的运动方程可以用广义坐标表示为

$$H\ddot{q} + C = \tau,$$

其中 C 计及作用在系统上的除作用在手柄上的力之外的所有力。设 v 、 a 和 f 分别表示手柄的空间速度和空间加速度, 以及作用在其上的空间力。如果 J 是根据 $v = J\dot{q}$ 把 $\dot{q}$ 映射到 v 的雅可比矩阵, 那么它就定义了 f 与 τ 之间以及 a 与 $\ddot{q}$ 之间的下列关系:

$$\tau = J^T f$$

和

$$a = J\ddot{q} + \dot{J}\dot{q}.$$

把这些方程合并起来得到

$$\begin{aligned} a &= JH^{-1}(J^T f - C) + \dot{J}\dot{q} \\ &= \Phi^A f + b^A, \end{aligned}$$

其中

$$\Phi^A = JH^{-1}J^T$$

和

$$b^A = \dot{J}\dot{q} - JH^{-1}C.$$

这些是铰接体逆惯性和偏置加速度的一般方程。它们适用于任何铰接体, 并且与广义坐标的选择无关。如果手柄具有完整的六个运动自由度, 则 Φ^A 可逆, 我们可以定义

$$f = I^A a + p^A,$$

其中

$$I^A = (JH^{-1}J^T)^{-1}$$

和

$$p^A = -I^A b^A.$$

铰接体惯性的许多基本性质都可以从式 (7.12) 推导出来。例如, 已知 H 是对称正定的, 则可推知 I^A 也必然是对称正定的; 而且显然 I^A 不依赖于速度项或力项。从式 (7.9) 推出的 Φ^A 的一些基本性质是:

$$\begin{aligned}\text{rank}(\Phi^A) &= \text{rank}(J), \\ \text{range}(\Phi^A) &= \text{range}(J), \\ \text{null}(\Phi^A) &= \text{null}(J^T).\end{aligned}$$

图 7.4: 由两个较小的铰接体构造一个较大的铰接体

7.2.2 多手柄

投影法很容易推广到具有多个手柄的铰接体。假设有 h 个手柄，并令 a_i 、 f_i 和 J_i 分别为与手柄 i 相关的加速度、力和雅可比矩阵。如果我们把量 a 、 f 和 J 重新定义为

$$a = \begin{bmatrix} a_1 \\ \vdots \\ a_h \end{bmatrix}, f = \begin{bmatrix} f_1 \\ \vdots \\ f_h \end{bmatrix} \text{ and } J = \begin{bmatrix} J_1 \\ \vdots \\ J_h \end{bmatrix},$$

那么式 (7.6) 到 (7.14) 立即适用于多手柄情形。

7.2.3 组合法 (The Assembly Method)

在组合法中，我们把给定的铰接体视为由较小的铰接体组装而成；并根据其组成部件的惯性和偏置力来计算它的惯性和偏置力。因此，组合法适用于递归算法。原则上，组合法是一般的。然而，我们这里只考虑每个铰接体都是一个浮动运动学树 (floating kinematic tree) ——即与固定基座没有连接的运动学树——这种特殊情况。这是铰接体算法所需的情形。这种情况的特殊性质是：每个铰接体有一个手柄；每个手柄都有完整的六个自由度；并且每次组装操作都由一个关节把一个手柄与另一个手柄连接起来。更一般的情形将在第 7.4 节和第 7.5 节中讨论。

图 7.4 显示了一对铰接体 A_1 和 A_2 正在被组装成更大的铰接体 A 。组装操作包括用一个关节连接两个手柄 B_1 和 B_2 。在组装之前， B_1 和 B_2 的铰接体方程为

$$f_1 = I_1^A a_1 + p_1^A$$

和

$$f_2 = I_2^A a_2 + p_2^A.$$

组装完成后， B_1 作为新铰接体的手柄，它的方程为

$$f = I^A a_1 + p^A.$$

目标是利用 I_1^A 、 I_2^A 、 p_1^A 、 p_2^A 以及新关节所施加的运动约束来表达 I^A 和 p^A 。

组装完成后，新关节引入一个新的力 f_J ，它是从 B_1 到 B_2 通过关节传递的总力。因此，力变量之间的关系为

$$\begin{aligned}f_1 &= f - f_J, \\ f_2 &= f_J.\end{aligned}$$

f_J 的值是未知的, 但它施加了运动约束

$$a_2 - a_1 = S\ddot{q} + c,$$

并且它满足

$$S^T f_J = \tau.$$

在这些方程中, $\ddot{q}$ 和 τ 分别是关节的加速度变量和力变量, S 是关节的运动子空间矩阵 (motion subspace matrix), $c = \dot{S}\dot{q}$ (如果关节依赖时间, 则为 $\dot{S}\dot{q} + \dot{\sigma}$)。 $\ddot{q}$ 是未知的, 但 τ 、 S 和 c 是给定的。通过把这些方程与式 (7.16) 合并, 我们可以按如下方式计算 $\ddot{q}$:

$$\begin{aligned}\tau &= S^T f_2 \\ &= S^T (I_2^A a_2 + p_2^A) \\ &= S^T (I_2^A (a_1 + c + S\ddot{q}) + p_2^A),\end{aligned}$$

所以

$$\ddot{q} = (S^T I_2^A S)^{-1} (\tau - S^T (I_2^A (a_1 + c) + p_2^A)).$$

矩阵 $S^T I_2^A S$ 是正定的, 因此可逆。解出 $\ddot{q}$ 之后, 我们现在可以构造一个把 f 与 a_1 联系起来的方程, 如下所示:

$$\begin{aligned}f &= f_1 + f_2 \\ &= I_1^A a_1 + I_2^A a_2 + p_1^A + p_2^A \\ &= I_1^A a_1 + I_2^A (a_1 + c + S\ddot{q}) + p_1^A + p_2^A \\ &= I_1^A a_1 + I_2^A (a_1 + c + S (S^T I_2^A S)^{-1} (\tau - S^T (I_2^A (a_1 + c) + p_2^A))) \\ &\quad + p_1^A + p_2^A.\end{aligned}$$

该方程与式 (7.17) 具有相同的形式, 并为 I^A 和 p^A 给出下列公式:

$$I^A = I_1^A + I_2^A - I_2^A S (S^T I_2^A S)^{-1} S^T I_2^A$$

和

$$p^A = p_1^A + p_2^A + I_2^A c + I_2^A S (S^T I_2^A S)^{-1} (\tau - S^T (I_2^A c + p_2^A)).$$

利用这些公式, 只要铰接体是浮动运动学树, 我们就可以计算任何铰接体中任何手柄的铰接体惯性和偏置力。

7.2.4 组装子树 (Assembling Subtrees)

图 7.4 中的三个铰接体被标记为 A_1 、 A_2 和 A 仅仅是为了标识, 数字 1 和 2 没有特别的意义。在实践中, 我们通常会定义一组铰接体 $A_1 \cdots A_{N_B}$, 使得 A_i 包含以物体 i 为根的子树中的所有物体, 并以物体 i 作为手柄 (例如见图 7.5)。在这种情况下, 最有用的组装公式是那些说明如何根据与物体 i 的子物体相关的惯性和偏置力来计算 I_i^A 和 p_i^A 的公式; 也就是说, 如何根据所有 $j \in \mu(i)$ 的量 I_j^A 和 p_j^A 来计算。不重复代数推导, 这些公式为

$$I_i^A = I_i + \sum_{j \in \mu(i)} I_j^a$$

和

$$p_i^A = p_i + \sum_{j \in \mu(i)} p_j^a,$$

其中 I_i 和 p_i 是物体 i 的刚体惯性和偏置力, 并且

$$I_j^a = I_j^A - I_j^A S_j (S_j^T I_j^A S_j)^{-1} S_j^T I_j^A$$

和

$$p_j^a = p_j^A + I_j^a c_j + I_j^A S_j (S_j^T I_j^A S_j)^{-1} (\tau_j - S_j^T p_j^A).$$

本质上, 这些方程是通过对物体 i 的每个子物体应用一次式 (7.19) 和 (7.20) 得到的。每次应用时, A_1 由物体 i 加上每个已经处理过的子子树组成, A_2 是下一个要添加的子子树。

量 I_j^a 和 p_j^a 是把 I_j^A 和 p_j^A 穿关节 j 传播的结果。它们可以被看作是一个无质量手柄的视在惯性和偏置力, 该无质量手柄通过关节 j 连接到物体 j 。在这种情况下, 式 (7.21) 和 (7.22) 中的公式的作用相当于把所有这些无质量手柄粘合到物体 i 上 (见图 7.2)。 I_j^a 和 p_j^a 还有一个有趣的性质: 它们把穿关节的力表示为父物体加速度的函数, 而 I_j^A 和 p_j^A 则把同一个力表示为子物体加速度的函数:

$$\begin{aligned} f_j &= I_j^A a_j + p_j^A \\ &= I_j^a a_{\lambda(j)} + p_j^a. \end{aligned}$$

图 7.5: 铰接体算法的各个方面: (a) 定义铰接体, (b) 求解关节 1 的加速度

I_j^a 在提高传播计算效率方面起到两个作用。第一, 它允许把式 (7.20) 中涉及 c 的两项合并成一项, 如式 (7.24) 所示。第二, 它具有 $I_j^a S_j = \mathbf{0}$ 的性质。根据 S_j 的值, 这个性质意味着 I_j^a 的一行或多行、一列或多列将为零。例如, 如果 $S_j = [0 \ 0 \ 1 \ 0 \ 0 \ 0]^T$ (即关节 j 是转动关节), 那么 I_j^a 的第 3 行和第 3 列将为零。可以利用这一事实来适度降低计算铰接体惯性的成本。

§7.3

铰接体算法 (The Articulated-Body Algorithm)

假设我们有一个包含 N_B 个刚体的运动学树。我们可以定义一组铰接体 $A_1 \cdots A_{N_B}$, 使得 A_i 包含以物体 i 为根的子树中的所有物体和关节。图 7.5(a) 显示了一个简单的例子。注意, 物体 i (B_i) 在 A_i 中, 但关节 i 不在。我们定义 B_i 为 A_i 的手柄; 并定义 I_i^A 和 p_i^A 为 B_i 在 A_i 中的铰接体惯性和偏置力。

考虑 B_1 的运动, 如图 7.5(b) 所示。如果 f_1 是穿过关节 1 传递的力, 那么 B_1 的运动方程为

$$f_1 = I_1^A a_1 + p_1^A.$$

现在, 关节 1 约束 B_1 的加速度满足

$$a_1 = a_0 + c_1 + S_1 \ddot{q}_1,$$

其中 a_0 是已知的基座加速度, $c_1 = \dot{S}_1 \dot{q}_1$ 。关节还约束传递的力满足

$$S_1^T f_1 = \tau_1.$$

由这三个方程我们得到

$$S_1^T (I_1^A (a_0 + c_1 + S_1 \ddot{q}_1) + p_1^A) = \tau_1,$$

它可以对 $\ddot{q}_1$ 求解, 得到

$$\ddot{q}_1 = (S_1^T I_1^A S_1)^{-1} (\tau_1 - S_1^T I_1^A (a_0 + c_1) - S_1^T p_1^A).$$

考虑我们刚刚完成了什么。仅仅知道 I_1^A 和 p_1^A 就使我们能够直接计算 $\ddot{q}_1$, 而不需要知道任何其他加速度变量的值。这是可能的, 因为式 (7.26) 已经计及了 A_1 中每个物体对 B_1 加速度响应的动力学效应。计算出 $\ddot{q}_1$ 后, 可以把它代入式 (7.27) 以得到 B_1 的空间加速度。这反过来又使我们能够对 B_1 的子物体重复这一过程, 依此类推。在一般情况下, 如果已知物体 $\lambda(i)$ 的加速度, 则物体 i 和关节 i 的加速度可以按如下方式计算:

$$\ddot{q}_i = (S_i^T I_i^A S_i)^{-1} (\tau_i - S_i^T I_i^A (a_{\lambda(i)} + c_i) - S_i^T p_i^A)$$

和

$$a_i = a_{\lambda(i)} + c_i + S_i \ddot{q}_i.$$

因此, 如果已知铰接体惯性和偏置力, 那么可以用这两个方程计算运动学树的正向动力学, 其中 i 从 1 取到 N_B 。

7.3.1 算法

铰接体算法正是按照上面概述的方式, 用 $O(N_B)$ 次算术运算计算运动学树的正向动力学。它对树进行三次遍历, 如下所示: 一次向外遍历 (从根到叶子) 计算速度和偏置项; 一次向内遍历计算铰接体惯性和偏置力; 第二次向外遍历计算加速度。

第 1 遍第一遍的工作是计算速度积加速度 (velocity-product acceleration) c_i 和刚体偏置力 p_i , 供后面的遍历使用。两者都是物体速度 v_i 的函数, 因此也有必要计算这些速度。第一遍的方程为:

$$v_{J_i} = S_i \dot{q}_i,$$

$$c_{J_i} = \dot{S}_i \dot{q}_i,$$

$$v_i = v_{\lambda(i)} + v_{J_i}, (v_0 = 0)$$

$$c_i = c_{J_i} + v_i \times v_{J_i},$$

$$p_i = v_i \times^* I_i v_i - f_i^x.$$

它们类似于递归牛顿-欧拉算法 (recursive Newton-Euler algorithm) 第一遍的方程, 如表 5.1 所示。如果关节 i 显式依赖于时间, 则向量 v_{J_i} 和 c_{J_i} 将分别取值为 $S_i \dot{q}_i + \sigma_i$ 和 $\dot{S}_i \dot{q}_i + \dot{\sigma}_i$ (见第 3.5 节); 但计算细节由 jcalc 处理 (第 4.4 节)。 f_i^x 是作用在物体 i 上的外力 (如果有的话)。

第 2 遍下一步是计算铰接体惯性和偏置力 I_i^A 和 p_i^A 。这用式 (7.21) 到 (7.24) 中的组装公式来完成, 这些公式转载如下。

$$I_i^A = I_i + \sum_{j \in \mu(i)} I_j^A$$

$$p_i^A = p_i + \sum_{j \in \mu(i)} p_j^A,$$

$$I_j^A = I_j^A - I_j^A S_j (S_j^T I_j^A S_j)^{-1} S_j^T I_j^A$$

$$p_j^A = p_j^A + I_j^A c_j + I_j^A S_j (S_j^T I_j^A S_j)^{-1} (\tau_j - S_j^T p_j^A).$$

对从 N_B 到 1 的每个 i 值执行这些计算。这意味着并非对每个 j 都计算 I_j^A 和 p_j^A , 而只对满足 $\lambda(j) \neq 0$ 的那些值计算。如果物体 i 没有子物体, 则 $I_i^A = I_i$ 且 $p_i^A = p_i$ 。

第 3 遍第三遍用式 (7.30) 和 (7.31) 计算加速度, 我们在这里重写为

$$\ddot{q}_i = (S_i^T I_i^A S_i)^{-1} (\tau_i - S_i^T I_i^A (\alpha_{\lambda(i)} + c_i) - S_i^T p_i^A)$$

$$a_i = a_{\lambda(i)} + c_i + S_i \ddot{q}_i.$$

$$(a_0 = -a_g)$$

通过把基座加速度初始化为 $-a_g$ 而不是 0，我们可以用虚构的向上加速度来模拟均匀重力场的效应。这比把重力当作外力来处理更高效。

7.3.2 通用子表达式

式 (7.39) 到 (7.42) 包含许多通用子表达式，可以通过定义下列中间结果来消除它们：

$$U_i = I_i^A S_i,$$

$$D_i = S_i^T U_i,$$

$$u_i = \tau_i - S_i^T p_i^A,$$

$$a_i' = a_{\lambda(i)} + c_i.$$

用这些量来表示，式 (7.39) 到 (7.42) 简化为

$$I_j^a = I_j^A - U_j D_j^{-1} U_j^T,$$

$$p_j^a = p_j^A + I_j^a c_j + U_j D_j^{-1} u_j,$$

$$\ddot{q}_i = D_i^{-1} (u_i - U_i^T a_i'),$$

$$a_i = a_i' + S_i \ddot{q}_i. (a_0 = -a_g)$$

7.3.3 体坐标方程

铰接体算法在体坐标 (body coordinates) 下效果最佳。要把上述方程转换到体坐标，只需定义每个与物体 i 相关的量都用物体 i 坐标表示，并在需要处插入坐标变换。所得方程见表 7.1。

7.3.4 算法细节

铰接体算法的伪代码见表 7.1。它组织为三遍，每遍一个 for 循环。量 $\mathbf{X}_j$ 、 $\mathbf{v}_j$ 和 $\mathbf{c}_j$ 是第一遍中的局部变量； $\mathbf{I}^a$ 和 $\mathbf{p}^a$ 是第二遍中的局部变量； $\mathbf{a}'$ 是第三遍中的局部变量。如果树包含显式依赖于时间的关节，则把时间作为 jcalc 的第四个参数。如果树包含速度变量不是位置变量导数的关节，则把符号 $\dot{q}_i$ 和 $\ddot{q}_i$ 替换为 α_i 和 $\dot{\alpha}_i$ 。如果没有外力，则不需要计算 ${}^i\mathbf{X}_0$ 。伪代码假定，一旦算出一个变换 $\mathbf{X}$ ，变换 $\mathbf{X}^*$ 、 $\mathbf{X}^{-1}$ 和 $(\mathbf{X}^*)^{-1}$ 立即可用。实现这一点的函数在附录 A 中描述。

式 (7.37) 和 (7.38) 已经从使用 $\mu(i)$ 的计算转换成了使用 $\lambda(i)$ 的计算。因此， I_i^A 和 p_i^A 在第一个 for 循环的底部被初始化为 I_i 和 p_i ，计算的其余部分在第二个 for 循环的 if 语句内进行（参见第 5.3 节中式 (5.20) 的实现）。

§7.4 替代组装公式

第 7.3 节描述了铰接体算法的标准版本，它基于第 7.2.2 节中的组装公式。然而，还有其他公式可以使用。本节提出一种系统性推导替代组装公式的方法，这些公式产生了铰接体算法的替代版本。在大多数情况下，替代版本比标准算法更复杂、效率更低。然而，它们可以解决更大一类问题。

在第 7.2.2 节中我们解决了以下问题：给定五个方程

$$f_1 = I_1^A a_1 + p_1^A,$$

$$f_2 = I_2^A a_2 + p_2^A,$$

$$f = f_1 + f_2,$$

$$a_2 = a_1 + c + S \ddot{q}$$

和

$$S^T f_2 = \tau,$$

表 7.1：铰接体方程与算法

求目标方程中的系数 I^A 和 p^A 的表达式

$$f = I^A a_1 + p^A.$$

按照几条简单规则，这五个给定的方程可以组合成下列复合方程：

$$\begin{bmatrix} 1 & -1 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -I_2^A & 0 \\ 0 & 0 & 0 & 1 & -S \\ 0 & 0 & S^T & 0 & 0 \end{bmatrix} \begin{bmatrix} f \\ f_1 \\ f_2 \\ a_2 \\ \ddot{q} \end{bmatrix} = \begin{bmatrix} 0 \\ p_1^A \\ p_2^A \\ c \\ \tau \end{bmatrix} + \begin{bmatrix} 0 \\ I_1^A \\ 0 \\ 1 \\ 0 \end{bmatrix} a_1.$$

构造该方程的规则是：

目标方程中的右侧未知量 (a_1) 成为复合方程中的右侧未知量；

目标方程中的左侧未知量 (f) 成为复合未知向量中的第一项；

维度不是六的未知向量 ($\ddot{q}$) 成为复合未知向量中的最后一项；

维度不是六的给定方程 ($S^T f_2 = \tau$) 占据复合方程的最后一行；以及

其余方程和未知量以任意顺序插入，只要使得 5×5 复合矩阵尽可能接近上三角形式。

构造出这个方程后，原来的问题可以用高斯消元 (Gaussian elimination) 加回代 (back-substitution) 的过程求解。一旦系数矩阵被化为上三角形式，我们立即就有一个把 $\ddot{q}$ 表示为 a_1

的函数的方程；而回代过程最终会产生一个把 f 表示为 a_1 的函数的方程。此时， a_1 的系数就是所要求的 I^A 的表达式，其余各项提供所要求的 p^A 的表达式。

这个过程在表 7.2 中“标准公式”（Standard Formulae）标题下加以说明。在高斯消元阶段结束时，第五行是

$$S^T I_2^A S \ddot{q} = \tau - S^T p_2^A - S^T I_2^A c - S^T I_2^A a_1,$$

这意味着

$$\ddot{q} = (S^T I_2^A S)^{-1} (\tau - S^T p_2^A - S^T I_2^A c - S^T I_2^A a_1)$$

（参见式 (7.18)）。已经用 a_1 表示出 $\ddot{q}$ 后，回代使我们能够用 a_1 表示 a_2 ，依此类推，直到最终得到用 a_1 表示的 f 表达式，从中可以提取出 I^A 和 p^A 的公式。

标准公式用 I_i^A 、 p_i^A 和 S 求得 $f = I^A a_1 + p^A$

$$\begin{aligned} \begin{bmatrix} 1 & -1 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -I_2^A & 0 \\ 0 & 0 & 0 & 1 & -S \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} f \\ f_1 \\ f_2 \\ a_2 \\ i \end{bmatrix} &= \begin{bmatrix} 0 \\ p_1^A \\ p_2^A \\ c \\ \tau \end{bmatrix} + \begin{bmatrix} 0 \\ I_1^A \\ 0 \\ 1 \\ 0 \end{bmatrix} a_1 \\ \begin{bmatrix} 1 & -1 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -I_2^A & 0 \\ 0 & 0 & 0 & 1 & -S \\ 0 & 0 & 0 & S^T I_2^A & 0 \end{bmatrix} \begin{bmatrix} f \\ f_1 \\ a_2 \\ q \end{bmatrix} &= \begin{bmatrix} 0 \\ p_1^A \\ p_2^A \\ c \\ \tau - S^T p_2^A \end{bmatrix} + \begin{bmatrix} 0 \\ I_1^A \\ 0 \\ 1 \\ 0 \end{bmatrix} a_1 \\ \begin{bmatrix} 1 & -1 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -I_2^A & 0 \\ 0 & 0 & 0 & 1 & -S \\ 0 & 0 & 0 & S^T I_2^A & 0 \end{bmatrix} \begin{bmatrix} f \\ f_1 \\ f_2 \\ \vdots \\ \tilde{q} \end{bmatrix} &= \begin{bmatrix} 0 \\ p_1^A \\ p_2^A \\ c \\ z \end{bmatrix} + \begin{bmatrix} 0 \\ I_1^A \\ 0 \\ 1 \\ -S^T I_2^A \end{bmatrix} a_1 \end{aligned}$$

其中 $z = \tau - S^T p_2^A - S^T I_2^A c$

结果：

$$\begin{aligned} \ddot{q} &= (S^T I_2^A S)^{-1} (z - S^T I_2^A a_1) \\ I^A &= I_1^A + I_2^A - I_2^A S (S^T I_2^A S)^{-1} S^T I_2^A \\ p^A &= p_1^A + p_2^A + I_2^A c + I_2^A S (S^T I_2^A S)^{-1} z \end{aligned}$$

变体公式 1 用 Φ_i^A 、 b_i^A 和 T 求得 $a_1 = \Phi^A f + b^A$

（续下页）

$$\begin{bmatrix} 1 & 0 & -\Phi_1^A & 0 & 0 \\ 0 & 1 & 0 & -\Phi_2^A & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & -T \\ 0 & 0 & 0 & T^T(\Phi_1^A + \Phi_2^A) & 0 \end{bmatrix} \begin{bmatrix} a_1 \\ a_1 \\ a_2 \\ f_1 \\ f_2 \\ \lambda \end{bmatrix} = \begin{bmatrix} b_1^A \\ b_2^A \\ 0 \\ T_a \tau \\ T^T(c + b_1^A - b_2^A) \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ T^T \Phi_1^A \end{bmatrix} f$$

$$\begin{bmatrix} 1 & 0 & -\Phi_1^A & 0 & 0 \\ 0 & 1 & 0 & -\Phi_2^A & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & -T \\ 0 & 0 & 0 & 0 & T^T(\Phi_1^A + \Phi_2^A)T \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ f_1 \\ f_2 \\ \lambda \end{bmatrix} = \begin{bmatrix} b_1^A \\ b_2^A \\ 0 \\ T_a \tau \\ z \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ T^T \Phi_1^A \end{bmatrix} f$$

其中 $z = T^T(c + b_1^A - b_2^A - (\Phi_1^A + \Phi_2^A)T_a \tau)$

结果:

$$\begin{aligned} \lambda &= (T^T(\Phi_1^A + \Phi_2^A)T)^{-1}(z + T^T \Phi_1^A f) \\ \Phi^A &= \Phi_1^A - \Phi_1^A T (T^T(\Phi_1^A + \Phi_2^A)T)^{-1} T^T \Phi_1^A \\ b^A &= b_1^A - \Phi_1^A T_a \tau - \Phi_1^A T (T^T(\Phi_1^A + \Phi_2^A)T)^{-1} z \end{aligned}$$

Variant Formulae 2 用 I_i^A 、 p_i^A 和 $\mathcal{T}$ 求得 $f = I^A a_1 + p^A$

$$\begin{bmatrix} 1 & -1 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -I_2^A & 0 \\ 0 & 0 & 1 & 0 & -T \\ 0 & 0 & 0 & T^T & 0 \end{bmatrix} \begin{bmatrix} f \\ f_1 \\ f_2 \\ f_2 \\ a_2 \\ \lambda \end{bmatrix} = \begin{bmatrix} 0 \\ p_1^A \\ p_2^A \\ T_a \tau \\ T^T c \end{bmatrix} + \begin{bmatrix} 0 \\ I_1^A \\ 0 \\ 0 \\ T^T \end{bmatrix} a_1$$

$$\begin{bmatrix} 1 & -1 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -I_2^A & 0 \\ 0 & 0 & 0 & T_2^T & 0 \end{bmatrix} \begin{bmatrix} f \\ f_1 \\ f_1 \\ f_2 \\ a_2 \\ \lambda \end{bmatrix} = \begin{bmatrix} 0 \\ p_1^A \\ p_2^A \\ T_a \tau - p_2^A \\ T^T c \end{bmatrix} + \begin{bmatrix} 0 \\ I_1^A \\ 0 \\ 0 \\ T^T \end{bmatrix} a_1$$

$$\begin{bmatrix} 1 & -1 & -1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -I_2^A & 0 \\ 0 & 0 & 0 & I_2^A & -T \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} f \\ f_1 \\ f_2 \\ a_2 \\ \lambda \end{bmatrix} = \begin{bmatrix} 0 \\ p_1^A \\ p_2^A \\ T_a \tau - p_2^A \\ z \end{bmatrix} + \begin{bmatrix} 0 \\ I_1^A \\ 0 \\ 0 \\ T^T \end{bmatrix} a_1$$

其中 $z = T^T(c - (I_2^A)^{-1}(T_a \tau - p_2^A))$

结果:

$$\begin{aligned}\lambda &= (T^T(I_2^A)^{-1}T)^{-1}(z + T^T a_1) \\ I^A &= I_1^A + T(T^T(I_2^A)^{-1}T)^{-1}T^T \\ p^A &= p_1^A + T_a \tau + T(T^T(I_2^A)^{-1}T)^{-1}z\end{aligned}$$

做这个练习的意义在于，它使求解过程系统化，从而可以应用于一组不同的给定方程。我们可以尝试两种明显的改变：

用它们的逆形式替换式 (7.51)、式 (7.52) 和式 (7.56)，即 $a_1 = \Phi_1^A f_1 + b_1^A$ 、 $a_2 = \Phi_2^A f_2 + b_2^A$ 和 $a_1 = \Phi^A f + b^A$ ；以及

用等价的隐式关节约束方程替换式 (7.54) 和式 (7.55)，即 $T^T a_2 = T^T(a_1 + c)$ 和 $f_2 = T_a \tau + T\lambda$ 。

表 7.2 显示了这种方法可以产生的两个替代公式：变体 1 是同时做两种改变的结果；变体 2 是只做第二种改变的结果。每一种都可以作为铰接体算法替代表述的基础。¹

注意，这个过程会产生一些可能有用的矩阵恒等式。例如， I^A 的变体公式必须等于标准公式，而且两者都必须等于 Φ^A 公式的逆。类似的评述也适用于 p^A 和 b^A 。

任何使用惯性而非逆惯性的公式都受惯性必须存在这一要求的限制。因此，标准公式和变体 2 的公式仅当两个手柄都有完整的六个自由度时才适用。相比之下，变体 1 的公式只要求矩阵 $T^T(\Phi_1^A + \Phi_2^A)T$ 满秩。从物理上讲，这等价于要求关节不施加冗余约束；也就是说，它不重复已经存在的约束。应用变体 1 的一个充分条件是两个手柄中至少有一个具有完整的六个自由度。因此，基于变体 1 的算法将能够计算任何运动学树以及少数闭环系统中物体的铰接体逆惯性，而标准算法仅限于浮动运动学树。

§7.5 多手柄

到目前为止，我们只把组合法应用于每个只有一个手柄的铰接体。这对于组装运动学树已经足够了。然而，如果我们希望组装一般的刚体系统，或者希望在组装顺序上有更大的灵活性，那么我们必须能够组装具有多个手柄的铰接体。作为能达到什么效果的例子，Featherstone (1999a,b) 描述了一种分治 (divide-and-conquer) 算法，用来组装运动学树和闭环系统，其方式允许在一台具有 $O(N_B)$ 个处理器的并行计算机上以 $O(\log(N_B))$ 的时间计算它们的动力学。因此，让我们现在为具有多个手柄的铰接体推导一个组装公式。

图 7.6：具有多个手柄的铰接体的组装

图 7.6 显示了两个铰接体，称为 A_{12} 和 A_{34} ，它们将被组装成更大的铰接体 A 。物体 A_{12} 包含两个手柄，编号为 1 和 2，而 A_{34} 包含手柄 3 和 4。想法是通过在手柄 2 和 3 之间连接一个关节来组装这些物体。组装操作被认为会消耗这两个手柄，使铰接体 A 只剩下手柄 1 和 4。我们将假定手柄 2 和 3 中至少有一个具有六个自由度。组装操作之前 A_{12} 和 A_{34} 的运动方程为

$$\begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} \Phi_1 & \Phi_{12} \\ \Phi_{21} & \Phi_2 \end{bmatrix} \begin{bmatrix} f_1 \\ f_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

和

$$\begin{bmatrix} a_3 \\ a_4 \end{bmatrix} = \begin{bmatrix} \Phi_3 & \Phi_{34} \\ \Phi_{43} & \Phi_4 \end{bmatrix} \begin{bmatrix} f_3 \\ f_4 \end{bmatrix} + \begin{bmatrix} b_3 \\ b_4 \end{bmatrix};$$

而组装操作之后 A 的运动方程为

$$\begin{bmatrix} a_1 \\ a_4 \end{bmatrix} = \begin{bmatrix} \Phi_1^A & \Phi_{14}^A \\ \Phi_{41}^A & \Phi_4^A \end{bmatrix} \begin{bmatrix} f_1 \\ f_4 \end{bmatrix} + \begin{bmatrix} b_1^A \\ b_4^A \end{bmatrix}.$$

(注意, 我们通过省略式 (7.57) 和 (7.58) 系数的上标简化了记号。) 这里我们使用逆惯性, 因为一个铰接体内的两个手柄之间有可能——而且很可能——并没有完整的 12 个运动自由度, 即使它们各自有 6 个自由度。在缺乏完整运动自由度的情况下, 这些方程中的系数矩阵将是奇异的。然而, 我们的假设——手柄 2 和 3 中至少有一个自身具有六个自由度——意味着矩阵 Φ_2 和 Φ_3 中至少有一个是正定的。这反过来又意味着和 $\Phi_2 + \Phi_3$ 是正定的。

组装完成后, 关节将对手柄 2 和 3 施加下列运动约束:

$$T^T(a_3 - a_2) = T^T c,$$

其中 $c = \dot{S}q$, 并且它还将施加力约束

$$f_3 = T\lambda = -f_2.$$

在表述式 (7.61) 时, 我们假定关节处的任何主动力都已经计入式 (7.57) 和 (7.58) 的偏置加速度中。换句话说, 主动关节力 $T_a\tau$ 已经被一对大小相等、方向相反的外力所代替——作用于手柄 3 的 $T_a\tau$ 和作用于手柄 2 的 $-T_a\tau$ ——并且这两个力对 A_{12} 和 A_{34} 运动的影响已经包含在向量 $b_1 \cdots b_4$ 中。

目标是用式 (7.57)、式 (7.58)、式 (7.60) 和式 (7.61) 的系数来表示式 (7.59) 的系数。求解过程的第一步是代入式 (7.60) 中的 a_3 和 a_2 , 得到

$$T^T(\Phi_3 f_3 + \Phi_{34} f_4 + b_3 - \Phi_{21} f_1 - \Phi_2 f_2 - b_2) = T^T c.$$

下一步是使用式 (7.61) 代入该方程中的 f_3 和 f_2 , 并按 λ 合并同类项, 得到

$$T^T(\Phi_2 + \Phi_3)T\lambda = T^T(\Phi_{21} f_1 - \Phi_{34} f_4 + c - b_3 + b_2).$$

鉴于 $\Phi_2 + \Phi_3$ 是正定的, λ 的系数保证是非奇异的, 因此我们可以按如下方式求解 λ :

$$\lambda = Y^{-1}T^T(\Phi_{21} f_1 - \Phi_{34} f_4 + \beta),$$

其中

$$Y = T^T(\Phi_2 + \Phi_3)T$$

和

$$\beta = c - b_3 + b_2.$$

最后一步是同时使用式 (7.61) 和式 (7.62)，代入 a_1 和 a_4 表达式（取自式 (7.57) 和式 (7.58)）中的 f_2 和 f_3 。这给出

$$\begin{aligned} a_1 &= \Phi_1 f_1 - \Phi_{12} T Y^{-1} T^T (\Phi_{21} f_1 - \Phi_{34} f_4 + \beta) + b_1 \\ a_4 &= \Phi_4 f_4 + \Phi_{43} T Y^{-1} T^T (\Phi_{21} f_1 - \Phi_{34} f_4 + \beta) + b_4. \end{aligned}$$

这两个方程把 a_1 和 a_4 表示为仅关于 f_1 和 f_4 的函数，因此是式 (7.59) 的组成部分。从而，式 (7.59) 系数的公式为

$$\begin{aligned} \Phi_1^A &= \Phi_1 - \Phi_{12} W \Phi_{21} & b_1^A &= b_1 - \Phi_{12} W \beta \\ \Phi_4^A &= \Phi_4 - \Phi_{43} W \Phi_{34} & b_4^A &= b_4 + \Phi_{43} W \beta \\ \Phi_{14}^A &= \Phi_{12} W \Phi_{34} = (\Phi_{41}^A)^T & b_4^A &= b_4 + \Phi_{43} W \beta \end{aligned}$$

其中 $W = T Y^{-1} T^T$ 。

7.5.1 推广

要把手柄数量推广，我们用四个手柄集合替换这四根单独的手柄。集合 1 包含 A_{12} 中所有在组装过程中不会连接到关节的手柄；集合 2 包含所有会连接到关节的手柄；依此类推。集合 2 中的每个手柄将通过一个关节连接到集合 3 中的一个手柄；因此集合 2 和集合 3 必须包含相同数量的手柄。为了允许同一物体上连接多个关节，我们允许一个物体有多个手柄。在这种背景下，手柄不应再被视为一个物体，而应被视为物体上的一个位置。任何运动学树都可以通过一系列组装操作得到，每次组装涉及一个关节（即集合 2 和集合 3 各包含一个手柄）。然而，闭环系统的构造需要多关节组装。

定义好这些集合后，下一步是定义 a_1 、 f_2 、 Φ_{34} 等为包含其集合中每个相关向量和矩阵的复合向量和复合矩阵。例如，如果集合 1 包含手柄 $1a$ 、 $1b$ 、 $1c$ 、……，其加速度为 a_{1a} 、 a_{1b} 、……，那么 a_1 就是复合向量 $[a_{1a}^T a_{1b}^T \cdots]^T$ 。注意， T 将是一个由各个关节的约束力子空间矩阵组成的块对角矩阵。

给定这些复合向量和复合矩阵，式 (7.63) 的推导过程与式 (7.57) 到 (7.62) 所示的方式相同，只是可能不再能假定 Y 可逆。Featherstone (1999b) 解释了 Y 奇异时该怎么办。它还解释了如何把约束稳定项（constraint stabilization，见第 8.3 节）纳入组装公式。

闭环系统

本章介绍含运动学回路（kinematic loops）的刚体系统的正动力学与逆动力学。运动学回路的存在给动力学问题带来了新的复杂度：需要新的公式化方法；出现了新的问题；系统表现出新的行为；尤其是逆动力学问题，在存在运动学回路时其性质发生了变化。构造闭环系统运动方程有两种主要策略，即：

从一组无约束的物体出发，同时施加所有关节约束；或者

以系统的一棵生成树（spanning tree）为出发点，施加回路闭合约束。

第三种可能性（此处不予考虑）是使用第 7.5 节所述的组装方法。方法 1 产生大型稀疏矩阵方程，将在第 8.13 节简要讨论。方法 2 是典型闭环系统的最佳选择，也是本章的主要内容。闭环系统的一些特殊性质将在第 8.10 节讨论；回路闭合函数的使用在第 8.11 节介绍；逆动力学在第 8.12 节介绍。

§ 8.1 运动方程

一般刚体系统（包括闭环系统）的运动方程已在第 3 章中引入。在本节中，我们从闭环系统的生成树的运动方程出发，推导其运动方程。这一过程可概括如下。

为闭环系统的生成树写出运动方程。

在此方程中加入表示回路关节对树所施力的项。

写出描述回路关节对树所施加运动约束的运动学方程。

合并这两个方程。

如果生成树有 n 个自由度，而回路关节对树施加了 n^c 个约束，那么这一过程将产生一个含 $n + n^c$ 个未知数的 $n + n^c$ 个方程组成的方程组。

考虑一个含 N_B 个物体、 N_J 个关节和 $N_L = N_J - N_B$ 个运动学回路的刚体系统。我们假设已经选定了生成树，并已据此对物体和关节进行编号。树关节变量数 n 和回路闭合约束数 n^c 由下列公式给出：

$$n = \sum_{i=1}^{N_B} n_i \text{ and } n^c = \sum_{k=N_B+1}^{N_J} n_k^c,$$

其中 n_i 是树关节 i 所允许的自由度数， n_k^c 是回路关节 k 所施加的约束数。注意，我们使用下标变量 i 和 k 分别遍历树关节编号和回路关节编号。更一般地，在与运动学回路相关的场合，我

们采用如下下标变量约定： i 和 j 遍历树关节和/或物体编号（1 到 N_B ）； l 遍历回路编号（1 到 N_L ）； k 标识闭合回路 l 的回路关节。按照我们的标准编号方案， k 与 l 通过 $k = l + N_B$ 相联系，因此 k 的取值范围为 $N_B + 1$ 到 N_f 。此后，凡 k 与 l 同时出现处，都应默认 $k = l + N_B$ 。

生成树的运动方程可以写为

$$H\ddot{q} + C = \tau,$$

其中 $\ddot{q}$ 和 τ 分别是树关节加速度变量和力变量的向量。现在，闭环系统与其生成树的唯一区别在于，前者包含回路关节，而后者不包含。因此，闭环系统的运动方程可以通过在式 8.2 中加入表示回路关节对树所施力的项来获得。于是，闭环系统的运动方程可写为

$$H\ddot{q} + C = \tau + \tau^c + \tau^a,$$

其中 τ^c 和 τ^a 分别表示回路关节所产生的约束力和主动力。主动力（active force）来自作用在回路关节处的弹簧、阻尼器、致动器等等。如果某一特定关节处不存在此类力，则称该关节为被动的（passive）；如果所有回路关节都是被动的，则 $\tau^a = 0$ 。在这种情况下，可以省略计算 τ^a 所需的代码，从而简化动力学计算软件。

τ^a 假定为已知；但 τ^c 是未知的，它可以表示为 n^c 个未知约束力变量的函数（如式 8.5 所示）。因此，式 8.3 是含 $n + n^c$ 个未知数的 n 个方程组成的方程组，求解前需要补充另外 n^c 个方程。这些方程来自运动学约束。

回路关节对树施加一组运动学约束，这些约束可以集中为一个矩阵方程，形式为

$$K\ddot{q} = k,$$

其中 K 是一个 $n^c \times n$ 矩阵。我们说此方程在加速度层面表达约束，理由是它已经被微分了足够的次数，使加速度变量得以出现。 K 和 k 的表达式在第 8.2 节中推导。

由于 τ^c 是树施加这些约束的力，因此 τ^c 可以表示为如下形式

$$\tau^c = K^T \lambda,$$

其中 λ 是 n^c 个未知约束力变量的向量。这一步在第 8.4 节中介绍。（另见式 3.15。）

现在可以将式 8.3、8.4 和 8.5 组装成单个矩阵方程，即闭环系统的完整运动方程：

$$\begin{bmatrix} H & K^T \\ K & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ -\lambda \end{bmatrix} = \begin{bmatrix} \tau - C + \tau^a \\ k \end{bmatrix}.$$

这是含 $n + n^c$ 个未知数的 $n + n^c$ 个方程组成的方程组。系数矩阵是对称的，但不是正定的。如果该矩阵满秩，则可同时求解出 $\ddot{q}$ 和 λ 。如果它不满秩，则 λ 的某些元素将无法确定，但方程仍可求出 $\ddot{q}$ 。系数矩阵的秩为 $n + \text{rank}(K)$ 。

§ 8.2 回路约束方程

现在让我们来构造回路约束方程（式 8.4）的系数。我们从跨过回路关节 k 的速度 v_{jk} 开始。该速度由方程给出

$$v_{jk} = v_{s(k)} - v_{p(k)},$$

其中 $p(k)$ 和 $s(k)$ 分别是关节 k 的前驱物体（predecessor）和后继物体（successor）。一般情况下，关节 k 将施加如下形式的约束

$$T_k^T v_{jk} = T_k^T \sigma_k,$$

其中 T_k 和 σ_k 分别表示关节 k 的约束力子空间和偏置速度（bias velocity）。（见式 3.38。）然而，为简单起见，我们假设所有回路关节都有 $\sigma_k = 0$ ，从而约束方程简化为

$$T_k^T v_{jk} = 0.$$

将式 8.7 与 8.8 结合，得到速度约束方程

$$T_k^T (v_{s(k)} - v_{p(k)}) = 0,$$

该式可以微分得到加速度约束方程：

$$T_k^T (a_{s(k)} - a_{p(k)}) + \dot{T}_k^T (v_{s(k)} - v_{p(k)}) = 0.$$

运动学树中任意物体的速度都可以用如下形式的方程以 $\dot{q}$ 表示

$$v_i = J_i \dot{q},$$

其中 v_i 是物体 i 的速度， J_i 是物体 i 的雅可比矩阵（Jacobian）。（见例 4.2。） J_i 由下式给出

$$J_i = [\epsilon_{i1} S_1 \epsilon_{i2} S_2 \cdots \epsilon_{iN_B} S_{N_B}],$$

其中 ϵ_{ij} 在每一个支撑物体 i 的关节 j 处取值为 1（即该关节位于物体与基座之间的路径上），否则取 0。用方程表示，即

$$\epsilon_{ij} = \begin{cases} 1 & \text{if } j \in \kappa(i) \\ 0 & \text{otherwise.} \end{cases}$$

同样地，运动学树中任意物体的加速度可以用式 8.11 的导数以 $\ddot{q}$ 表示：

$$\begin{aligned} a_i &= J_i \ddot{q} + \dot{J}_i \dot{q} \\ &= J_i \ddot{q} + a_i^{vp}. \end{aligned}$$

a_i^{vp} 是物体 i 的“速度积”加速度（velocity-product acceleration），即当所有树关节加速度变量均为零时物体将具有的加速度。它可作为计算 C 的副产品之一获得。（见第 8.6 节。）

将式 8.10 与 8.13 结合, 得到

$$T_k^T (J_{s(k)} - J_{p(k)}) \ddot{q} + T_k^T (a_{s(k)}^{vp} - a_{p(k)}^{vp}) + \dot{T}_k^T (v_{s(k)} - v_{p(k)}) = \mathbf{0},$$

可将其简化为

$$T_k^T (J_{s(k)} - J_{p(k)}) \ddot{q} = k_l$$

其中定义

$$k_l = -T_k^T (a_{s(k)}^{vp} - a_{p(k)}^{vp}) - \dot{T}_k^T (v_{s(k)} - v_{p(k)}).$$

式 8.14 是回路关节 k 对生成树施加的加速度约束, 每个运动学回路有一个这样的方程。把这些方程集中起来, 得到

$$\begin{bmatrix} T_{N_B+1}^T (J_{s(N_B+1)} - J_{p(N_B+1)}) \\ \vdots \\ T_{N_I}^T (J_{s(N_I)} - J_{p(N_I)}) \end{bmatrix} \ddot{q} = \begin{bmatrix} k_1 \\ \vdots \\ k_{N_I} \end{bmatrix}.$$

将此式与式 8.4 比较, 可以看出

$$K = \begin{bmatrix} T_{N_B+1}^T (J_{s(N_B+1)} - J_{p(N_B+1)}) \\ \vdots \\ T_{N_I}^T (J_{s(N_I)} - J_{p(N_I)}) \end{bmatrix} \text{ and } k = \begin{bmatrix} k_1 \\ \vdots \\ k_{N_I} \end{bmatrix}.$$

表达式 $J_{s(k)} - J_{p(k)}$ 定义了一个回路雅可比矩阵 (loop Jacobian) —— 一个将 $\ddot{q}$ 映射到跨过回路闭合关节速度的矩阵。回路 l 的回路雅可比矩阵 J_{Ll} 可定义如下:

$$\begin{aligned} J_{Ll} &= J_{s(k)} - J_{p(k)} \\ &= [\varepsilon_{l1} S_1 \varepsilon_{l2} S_2 \cdots \varepsilon_{lN_B} S_{N_B}] \end{aligned}$$

其中 $\varepsilon_{lj} = \mathbf{e}_{s(k)j} - \mathbf{e}_{p(k)j}$ 。这些数标识参与每个回路的树关节。如果我们将回路 l 的根定义为 $p(k)$ 与 $s(k)$ 共同祖先中编号最高的物体, 那么参与回路 l 的树关节集合是两个子集的并集: 连接根到 $p(k)$ 的关节, 以及连接根到 $s(k)$ 的关节。 ε_{lj} 在第一个子集中的每个关节处取 -1 , 在第二个子集中的每个关节处取 $+1$, 在所有其他关节处取 0 。利用式 8.18, 我们可以构造出 K 的另一种表达式, 即用于计算它的表达式:

$$K = \begin{bmatrix} K_{11} & \cdots & K_{1N_B} \\ \vdots & & \vdots \\ K_{N_L1} & \cdots & K_{N_LN_B} \end{bmatrix}$$

其中

$$K_{lj} = \varepsilon_{lj} T_k^T S_j.$$

§ 8.3 约束稳定化

式 8.16 在理论上正确，但在数值积分过程中不稳定。问题在于：原则上，式 8.16 表现得像如下形式的微分方程

$$\ddot{e} = 0,$$

其中 e 表示回路闭合位置误差；但在实践中它表现得像

$$\ddot{e} = \text{noise},$$

其中 noise 是一个量值很小的信号，代表数值误差，例如数值积分过程中的截断误差。因此，使用式 8.16 将保证加速度误差保持很小，但没有任何机制阻止位置和速度误差的无界累积。这个问题可以通过在式 8.4（因此也在式 8.6）中增加稳定化项 k_{stab} 来解决，使方程变为

$$K\ddot{q} = k + k_{stab}.$$

k_{stab} 的目的是在数值积分下修正约束方程的行为，使它表现得像如下形式的方程

$$\ddot{e} + 2\alpha\dot{e} + \beta^2 e = \text{noise},$$

其中 $\alpha > 0$ 且 $\beta \neq 0$ 。该方法归功于 Baumgarte，有时称为 Baumgarte 稳定化（Baumgarte, 1972）。稳定化常数通常按下式选取

$$\alpha = \beta = 1/T_{stab},$$

其中 T_{stab} 是位置和速度误差衰减的期望时间常数。为达到此效果， k_{stab} 定义如下：

$$k_{stab} = -2\alpha \begin{bmatrix} T_{N_B+1}^T v_{J_{N_B+1}} \\ \vdots \\ T_{N_I}^T v_{J_{N_I}} \end{bmatrix} - \beta^2 \begin{bmatrix} \delta_1 \\ \vdots \\ \delta_{N_L} \end{bmatrix}$$

其中

$$\delta_l = \text{jcalc}(\text{jtype}(k), {}^{s(k),k}\mathbf{X}_{p(k),k}).$$

δ_l 是生成树位置违背回路关节 k 所施加约束程度的度量。它是形如 $\delta_l = \mathbf{T}_k^T \mathbf{d}_l$ 的量，其中 $\mathbf{d}_l$ 是一个空间向量，测量绕回路 l 的位置误差。该向量仅取决于关节类型以及将关节后继坐标系定位于其前驱坐标系相对位置的变换。后者由下式给出

$$\begin{aligned} s(k), {}^k\mathbf{X}_{p(k),k} &= s(k), \bar{k}\mathbf{X}_{s(k)} s(\bar{k})\mathbf{X}_{p(k)} p(k)\mathbf{X}_{p(k),k} \\ &= \mathbf{X}_S(l) s(k)\mathbf{X}_{p(k)} \mathbf{X}_P(l)^{-1}, \end{aligned}$$

其中 $\mathbf{X}_P$ 和 $\mathbf{X}_S$ 是系统模型中存储的回路关节前驱和后继坐标系变换数组。（见 §4.2。）

Baumgarte 稳定化的一个问题在于, 选择 T_{stab} 没有硬性规定。一个合理的策略是: 询问你需要每秒多少帧运动系统的快照才不会错过任何重要内容, 然后选择一个与快照间隔相近或略短于该间隔的 T_{stab} 值。例如, 如果系统是一台大型工业机器人, 那么 $T_{stab} = 1$ 太大, $T_{stab} = 0.1$ 是合理的, 而 $T_{stab} = 0.01$ 略小, 但仍然合理。如果 T_{stab} 选得太小, 微分方程会变得不必要地刚性 (stiff)。 T_{stab} 的精确值并不关键——将其改变一个因子 2 只会造成很小的差异。

有一种诱惑是把 T_{stab} 选得尽可能小, 使位置和速度误差尽快衰减, 以为这样能最大化仿真的精度。这是一种糟糕的策略。约束稳定化的目的是实现稳定性, 而不是精度。如果仿真精度不够, 改进它的最好方法是使用更好的积分方法和/或更短的积分时间步长。

8.3.1 计算 δ

设 ${}^s\mathbf{X}_p$ 是由生成树的位置变量确定的、从回路关节前驱坐标系到后继坐标系的坐标变换。一般来说, 该变换不会精确地符合回路关节所施加的运动约束。因此, 让我们把这个矩阵因式分解如下:

$${}^s\mathbf{X}_p = \mathbf{X}_{err} \mathbf{X}_J,$$

其中 $\mathbf{X}_J$ 精确符合关节约束, 而 $\mathbf{X}_{err}$ 表示约束误差, 并假设它很小。这种因式分解必须利用关节所允许运动类型的知识进行符号求解, 以得到 $\mathbf{X}_{err}$ 的元素关于 ${}^s\mathbf{X}_p$ 的表达式。这些表达式因关节类型而异。给定 $\mathbf{X}_{err}$ 表示一个小位移, 存在一个满足下式的运动向量 $\mathbf{d} \in \mathbb{M}^6$

$$\mathbf{1} - \mathbf{d} \times \simeq \mathbf{X}_{err}.$$

该向量 (近似) 是后继坐标系本应处的位置到其实际位置之间的位移。获得 $\mathbf{d}$ 之后, δ 即由下式给出

$$\delta = \mathbf{T}^T \mathbf{d}.$$

各种常见关节类型 δ 的公式如表 8.1 所示。注意, 零自由度关节可以充当通用的回路关节, 因为任何闭环系统都可以通过添加无质量物体和零自由度关节进行改造, 使每个回路闭合关节都是零自由度关节。

例 8.1 表 8.1 中圆柱关节的公式可以如下获得。结合式 8.25 和 8.26, 并为圆柱关节选择适当的 $\mathbf{X}_J$ 和 $\mathbf{d}$ 表达式, 我们得到如下近似方程:

表 8.1: $\delta = \text{jcalc}(\text{type}, \mathbf{X})$ 的公式

近似方程:

$${}^s\mathbf{X}_p \simeq \begin{bmatrix} 1 & 0 & -d_2 & 0 & 0 & 0 \\ 0 & 1 & d_1 & 0 & 0 & 0 \\ d_2 & -d_1 & 1 & 0 & 0 & 0 \\ 0 & 0 & -d_5 & 1 & 0 & -d_2 \\ 0 & 0 & d_4 & 0 & 1 & d_1 \\ d_5 & -d_4 & 0 & d_2 & -d_1 & 1 \end{bmatrix} \begin{bmatrix} c & s & 0 & 0 & 0 & 0 \\ -s & c & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & z & 0 & c & s & 0 \\ -z & 0 & 0 & -s & c & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix},$$

其中 $c = \cos(\theta)$, $s = \sin(\theta)$, θ 和 z 是圆柱关节的角位移和线位移（即关节位置变量）。注意，我们置 $d_3 = d_6 = 0$ ，因为它们是 $\mathbf{d}$ 在关节运动自由度方向上的坐标。如果执行乘法并检查结果，我们会发现 $d_1 = ({}^s X_p)_{23}$, $d_2 = -({}^s X_p)_{13}$ ，等等。最后， $\delta = [d_1 d_2 d_4 d_5]^T$ 。

§ 8.4 回路关节力

设 f_k 表示闭环系统中跨过回路关节 k 传递的力。因此，关节 k 对生成树的作用是在物体 $s(k)$ 上施加力 f_k ，在物体 $p(k)$ 上施加力 $-f_k$ 。现在，如果一个空间力 f 施加在运动学树中的物体 i 上，那么它对树的作用等效于如下关节空间力 τ

$$\tau = J_i^T f.$$

因此，关节 k 对生成树的作用等效于关节空间力

$$\tau = (J_{s(k)}^T - J_{p(k)}^T) f_k = J_{Ll}^T f_k.$$

让我们将 f_k 分解为主动力 f_k^a 与约束力 f_k^c 之和，并分别考虑它们的作用。如果 τ^a 表示回路关节处所有主动力对生成树的净作用，则 τ^a 由下式给出

$$\tau^a = \sum_{l=1}^{N_L} J_{Ll}^T f_k^a.$$

同样，如果 τ^c 表示回路关节处所有约束力的净作用，则 τ^c 由下式给出

$$\tau^c = \sum_{l=1}^{N_L} J_{Ll}^T f_k^c.$$

然而，关节 k 处的约束力也可以表示为如下形式

$$f_k^c = T_k \lambda_k,$$

其中 λ_k 是 n_k^c 个未知约束力变量的向量（参照式 3.34），因此 τ^c 也可以表示为

$$\tau^c = \sum_{l=1}^{N_L} J_{Ll}^T T_k \lambda_k.$$

将此式与式 8.17 中 K 的定义比较，可以看出

$$\tau^c = K^T \lambda$$

其中

$$\lambda = [\lambda_{N_B+1}^T \cdots \lambda_{N_f}^T]^T.$$

如前所述，主动力假定为已知。将 τ^a 纳入运动方程的最佳方式是修改计算 $\mathbf{C}$ 的算法，使其改为计算 $\mathbf{C} - \tau^a$ 。第 8.6 节给出了一个合适的算法。

§8.5 求解运动方程

在写出运动方程中的每一项之后，下一步是求解树关节加速度。回顾一下，生成树在回路闭合力作用下的运动方程是

$$H\ddot{q} + C = \tau + K^T\lambda + \tau^a;$$

回路闭约束方程是

$$K\ddot{q} = k + k_{stab};$$

这两个方程可以合并为一个复合方程，

$$\begin{bmatrix} H & K^T \\ K & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ -\lambda \end{bmatrix} = \begin{bmatrix} \tau - C + \tau^a \\ k + k_{stab} \end{bmatrix}.$$

在这些方程中， H 是 $n \times n$ 对称正定矩阵，而 K 是一般 $n^c \times n$ 矩阵，可能秩亏。因此，式 8.36 中的系数矩阵是对称的，其尺寸为 $(n + n^c) \times (n + n^c)$ ；但它不是正定的，其秩为 $n + r$ ，其中 $r = \text{rank}(K)$ ，所以每当 $r < n^c$ 时它就是奇异的。

如果 $r = n^c$ ，则 $\ddot{q}$ 和 λ 都由式 8.36 唯一确定。然而，如果 $r < n^c$ ，则回路关节对生成树施加的约束中有 $n^c - r$ 个约束线性依赖于其余 r 个约束。结果是， λ 的某些元素将无法确定，但 $\ddot{q}$ 仍由方程唯一确定。 $r < n^c$ 时可能出现的另一个问题是，式 8.35 可能略微不一致；即 $k + k_{stab}$ 可能略微落在 $\text{range}(K)$ 之外。这个问题由 K 、 k 和 k_{stab} 计算中的数值误差，以及位置和速度对回路闭约束的不精确满足所引起。当式 8.35 略微不一致时，以最小二乘意义求解是可以接受的。

求解上述方程得到 $\ddot{q}$ 有多种方法。现在让我们考察其中一些。特别地，让我们考虑以下三种方法：

直接求解式 8.36；

在许多方法中求解式 8.00；

先求解式 8.36 中的 λ ，再利用结果求出 $\ddot{q}$ ；或者

先求解式 8.35，将 $\ddot{q}$ 表示为一组独立加速度变量的向量的函数，然后再求解式 8.34。

8.5.1 方法 1：直接求解式 8.36

如果式 8.36 中的系数矩阵非奇异，则此方程可以用通用线性方程求解器，或专为对称不定矩阵设计的求解器直接求解。这是最简单的求解方法，但不是最高效的。因此，凡人工成本比计算效率更重要时，它就适用。当 $r = n^c$ 时，或者当 $n^c - r$ 个线性相关的行在并入式 8.36 之前已从式 8.35 中修剪掉时，系数矩阵将是非奇异的。如果碰巧预先知道要删除哪些行，后者是可行的。这种修剪策略相当于把 n^c 减小到 $n^c = r$ 。修剪也可以与方法 2 和方法 3 结合使用。

8.5.2 方法 2：先求解 λ

在式 8.36 中，从第二行减去第一行的 KH^{-1} 倍，我们得到如下形式的 λ 方程

$$A\lambda = b,$$

其中

$$A = KH^{-1}K^T$$

以及

$$b = k + k_{stab} - KH^{-1}(\tau - C + \tau^a).$$

系数矩阵 A 是对称半正定的。其尺寸为 $n^c \times n^c$ ，秩为 r 。如果 $r = n^c$ ，则 A 可逆，式 8.37 有唯一解

$$\lambda = A^{-1}b.$$

如果 $r < n^c$ ，则 A 奇异，式 8.37 有无穷多个解。在这种情况下，解的一般形式为

$$\lambda = A^+b + (1 - A^+A)z,$$

其中 A^+ 是 A 的伪逆 (pseudoinverse)， z 是任意向量，算子 $1 - A^+A$ 把向量投影到 A 的零空间上。如果式 8.37 是一致的，则式 8.41 描述一族精确解；否则，它描述一族最小二乘解。式 8.37 一致当且仅当式 8.35 一致。

获得 λ 的值之后，最后一步是将其代入式 8.34，并求解所得方程得到 $\ddot{q}$ 。注意， z 对 $\ddot{q}$ 没有影响，因为

$$K^T(1 - A^+A) = 0.$$

因此可以自由地为 z 选择任何方便的值，例如 $z = 0$ 。

下面的过程将以高效的方式计算 $\ddot{q}$ 。它使用第 6.5 节所述的 $L^T L$ 因式分解和回代算法，因此利用了 H 中可能存在的任何由分支引起的稀疏性。它可以很容易地修改为使用 $L^T D L$ 因式分解。

将 H 因式分解为 $H = L^T L$ 。

计算 $\tau' = \tau - C + \tau^a$ 。

利用回代，计算 $Y = L^{-T}K^T$ 和 $z = L^{-T}\tau'$ 。

计算 $A = Y^T Y$ 和 $b = k + k_{stab} - Y^T z$ 。

用任何合适的方法求解 $A\lambda = b$ 得到 λ 。

利用第 1 步的因子求解 $H\ddot{q} = \tau' + K^T\lambda$ 得到 $\ddot{q}$ 。

8.5.3 方法 3：先求解式 8.35

假设给我们一个一般的线性方程 $Ax = b$ ，其中 A 是一个秩为 r 的 $m \times n$ 矩阵，且 $b \in \text{range}(A)$ （即方程是一致的）。任务是求解 x 。如果 $r = n$ ，则只有一个解，由公式 $x = A^+b$ 给出。如果 $r < n$ ，则有无穷多个解，由公式 $x = Cy + d$ 给出。在此方程中， y 是一个 $(n - r)$ 维未知向量， d 是任一满足 $Ad = b$ 的向量， C 是满足 $AC = 0$ 性质的满秩矩阵。每个解都有唯一的对应 y 值，反之亦然（即两者之间存在 1:1 对应）。

让我们把这个公式应用到式 8.35。该方程的系数矩阵是 $n^c \times n$ 的，秩为 r 。如果 $r = n$ ，则方程有唯一解，即 $\ddot{\mathbf{q}} = \mathbf{K}^+(k + k_{stab})$ ；但如果 $r < n$ ，则有无穷多个解，全部由公式给出

$$\ddot{\mathbf{q}} = \mathbf{G}\ddot{\mathbf{y}} + \mathbf{g}.$$

在此方程中， $\mathbf{G}$ 是满足下述性质的 $n \times (n - r)$ 满秩矩阵

$$\mathbf{K}\mathbf{G} = \mathbf{0},$$

$\ddot{\mathbf{y}}$ 是 $(n - r)$ 维独立加速度变量向量， $\mathbf{g}$ 是任一满足下式的向量

$$\mathbf{K}\mathbf{g} = k + k_{stab}.$$

通常， $\ddot{\mathbf{y}}$ 被选为 $\ddot{\mathbf{q}}$ 中元素的子集。因此，它可以被视为闭环系统的独立关节加速度变量向量。计算 $\mathbf{G}$ 和 $\mathbf{g}$ 的算法在第 8.8 节中给出。

现在我们有了一个将 $\ddot{\mathbf{q}}$ 表示为未知向量 $\ddot{\mathbf{y}}$ 的函数的方程。下一步是求 $\ddot{\mathbf{y}}$ 的表达式。将式 8.34 左乘 $\mathbf{G}^T$ 会使 λ 被消去，得到

$$\mathbf{G}^T \mathbf{H} \ddot{\mathbf{q}} = \mathbf{G}^T (\boldsymbol{\tau} - \mathbf{C} + \boldsymbol{\tau}^a);$$

将式 8.42 代入此方程产生

$$\mathbf{G}^T \mathbf{H} \mathbf{G} \ddot{\mathbf{y}} = \mathbf{G}^T (\boldsymbol{\tau} - \mathbf{C} + \boldsymbol{\tau}^a - \mathbf{H}\mathbf{g})$$

(参照式 3.20)。此方程可以看作闭环系统的运动方程，以 $\ddot{\mathbf{y}}$ 中的独立关节加速度变量表示。矩阵 $\mathbf{G}^T \mathbf{H} \mathbf{G}$ 是对称正定的，因此可以立即求解式 8.45 得到 $\ddot{\mathbf{y}}$ ，即

$$\ddot{\mathbf{y}} = (\mathbf{G}^T \mathbf{H} \mathbf{G})^{-1} \mathbf{G}^T (\boldsymbol{\tau} - \mathbf{C} + \boldsymbol{\tau}^a - \mathbf{H}\mathbf{g}).$$

§8.6 计算 $\mathbf{C} - \boldsymbol{\tau}^a$ 的算法

处理 $\boldsymbol{\tau}^a$ 的最佳策略是选择生成树，使得每个回路关节都是被动的。在这种情况下，每个回路关节都有 $\mathbf{f}_k^a = \mathbf{0}$ ，从而 $\boldsymbol{\tau}^a = \mathbf{0}$ 。次佳策略是修改计算 $\mathbf{C}$ 的代码，使其改为计算 $\mathbf{C} - \boldsymbol{\tau}^a$ 。

表 8.2 展示了递归牛顿-欧拉算法 (recursive Newton-Euler algorithm) 的基本方程，如它们出现在表 5.1 中的那样，以及计算 $\mathbf{C} - \boldsymbol{\tau}^a$ 的方程。将原始方程与修改后的方程比较，可以看出以下差异。

已去掉项 $S_i \ddot{\mathbf{q}}_i$ ，因为在计算 $\mathbf{C}$ 时加速度变量被置为零。

变量名 a_i 和 τ_i 已改为 a_i^{vp} 和 C'_i (其中 $C' = \mathbf{C} - \boldsymbol{\tau}^a$)。这纯粹是外观上的改动。¹

关节力计算中加入了一个新项，它在与一般外力被减去的同一位置减去回路关节的主动力。

从生成树的角度看，回路关节的主动力确实是外力，因为它们来自不属于树的东西，而这正是它们被处理的方式。系数 e_{ik} 确保 f_k^a 只从 $f_{s(k)}$ 中减去，而 $-f_k^a$ 从 $f_{p(k)}$ 中减去。

表 8.3 展示了按照表 8.2 中的方程计算 $C - \tau^a$ 的算法伪代码。计算在物体坐标系中进行。该算法中的第一个和第三个循环与表 5.1 中相应的代码非常相似，但第二个循环是新增的。循环 1 的最后一行将每个向量 f_i 初始化为表达式 $f_i^B - f_i^x$ 。循环 2 的目的则是在循环 3 开始使用这些量之前，从每个 f_i 中减去项 $\sum_{k=N_B+1}^{N_I} e_{ik} f_k^a$ 。

循环 2 中的第二行使用 jcalc 计算 T_k^a ，并放入局部变量 T^a 。这里有一个假设，即 T_k^a 是常数，因此

表 8.3: 计算 $C - \tau^a$ 的算法

jcalc 只需知道关节 k 的类型说明符即可算出正确的值。如果不作此假设，就必须算出关节 k 从前驱坐标系到后继坐标系的变换，以便将其作为参数提供给 jcalc。

表达式 $T^a \tau_k$ 给出关节 k 后继坐标系坐标下的 f_k^a ，而 T^a 正是在该坐标系下提供的。循环 2 中的第三行计算后继坐标系坐标下的 f_k^a ，将其变换到物体 $s(k)$ 坐标系，并将结果存入局部变量 f^a 。接下来两行将这个量从 $f_{s(k)}$ 中减去，并加到 $f_{p(k)}$ 上。注意，在执行加法之前，必须将 f^a 从 $s(k)$ 坐标系变换到 $p(k)$ 坐标系。

§ 8.7 计算 K 和 k 的算法

表 8.4 展示了通过式 8.19 和 8.20 计算 K 、通过式 8.15 计算 k 、以及通过式 8.22 和 8.23 计算 k_{stab} 的算法伪代码。这些计算涉及组合最初在各种不同坐标系中获得的向量和矩阵。因此，在组合之前，至少必须对其中一些量施加坐标变换。本算法采用的策略是把所有量都变换到基座坐标系，并在那里组合。这是最简单的策略，但不一定是最好的。

局部变量 X_p 和 X_s 分别包含从基座坐标系到回路关节 k 的前驱坐标系和后继坐标系的 Plücker 变换；因此 $X_p = p^{(k),k} X_0$ 且 $X_s = s^{(k),k} X_0$ 。表达式 $X_s X_p^{-1}$ 因此是从关节前驱坐标系到后继坐标系的变换，即式 8.23 中 jcalc 的参数。对 jcalc 的调用同时请求回路的位置

```

 $K = 0$ 
for  $l = 1$  to  $N_L$  do
 $k = l + N_B$ 
 $X_p = X_P(l)^{p(k)} X_0$ 
 $X_s = X_S(l)^{s(k)} X_0$ 
 $v_p = {}^0 X_{p(k)} v_{p(k)}$ 
 $v_s = {}^0 X_{s(k)} v_{s(k)}$ 
 $a_p = {}^0 X_{p(k)} a_{p(k)}^{vp}$ 
 $a_s = {}^0 X_{s(k)} a_{s(k)}^{vp}$ 
 $[\delta, T] = \text{jcalc}(\text{jtype}(k), X_s X_p^{-1})$ 
 $T = X_s^T T$ 
 $k_{stab} = 2\alpha T^T (v_s - v_p) + \beta^2 \delta$ 
end
inter

```

```

:
 $k_l = -T^T (a_s - a_p + v_s \times v_p) - k_{stab}$ 
 $i = p(k)$ 
 $j = s(k)$ 
while  $i \neq j$  do
  if  $i > j$  then
 $K_{li} = -T^{T0} X_i S_i$ 
 $i = \lambda(i)$ 
  else
 $j = \lambda(j)$ 
  end
end
end
end

```

误差 δ 和约束力子空间矩阵 $\mathbf{T}$ 。紧随此调用之后的一行将 $\mathbf{T}$ 从后继坐标系坐标变换到基座坐标系；但对 δ 不施加变换，因为它表示形如 $\mathbf{T}^T \mathbf{d}$ 的量，其中 $\mathbf{d} \in \mathbb{M}^6$ 是无穷小位移。

本算法假设每个回路关节都有 $\dot{\mathbf{T}} = \mathbf{0}$ 且 $\sigma = \dot{\sigma} = \mathbf{0}$ 。不作此假设的话，jcalc 就需要更多参数，并要求返回更多量。假设 $\dot{\mathbf{T}} = \mathbf{0}$ 意味着 $\dot{\mathbf{T}} = \mathbf{v}_s \times^* \mathbf{T}$ ，这使我们可以如下简化式 8.15 的右端：

$$\begin{aligned} \mathbf{k}_l &= -\mathbf{T}_k^T (\mathbf{a}_{s(k)}^{vp} - \mathbf{a}_{p(k)}^{vp}) - \dot{\mathbf{T}}_k^T (\mathbf{v}_{s(k)} - \mathbf{v}_{p(k)}) \\ &= -\mathbf{T}_k^T (\mathbf{a}_{s(k)}^{vp} - \mathbf{a}_{p(k)}^{vp}) - (\mathbf{v}_{s(k)} \times^* \mathbf{T}_k)^T (\mathbf{v}_{s(k)} - \mathbf{v}_{p(k)}) \\ &= -\mathbf{T}_k^T (\mathbf{a}_{s(k)}^{vp} - \mathbf{a}_{p(k)}^{vp}) + \mathbf{T}_k^T \mathbf{v}_{s(k)} \times (\mathbf{v}_{s(k)} - \mathbf{v}_{p(k)}) \\ &= -\mathbf{T}_k^T (\mathbf{a}_{s(k)}^{vp} - \mathbf{a}_{p(k)}^{vp}) + \mathbf{v}_{s(k)} \times \mathbf{v}_{p(k)}. \end{aligned}$$

这就是算法中使用的表达式。

$\mathbf{K}$ 的计算方式是先将整个矩阵初始化为零，然后计算非零子矩阵。对于每个回路 l ，子矩阵 $\mathbf{K}_{li}$ 对位于回路路径上的每个树关节 i 都是非零的。这些量在一个使用两个循环变量 i 和 j 的 while 循环中计算。这两个变量分别初始化为 $p(k)$ 和 $s(k)$ ，然后按保证它们在 $p(k)$ 和 $s(k)$ 的最近共同祖先处相遇的顺序向基座方向回退。此时，回路 l 路径上的每个关节都已处理到，且 $i = j$ ，这即为循环的终止条件。

表 8.4 中给出的算法可能涉及一些重复计算。例如，如果关节 i 参与多个回路，那么 $\mathbf{S}_i$ 将被多次变换到基座坐标系。同样，如果物体 i 是多个回路关节的后继或前驱，那么 $\mathbf{v}_i$ 和 $\mathbf{a}_i^{vp}$ 将被多次变换到基座坐标系，并且相关变换 ($\mathbf{X}_p$ 或 $\mathbf{X}_s$) 将被多次计算。这类重复对整体效率的影响相对较小。不过，如果要求最大速度，代码可以改写以避免此类重复。

8.7.1 舍入误差问题

该算法使用基座坐标系实现两个目的：它使算法保持简单，并使所需的变换矩阵数量最小化。然而，这是有代价的：大多数浮点计算都涉及舍入误差，其中一些误差对执行计算的坐标系与计算所涉及物体位置之间的距离很敏感。因此，如果基座坐标系离运动学回路很远，计算就会损失精度。这种现象对大多数刚体系统不是问题；但对于浮动基座系统，如果它行驶到离其固定基座原点很远的地方，这可能成为问题。（想想飞机或航天器。）在这种情况下，一个简单的补救措施是对 $\mathbf{K}$ 、 $\mathbf{k}$ 和 $\mathbf{k}_{stab}$ 的计算改用浮动基座坐标系，而不是固定基座坐标系。

§8.8 计算 $\mathbf{G}$ 和 $\mathbf{g}$ 的算法

本节展示如何求解如下形式的线性方程

$$\kappa \mathbf{x} = \mathbf{k}$$

以得到如下形式的解

$$\mathbf{x} = \mathbf{G}\mathbf{y} + \mathbf{g}.$$

该方法适用于求解式 8.35 以获得式 8.42 的任务。在式 8.48 和 8.49 中, K 是秩为 r 的 $m \times n$ 矩阵, $k \in \text{range}(K)$ (因此式 8.48 是一致的), G 是 $n \times (n-r)$ 满秩矩阵, 式 8.49 产生式 8.48 的所有可能解。事实上, 式 8.49 定义了 y 与式 8.48 解集之间的 1:1 映射。如果 $r = n$, 则式 8.49 简化为 $x = g$, 且 g 唯一。如果 $r < n$, 则 G 和 g 不唯一, 我们寻求能解决问题的一对值即可。

鉴于 $\text{rank}(K) = r$, 可知 K 必含有一个 $r \times r$ 可逆子矩阵。因此, 存在 K 的一个置换, 使该子矩阵的元素占据左上角。于是我们可以写

$$K = Q_1 \begin{bmatrix} K_{11} & K_{12} \\ K_{21} & K_{22} \end{bmatrix} Q_2,$$

其中 K_{11} 是 $r \times r$ 可逆矩阵, Q_1 和 Q_2 是置换矩阵 (permutation matrix)。将此置换扩展到整个式 8.48, 得到

$$Q_1 \begin{bmatrix} K_{11} & K_{12} \\ K_{21} & K_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = Q_1 \begin{bmatrix} k_1 \\ k_2 \end{bmatrix},$$

其中

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = Q_2 x \text{ and } Q_1 \begin{bmatrix} k_1 \\ k_2 \end{bmatrix} = k.$$

同样, 鉴于 $\text{rank}(K) = r$ 且式 8.48 是一致的, 可知式 8.50 中的底行是顶行的线性倍数; 因此必然存在矩阵 Y , 使得 $K_{21} = YK_{11}$ 、 $K_{22} = YK_{12}$ 且 $k_2 = Yk_1$ 。因此我们可以把式 8.50 改写为

$$Q_1 \begin{bmatrix} K_{11} & K_{12} \\ YK_{11} & YK_{12} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = Q_1 \begin{bmatrix} k_1 \\ Yk_1 \end{bmatrix}.$$

通过观察, 可以看出此方程的一个可能解是

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} -K_{11}^{-1}K_{12} \\ \mathbf{1} \end{bmatrix} x_2 + \begin{bmatrix} K_{11}^{-1}k_1 \\ \mathbf{0} \end{bmatrix}.$$

因此我们可以得出结论, 原问题的一个解是

$$G = Q_2^T \begin{bmatrix} -K_{11}^{-1}K_{12} \end{bmatrix} \text{ and } g = Q_2^T \begin{bmatrix} K_{11}^{-1}k_1 \end{bmatrix}.$$

这个特定解将 y 等同于 x_2 , 这意味着 y 是 x 中元素的子集。

式 8.51 中的表达式可以用标准的高斯消元 (Gaussian elimination) 算法或 LU 因式分解计算, 并作如下修改。

必须使用完全主元 (complete pivoting), 既交换行也交换列。必须记录列交换, 因为这些数据决定 Q_2 的值。

如果没有提供 r 作为参数, 则需要行秩检验, 以检测最后一个独立行何时已处理完毕。

因式分解过程在处理完最后一个独立行后即终止。在此阶段, K 的左上 $r \times r$ 子矩阵包含上三角矩阵 U , 其中 $LU = K_{11}$; 右上 $r \times (n-r)$ 子矩阵包含 $L^{-1}K_{12}$; k 的顶部 r 行包含 $L^{-1}k_1$ 。因此, 对顶部 r 行进行回代即可得到 $K_{11}^{-1}K_{12}$ 和 $K_{11}^{-1}k_1$ 。这个基本算法可以添加各种特性。例如, 可以提供掩码来标识 K 中哪些列对应于不参与任何运动学回路的关节变量。这些列只包含零, 因

此因式分解过程不改变它们。另一种可能性是允许用户指定自己选择的独立变量。这消除了完全主元的需要，从而可以改用部分主元（partial pivoting）。

§8.9 利用 K 和 G 中的稀疏性

很明显, K 和 G 都可能包含许多零。例如, 式 8.20 意味着 K 的每个子矩阵 K_{lj} 对所有使 $\varepsilon_{lj} = 0$ 的 l 和 j 都为零; 而式 8.51 意味着 G 至少包含 $(n-r)(n-r-1)$ 个零, 这正是 $(n-r) \times (n-r)$ 单位矩阵中的零的个数。然而, 这两个矩阵中很大一部分稀疏性可以归结于一个事实, 即本节的主题: 它们都是分块对角 (block-diagonal) 的。

图 8.1: 回路簇以及 K 和 G 的稀疏模式

为了揭示分块对角结构, 我们首先必须把运动学回路组织为一组极小回路簇 (minimal loop cluster)。回路簇是任何一组具有如下性质的回路: 簇内的任何回路都不与簇外的任何回路耦合; 而极小回路簇是不能被分成两个更小簇的回路簇。如果两个运动学回路共有一个关节, 则它们之间存在耦合。给定一组极小回路簇, 可以为生成树设计一种正则编号 (regular numbering), 使其具有如下性质: 每个簇内的树关节按顺序连续编号。正是这种编号方案揭示了 K 和 G 的分块对角结构。

图 8.1 给出了一个例子。该图显示了一个含四个运动学回路的闭环系统的连接图 (connectivity graph)。回路编号为 1 到 4, 分别由回路关节 21 到 24 闭合。(图中未显示回路编号。) 回路 1 与回路 2 耦合, 但系统中没有其他耦合。因此我们可以识别出如下三个极小簇: 簇 1 包含回路 1 和 2; 簇 2 只包含回路 3; 簇 3 只包含回路 4。识别出这三个簇之后, 我们可以设计一种正则编号, 使每个簇内的树关节连续编号。在本例中, 簇 1 中的关节编号为 3 到 7, 簇 2 中的编号为 10 到 12, 簇 3 中的编号为 19 和 20。其余树关节不位于任何运动学回路的路径上, 因此不属于任何簇。它们的编号可以与正则编号的规则一致地任意进行。

采用这种编号方案后, K 呈现一种带间隙的不规则分块对角结构: 每个簇有一个块; 块一般是矩形的; 每个块的大小和形状可以与其他块不同; 块之间可以存在大小不一的水平间隙。这些间隙由不属于任何簇的树关节造成。 K 中的这种稀疏模式使 G 呈现一种无间隙的不规则分块对角结构: K 中的每个块 K_i 在 G 中产生一个块 G_i ; K 中的每个间隙在 G 中产生一个单位矩阵块。为便于识别, 连接图中的关节被分成六个集合, 标记为 a 到 f , 同样的划分也显示在 K 的列和 G 的行旁边。

分块对角稀疏性可以在计算 $KH^{-1}K^T$ 和 G^THG , 以及从 K 计算 G 时加以利用。特别是, G 可以从 K 逐块计算: 由于 $x = G_i y + g_i$ 是 $K_i x = k_i$ 的解, 每个 G_i 和 g_i 都可以用第 8.8 节的算法直接从 K_i 和 k_i 计算。

§8.10 闭环系统的一些性质

闭环系统展现出的性质和行为的多样性比运动学树更大, 这可能给动力学算法带来新的困难。本节考察闭环系统与动力学相关的一些特殊性质。

可动度 (Mobility): 可动度是刚体系统的运动自由度。运动学树的可动度就是 n ; 但闭环系统的可动度由一般公式给出

$$mobility = n - r.$$

如果 y 是刚体系统的独立坐标向量, 那么 y 的维数是 $n - r$ 。具有 $r = n^c$ 性质的系统称为适当约束 (properly constrained) 系统。对于这样的系统, 可动度可以用公式表示

$$mobility = n_{total} - 6N_L,$$

图 8.2: 可变可动度 (a) 和构型歧义 (b) 的例子

其中 n_{tot} 是系统中所有关节的自由度之和, 而不仅仅是树关节。该公式适用于三维欧几里得空间中的刚体系统。二维空间中的等效公式是

$$mobility = n_{total} - 3N_L.$$

可变可动度 (Variable Mobility): 运动学树的可动度是固定的; 但闭环系统的可动度取决于 r , 而 r 可以随构型变化。可变可动度的例子见图 8.2(a)。只要角 θ 非零, 机构的可动度就是 1; 但如果 $\theta = 0$, 则两个臂能够独立运动, 可动度为 2。此外, 在中间图所示的构型 (这是两种运动机制之间的边界) 下, 机构具有瞬时可动度 3。用 n 和 r 表示, 全程 $n = 5$, 但左图中 $r = 4$, 右图中 $r = 3$, 中间图中 $r = 2$ 。

构型歧义 (Configuration Ambiguity): 如果 q 和 y 分别是运动学树和闭环系统的独立位置变量向量, 那么 q 总是标识树的一个唯一构型, 但 y 不一定标识闭环系统的唯一构型。图 8.2(b) 显示了一个简单的例子: 知道独立变量 θ 的值不足以唯一确定系统的构型。解决此问题有两种方法:

用 q 代替 y , 其中 q 是生成树的位置向量; 或者

提供一个函数, 把每个 y 值映射到一个唯一的 q 值。

第一种选择是前面各节使用的, 第二种选择是第 8.11 节的主题。对于图 8.2(b) 所示的系统, 如果我们预先知道两种构型中的一种无法到达, 则方法 2 适用。否则, 我们必须尝试不同的独立变量, 或者求助于方法 1。

过约束 (Overconstraint): 如果 $r < n^c$, 则称闭环系统为过约束的。这样的系统有 $n^c - r$ 个冗余约束, 这意味着运动方程中将有 $n^c - r$ 个不确定的约束力。它们相对于适当约束系统还有多余的可动度, 因为可动度公式可以写为

$$mobility = n_{total} - 6N_L + (n^c - r).$$

过约束系统实际上非常普遍。例如, 任何包含平面运动学回路的系统都是过约束的。图 8.2(b) 所示的平面回路包含四个旋转关节, 因此该系统的数字为 $n = 3$ 和 $n^c = 5$; 但可动度为 1, 这意味着 $r = 2$ 。

过约束系统的动力学比适当约束系统更难计算。因此, 只要可能, 把前者转换为后者是有用的。转换的实现方式是: 用施加较少约束的关节替换原来的回路关节, 从而减小 n^c 的值。例如, 如果平面运动学回路中的回路闭合旋转关节被替换为球柱关节 (sphere-in-cylinder joint), 后者只施加两个约束, 那么 n^c 将从 5 减小到 2, 这意味着 $n^c = r$ 。

§8.11 回路闭合函数

设 q 是给定闭环系统生成树的位置变量向量，设 y 是同一系统的独立位置变量向量。 y 通常是 q 中元素的子集，但这并非必要。我们假设 y 唯一地定义 q 。为使此假设成立，可能需要对 y 的值施加限制。因此我们定义一组可接受的 y 值集合 $C \subseteq \mathbb{R}^{(n-r)}$ ，并假设 $y \in C$ 。在这些假设下，存在一个函数 γ 满足

$$q = \gamma(y)$$

对所有 $y \in C$ 。（参照式 3.11。）对方程求微分得到

$$\dot{q} = G \dot{y},$$

其中

$$G = \frac{\partial \gamma}{\partial y},$$

再微分一次得到

$$\ddot{q} = G \ddot{y} + g,$$

其中

$$g = \dot{G} \dot{y}.$$

式 8.59 与式 8.42 具有相同的形式，当回路闭合误差为零时，两个方程相同。

现在我们有了一种获得 G 和 g 的新方法，其中它们由独立变量向量计算。该方法可概括如下。

定义 y ，并确定 C 。

在 $y \in C$ 的前提下，找出把 y 唯一映射到 q 的函数 γ 的表达式。

符号化地对这个表达式求微分，得到 G 和 g 的表达式。

编写代码来求值这些表达式。

这个方法不如第 8.5 节描述的那个方法通用，主要是因为第 2 步并非总是可行。它也不那么方便，因为它要求用户在系统模型之外再提供 G 和 g 的表达式（或代码）。尽管如此，它适用于各种各样实际的闭环系统，而且用这种方法计算 G 和 g 可能比使用第 8.7 和 8.8 节的算法便宜得多。

这种方法的另一个优点是回路闭合误差不可能发生。这是因为我们不对 $\ddot{q}$ 求积分得到 $\dot{q}$ 和 q ，而是对 $\ddot{y}$ 求积分得到 $\dot{y}$ 和 y 。这样，向量 q 和 $\dot{q}$ 用式 8.56 和 8.57 由 y 和 $\dot{y}$ 计算，这意味着它们的值自动满足回路约束。因此不需要约束稳定化项。

8.11.1 一个完整例子

图 8.3 显示了一个含四个物体、五个关节和一个运动学回路的刚体系统。该系统是一个平面机构，其可动度为 2。物体 B_1 和 B_4 构成一个两连杆臂，而 B_2 和 B_3 分别是一个线性致动器的缸体和活塞。关节 3 是移动副（prismatic），其余都是旋转副（revolute）。关节 1、2、3 和 5 参与运动学回路，其中关节 5 是回路关节。因此，树关节变量向量为 $\mathbf{q} = [q_1 q_2 q_3 q_4]^T$ ，其中 q_3 是距离，其余是角度。在本例中，我们选择 q_1 和 q_4 作为独立变量；也就是说，我们定义独立变量向量 $\mathbf{y} = [y_1 y_2]^T$ ，使得 $y_1 = q_1$ 、 $y_2 = q_4$ 。主要任务随后是把 q_2 和 q_3 表示为 y_1 的函数。

8.11.2 三角形公式：

$$d > 0$$

$$\sin(\theta_2) = \frac{l_2 \sin(\theta_1)}{d}$$

$$\cos(\theta_2) = \frac{l_2 \cos(\theta_1) - l_1}{d}$$

解：

一

$$\gamma(\mathbf{y}) = \begin{bmatrix} y_1 \\ \theta_2 \\ d \\ y_2 \end{bmatrix}$$

其中

$$\theta_2 = \tan^{-1} \left(\frac{l_2 \sin(y_1)}{l_2 \cos(y_1) - l_1} \right)$$

$$d = \sqrt{l_1^2 + l_2^2 - 2l_1 l_2 \cos(y_1)}$$

$$G = \frac{\partial \gamma}{\partial \mathbf{y}} = \begin{bmatrix} 1 & 0 \\ u_1 & 0 \\ u_2 & 0 \\ 0 & 1 \end{bmatrix}$$

其中

$$u_1 = \frac{l_2^2 - l_1 l_2 \cos(y_1)}{d^2}$$

$$u_2 = \frac{l_1 l_2 \sin(y_1)}{d}$$

$$\mathbf{g} = \mathbf{G} \dot{\mathbf{y}} = \begin{bmatrix} 0 \\ u_3 \\ u_4 \\ 0 \end{bmatrix}$$

其中

$$u_3 = \frac{(l_1^2 - l_2^2) l_1 l_2 \sin(y_1)}{d^4} \dot{y}_1^2$$

$$u_4 = \frac{(l_1 l_2 \cos(y_1) - u_2^2)}{d} \dot{y}_1^2$$

图 8.3：为简单闭环系统建立 γ 、 G 和 g 的公式

运动学回路的几何非常简单：只是一个三角形。因此可以使用标准三角公式把 q_2 和 q_3 （在三角形上标为 θ_2 和 d ）表示为 y_1 的函数。为使映射唯一，我们施加约束 $d > 0$ ，并使用四象限

版本的反正切函数（取两个参数的那种）来计算 θ_2 。如果 $l_1 = l_2$ ，则要求 $\theta_1 \neq 0$ 。在这种情况下， C 必须排除所有满足 $y_1 = 0$ 的向量 $\mathbf{y}$ 。当一个运动学回路具有显式解公式时，就称它有闭式解（closed-form solution）。

图 8.3 还给出了 γ 、 G 和 g 的显式公式。可以看出， G 是分块对角的，包含一个 3×1 块和一个 1×1 块。注意，该图中的方程都是标量方程，与第 8.7 和 8.8 节中的向量和矩阵方程形成对比。因此，用这里给出的方程计算 G 和 g 要便宜得多。

总之，这种方法给用户增加了提供计算 γ 、 G 和 g 代码的额外负担，但作为回报，动力学计算中这一部分的效率可能大幅提高。该方法适用于含有具有简单闭式解的运动学回路的系统，例如挖掘机臂、大型工业机器人、并联机器人和 Stuart-Gough 平台（飞行模拟器）上可能遇到的那些。

§8.12 逆动力学

逆动力学（inverse dynamics）是计算产生给定加速度所需力的问题。在运动学树中， τ 与 $\ddot{q}$ 之间存在 1:1 关系，从 $\ddot{q}$ 计算 τ 是直接的。在闭环系统中，给定的 $\ddot{q}$ 值必须符合回路闭约束，而且有无限多个 τ 值产生相同的加速度。因此，运动学回路的存在显著改变了逆动力学问题的性质。

现在让我们为闭环系统建立逆动力学方程。我们从生成树的运动方程开始，即

$$\boldsymbol{\tau} = H\ddot{\mathbf{q}} + \mathbf{C} - \boldsymbol{\tau}^a - \mathbf{K}^T \boldsymbol{\lambda}.$$

在此方程中， $\boldsymbol{\tau}$ 是作用在树关节处的未知力向量。任何已知力，如弹簧和阻尼器产生的力，都假定已包含在 $\mathbf{C}$ 中。还假设回路关节处没有未知的主动动力。因此，如果 $\boldsymbol{\tau}^a$ 出现的话，它完全由已知力组成。

式 8.61 包含两个未知量 τ 和 λ 。由于我们只对 τ 感兴趣，下一步是消去 λ 。这通过将方程乘以 G^T 完成，得到

$$G^T \boldsymbol{\tau} = G^T (H\ddot{\mathbf{q}} + \mathbf{C} - \boldsymbol{\tau}^a).$$

此方程可以写为

$$G^T \boldsymbol{\tau} = G^T \boldsymbol{\tau}_{\text{ID}},$$

其中 $\boldsymbol{\tau}_{\text{ID}}$ 是向生成树应用逆动力学函数所计算的力；也就是说，

$$\begin{aligned} \boldsymbol{\tau}_{\text{ID}} &= H\ddot{\mathbf{q}} + \mathbf{C} - \boldsymbol{\tau}^a \\ &= \text{ID}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}). \end{aligned}$$

在此语境中， $\text{ID}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$ 是第 8.6 节所述算法在放回关节加速度项后的输出。请记住， $\mathbf{q}$ 、 $\dot{\mathbf{q}}$ 和 $\ddot{\mathbf{q}}$ 都必须符合回路闭约束。

式 8.62 是闭环系统的基本逆动力学方程。由于 $\mathbf{G}^T$ 是 $(n-r) \times n$ 矩阵，此方程对 n 维未知向量施加 $n-r$ 个约束，留下 r 个自由选择。换句话说，有 ∞^r 个不同的 $\boldsymbol{\tau}$ 值产生相同的加速度。如果我们想要问题的唯一解，就必须施加更多约束或应用最优性准则。

8.12.1 致动器力

逆动力学的一个典型用途是计算产生给定加速度的致动器力向量。现在，并非每个关节都由致动器驱动，因此让我们定义 p 为树中被驱动的自由度数， $\mathbf{u}$ 为 p 维致动器力值向量。 $\mathbf{u}$ 与 $\boldsymbol{\tau}$ 的关系由方程给出

$$= \mathbf{Q} \begin{bmatrix} \mathbf{u} \\ \mathbf{0} \end{bmatrix},$$

其中 $\mathbf{Q}$ 是 $n \times n$ 置换矩阵。此方程只是说明 $\boldsymbol{\tau}$ 的元素是 $\mathbf{u}$ 的 p 个元素与 $n-p$ 个零的置换。 $\boldsymbol{\tau}$ 中值为零的元素属于未被驱动的自由度。让我们还定义两个矩阵 $\mathbf{G}_u$ 和 $\mathbf{G}_0$ ，分别包含 $\mathbf{G}$ 中对应于被驱动和未驱动自由度的行。这些矩阵与 $\mathbf{G}$ 的关系由方程给出

$$\mathbf{G} = \mathbf{Q} \begin{bmatrix} \mathbf{G}_u \\ \mathbf{G}_0 \end{bmatrix}.$$

由这些定义可得

$$\mathbf{G}^T \boldsymbol{\tau} = \begin{bmatrix} \mathbf{G}_u^T & \mathbf{G}_0^T \end{bmatrix} \mathbf{Q}^T \mathbf{Q} \begin{bmatrix} \mathbf{u} \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} \mathbf{G}_u^T & \mathbf{G}_0^T \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \mathbf{0} \end{bmatrix} = \mathbf{G}_u^T \mathbf{u};$$

将此结果代入式 8.62 得到

$$\mathbf{G}_u^T \mathbf{u} = \mathbf{G}^T \boldsymbol{\tau}_{\text{ID}},$$

这就是致动器力的逆动力学方程。求解此方程时，要考虑三种情况。

$\text{rank}(\mathbf{G}_u) < n-r$ 。在这种情况下，系统欠驱动 (underactuated)：它的运动自由度多于致动器能控制的数量。

$p > \text{rank}(\mathbf{G}_u)$ 。在这种情况下，系统冗余驱动 (redundantly actuated)：有无限多个不同的 $\mathbf{u}$ 值产生相同的加速度。

$p = \text{rank}(\mathbf{G}_u) = n-r$ 。在这种情况下，系统适当驱动 (properly actuated)： $\mathbf{G}_u$ 可逆，式 8.63 有唯一解，由 $\mathbf{u} = \mathbf{G}_u^{-T} \mathbf{G}^T \boldsymbol{\tau}_{\text{ID}}$ 给出。

情况 1 和 2 并不互斥。

例 8.2 考虑为图 8.3 所示的机构分配致动器。该机构有两个自由度，因此需要两个致动器。 $\mathbf{G}_u$ 将是包含 $\mathbf{G}$ 两行的 2×2 矩阵；必须选择这些行，使 $\mathbf{G}_u$ 可逆。根据图中所示的 $\mathbf{G}$ 公式， $\mathbf{G}_u$ 必须包含 $\mathbf{G}$ 的第 4 行以及其他三行中的任意一行。这意味着关节 4 处必须有一个致动器，并且其他三个关节中的任意一个处也要有。（由于该机构非常简单，这个结果也可以通过观察得到。）

让我们假设关节 3 和 4 被驱动，这意味着 $\mathbf{G}_u = \begin{bmatrix} u_2 & 0 \\ 0 & 1 \end{bmatrix}$ 。该系统的逆动力学公式将是

$$\begin{aligned}
u &= G_u^{-T} G^T \tau_{ID} \\
&= \begin{bmatrix} u_2 & 0 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 1 & u_1 & u_2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tau_{ID} \\
&= \begin{bmatrix} 1/u_2 & u_1/u_2 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tau_{ID},
\end{aligned}$$

其中

$$\tau_{ID} = ID(\gamma(y), G\ddot{y}, G\ddot{y} + g).$$

注意, u 不是该系统的广义力向量, 因为表达式 $u^T \dot{y}$ 不等于 u 所传递的功率。这种差异是由于 $\dot{q}_1$ 是 $\dot{y}$ 中的第一个元素, 而 τ_3 是 u 中的第一个元素。如果你想要 u 成为广义力向量, 那么 u 的元素必须是 τ 中与 $\dot{y}$ 是 $\dot{q}$ 的子集相同的子集。

§8.13 稀疏矩阵方法

到目前为止, 我们研究的所有算法都是通过对生成树施加一组回路闭合约束来工作的。另一种策略是放弃生成树, 直接处理系统中的各个物体和关节。这种方法产生的方程具有大型稀疏矩阵; 任何基于这些方程的算法都必须利用稀疏性才能具有竞争力。这类算法往往不如基于生成树的算法高效, 但在特殊情况下它们可以更高效, 如下文所示。

一个含 N_B 个物体的物体集合的运动方程为

$$\begin{bmatrix} I & PT \\ T^T P^T & 0 \end{bmatrix} \begin{bmatrix} a \\ -\lambda \end{bmatrix} = \begin{bmatrix} PT^a \tau - v \times^* I v + f^x \\ -\dot{T}^T P^T v \end{bmatrix}.$$

该方程在第 3.7 节中推导。 v 、 a 和 f^x 分别是物体速度、加速度和外力的向量; λ 是关节约束力变量的向量; $T^a \tau$ 是跨关节传递的主动力向量; P^T 是从物体速度映射到关节速度的矩阵 (即 $P^T v$ 是关节速度向量); I 和 T 分别是物体惯量和关约束力子空间的复合矩阵。 a 和 λ 是未知量; 系数矩阵的尺寸为 $6N_B + n_{\text{tot}}^c$ 平方, 其中 n_{tot}^c 是关节施加的约束总数。

我们主要的兴趣在于系数矩阵的稀疏模式。

$$I = \text{diag}(I_1, \dots, I_{N_B}) \text{ and } T = \text{diag}(T_1, \dots, T_{N_J}).$$

P 也是稀疏的, 但不是分块对角的。它的定义如下

$$P_{ij} = \begin{cases} \mathbf{1}_{6 \times 6} & \text{if } i = s(j) \\ -\mathbf{1}_{6 \times 6} & \text{if } i = p(j) \\ \mathbf{0}_{6 \times 6} & \text{otherwise.} \end{cases}$$

因此 P 是 $6N_B \times 6N_J$ 矩阵, 组织成 $N_B \times N_J$ 的 6×6 块矩阵。除了直接连接到基座的关节所对应的列之外, 它的每一列都有两个非零块, 而直接连接基座的关节的列只有一个非零块。由

这些定义可知，系数矩阵在主对角线上包含 N_B 个非零块，以及最多 $4N_J$ 个非零的非对角块。后者的确切位置由连接性决定。

式 8.64 是稀疏矩阵运算软件库（sparse matrix arithmetic software library）被设计用来求解的那种方程的典型代表，使用这样的库可能是最好的做法。稀疏矩阵库通常提供如下功能：

一个分析矩阵稀疏模式的函数，作为预处理步骤，以确定最佳因式分解顺序，以及哪些零值元素在因式分解过程中会被填充为非零值；

一种紧凑存储矩阵中所有非零值以及因式分解过程中将被改变的所有零值元素的数据结构；

一个按预处理步骤确定的顺序执行因式分解的函数；以及

一个利用所得因子执行回代的函数。

如果一个 $n \times n$ 矩阵只包含 $O(n)$ 个非零元素，那么有时可以在仅 $O(n)$ 次算术运算内对该矩阵进行因式分解。如果系统是运动学树，则式 8.64 的 $O(n)$ 因式分解总是可能的。² 然而，也有许多闭环系统可以进行 $O(n)$ 因式分解，其中包括一些任何使用生成树的算法都无法在 $O(n)$ 时间内求解的系统。在这种情况下，对于足够大的 n ，稀疏矩阵算法将优于生成树算法。

图 8.4 显示了通过稀疏矩阵方法具有 $O(n)$ 动力学的闭环系统示例。这种连接图有时被称为梯子（ladder）。通过在右侧添加更多节点，它可以无限扩展。该图还显示了系数矩阵对称置换的稀疏模式。选择这种置换是为了使所得矩阵具有固定带宽（fixed bandwidth）——每个非零块都出现在主对角线上，或两侧三条最近对角线之一上。具有此性质的矩阵总是可以在 $O(n)$ 次运算内因式分解。

为了解释这种置换，图中的节点按它们在矩阵中出现的相同顺序编号，关节被赋予反映其连接性的编号对：关节 ij 是从物体 i 连接到物体 j 的关节。置换后矩阵的第 1 行是原矩阵中包含 $\mathbf{I}_1$ 的行（物体编号如图中所示）；第 2 行是包含 $\mathbf{I}_2$ 的行；第 3 行（标为 12）是原矩阵中 $\mathbf{T}^T \mathbf{P}^T$ 中与关节 12 相关的那一行；依此类推。注意，每个以物体编号标记的行在主对角线上包含一个非零块，而每个以编号对 ij 标记的行只在第 i 列和第 j 列包含非零块。

如果我们对这个系统应用生成树算法，我们会遇到如下问题：无论我们选择什么生成树，关节空间惯量矩阵 H 中总有 $O(n^2)$ 个非零元素，因此不可能仅在 $O(n)$ 次运算内计算动力学。

最后，还应提及以下细节。首先，如果该算法用于动力学仿真器，则需要约束稳定化。式 8.64 中没有包含所需的额外项。其次，我们假设系数矩阵满秩；也就是说，我们假设闭环系统是适当约束的。如果矩阵奇异，则稀疏因式分解可能无法工作。

图 8.4：由梯子状连接图产生的稀疏模式

混合动力学及其他专题

本章涵盖以下主题：混合动力学（hybrid dynamics）、浮动基座（floating bases）、齿轮（gears）与动力学等价（dynamic equivalence）。混合动力学是正向动力学与逆向动力学的一种推广，其中部分关节的力为已知量，其余关节的加速度为已知量，任务是计算未知的力与加速度。实际上，它是在一部分关节处执行正向动力学，而在其余关节处执行逆向动力学。浮动基座系统（floating-base system）是指基座为运动刚体的系统。通过在固定基座与代表浮动基座的刚体之间安装一个六自由度关节，可将此类系统建模为固定基座系统。然而，浮动基座系统是刚体系统中的一个重要类别，值得专门处理。第 9.1 至 9.5 节介绍若干用于混合动力学与浮动基座的算法。齿轮会在关节变量之间引入约束，因此它们类似于运动学回路（kinematic loops），第 9.6 节将展示如何利用第 8 章为运动学回路发展出的一种技术，把齿轮约束并入刚体系统。若两个不同的刚体系统具有相同的运动方程，则称二者动力学等价（dynamically equivalent）。第 9.7 节探讨这一概念，并给出一种在保持运动方程不变的前提下修改刚体系统惯性参数的方法。

§ 9.1

混合动力学（Hybrid Dynamics）

对于每个关节 i ，都有一个力变量 τ_i 和一个加速度变量 $\ddot{q}_i$ 。在正向动力学问题中，每个关节的 τ_i 均为已知，任务是计算加速度。在逆向动力学问题中，每个关节的 $\ddot{q}_i$ 均为已知，任务是计算力。在混合动力学问题中，每个关节的 τ_i 或 $\ddot{q}_i$ 二者之一是已知量，任务是计算未知的加速度与力。实际上，混合动力学就像在部分关节处执行正向动力学、在其余关节处执行逆向动力学

一样。因此，混合动力学问题把正向动力学问题与逆向动力学问题都作为其特例而包含在内。

混合动力学用于将规定的运动（prescribed motion）引入刚体系统。例如，它使你得以研究机械连杆机构或传动装置在输入端给定运动条件下的动力学，或研究复摆（compound pendulum）对给定运动激励的响应，或研究动画角色在空中翻转（tumbling）的行为。它还可用于辅助设计欠驱动系统（underactuated system）的轨迹；例如，为动画角色设计一套肢体动作序列，使其完成跳跃、空翻并双脚落地。

另一个用途是通过去除不重要和/或高频的动力学来简化仿真。例如，机器人关节的运动通常由高增益反馈控制系统（high-gain feedback control system）来控制。这些控制系统不断把关节的实际运动与指令运动进行比较，并调整执行器的力输出，使二者之差迅速趋于零。在完整机器人的仿真中，仿真器必须连同机械部件一起计算执行器与控制系统的动力学。然而，如果某个控制系统可以被视为理想系统，即它所控制的关节的实际运动几乎等于指令运动，那么就可以

省去该控制系统及其执行器，直接规定受影响关节的运动。如果被去除的部件在原系统中承担了最高频的动力学，那么仿真器现在就可以采用更长的积分时间步长。这一技术的一个例子可参见 Hu 等人 (2005) 的文献。

9.1.1 方程 (Equations)

设 fd 表示“正向动力学关节” (forward-dynamics joints) 的集合，即力为已知、加速度为未知的那些关节的集合。凡不属于 fd 的关节即为“逆向动力学关节” (inverse-dynamics joints)，其加速度为已知、力为未知。再设 Q 为置换矩阵 (permutation matrix)，它对 $\ddot{q}$ 与 τ 的元素重新排序，使正向动力学的变量排在前面。于是有

$$Q\ddot{q} = \begin{bmatrix} \ddot{q}_1 \\ \ddot{q}_2 \end{bmatrix} \text{ and } Q\tau = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix},$$

其中 $\ddot{q}_1$ 与 τ_1 包含与正向动力学关节相关的加速度与力变量，而 $\ddot{q}_2$ 与 τ_2 包含与逆向动力学关节相关的加速度与力变量。此时，运动学树 (kinematic tree) 的运动方程可以写成

$$\begin{bmatrix} H_{11} & H_{12} \\ H_{21} & H_{22} \end{bmatrix} \begin{bmatrix} \ddot{q}_1 \\ \ddot{q}_2 \end{bmatrix} = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix} - \begin{bmatrix} C_1 \\ C_2 \end{bmatrix},$$

其中

$$\begin{bmatrix} H_{11} & H_{12} \\ H_{21} & H_{22} \end{bmatrix} = Q H Q^T \text{ and } \begin{bmatrix} C_1 \\ C_2 \end{bmatrix} = Q C.$$

式 9.1 只是标准运动方程 $H\ddot{q} = \tau - C$ 的一个置换 (例如参见式 6.1)。该方程中的两个未知量是 $\ddot{q}_1$ 与 τ_2 。若把它们移到等号左端，则得到

$$\begin{bmatrix} H_{11} & 0 \\ H_{21} & -1 \end{bmatrix} \begin{bmatrix} \ddot{q}_1 \\ \tau_2 \end{bmatrix} = \begin{bmatrix} \tau_1 \\ 0 \end{bmatrix} - \begin{bmatrix} C'_1 \\ C'_2 \end{bmatrix},$$

其中

$$\begin{bmatrix} C'_1 \\ C'_2 \end{bmatrix} = \begin{bmatrix} C_1 + H_{12}\ddot{q}_2 \\ C_2 + H_{22}\ddot{q}_2 \end{bmatrix} = Q C'.$$

这就是混合动力学的基本方程。从物理意义上看， C' 是使每个正向动力学关节的加速度为零、并使每个逆向动力学关节具有给定加速度所需的力。因此， C' 可以通过逆向动力学函数按下式计算：

$$C' = \text{ID}(q, \dot{q}, Q^T \begin{bmatrix} 0 \\ \ddot{q}_2 \end{bmatrix}).$$

例 9.1 式 9.2 把 $\ddot{q}_2$ 隐藏在 C' 之中。若想使 $\ddot{q}_2$ 显式可见，则该方程可以写成

$$\begin{bmatrix} H_{11} & 0 \\ H_{21} & -1 \end{bmatrix} \begin{bmatrix} \ddot{q}_1 \\ \tau_2 \end{bmatrix} = \begin{bmatrix} 1 & -H_{12} \\ 0 & -H_{22} \end{bmatrix} \begin{bmatrix} \tau_1 \\ \ddot{q}_2 \end{bmatrix} - \begin{bmatrix} C_1 \\ C_2 \end{bmatrix}.$$

将该方程左端的系数矩阵求逆并左乘后，即可得到 Jain 与 Rodriguez (1993) 的式 10。

9.1.2 算法 (Algorithms)

式 9.2 可以按如下四步求解。

校对标注：下方四步求解过程原以公式形式给出 (LaTeX 已损坏)，现以文本形式保留。

1. 利用式 (9.2) 计算 C ；
2. 计算 H_{δ} (可用组合刚体算法求得 H 后提取子矩阵 H_{11})；
3. 求解 $H_{\delta} \cdot \ddot{q} = \tau - C_{\delta}$ ，得到 $\ddot{q}$ ；
4. 通过 $\tau = C + ID_{\delta}(\cdot)$ 计算 τ 。

第 4 步中的符号 ID_{δ} 表示表 6.1 中所描述的微分逆向动力学函数 (differential inverse-dynamics function)，它将其自变量乘以 H 。因此，第 4 步中的方程可化为

$$\tau = C' + Q^T \left[\begin{array}{c} H_{11} \ddot{q}_1 \\ H_{21} \ddot{q}_1 \end{array} \right]'$$

完成第 2 步最简单的方法是借助组合刚体算法 (composite rigid body algorithm, §6.2) 计算 H ，然后提取子矩阵 H_{11} 。然而，如果需要更高的效率，则可以对算法加以修改，使其只执行获得 H_{11} 所需的最少计算量。修改后的算法如表 9.1 所示。该算法中出现了一个新量： $v(fd)$ ，即由至少一个正向动力学关节所支撑的刚体集合。它由以下公式定义

$$v(fd) = \bigcup_{i \in fd} v(i),$$

并且可以由 fd 与 λ 按下法计算：1

```

 $v(fd) = fd$ 
for  $i = 1$  to  $N_B$  do
  if  $\lambda(i) \in v(fd)$  then
     $v(fd) = v(fd) \cup \{i\}$ 
  end
end

```

计算 $v(fd)$ 的代价很小，因此混合动力学例程每次被调用时都实际计算一次 $v(fd)$ 是可行的。不过，由于 $v(fd)$ 只依赖于 fd 与 λ ，其值在一次仿真运行过程中通常不会改变；因此也可以在运行开始时仅计算一次，并存储起来供以后使用。

将修改后的算法与表 6.2 中的原始算法相比较可以看出，二者结构相同，但修改后的算法只执行其中的一部分计算。例如， I_i^c 只对满足 $i \in v(fd)$ 的那些 i 值进行计算，而 H_{ij} 只对满足 $i, j \in fd$ 的那些 i 与 j 值进行计算。

图 9.1： H_{11} 中由分支结构引起的稀疏性

对满足 $i \in v(fd)$ 的那些 i 值进行计算，而 H_{ij} 只对满足 $i, j \in fd$ 的那些 i 与 j 值进行计算。

再来看第 3 步。 H_{11} 是一个对称正定矩阵，因此第 3 步中的方程可以用任何针对此类矩阵的标准线性方程求解器来求解。然而，如图 9.1 所示， H_{11} 继承了 H 中一部分由分支结构引起的稀疏性。因此，我们有机会利用第 6.5 节所述的某种算法来开发这种稀疏性。

开发 H_{11} 中稀疏性的前提，是获得一个描述该矩阵稀疏性模式的修改后的父数组（parent array） λ' 。图 9.1 展示了如何做到这一点。从原系统的连通图（connectivity graph）出发，把每一对由逆向动力学关节相连的节点合并起来，即可构造出修改后的图。合并完成后，剩余的节点必须重新编号，因为节点数目已经减少了。在图中所示的例子中，关节 1 和 2 是逆向动力学关节，因此节点 1 与 2 并入节点 0，节点 3 至 7 重新编号为 1 至 5。

图 9.1 还展示了一个从 λ 与 fd 计算 λ' 的算法。作为副产品，该算法还计算了从原图到修改图的节点编号映射： $map(i)$ 是原节点 i 的新编号。

要理解该算法，最好先知道： i 在原图中对节点编号计数，而 j 在修改图中对节点编号计数。请注意，如果修改图中的任何关节具有一个以上的自由度，那么 λ' 在使用之前必须像第 6.5 节所解释的那样进行展开。

§9.2

铰接体混合动力学（Articulated-Body Hybrid Dynamics）

铰接体算法（articulated-body algorithm）很容易改编为执行混合动力学：只需对算法做如下修改——在每个正向动力学关节处使用正常的铰接体方程（如 §7.3 所述），而在每个逆向动力学关节处使用下文推导的方程。两组方程在表 9.2 中并排展示，伪代码则如表 9.3 所示。后者改编自表 7.1 中的代码。

设 f_i 为穿过关节 i 传递的空间力（spatial force）。 f_i 与刚体 i 的加速度之间的关系为

$$f_i = I_i^A a_i + p_i^A;$$

它还与刚体 $\lambda(i)$ 的加速度存在如下关系

$$f_i = I_i^a a_{\lambda(i)} + p_i^a.$$

（参见式 7.25。）此外， $a_{\lambda(i)}$ 与 a_i 的关系为

$$a_i = a_{\lambda(i)} + \dot{S}_i \dot{q}_i + S_i \ddot{q}_i.$$

在铰接体算法中，主要步骤是从 I_i^A 和 p_i^A 计算 I_i^a 与 p_i^a 。若关节 i 是正向动力学关节，那么首要的问题在于 $\ddot{q}_i$ 未知，所以第一步是为 $\ddot{q}_i$ 找到一个表达式。然而，对于逆向动力学关节，并不存在这样的困难。把式 9.8 代入式 9.6，可得

$$f_i = I_i^A (a_{\lambda(i)} + \dot{S}_i \dot{q}_i + S_i \ddot{q}_i) + p_i^A;$$

将此方程与式 9.7 比较可知

$$I_i^a = I_i^A$$

以及

$$p_i^a = p_i^A + I_i^A (\dot{S}_i \dot{q}_i + S_i \ddot{q}_i).$$

这两个方程取代了第二遍 (Pass 2) 中的式 7.39 与 7.40。除此之外, 表 9.2 的逆向动力学方程中还可以看到另外两处变化: 项 $S_i \ddot{q}_i$ 被加到了 c_i 上 (这意味着 a'_i 不再出现在第三遍中), 并且第三遍中新增了一个方程, 通过 $\tau_i = S_i^T f_i$ 与式 9.6 来计算 τ_i 。(我们本来也可以使用式 9.7。)

表 9.2: 铰接体混合动力学方程

表 9.2 的两组方程 (原稿 LaTeX 已损坏, 以下为修复后的关键方程)。

逆向动力学关节——第二遍: $I_i^a = I_i^A, \quad p_i^a = p_i^A + I_i^A (\dot{S}_i \dot{q}_i + S_i \ddot{q}_i),$

逆向动力学关节——第三遍: $a_i = {}^i X_{\lambda(i)} a_{\lambda(i)} + c_i, \quad \tau_i = S_i^T (I_i^A a_i + p_i^A).$

:

$a_0 = -a_g$

for $i = 1$ to N_B do

if $i \in fd$ then

$a' = {}^i X_{\lambda(i)} a_{\lambda(i)} + c_i$

$\ddot{q}_i = D_i^{-1} (u_i - U_i^T a')$

$a_i = a' + S_i \ddot{q}_i$

else

$a_i = {}^i X_{\lambda(i)} a_{\lambda(i)} + c_i$

$\tau_i = S_i^T (I_i^A a_i + p_i^A)$

end

end

$v_0 = 0$

for $i = 1$ to N_B do

$[X_J, S_i, v_J, c_J] = \text{jcalc}(\text{jtype}(i), q_i, \dot{q}_i)$

${}^i X_{\lambda(i)} = X_J X_T(i)$

if $\lambda(i) \neq 0$ then

${}^i X_0 = {}^i X_{\lambda(i)} \lambda(i) X_0$

end

$v_i = {}^i X_{\lambda(i)} v_{\lambda(i)} + v_J$

if $i \in fd$ then

$c_i = c_J + v_i \times v_J$

else

$c_i = c_J + v_i \times v_J + S_i \dot{q}_i$

end

$I_i^A = I_i$

$p_i^A = v_i \times {}^* I_i v_i - {}^i X_0^* f_i^x$

end

for $i = N_B$ to 1 do

if $i \in fd$ then

```

 $U_i = I_i^A S_i$ 
 $D_i = S_i^T U_i$ 
 $u_i = \tau_i - S_i^T p_i^A$ 
if  $\lambda(i) \neq 0$  then
 $I^a = I_i^A - U_i D_i^{-1} U_i^T$ 
 $I_{\lambda(i)}^A = I_{\lambda(i)}^A + \lambda(i) X_i^* I^a X_{\lambda(i)}$ 
 $p^a = p_i^A + I^a c_i + U_i D_i^{-1} u_i$ 
 $p_{\lambda(i)}^A = p_{\lambda(i)}^A + \lambda(i) X_i^* p^a$ 
end
else
else
if  $\lambda(i) \neq 0$  then
 $I^a = I_i^A$ 
 $I_{\lambda(i)}^A = I_{\lambda(i)}^A + \lambda(i) X_i^* I^a X_{\lambda(i)}$ 
 $p^a = p_i^A + I^a c_i$ 
 $p_{\lambda(i)}^A = p_{\lambda(i)}^A + \lambda(i) X_i^* p^a$ 
end
end
end
end

```

§ 9.3 浮动基座 (Floating Bases)

浮动基座刚体系统是指基座刚体（刚体 0）可以自由运动而非固定于空间的系统。任何浮动基座系统都可以通过以下改动转换为等价的固定基座系统：

把所有刚体与关节的编号都加 1，使浮动基座成为刚体 1，编号最小的关节成为关节 2。

向系统添加一个固定基座，并在固定基座与浮动基座之间连接一个新的六自由度关节；前者为新的刚体 0，后者为新的关节 1。

作为第 2 步的结果， N_B 与 N_J 各加 1， n 加 6。

完成这些改动之后，就可以用任何为固定基座系统设计的技巧与算法来分析并计算浮动基座系统的动力学。尽管如此，浮动基座系统仍然是一个重要的特殊情形，也确实值得专门处理。

下面我们来推导浮动基座运动学树的运动方程。最快的途径是从等价固定基座系统的运动方程入手。该方程可以写成

$$\begin{bmatrix} H_{11} & H_{1*} \\ H_{*1} & H_{**} \end{bmatrix} \begin{bmatrix} \ddot{q}_1 \\ \ddot{q}_* \end{bmatrix} + \begin{bmatrix} C_1 \\ C_* \end{bmatrix} = \begin{bmatrix} \tau_1 \\ \tau_* \end{bmatrix},$$

其中下标 1 与 * 分别指关节 1 与“所有其他关节”。这个方程已经是浮动基座系统正确的运动方程——只需要把它整理成更合适的记号，这正是我们接下来要做的。

关节 1 的特殊性质在于它有六个自由度。因此, 它的运动子空间矩阵 (motion-subspace matrix) S_1 必须是 6×6 满秩矩阵。事实上, S_1 可以是任意 6×6 满秩矩阵, 因为对 S_1 的两种不同选取只相当于对速度变量的两种不同选取, 而不会引起方程的根本性变化。因此我们选取最方便的值, 即单位矩阵。特别地, 我们选取 S_1 为浮动基座坐标下的单位矩阵。²

确定了 $S_1 = \mathbf{1}_{6 \times 6}$ 之后, 若干其他量也取特殊值。先看 $\dot{\mathbf{q}}_1$ 、 $\ddot{\mathbf{q}}_1$ 与 $\boldsymbol{\tau}_1$, 有

$$\begin{aligned} \mathbf{v}_1 &= S_1 \dot{\mathbf{q}}_1 & \mathbf{a}_1 &= \dot{S}_1 \dot{\mathbf{q}}_1 + S_1 \ddot{\mathbf{q}}_1 & \text{and} & & \boldsymbol{\tau}_1 &= S_1^T \mathbf{f}_1 \\ &= \mathbf{1}_{6 \times 6} \dot{\mathbf{q}}_1 & &= \mathbf{v}_1 \times S_1 \dot{\mathbf{q}}_1 + S_1 \ddot{\mathbf{q}}_1 & & & &= \mathbf{1}_{6 \times 6} \mathbf{f}_1 \\ &= \dot{\mathbf{q}}_1, & &= \mathbf{v}_1 \times \mathbf{v}_1 + \mathbf{1}_{6 \times 6} \ddot{\mathbf{q}}_1 & & & &= \mathbf{f}_1. \\ & & & & & & &= \ddot{\mathbf{q}}_1 \end{aligned}$$

于是, $\dot{\mathbf{q}}_1$ 、 $\ddot{\mathbf{q}}_1$ 与 $\boldsymbol{\tau}_1$ 分别包含空间向量 $\mathbf{v}_1$ 、 $\mathbf{a}_1$ 与 $\mathbf{f}_1$ 的普吕克坐标 (Plücker coordinates)。接下来我们注意到, $\mathbf{f}_1$ 与 $\mathbf{f}_1^x$ 的意义基本相同, 因为它们都是作用在浮动基座上的外力; 因此总的外力实际上由和 $\mathbf{f}_1 + \mathbf{f}_1^x$ 给出。在下面的方程中, $\mathbf{f}_1$ (因而也包括 $\boldsymbol{\tau}_1$) 已被置为零; 但我们始终可以自由选择如何在外力 $\mathbf{f}_1^x$ 与 $\mathbf{f}_1$ 之间分配。

由于 $\mathbf{C}$ 是广义偏置力 (generalized bias force), 故 $\mathbf{C}_1$ 就是为使浮动基座处产生零关节加速度、同时假定所有其他关节加速度为零时 $\boldsymbol{\tau}_1$ 所必须取的值。鉴于 $\ddot{\mathbf{q}}_1 = \mathbf{a}_1$ 且 $\boldsymbol{\tau}_1 = \mathbf{f}_1$, 这意味着 $\mathbf{C}_1$ 也是由整个浮动基座系统构成的组合刚体 (composite rigid body) 的空间偏置力。我们将用符号 p_1^c 表示这个量。最后, 由式 6.14 中 H_{ij} 的定义, 有

$$H_{1i} = S_1^T I_i^c S_i = I_i^c S_i, \quad H_{11} = S_1^T I_1^c S_1 = I_1^c.$$

把这些变化考虑进去, 式 9.11 可以写成

$$\begin{bmatrix} I_1^c & H_{1*} \\ H_{*1} & H_{**} \end{bmatrix} \begin{bmatrix} a_1 \\ \ddot{\mathbf{q}}_* \end{bmatrix} + \begin{bmatrix} p_1^c \\ C_* \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \boldsymbol{\tau}_* \end{bmatrix}.$$

现在回到原来的浮动基座系统, 并相应调整记号。首先, N_B 、 N_J 与 n 恢复其原来的数值, 刚体与关节也恢复原来的编号。因此符号 a_1 、 p_1^c 、 I_1^c 等分别变成 a_0 、 p_0^c 、 I_0^c 等。此外, 由于六自由度关节 1 已不复存在, 符号 H_{**} 、 C_* 、 $\ddot{\mathbf{q}}_*$ 与 $\boldsymbol{\tau}_*$ 分别简化为 H 、 C 、 $\ddot{\mathbf{q}}$ 与 $\boldsymbol{\tau}$ 。我们把 H_{1*} 替换成一个新的符号 F , 其定义如下:

$$F = [F_1 F_2 \cdots F_{N_B}]$$

其中

$$F_i = I_i^c S_i.$$

F 是一个 $6 \times n$ 矩阵, 其各列是在浮动基座处为支撑各关节变量的单位加速度所需的空间力 (参见 §6.3)。经过这些改动, 浮动基座系统的运动方程现在写为

$$\begin{bmatrix} I_0^c & F \\ F^T & H \end{bmatrix} \begin{bmatrix} a_0 \\ \ddot{\mathbf{q}} \end{bmatrix} + \begin{bmatrix} p_0^c \\ C \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \boldsymbol{\tau} \end{bmatrix}.$$

这是一个含 $n+6$ 个方程、 $n+6$ 个未知量的方程组，反映出浮动基座系统具有 $n+6$ 个运动自由度。注意，对该系统加速度的完整描述需要两个向量 $\mathbf{a}_0$ 与 $\dot{\mathbf{q}}$ 。同样地，对速度的完整描述需要 $\mathbf{v}_0$ 与 $\dot{\mathbf{q}}$ ，而对位置的完整描述需要量 ${}^0\mathbf{X}_{ref}$ 与 $\mathbf{q}$ ，其中 ${}^0\mathbf{X}_{ref}$ 是从惯性参考系到浮动基座坐标系的普吕克变换（Plücker transform）。

例 9.2 从式 9.13 出发，假设我们从第二行减去 $F^T(I_0^\xi)^{-1}$ 倍的第一行。得到的方程为

$$\begin{bmatrix} I_0^\xi & F \\ 0 & H - F^T(I_0^\xi)^{-1}F \end{bmatrix} \begin{bmatrix} \mathbf{a}_0 \\ \ddot{\mathbf{q}} \end{bmatrix} + [C - F^T(I_0^\xi)^{-1}p_0^\xi] = \begin{bmatrix} 0 \\ \tau \end{bmatrix}.$$

取出最下面一行，我们得到

$$H^{fl} \ddot{\mathbf{q}} + C^{fl} = \tau,$$

其中

$$H^{fl} = H - F^T(I_0^\xi)^{-1}F$$

以及

$$C^{fl} = C - F^T(I_0^\xi)^{-1}p_0^\xi.$$

式 9.14 之所以有趣，是因为它给出了 $\ddot{\mathbf{q}}$ 与 τ 之间的直接关系，并且系数 $\mathbf{H}^{fl}$ 和 $\mathbf{C}^{fl}$ 可以看作标准运动方程系数的浮动基座对应物。然而，该方程不一定是计算上的好选择，因为在式 9.15 中从 $\mathbf{H}$ 中减去 $F^T(I_0^\xi)^{-1}F$ 会抹掉 $\mathbf{H}$ 中可能存在的、由分支结构引入的稀疏性。因此， $\mathbf{H}^{fl}$ 是一个稠密矩阵；尽管 $\mathbf{H}^{fl}$ 比 $\mathbf{H}$ 小，但它完全可能拥有比 $\mathbf{H}$ 更多的非零元素。

§9.4

浮动基座正向动力学（Floating-Base Forward Dynamics）

如果有现成软件可以计算固定基座动力学（fixed-base dynamics），并且该软件支持六自由度（6-DoF）关节，那么就可以用它来计算浮动基座动力学。或者，如果需要专用的浮动基座软件，那么铰接体算法（articulated-body algorithm）和复合刚体算法（composite-rigid-body algorithm）都可以改造来胜任这一任务。

表 9.4 给出了浮动基座铰接体算法的伪代码。将该算法与表 7.1 中的算法进行比较，可以看到以下区别： $\mathbf{v}_0$ 现在是一个输入；第二遍扫描被扩展为计算 I_0^A 和 p_0^A ； $\mathbf{a}_0$ 由如下方程计算

$$I_0^A \mathbf{a}_0 + p_0^A = 0;$$

并且引力加速度在最后直接加到 $\mathbf{a}_0$ 上。该实现期望向量 f_i^x 和 $\mathbf{a}_g$ 以浮动基座坐标系提供，而非参考坐标系，因此无需知道 ${}^0\mathbf{X}_{ref}$ 的值。相应地使用了符号 ${}^0f_i^x$ 和 ${}^0\mathbf{a}_g$ 。注意， ${}^0\mathbf{a}_g$ 不是常数，而会随浮动基座的转动而变化。（ ${}^0\mathbf{a}_g = {}^0\mathbf{X}_{ref}^{ref} \mathbf{a}_g$ 。）

均匀引力场对自由浮动刚体系统的作用，是以相同的量修改系统中每个刚体的

Table 9.4: 浮动基座铰接体动力学算法

加速度。因此，任意两个刚体之间的相对加速度与重力无关。表 9.4 中的算法正是利用了这一事实：它先计算系统在无引力场情况下的运动，然后在最后把 a_g 加到 a_0 上。如果你希望该算法返回系统中其他刚体的加速度，那么在这些加速度被返回之前，也必须把 a_g 加到它们上。

表 9.5 给出了递归牛顿-欧拉算法 (recursive Newton-Euler algorithm) 和复合刚体算法的浮动基座版本。两者共同计算式 9.13 的系数，从而可以解出 a_0 和 $\ddot{q}$ 。注意，该方程中的系数矩阵具有与式 9.11 中矩阵相同的由分支结构引入的稀疏模式，因此可以用 6.5 节中的算法进行分解。

将修改后的牛顿-欧拉算法与表 5.1 中的原算法进行比较，可以看到以下区别： v_0 现在是一个输入； a_i 表达式中的 $S_i \ddot{q}_i$ 项被移除，该表达式相应地更名为 a_i^{vp} ，因为它只包含速度积加速度 (velocity-product acceleration) 项；第二遍扫描被扩展为计算 f_0 ；并增加了一行来设置 $p_0^c = f_0$ 。同样，向量 a_g 和 f_i^x 以浮动基座坐标系提供，因此不需要 ${}^0X_{\text{ref}}$ 的值，伪代码中相应地使用了符号 0a_g 和 ${}^0f_i^x$ 。在一开始令 $a_0^{vp} = -a_g$ 会使引力项出现在 C 和 p_0^c 中，最终的效果是：当解式 9.13 时，正确的引力加速度项会出现在 a_0 中。

在将修改后的复合刚体算法与表 6.2 中的原始算法进行比较时，可以看到以下区别：

9.4.1 计算 C 与 p_0^c ：

校对标注：下方伪代码原以公式形式给出 (LaTeX 已损坏)，现以文本形式保留其内容。

- 计算 a_i 相关的量；
- for $i = 1$ to N_B : 由 jcalc 计算关节变换及速度/加速度量；
- 若 $\lambda(i) \neq 0$ ，则进行相应坐标变换。

计算 H 、 F 与 I_0^c ：

```

$H = 0$
for $i = 0$ to $N_B$ do
    $I_i^c = I_i$
end
for $i = N_B$ to $1$ do
    $I_{\{\lambda(i)\}}^c = I_{\{\lambda(i)\}}^c + \{\}^{\{\lambda(i)\}}X_i^* I_i^c \{\}^iX_{\{\lambda(i)\}}$
    $F_i = I_i^c S_i$
    $H_{\{ii\}} = S_i^T F_i$
    $j = i$
    while $\lambda(j) \neq 0$ do
        $F_i = \{\}^{\lambda(j)}X_j^* F_i$
        $j = \lambda(j)$
        $H_{\{ij\}} = F_i^T S_j$
        $H_{\{ji\}} = H_{\{ij\}}^T$
    end
    $F_i = \{\}^0X_j^* F_i$
end
end

```

计算 I_i^c 的过程已扩展为包含 I_0^c ；原来的局部变量 F （不可与式 9.12 中新出现的 $6 \times n$ 矩阵 F 混淆）已被变量 F_i 取代， F_i 正是式 9.12 中矩阵 F 的分块列（block column）；并且在该 while 循环之后增加了一行，将每个 F_i 变换到浮动基座坐标系。（每个 F_i 最初以刚体 i 的坐标系表示，随后被逐步变换到浮动基座坐标系。）

这些表中的算法并不是解决该问题的唯一途径。其他一些方法的示例可参见 Hooker and Margulies (1965)；Hooker (1970)；Roberson and Schwertassek (1988)；Wittenburg (1977)。

§9.5

浮动基座逆向动力学 (Floating-Base Inverse Dynamics)

浮动基座系统的逆向动力学问题实际上是一个混合动力学（hybrid dynamics）问题：关节加速度已知，而基座加速度未知。因此，可以用 9.1 节和 9.2 节中的通用混合动力学算法之一来求解该问题。或者，我们也可以设计一种专用算法，它能更高效地求解该问题。本节就介绍这样一种算法。

令 f_i 为刚体 i 的父刚体作用在它上面的空间力（spatial force）。对于刚体 0 的情况，由于该刚体没有父刚体，因此 $f_0 = 0$ 。在所有其他情况下， f_i 是穿过关节 i 传递的力。在任何运动学树（kinematic tree）中， f_i 支撑着集合 $v(i)$ 中所有刚体的运动，因此我们有

$$f_i = \sum_{j \in v(i)} (I_j a_j + v_j \times^* I_j v_j - f_j^x).$$

我们的目标是解此方程求出 a_0 。为此，我们先把 a_i 表示为

$$a_i = a_0 + a_i^r,$$

其中 a_i^r 是刚体 i 的相对加速度，然后把这个表达式代入式 9.18，得到

$$f_i = \left(\sum_{j \in v(i)} I_j \right) a_0 + \sum_{j \in v(i)} (I_j a_j^r + v_j \times^* I_j v_j - f_j^x).$$

该方程中 a_0 的系数就是复合刚体惯量（composite-rigid-body inertia） I_i^c 。如果我们把等式右边的其余部分视为偏置力（bias force），那么

$$f_i = I_i^c a_0 + p_i^c,$$

其中

$$p_i^c = \sum_{j \in v(i)} (I_j a_j^r + v_j \times^* I_j v_j - f_j^x).$$

p_i^c 是当 a_0 恰好为零时，支撑 $v(i)$ 中所有刚体运动所需的力。它可以用如下方程递归计算

$$p_i^c = p_i + \sum_{j \in \mu(i)} p_j^c$$

其中

$$p_i = I_i a_i^r + v_i \times^* I_i v_i - f_i^x.$$

当 $i = 0$ 时, 式 9.20 可以直接解出 a_0 , 得到

$$a_0 = -(I_0^c)^{-1} p_0^c.$$

一旦 a_0 已知, 就可以用下式计算关节力

$$\tau_i = S_i^T f_i = S_i^T (I_i^c a_0 + p_i^c).$$

表 9.6 给出了基于式 9.22 至式 9.25, 以及刚体速度、刚体加速度和复合刚体惯量常用方程的算法及其伪代码。这些方程以刚体坐标系表示, 并在必要之处包含坐标变换。与其他浮动基座算法一致, 该算法期望 a_g 和 f_i^x 以

Table 9.6: 浮动基座逆向动力学方程与算法

浮动基座坐标系提供, 因此相应地使用了符号 ${}^0 a_g$ 和 ${}^0 f_i^x$ 。该算法通过把 a_0^r 初始化为 $-a_g$ 而非零来处理重力。这会使引力项出现在 p_i^c 中, 进而使正确的引力加速度出现在 a_0 中。另一种做法是, 在第一遍扫描开始时把 a_0^r 设为零, 然后在第三遍扫描结束时把 a_g 加到 a_0 上。

关于这一主题及相关主题的更多内容, 可参见 Dubowsky and Papadopoulos (1993); Fang and Pollard (2003); Longman et al. (1987); Umetani and Yoshida (1989); Vafa and Dubowsky (1990a,b) 以及许多其他论文。

例 9.3 刚体系统的总空间动量 (spatial momentum) 是其各刚体动量之和。因此, 浮动基座系统的动量由下式给出

$$= \sum_{i=0}^{N_B} I_i v_i.$$

在没有外力的情况下, 这个量是守恒的。事实证明, 动量也可以用表达式 $I_0^c v_0 + F \dot{q}$ 给出。我们可以如下证明:

$$\begin{aligned} \sum_{i=0}^{N_B} I_i v_i &= \sum_{i=0}^{N_B} I_i \left(v_0 + \sum_{j \in \kappa(i)} S_j \dot{q}_j \right) \\ &= \sum_{i=0}^{N_B} I_i v_0 + \sum_{j=1}^{N_B} \sum_{i \in \nu(j)} I_i S_j \dot{q}_j \\ &= I_0^c v_0 + \sum_{j=1}^{N_B} I_j^c S_j \dot{q}_j = I_0^c v_0 + F \dot{q}. \end{aligned}$$

(关于对 $\nu(j)$ 的求和, 参见式 4.3。)用类似的论证, 我们可以证明浮动基座系统的动能为

$$T = \frac{1}{2} \sum_{i=0}^{N_B} v_i^T I_i v_i = \frac{1}{2} \begin{bmatrix} v_0^T & \dot{q}^T \end{bmatrix} \begin{bmatrix} I_0^c & F \\ F^T & H \end{bmatrix} \begin{bmatrix} v_0 \\ \dot{q} \end{bmatrix}.$$

例 9.4 在固定基座系统中, 刚体 i 的加速度由 $a_i = J_i \ddot{q} + \dot{J}_i \dot{q}$ 给出, 其中 J_i 是刚体 i 的雅可比矩阵 (Jacobian)。在浮动基座系统中, 这个表达式变为

$$a_i = J_i \ddot{q} + \dot{J}_i \dot{q} + a_0.$$

然而, 由式 9.13 可得

$$I_0^c a_0 + F \ddot{q} + p_0^c = 0.$$

将这两个方程合在一起, 得到

$$\begin{aligned} a_i &= J_i \ddot{q} + \dot{J}_i \dot{q} - (I_0^c)^{-1} (F \ddot{q} + p_0^c) \\ &= J_i^{fl} \ddot{q} + \dot{J}_i \dot{q} - (I_0^c)^{-1} p_0^c, \end{aligned}$$

其中

$$J_i^{fl} = J_i - (I_0^c)^{-1} F.$$

量 J_i^{fl} 实际上是一个浮动基座雅可比矩阵, 因为它描述了 a_i 对 $\ddot{q}$ 的线性依赖关系 (参见 Umetani and Yoshida (1989)). 这个矩阵还把 $\dot{q}$ 的变化与 v_i 的变化联系起来。例如, 如果系统内部的冲量使 $\dot{q}$ 变化了 $\delta \dot{q}$, 那么 v_i 随之发生的变化为

$$\delta v_i = J_i^{fl} \delta \dot{q}.$$

§9.6 齿轮 (Gears)

许多机械系统包含齿轮副 (gear pair), 或功能类似的部件。从数学角度看, 齿轮副是两个关节变量之间的一个代数

Figure 9.2: 由电机和齿轮驱动的两连杆机械臂

方程。通常, 其中一个变量是另一个变量的常数倍, 但非线性关系也是可能的。另一种可能是涉及两个以上的变量。例如, 差速齿轮箱 (differential gear box) 通常有两个输出和一个或两个输入, 因此总共涉及三个或四个变量。

可以采用与 8.11 节处理运动学回路 (kinematic loop) 相同的技术, 把齿轮约束纳入运动学树; 即选取一个独立关节变量向量 y , 并定义一个函数 γ , 把 y 映射为 q 。下面我们借助一个例子来研究这一技术。

图 9.2 展示了一个简单的两连杆机械臂, 其两个关节都由电机通过齿轮副驱动。该系统包含以下部件: 一个固定基座、一个上臂、一个前臂、两个电机、两个大齿轮和两个小齿轮。每个电机由两部分组成: 一个定子 (stator) 和一个转子 (rotor)。两个定子分别固定在基座和上臂上, 两个转子分别固定在各自的小齿轮上。两个大齿轮分别固定在上臂和前臂上。因此, 该系统共包含四个运动刚体, 定义如下:

- 刚体 1: 上臂、第一个大齿轮以及第二个电机的定子;
- 刚体 2: 第一个小齿轮以及第一个电机的转子;
- 刚体 3: 前臂以及第二个大齿轮;

刚体 4: 第二个小齿轮以及第二个电机的转子。

因此, 关节 1 和关节 3 分别是机械臂的肩关节和肘关节, 而关节 2 和关节 4 是电机内部的轴承, 它们允许转子相对于定子转动。两个齿轮副对关节变量施加如下约束:

$$q_2 = \rho_1 q_1 \text{ and } q_4 = \rho_2 q_3,$$

其中 ρ_1 和 ρ_2 是传动比 (gear ratio)。图 9.2 还展示了该系统的连接图 (connectivity graph), 以及各关节之间的齿轮关系。箭头仅用于表示所标传动比的作用方向。因此, 标有 ρ_1 的箭头从关节 1 指向关节 2, 因为 q_2 是 q_1 的 ρ_1 倍。

为了把齿轮约束纳入运动方程, 我们首先定义一个独立变量向量 $\mathbf{y}$, 然后定义一个函数 γ , 它把 $\mathbf{y}$ 映射为 $\mathbf{q}$ 。让我们选择 q_1 和 q_3 作为独立变量。于是 $\mathbf{y}$ 是向量 $[y_1 \ y_2]^T$, 其中 $y_1 = q_1$, $y_2 = q_3$ 。按此变量选择, γ 定义如下:

$$\gamma(\mathbf{y}) = \begin{bmatrix} y_1 \\ \rho_1 y_1 \\ y_2 \\ \rho_2 y_2 \end{bmatrix}.$$

下一步是从 γ 推导量 $G = \partial\gamma/\partial\mathbf{y}$ 和 $\mathbf{g} = \dot{G}\mathbf{y}$ 。对于我们的例子, 这些量为

$$G = \frac{\partial\gamma}{\partial\mathbf{y}} = \begin{bmatrix} 1 & 0 \\ \rho_1 & 0 \\ 0 & 1 \\ 0 & \rho_2 \end{bmatrix} \text{ and } \mathbf{g} = \dot{G}\mathbf{y} = 0.$$

(只有当系统中存在非线性齿轮副时, $\mathbf{g}$ 才会非零。) 最后, 考虑齿轮约束的运动学树运动方程为

$$G^T H G \ddot{\mathbf{y}} + G^T (C + H\mathbf{g}) = G^T \boldsymbol{\tau},$$

其中 H 和 C 是在没有齿轮约束情况下的运动方程系数。(参见式 8.45。) 由于电机绕关节 2 和关节 4 施加力矩, $\boldsymbol{\tau}$ 中的元素 τ_2 和 τ_4 将包含电机力矩, 而 $\tau_1 = \tau_3 = 0$ 。带齿轮的运动学树的逆向动力学可以用 8.12 节所述方法计算。

G 通常是一个非常稀疏的矩阵。事实上, 它每一行通常只含一个非零元素, 而且其中许多元素为 1。因此建议利用这种稀疏性。举例来说, 如果我们把 G 和 H 当作稠密矩阵处理, 那么对一个 4×2 的矩阵 G 计算 $G^T H G$ 的代价是 44 次乘法和 33 次加法。相比之下, 如果对上述例子使用 G 和 H 的实际值, 则代价仅为 5 次乘法和 3 次加法, 如下所示。显然, 矩阵越大, 节省的开销就越多。

$$\begin{aligned} G^T H G &= \begin{bmatrix} 1 & \rho_1 & 0 & 0 \\ 0 & 0 & 1 & \rho_2 \end{bmatrix} \begin{bmatrix} H_{11} & 0 & H_{13} & H_{14} \\ 0 & H_{22} & 0 & 0 \\ H_{13} & 0 & H_{33} & 0 \\ H_{14} & 0 & 0 & H_{44} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \rho_1 & 0 \\ 0 & 1 \\ 0 & \rho_2 \end{bmatrix} \\ &= \begin{bmatrix} H_{11} + \rho_1^2 H_{22} & H_{13} + \rho_1 \rho_2 H_{14} \\ H_{13} + \rho_2 H_{14} & H_{33} + \rho_2^2 H_{44} \end{bmatrix}. \end{aligned}$$

例 9.5 在典型的机器人机构中，电机的转子比机构的其他部件小得多，因此只占总质量的很小一部分。然而，它们的转速也比它们所驱动的刚体高得多，因此能对整个系统的动力学产生显著影响。

本节介绍的方法是计入转子动力学效应的正确方式，但机器人学中还有一种广受欢迎的简易（quick-and-dirty）方法。按这种方法，我们舍弃转子和齿轮箱，把执行器（actuator）建模为一个虚构的电机：它产生真电机 ρ 倍的力矩，以真电机 $1/\rho$ 倍的速度运转，并具有真电机 ρ^2 倍的转子惯量，其中 ρ 是减速传动比（step-down gear ratio）。这些惯量是标量常数，直接加到 $\mathbf{H}$ 的对角元素上。特别地，如果 I_i^{rot} 是驱动关节 i 的电机的缩放后转子惯量，那么 I_i^{rot} 就被加到 $\mathbf{H}_{ii}$ 上。为使这一方法成立，关节 i 必须是单自由度（1-DoF）关节，这样 $\mathbf{H}_{ii}$ 才是 1×1 矩阵。

这种方法捕捉了转子最大的惯性效应，但忽略了较小的惯性效应以及所有的陀螺效应（gyroscopic effect）。这种方法为何有效，可以从式 9.27 中窥见端倪。如果转子很小，那么与 H_{11} 、 H_{13} 和 H_{33} 相比， H_{22} 、 H_{14} 和 H_{44} 将很小，但 ρ_1 和 ρ_2 会适度地大。（100:1 的传动比相当典型。）因此，两个转子对两个对角元素的影响最大，因为在那里它们被乘以传动比的平方。

这种简易方法也适用于铰接体算法。在这种情况下，我们把式 7.44 修改为

$$D_i = S_i^T U_i + I_i^{rot}.$$

同样，关节 i 必须是单自由度关节，这样 D_i 才是 1×1 矩阵。

§9.7

动力学等价（Dynamic Equivalence）

两个不同的刚体系统完全有可能具有相同的运动方程。我们称这样的系统为动力学等价（dynamically equivalent）。一个重要的特例是两个系统仅在惯性参数（inertia parameter）上不同。这类等价的一个直接推论是，刚体系统的惯性参数无法从其动力学行为的测量中辨识出来：人们只能辨识一个可观测的子集，称为基惯性参数（base inertia parameters, Khalil and Dombre, 2002）。另一个推论——也是本节的主题——是，可以在不改变系统行为的前提下调整系统模型的惯性参数。如果调整能使非零参数的个数减少，那么计算动力学的开销就可以降低。

Figure 9.3: 一个两刚体浮动刚体系统

考虑图 9.3 所示的刚体系统。它由两个刚体 B_1 和 B_2 组成，它们的惯量分别为 I_1 和 I_2 ，两者由一个运动子空间（motion subspace）为 $\mathbf{S}$ 的关节连接。一个空间力 $\mathbf{f}$ 作用在 B_1 上，一个广义力（generalized force） τ 作用在关节处。该系统的运动方程为

$$\begin{bmatrix} I_1 + X_J^T I_2 X_J & X_J^T I_2 S \\ S^T I_2 X_J & S^T I_2 S \end{bmatrix} \begin{bmatrix} a_1 \\ \ddot{q} \end{bmatrix} + \begin{bmatrix} C_f \\ C_\tau \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix},$$

其中

$$\begin{bmatrix} C_f \\ C_\tau \end{bmatrix} = \begin{bmatrix} v_1 \times^* I_1 v_1 + X_J^T (I_2 \dot{S} \dot{q} + v_2 \times^* I_2 v_2) \\ S^T (I_2 \dot{S} \dot{q} + v_2 \times^* I_2 v_2) \end{bmatrix}$$

且 $\mathbf{v}_2 = \mathbf{X}_J \mathbf{v}_1 + \mathbf{S} \dot{\mathbf{q}}$ 。除了记号上的差异，该方程不过是把式 9.13 应用到两刚体系统上。在构建该方程时，我们选择关节的前继坐标系 (predecessor frame) 和后继坐标系 (successor frame) 作为连杆坐标系，因此 ${}^2\mathbf{X}_1 = \mathbf{X}_J$ 。

现在我们修改这个系统：给 I_1 加上一个空间惯量 (spatial inertia) I_Δ ，并从 I_2 中减去 I_Δ 。如果我们希望这个修改不改变式 9.29 的系数，那么 I_Δ 必须满足以下条件：³

$$I_\Delta \mathbf{S} \equiv 0, I_\Delta \equiv \mathbf{X}_J^T I_\Delta \mathbf{X}_J \text{ and } \mathbf{S}^T \mathbf{v}_1 \times {}^* I_\Delta \mathbf{v}_1 \equiv 0.$$

这些条件必须在每个关节位形下、对每个 $\mathbf{v}_1 \in \mathbb{M}^6$ 都成立。因此可以推出，这些条件只取决于关节类型，而且在除了关节类型之外对系统一无所知的情况下，也能找到合适的 I_Δ 值。

Table 9.7: 不改变运动方程的 I_Δ 取值

表 9.7 给出了对某些关节类型满足式 9.30 中条件的 I_Δ 取值。让我们考察这些结果的物理意义。对于球关节 (spherical joint)， I_Δ 是位于关节旋转中心的一个质量为 m 的质点的惯量。这个结果表明，你可以分别在 B_1 和 B_2 上添加质量为 m 和 $-m$ 的质点，两者都位于旋转中心，由于这两个质点始终重合，因此它们恰好相互抵消。对于转动关节 (revolute joint)， I_Δ 是位于关节轴（即 z 轴）上不同位置的一对质点的惯量。 m 和 c_z 分别是这对质点的总质量和质心，而 I 取决于它们之间的距离。同样，由于这些质点位于转动轴上，如果我们把它们加到 B_1 上、把它们的相反量加到 B_2 上，那么它们始终重合，因此始终相互抵消。对于移动关节 (prismatic joint)， I_Δ 是一个无形的纯转动惯量。这类惯量具有平移不变性；因此，如果我们把 I_Δ 加到 B_1 上并从 B_2 中减去，那么只要 B_2 相对于 B_1 从不转动，两者就会始终相互抵消。

显然，从 B_2 中减去一个质量足够大的质点会导致负质量，减去一个足够大的纯转动惯量会导致负转动惯量。因此， B_2 最终可能得到一个物理上不可实现的刚体惯量。尽管如此，修改后系统的运动方程与原系统完全相同。

如果 B_1 和 B_2 是一个更大刚体系统的一部分，那么上述结果仍然适用——把一个两刚体子系统替换为动力学等价的子系统，不会改变整个系统的运动方程。

9.7.1 刚体系统的简化 (Simplification of Rigid-Body Systems)

表 9.7 的结果使动力学仿真器能够简化任意给定刚体系统的惯性参数：通过加减经过设计的 I_Δ 值，使尽可能多的惯性参数变为零。其过程如下：

```
for $i = N_B$ to $1$ do
  $I_{\Delta} = \mathrm{best\_match}(\mathrm{jtype}(i), I_i)$
  $I_i = I_i - I_{\Delta}$
  if $\lambda(i) \neq 0$ then
    $I_{\lambda(i)} = I_{\lambda(i)} + I_{\Delta}$
  end
end

for $i = N_B$ to $1$ do $I_\Delta = \mathrm{best\_match}(\mathrm{jtype}(i), I_i)$; $I_i = I_i - I_\Delta$; if $\lambda(i) \neq 0$ then $I_{\lambda(i)} = I_{\lambda(i)} + I_\Delta$; end; end
```

函数 best match (最佳匹配) 首先在 I_Δ 公式表中查找关节类型 $\mathrm{jtype}(i)$ 对应的条目。如果找不到条目，就返回值 $0_{6 \times 6}$ 。但如果找到了公式，就返回使表达式 $I_i - I_\Delta$ 中取零值的惯性参数个

数最大化的 I_{Δ} 值。例如，如果关节 i 是球关节，那么返回的 I_{Δ} 值与 I_i 质量相同，从而使 $I_i - I_{\Delta}$ 的质量为零。

这个过程的意义在于，它会产生一种可预测的零值参数模式，可以针对每种特殊情况编写代码来加以利用。⁴ 假设大多数关节是转动关节，这一技术可以把递归牛顿-欧拉算法的开销降低约 15

一个动力学算法在修改后的系统上可能失败，而在原系统上却可以成功。例如，如果算法使用了刚体惯量的逆，那么只有当这些逆存在时算法才能工作。如果修改已使刚体 i 的质量变为零，那么它的修改后惯量将是奇异的。

关于这一主题及相关主题的更多内容，可参见 Gautier and Khalil (1990); Khalil and Kleinfinger (1987); Khalil and Dombre (2002)。

9.7.2 增广刚体 (Augmented Bodies)

上述技术类似于一种更古老的技术，称为增广刚体 (augmented body)。两者都涉及惯性参数，并且目标相同：降低开销。不过，增广刚体技术依赖于对运动方程的重写，把惯性参数汇总为可以预先计算的常数子表达式。这些预先计算的值可以看作一组增广刚体的惯性参数。

从一个运动学树出发，增广刚体 i 是这样得到的：对于连接到刚体 i 上的每一个关节，向刚体 i 添加一个质点。这些质点的位置取决于关节，但每个质点的质量等于通过该关节连接到刚体 i 的那棵子树的质量。在增广刚体的一种变体中，只对 $\mu(i)$ 中的每个关节添加一个质点。前者往往用于浮动基座系统，后者用于固定基座系统。关于这一主题的更多内容，可参见 Balafoutis et al. (1988); Balafoutis and Patel (1989, 1991); Hooker and Margulies (1965); Hooker (1970); Roberson and Schwertassek (1988); Wittenburg (1977)。

精度与效率

从理论上讲，本书介绍的算法都是精确的。然而，计算过程中存在的舍入误差（round-off error）确保结果很少真正精确。其他误差来源包括数值积分的运用，以及刚体系统与其所代表的物理系统之间在行为上的差异。第 10.1 节考察了这些不同的误差来源，尤其关注动力学计算中舍入误差的来源以及避免它们的方法。

如果刚体系统的加速度会因作用力的微小变化而发生显著改变，则该系统被称为敏感的（sensitive）。如果你想精确地知道这样一个系统的加速度，那么作用力就必须被更精确地知晓。事实证明，许多运动学树（kinematic tree）都具有这种特性，这一现象将在第 10.2 节中考察。

第 10.3 节探讨了效率（efficiency）这一主题。它解释了如何衡量动力学算法的效率；并给出了若干张表格和曲线图，用以比较运动学树三种主要算法的多个版本的效率。复合刚体算法（composite-rigid-body algorithm）的效率被详细考察，部分是为了展示运算次数公式（operations-count formula）是如何得出的，部分是因为这种特定算法的效率会随树中分支的多寡而发生巨大变化。本节还表明，复合刚体算法的真实渐近复杂度是 $O(nd)$ ，其中 d 是连接树（connectivity tree）的深度，而所谓 $O(n^3)$ 算法的真实复杂度是 $O(nd^2)$ 。

基于模型的算法（model-based algorithm）的缺点之一是，它们必须兼顾一般情形。因此，它们会执行许多在一般情况下确实必需的运算，但对于某个特定的系统模型来说常常并非必要。这一问题的解决方案之一是自动生成定制化的动力学例程——即通用算法的精简版本，只执行对某个特定给定系统模型所必需的运算。第 10.4 节简要介绍了一种实现此目的的技术，称为符号化简（symbolic simplification）。

§ 10.1

误差来源

动力学计算受三类误差影响：舍入误差，它是使用有限精度浮点运算的结果；截断误差（truncation error），它是用有限项近似无穷级数的结果；以及建模误差（modelling error），它指的是数学模型的行为与其所代表的物理系统的行为之间的差异。每类误差都与其他类不同，因此让我们分别加以考察。

10.1.1 舍入误差

大多数浮点运算都会产生舍入误差，但它们通常很小。当两个大数值的数相加或相减并产生一个较小数值的结果时，可能会出现大的误差，尤其是当这种情况在一次计算过程中多次出现时。防范舍入误差的最简单手段是对所有计算都使用双精度浮点运算，但这并不能解决所有问题。

在动力学计算过程中，可能由于多种原因产生较大的舍入误差。其中较常见的一些原因是：
使用远离物体的坐标系（far-away coordinate system）；

大速度（large velocity）；

大的惯量比（inertia ratio）；以及

接近奇异的闭链约束（nearly singular loop-closure constraint）。

如果坐标系原点与质心之间的距离是物体回转半径（radius of gyration）的许多倍，那么这个坐标系对于该刚体来说就是远离的。理想情况下，这段距离不应超过大约一到两个回转半径。这一效应正是动力学算法在物体坐标系（body coordinates）中实现比在绝对坐标系（absolute coordinates）中实现更精确的原因。大速度问题出现在具有较大平均速度的系统中，例如航天器，其中各组成部分的相对速度与它们的绝对速度相比很小。这个问题可以通过使用一个惯性参考系（inertial reference frame）来解决，该参考系的速度接近系统的平均线速度。惯量的大差异源于相对较小的尺寸差异。例如，如果两个实心球体由同种物质制成，且其中一个比另一个大 10 倍，那么较大的球体的质量将是较小者的 1000 倍，转动惯量则是后者的 100000 倍。

让我们更详细地考察其中一些舍入误差是如何产生的。图 10.1 显示了一个平面刚体，其质心位于点 C，点 C 相对于原点的位置由矢量 $\mathbf{c} = \overrightarrow{OC} = [c_x \ c_y]^T$ 确定。在这个例子中我们将使用平面矢量，以保持矩阵较小，但舍入问题对于平面矢量和空间矢量而言本质上是相同的。该物体具有单位质量，并且关于 C 具有单位转动惯量，因此其回转半径

Figure 10.1: 计算平面刚体的加速度

为 1。用 C 坐标表示，该物体的速度为 $\mathbf{v}_C = [1 \ 0]^T$ ，并且它受到一个作用力 $\mathbf{f}_C = [1 \ -1]^T$ 。前者是绕 C 的单位转动，后者是一个纯力，其大小为 $\sqrt{2}$ ，作用线通过相对于 C 的点 (0.5, 0.5)。用 O 坐标表示，该物体的运动方程为

$$I_O \mathbf{a}_O = \mathbf{f}_O - \mathbf{v}_O \times^* I_O \mathbf{v}_O,$$

其中

$$\mathbf{v}_O = \begin{bmatrix} 1 \\ c_y \\ -c_x \end{bmatrix}, \mathbf{v}_O \times^* = \begin{bmatrix} 0 & c_x & c_y \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}, \mathbf{f}_O = \begin{bmatrix} 1 + c_x + c_y \\ -1 \\ 1 \end{bmatrix}$$

以及

$$I_O = \begin{bmatrix} 1 + c_x^2 + c_y^2 & -c_y & c_x \\ -c_y & 1 & 0 \\ c_x & 0 & 1 \end{bmatrix}.$$

该方程的精确解为

$$\mathbf{a}_O = \begin{bmatrix} 1 \\ c_y - 1 \\ 1 - c_x \end{bmatrix}.$$

假设我们设定 $c_x = 100$ 和 $c_y = 10$ ，并让计算机使用 IEEE 双精度浮点运算来求 $\mathbf{a}_O$ ，这种运算的精度为 10^{16} 分之一。计算机将通过求解线性方程

$$\begin{bmatrix} 10101 & -10 & 100 \\ -10 & 1 & 0 \\ 100 & 0 & 1 \end{bmatrix} \mathbf{a}_O = \begin{bmatrix} 111 \\ -1 \\ 1 \end{bmatrix},$$

来完成这一工作，而计算值与精确值将相差约 10^{12} 分之一。这样，就损失了四位有效数字的精度。损失的原因

Figure 10.2: 在 C 处测得的 $\mathbf{a}_O$ 值随 $|\mathbf{c}|$ 变化的舍入误差

在于系数矩阵是高度病态的 (ill-conditioned)，其条件数 (condition number) 为 1.02×10^8 。为了说明舍入误差如何随 $|\mathbf{c}|$ 变化，图 10.2 将 $\|\mathbf{a}_C - {}^C\mathbf{X}_O \mathbf{a}_O\|_\infty$ 对 $|\mathbf{c}|$ 作图，其中 $\mathbf{a}_C$ 是精确值， $\mathbf{a}_O$ 是计算值。可以看出，当 $|\mathbf{c}| < 1$ (即当 $|\mathbf{c}|$ 小于回转半径) 时，舍入误差近似恒定，而当 $|\mathbf{c}| > 1$ 时，误差大致与 $|\mathbf{c}|^2$ 成正比增长。

在这个例子中，物体质心的线速度为零 ($v_{Cx} = v_{Cy} = 0$)。如果物体反而具有很大的线速度，那么即使在 $|\mathbf{c}|$ 相对较小的情况下，计算式 10.1 的右端时也可能出现可观的舍入误差。

10.1.2 动力学算法中的舍入误差

已知舍入误差会随物体数量的增加而增大，并且对某些算法增大得比另一些算法更快。已知运动学树的三种主要算法即使系统中有数千个物体也能保持精确；此外还已知，铰接体算法 (articulated-body algorithm) 比复合刚体算法更精确。然而，许多其他算法的舍入行为尚不为人知。关于这一主题的两篇相关文章是 Ascher 等人 (1997 年) 和 Featherstone (1999b 年)。

10.1.3 截断误差

截断误差是使用无穷级数、连续曲线、连续函数等的有限近似的结果。在刚体动力学中，截断误差的主要来源是数值积分 (numerical integration) 过程，它只是精确数学积分的近似。就其本质而言，对截断误差的管理涉及在效率

与精度之间的折衷：更好的近似会减小误差，但计算代价更高。对积分误差的管理涉及对积分方法和时间步长做出恰当的选择。

如果将仿真刚体系统的加速度变量对时间作图，那么往往会发现所得图形包含大的尖峰：即加速度远高于其他时刻的短暂时间段。在这种情况下，采用一种能自动调整步长以维持规定精度的积分方法可能是有益的。这类方法在变化不多时采取大步长，而在加速度很高时采取极小步长。这一策略可以减少动力学必须被计算的次数，从而可能提高整个仿真运行的总体效率。

10.1.4 建模误差

建模误差是那些使数学模型的行为有别于物理系统行为的误差。如果数学模型是刚体系统，那么建模误差的主要来源是：

刚体假设，
理想关节假设（精确的运动学约束），
未建模的动力学（unmodelled dynamics），以及
模型参数的不准确。

在现实中，并不存在真正的刚体这种东西，真实的关节也会遭受诸如摩擦、柔顺性和间隙之类的缺陷。未建模的动力学指的是在建立模型时被忽略的物理系统的那些方面。例如，人们可以选择忽略链-链轮传动中链条的动力学，或者由于缺乏数据而被迫忽略某个重要关节轴承处与温度相关或与位置相关的摩擦变化。大多数模型参数要么涉及几何，要么涉及惯量。机械系统的几何参数通常能以合理的精度获知，但惯量参数可能精度较低，甚至完全未知。如果系统可以拆卸，那么可以测量各个部件的惯量；否则，可以使用参数辨识（parameter-identification）技术，例如 Khalil 和 Dombre（2002 年）所描述的那些技术。

§ 10.2 灵敏度问题

某些刚体系统对作用力的小变化，或模型参数的小变化可能极度敏感。对几何参数的灵敏度，在具有特殊几何结构、或者接近特殊的通用几何结构的闭链系统中，原本就是意料之中的。然而，这个问题

Figure 10.3: 处于折线（zigzag）构型的 6 连杆平面机构

远比这更广泛，而且，它尤其是既影响运动学树也影响闭链系统的问题。

为了说明这个问题，请考虑图 10.3 所示的 6 连杆平面机器人机构。每个连杆都具有单位长度、单位质量，其质心位于连杆长度的一半处，关于其质心的转动惯量为 $1/12$ 。关节都是转动关节，关节角如图示在 $+75^\circ$ 和 -75° 之间交替。机器人初始静止，除关节力之外没有其他力作用其上。因此运动方程为

$$\tau = H\ddot{q}.$$

假设我们希望给该机器人赋予期望加速度 $\ddot{q}_d = [1 \ 1 \ 1 \ 1 \ 1 \ 1]^T$ 。做到这一点所需的精确力为

$$\tau_d = H\ddot{q}_d = \begin{bmatrix} 126.4936 \cdots \\ 97.4663 \cdots \\ 69.9762 \cdots \\ 43.7998 \cdots \\ 21.9371 \cdots \\ 6.1646 \cdots \end{bmatrix}$$

如果我们对该机器人精确施加这个力，那么我们将得到精确的期望加速度。现在假设机器人的驱动器稍有缺陷，以至于它们实际施加的力等于被舍入到三位有效数字的命令力。那么实际施加到机器人上的力为

$$\tau_a = \begin{bmatrix} 126 \\ 97.5 \\ 70.0 \\ 43.8 \\ 21.9 \\ 6.16 \end{bmatrix},$$

而机器人对这个力的响应将是

$$\ddot{q}_a = H^{-1}\tau_a = \begin{bmatrix} 0.6952 \\ 1.3654 \\ 1.3808 \\ 0.5894 \\ 0.9057 \\ 1.0705 \end{bmatrix}.$$

请注意，尽管施加的力全部精确到优于 0.5

这个问题已在 Featherstone (2004 年) 中研究过，其中 H 的条件数被用作该机构潜在灵敏度的一个指标。该论文表明，条件数可以随物体数量的四次方成正比增长，并且对于由相同连杆组成的链，如图 10.3 中的机构，最坏情况的条件数约为 $4N_B^4$ 。因此，6 连杆机器人的最坏情况条件数约为 5184，尽管上例中 H 的实际条件数是 725；而 10 连杆机器人的最坏情况是 40000。诸如此类的数字表明，人们应当谨慎地仿真长刚体链，并对结果的精度持怀疑态度。

该论文还考察了分支运动学树，并表明在其他条件相同的情况下，灵敏度随树的深度而增加。因此，由同一组物体构成的分支运动学树通常比无分支树具有更低的条件数。一个明显的推论是，对于闭链机构，人们应当选择一棵最小深度生成树 (minimum-depth spanning tree)。

§ 10.3 效率

为了比较不同算法的效率，有必要具备一个共同的效率衡量标准。就动力学算法而言，最广泛使用的衡量标准是浮点运算次数 (floating-point operations count)。这个计数通常被分为两个小计：乘法和除法的次数，统称为乘法 (multiplications)；以及加法和减法的次数，统称为加法 (additions)。此外，这些小计被表示为某个参数的函数，该参数度量刚体系统某一方面的规模，例如物体数量或关节变量数目。

表 10.1 展示了运算次数公式的一些例子。符号 m 和 a 分别代表乘法和加法， n 是关节变量的数目。为了获得这样的数字，有必要就一组标准的刚体系统达成一致，每个 n 值对应一个系统，从而使每个算法都应用于同一组系统。使用最广泛的标准

10.3.1 缩写

RNEA 递归牛顿-欧拉算法 CRBA 复合刚体算法 F&S 对方程 $H\ddot{q} = \tau - C$ 进行分解与求解 ABA 铰接体算法

10.3.2 来源

1 Balafoutis 等人 (1988 年), 表 II 中的算法 32 Balafoutis 和 Patel (1989 年), 表 II 中的算法 I3 Balafoutis 和 Patel (1991 年), 表 5.1 和 5.2 中的算法 5.74 Balafoutis 和 Patel (1991 年), 表 6.2 中的算法 6.25 Brandl 等人 (1988 年) 6 Featherstone (1987 年), 表 8-67 Featherstone (2005 年), 式 20, 其中 $D_{0a} = n - 1$, $D_{1a} = (n^2 - n)/2$, $D_{0b} = D_{1b} = 08$ Hollerbach (1980 年), 表 I 中的牛顿-欧拉方法 9 Lilly 和 Orin (1991 年), 表 7 中的方法 IV10 Lilly (1993 年), 表 3.7 中的方法 IV11 McMillan 等人 (1995 年), 表 II (固定基座) 12 Walker 和 Orin (1982 年), 方法 313 表 6.6 与式 6.2814 式 10.3

表 10.1: 若干已发表的动力学算法的运算次数

Figure 10.4: 各种动力学算法的运算次数随 n 的变化

是一组无分支运动学链, 其中所有关节都是转动关节, 且所有物体都具有一般惯量和几何参数。为便于引用, 让我们把这组称为 GU (一般无分支, **general unbranched**), 并将其成员称为 $GU(1)$ 、 $GU(2)$ 等等。那么 $GU(n)$ 就是具有 n 个关节变量、因此也具有 n 个关节和 n 个运动物体 (即 $N_J = N_B = n$) 的无分支链。

鉴于 GU 中的关节是转动关节, 可以理解每个关节都需要一次正弦和一次余弦计算。由于这些计算对于所有需要它们的算法都是一样的, 因此它们被排除在运算次数之外。需要它们的算法是递归牛顿-欧拉算法和铰接体算法。

为了对各算法的相对效率给出直观印象, 图 10.4 将每种算法最佳版本的总运算次数对 n 作图。它还画出了 $O(n^3)$ 算法的三个组成部分, 以便可以估量它们的相对贡献。严格来说, 这些曲线图中的曲线只有在执行一次加法的代价与执行一次乘法的代价相同时才是正确的。然而在实践中

, 无论将一次加法的代价算作一次乘法、0.5 次乘法还是 0.25 次乘法, 差别都很小: 除了纵轴比例之外, 曲线图看起来几乎相同。

根据这些曲线图, 对于 $n \leq 8$, $O(n^3)$ 算法略快于铰接体算法, 而到 $n = 18$ 时, 其代价已上升为铰接体算法代价的约 1.6 倍。第二幅图表明, 从 $n = 8$ 一直到远超过 $n = 30$, 复合刚体算法 (其为 $O(n^2)$) 的代价是 $O(n^3)$ 算法中的主导项。(F&S 曲线在大约 $n = 45$ 处与 CRBA 曲线相交。) 它还表明, 在 $n \leq 18$ 时, 分解并求解运动方程的代价实际上是最小的一项。因此, 在 n 较小时, n^3 项对总代价的贡献相对而言是无关紧要的。

10.3.3 优化

表 10.1 中引用的数字是大量优化的结果, 这些优化旨在使每种算法呈现出尽可能好的面貌。其中最重要的优化如下。¹

使用 Denavit-Hartenberg 坐标系。(参见图 4.8。) 这使优化 2 和 3 得以实现。

利用这样一个事实: 在连杆坐标中 $S_i = [0 \ 0 \ 1 \ 0 \ 0 \ 0]^T$ 。

将连杆到连杆的坐标变换实现为两个轴向螺旋变换 (axial-screw transform) 的序列。

轴向螺旋变换是绕单个坐标轴转动并沿该轴平移的组合。这种变换实施起来特别便宜，其公式和代价在 A.4 节中给出。从一个 Denavit-Hartenberg 坐标系到另一个坐标系的变换可以通过两个轴向螺旋变换完成：一个绕 x 轴，一个绕 z 轴。前者是常数，而后者包含关节变量。

为了了解这些优化是如何应用的，让我们计算复合刚体算法的运算次数公式。表 10.2 再现了该算法的伪代码（取自表 6.2），并附上了一些注释。符号 $\mathbf{ra}$ 、 $\mathbf{rx}$ 和 $\mathbf{vx}$ 分别表示相加两个刚体惯量、变换一个刚体惯量和变换一个矢量的代价。我们稍后将计算出这些代价。

第一步是识别哪些行包含浮点计算。赋值和转置操作不算作计算，因此在第 1、3 或 16 行没有浮点计算。利用优化 2，第

算法：注释：

```

1  $H = 0$ 
2 for  $i = 1$  to  $N_B$  do
3  $I_i^c = I_i$ 
4 end
5 for  $i = N_B$  to 1 do
6 if  $\lambda(i) \neq 0$  then
7  $I_{\lambda(i)}^c = I_{\lambda(i)}^c + \lambda(i) X_i^* I_i^c X_{\lambda(i)}$  操作： $\mathbf{ra} + \mathbf{rx}$ 
8 end
9  $F = I_i^c S_i$ 
10  $H_{ii} = S_i^T F$ 
11  $j = i$ 
12 while  $\lambda(j) \neq 0$  do
13  $F = \lambda(j) X_j^* F$ 
14  $j = \lambda(j)$ 
15  $H_{ij} = F^T S_j$ 
16  $H_{ji} = H_{ij}^T$ 
17 end
18 end
```

表 10.2：复合刚体算法执行的运算

9 行上的乘法 $I_i^c S_i$ 简化为从 I_i^c 中提取第 3 列，这不需要任何计算。同样，第 10 和 15 行上的乘法 $S_i^T F$ 和 $F^T S_j$ 都简化为从 F 中提取元素 3，这也不需要任何计算。因此，只有两行会发生浮点计算：第 7 行，它执行操作 $\mathbf{ra}$ 和 $\mathbf{rx}$ ；以及第 13 行，它执行操作 $\mathbf{vx}$ 。

如果将该算法应用于机构 $GU(n)$ ，那么循环 2 的循环体将被执行 n 次。条件 $\lambda(i) \neq 0$ 对除 $i = 1$ 之外的每个 i 值都为真，因此第 7 行总共被执行 $n - 1$ 次。在循环 2 的每次迭代中，循环 3 的循环体将被执行 $i - 1$ 次（因为 $\lambda(j) = j - 1$ ）；因此第 13 行总共被执行 $\sum_{i=1}^n (i - 1)$ 次，算下来是 $n(n - 1)/2$ 次。因此，复合刚体算法的代价公式为

$$\text{cost} = (n - 1)(\mathbf{ra} + \mathbf{rx}) + \frac{n(n - 1)}{2} \mathbf{vx}.$$

下一步是用 $\mathbf{m}$ 和 $\mathbf{a}$ 来表达 $\mathbf{ra}$ 、 $\mathbf{rx}$ 和 $\mathbf{vx}$ 。如果刚体惯量按 A.3 节所述以紧凑形式存储，那么相加两个惯量的代价是十次浮点加法；因此 $\mathbf{ra} = 10\mathbf{a}$ 。为了获得 $\mathbf{rx}$ 和 $\mathbf{rx}$ 的最低运算次数，我们使用优化 1 和 3。根据 A.4 节的数据，对空间矢量进行单个轴向螺旋变换的代价是 $10\mathbf{m} + 6\mathbf{a}$ ，但 $\mathbf{vx}$ 需要

两个, 因此 $\mathbf{v}_x = 20\mathbf{m} + 12\mathbf{a}$ 。对刚体惯量进行轴向螺旋变换的基本代价是 $16\mathbf{m} + 15\mathbf{a}$, 每当角度变化时还要加上 $2\mathbf{m} + 3\mathbf{a}$ 。然而, 我们可以通过预计算质量与螺旋变换距离参数的乘积来从这个数字中削减 $1\mathbf{m}$, 因为它们都是常数。这样得到的 $\mathbf{r}_x$ 代价是 $2(15\mathbf{m} + 15\mathbf{a}) + 2\mathbf{m} + 3\mathbf{a} = 32\mathbf{m} + 33\mathbf{a}$ 。将这些代价代入式 10.2 得

$$\text{cost} = (10n^2 + 22n - 32)\mathbf{m} + (6n^2 + 37n - 43)\mathbf{a}.$$

这个公式是我们愿意把优化过程做到的程度, 但效率还有进一步提升的空间。例如, 可以按照第 9.7 节所述修改惯量参数, 从而适度降低 $\mathbf{r}_a$ 和 $\mathbf{r}_x$ 两者的代价。另一个节省代价的方法是通过观察: 计算 H_{ij} 只需要 F 的元素 3, 因此在循环 3 的每次调用中, 第 13 行的最后一次执行可以用一个只计算变换后矢量元素 3 的更简单的计算来替代。

即便如此, 也还不是效率的最终结论。不过, 手动实现这类琐碎的优化实在不值得。更好的策略是让计算机来替你完成优化, 这就是第 10.4 节的主题。

10.3.4 分支

把 GU 用作基准刚体系统组的一个问题是, 它不包含任何分支树, 因此无法看到分支的影响。事实证明, 分支对递归牛顿-欧拉算法和铰接体算法的影响非常小, 但它们可以极大地降低复合刚体算法的代价, 以及分解并求解运动方程的代价。在两种情况下, 代价的节省都是由分支引起的关节空间惯量矩阵 (joint-space inertia matrix) 中的稀疏性 (sparsity) 造成的。(参见第 6.4 节。)

因此可以得出结论, 表 10.1 中的代价公式和图 10.4 中的曲线图通过只陈述最坏情况性能, 使 $O(n^3)$ 算法呈现了一种过分悲观的面貌。事实上, 连 “ $O(n^3)$ 算法” 这个名字都具有误导性, 因为该算法的真实复杂度是 $O(nd^2)$, 其中 d 是连接树的深度。如果没有分支, 那么 $d = n$; 但如果存在分支, 那么 $d = n$ 可能比 n 小得多; 而如果 d 受一个固定上限约束, 那么该算法的复杂度就只是 $O(n)$ 。

让我们重新计算复合刚体算法的代价公式, 假设它被应用于一棵具有 n 个转动关节的一般运动学树, 并且关节轴在连杆坐标中满足 $S_i = [0 \ 0 \ 1 \ 0 \ 0 \ 0]^T$ 。回到表 10.2, 第 7 和 13 行仍然是仅有的两行执行浮点计算的行, 但这些行被执行的次数已经改变。就第 7 行而言, 表达式 $\lambda(i) \neq 0$

对每个作为物体 0 的子物体的物体 i 为假; 也就是说, 对每个满足 $i \in \mu(0)$ 的 i 为假。因此第 7 行被执行 $n - |\mu(0)|$ 次。就第 13 行而言, 循环 3 遍历物体 i 的每一个祖先, 从物体 i 本身开始, 在离根最近的祖先 (即满足 $j \in \mu(0)$ 的祖先) 处终止。因此, 在循环 2 的每次迭代中, 循环 3 的循环体被执行 $|\kappa(i)| - 1$ 次; 所以第 13 行总共被执行 $\sum_{i=1}^n (|\kappa(i)| - 1)$ 次。

让我们定义量 D_0 和 D_1 如下:

$$D_0 = n - |\mu(0)|$$

以及

$$D_1 = \sum_{i=1}^n (|\kappa(i)| - 1).$$

于是复合刚体算法的代价可以表示为

$$cost = D_0(\mathbf{r}\mathbf{a} + \mathbf{r}\mathbf{x}) + D_1\mathbf{v}\mathbf{x}.$$

此外, 由于 $|\mu(0)|$ 受 $1 \leq |\mu(0)| \leq n$ 约束, $|\kappa(i)|$ 受 $1 \leq |\kappa(i)| \leq i$ 约束, 因此 D_0 和 D_1 受

$$0 \leq D_0 \leq n - 1$$

以及

$$0 \leq D_1 \leq n(n-1)/2.$$

约束。在一个每个物体都直接连接到基座的系统中, 这两个量都达到其下界。在这样的系统中, $\mathbf{H}$ 是对角且恒定的, 计算它的代价为零。在另一个极端, 在一个无分支运动学树中, 这两个量都达到其上界。在这种情况下, 式 10.6 与式 10.2 完全相同。如果 d 是连接树的深度, 那么 $|\kappa(i)|$ 也受 $|\kappa(i)| \leq d$ 约束, 这意味着

$$0 \leq D_1 \leq n(d-1).$$

因此, 复合刚体算法的真实复杂度实际上是 $O(nd)$, 而表达式 $O(n^2)$ 只适用于最坏情况。如果我们将它与第 6.5 节中分解算法的 $O(nd^2)$ 复杂度结合起来, 那么完整的前向动力学算法的复杂度为 $O(nd^2)$ 。

现在让我们用 $\mathbf{m}$ 和 $\mathbf{a}$ 来表示式 10.6 中的代价公式。这时出现了一个新的复杂情况: 每个非末端物体 (nonterminal body) 只有一个子物体能够从 Denavit-Hartenberg 坐标系带来的代价节省中受益。因此我们将式 10.6 重写如下:

$$cost = D_0\mathbf{r}\mathbf{a} + D_0^{dh}\mathbf{r}\mathbf{x}^{dh} + D_0^g\mathbf{r}\mathbf{x}^g + D_1^{dh}\mathbf{v}\mathbf{x}^{dh} + D_1^g\mathbf{v}\mathbf{x}^g,$$

其中上标 dh 和 g 分别指 Denavit-Hartenberg 变换和一般坐标变换。于是 $\mathbf{r}\mathbf{x}^{dh}$ 是在 ${}^i\mathbf{X}_{\lambda(i)}$ 为 Denavit-Hartenberg 变换 (两个轴向螺旋) 时计算 $\lambda^{(i)}\mathbf{X}_i^*\mathbf{I}_i^*\mathbf{i}\mathbf{X}_{\lambda(i)}$ 的代价; $\mathbf{v}\mathbf{x}^g$ 是在 $\lambda^{(j)}\mathbf{X}_j^*$ 为一般坐标变换时计算 $\lambda^{(j)}\mathbf{X}_j^*\mathbf{F}$ 的代价; D_0^g 是执行 $\mathbf{r}\mathbf{x}^g$ 的次数; 等等。请注意, $D_0^{dh} + D_0^g = D_0$ 且 $D_1^{dh} + D_1^g = D_1$ 。

为了获得各个 D 量的表达式, 我们引入两个新集合: π^{dh} 和 π^g 。 π^{dh} 是所有满足 $\lambda(i) \neq 0$ 且 ${}^i\mathbf{X}_{\lambda(i)}$ 是 Denavit-Hartenberg 变换的物体 i 的集合; π^g 是所有满足 $\lambda(i) \neq 0$ 且 ${}^i\mathbf{X}_{\lambda(i)}$ 是一般变换的物体 i 的集合。因此, $\pi^{dh} \cup \pi^g$ 是所有不是根节点子物体 (即不属于 $\mu(0)$ 的元素) 的物体的集合, 并且 $|\pi^{dh} \cup \pi^g| = n - |\mu(0)| = D_0$ 。有了这些集合, 我们可以如下定义各 D 量:

$$D_0^{dh} = |\pi^{dh}|, D_1^{dh} = \sum_{i \in \pi^{dh}} |\nu(i)|,$$

$$D_0^g = |\pi^g|, D_1^g = \sum_{i \in \pi^g} |\nu(i)|.$$

D_1^{dh} 和 D_1^g 的表达式按如下方式获得。当我们第一次求 D_1 时, 我们问的问题是“循环 3 被执行多少次?”。另一种方法是问, 对于每个可能的 j 值, $\lambda^{(j)}\mathbf{X}_j^*$ 被使用了多少次。对这个问题的回答是: 如果 $\lambda(j) = 0$, 那么这个变换从不会被使用; 否则, 它被 $\nu(j)$ 中的每个物体各使用一次, 这意味着它被使用 $|\nu(j)|$ 次。式 10.11 中的表达式就是这些使用次数的总和。

最后一步是用 m 和 a 来表达 ra 、 rx^{dh} 等。根据 A.3 和 A.4 节，这些表达式为： $ra = 10a$ ， $rx^{dh} = 32m + 33a$ ， $rx^g = 47m + 48a$ （三个轴向螺旋）， $rx^{dh} = 20m + 12a$ ， $rx^g = 24m + 18a$ 。将这些表达式代入式 10.10 得

$$\begin{aligned} cost = & D_0^{dh}(32m + 43a) + D_0^g(47m + 58a) + \\ & D_1^{dh}(20m + 12a) + D_1^g(24m + 18a). \end{aligned}$$

这就是复合刚体算法应用于所有关节都是转动关节的一般运动学树时的代价。在无分支树上，有 $D_0^{dh} = n - 1$ ， $D_1^{dh} = n(n - 1)/2$ ， $D_0^g = D_1^g = 0$ ，这使得式 10.12 与式 10.3 相同。关于这一主题的更多内容，包括浮动基座运动学树的代价公式，可以在 Featherstone（2005 年）中找到。

为了说明分支能带来多大的差异，图 10.5 再现了图 10.4 中铰接体算法和 $O(n^3)$ 算法（现在标注为 $O(nd^2)$ ）的代价曲线，并添加了一条新曲线：将 $O(nd^2)$ 算法应用于二叉树（binary tree）的代价。这条曲线与铰接体算法的曲线几乎相同，而与应用于无分支链的 $O(nd^2)$ 算法的曲线截然不同。原因在于，在二叉树上 d 是 $O(\log(n))$ ，因此 $O(nd^2)$ 算法在二叉树上的复杂度仅为 $O(n \log(n)^2)$ ，这非常接近 $O(n)$ 。

Figure 10.5: 分支对 $O(nd^2)$ 算法代价的影响

§ 10.4 符号化简

基于模型的动力学软件（model-based dynamics software）的优点是，每个算法可以被实现一次，所得的代码能对作为参数提供给它的任何刚体系统正常工作。缺点是代码必须兼顾系统模型所允许的每一种可能性，而不能依赖对系统的先验知识来走捷径。特别是，典型系统中的许多参数将为零，而基于模型的软件会花大量时间计算那些必然会求值为零的项，因为它们的一个因数是值为零的模型参数。

另一种做法是编写只针对单个刚体系统、或一小类相似刚体系统的定制代码。这类代码可以利用关节类型、几何和惯量参数值等的先验知识，最终结果是与同一算法的基于模型的编码相比，执行计算所需的算术运算更少。

手动构造定制代码并不实际，除非是极其简单的刚体系统；但自动构造此类代码则完全可行，而实现这一过程的程序就叫符号化简。图 10.6 展示了这一过程。在第一阶段，一个算法和一个系统模型被送入一个符号执行器（symbolic executable）。这是一个将算法应用于数据的程序，但它不是真正执行数值计算，而只是记录这些计算，并将记录作为数据文件输出。该文件的内容是一列赋值语句，详述了给定算法在真实执行时本会在给定模型上执行的浮点计算。该文件还标明了输入和输出变量。

在第二阶段，赋值语句文件被传给一个符号化简程序（symbolic simplifier）。该程序读取文件，应用几条基本的化简规则

Figure 10.6: 符号化简过程

则，并以合适的编程语言表示的计算机源代码形式输出结果。最重要的化简如下：

删除任何计算了从未被使用的量的赋值语句。（作为输出被使用也算作被使用。）

删除形如 $a = b$ 的赋值语句，其中 a 不是输出变量， b 是常数或变量，并用 b 替换其后所有出现 a 的地方。

执行一些基本的代数化简，例如： $x \cdot 0 \rightarrow 0$ ， $x \cdot 1 \rightarrow x$ ， $x + 0 \rightarrow x$ ，等等。

求值常量表达式。

这些规则必须被穷尽地应用，因为应用一条规则可能为应用另一条规则创造新的机会。它们还必须廉价，因为列表中的赋值很容易有数万个。

表 10.3 给出了该过程如何运作的一个例子。在这个例子中， v 是一个输入变量， τ_{out} 是一个输出变量， I 和 s 是在系统模型中定义的量。算法规定了如何从 v 计算 τ_{out} 。如果我们直接实现这个算法，那么它将需要一次 $3 \times 3 \times 1$ 矩阵乘法、一次 3D 矢量叉积和一次 3D 矢量点积。这总共加起来是 29 次浮点算术运算，可以在赋值语句文件中看到。符号执行器只处理标量，因此它把矢量运算展开成等价的标量运算。它还一次只处理一个矢量运算，因此会根据需要引入临时变量。除此之外，它唯一做的另一件事是用模型参数的实际值替换它们。

仅使用上面列出的化简，化简器就能够把总运算次数从 29 削减到 5。如果化简器使用了更表 10.3：符号化简示例

复杂精巧的规则集，那么它可能已经找到了最优解，即 $\tau_{out} = 0.1v_1v_2$ ，但该过程将需要多得多的 CPU 时间。即便如此，简单的规则集也足以找到 90

上述方案还可以有多种变体。例如，可以把一组预定义的算法内置到符号执行器中，这样用户只需提供系统模型即可。这使软件更易于使用，也更易于实现，但它剥夺了用户提供自己算法的自由。另一种变体是，符号执行器和化简器可以是同一程序的两个部分，而不是两个独立的程序。在这种情况下，赋值语句文件将不存在，不过仍然需要等价的内存数据结构。另一种可能性是允许模型只定义部分参数的值。未定义的参数随后将被视为要在运行时提供的未知量。这个特性允许创建针对一组刚体系统的定制代码，这些系统彼此之间仅在某些特定参数的取值上有区别。

符号化简的一个潜在缺点在于它把矢量运算化简为标量运算。在配备矢量运算硬件的计算机上，所得代码可能无法充分利用可用的处理能力，并且最终可能比原始的面向矢量的代码更慢。另一个可能的缺点是定制代码的大小随运算次数线性增长（因为它只是一列赋值语句）。对于一个足够复杂的刚体系统，定制代码会太大而无法装入计算机的指令缓存（instruction cache）。这时，计算机被迫从主存储器获取指令，这是一个慢得多的过程。净效应是执行速度下降，因为 CPU 反复不得不等待下一批指令。

已经编写了多个实现这种技术或类似技术的程序，其中两个已可商用：SD/FAST 和 Autolev。

² 关于动力学方程自动生成与化简这一主题的更多内容，可以参见 Murray 和 Neuman (1984 年)；Neuman 和 Murray (1987 年)；Wittenburg 和 Wolz (1985 年)。

接触与冲击

当两个刚体相互接触时，它们便受制于一个接触约束（contact constraint），该约束规定两个刚体不得相互穿透。若 ϕ 度量它们之间的带符号距离，即当它们重叠时 $\phi < 0$ ，则接触约束可由不等式 $\phi \geq 0$ 表示。施加这一约束的力同样是单侧的（one-sided）：它们可以阻止穿透，但不能阻止分离——它们可以排斥，却不能吸引。若两个刚体以与二者之间接触约束不一致的速度相遇，则会产生一个冲量（impulse），使二者的速度发生阶跃变化。我们称这一事件为冲击（impact）。

本章前四节推导了含有单点接触（point contact）的刚体系统的运动方程。随后，11.5 节研究求解这些方程的方法，11.6 节考察问题的一些几何方面，包括如何用点接触来表示线接触和面接触。11.7 节推导刚体系统的冲击运动方程以及两刚体之间的冲击方程。最后，11.8 节给出刚体接触与冲击动力学的另一种方案，即将柔性（compliance）引入接触表面。

§ 11.1 单点接触

考察图 11.1 所示的刚体系统。该系统包含一个刚体 B ，它在单个点 C 处与一个固定表面相接触。该刚体的速度为 $\mathbf{v}$ ，加速度为 $\mathbf{a}$ ；它同时受到外加力 $\mathbf{f}$ 和接触力 $\mathbf{f}_c$ 的作用。该接触由一个接触法线向量（contact normal vector） $\mathbf{n} \in \mathbf{F}^6$ 表征， $\mathbf{n}$ 是沿接触法线方向作用的单位力。我们定义 $\mathbf{n}$ 指向表面外侧，因此 $\mathbf{n}$ 的正标量倍数将把刚体从表面推开。假设接触是无摩擦的（frictionless）；并假设在当前瞬间及其前后紧邻时刻，刚体只受有限力作用（即不受冲量）。该

图 11.1：一个在单个点处与固定表面接触的刚体

系统中的两个未知量是 $\mathbf{f}_c$ 和 $\mathbf{a}$ ，问题就在于求出它们。

设 C' 表示刚体表面上在当前瞬间与 C 重合的点。固定表面对刚体施加约束力 $\mathbf{f}_c$ ，其作用是阻止 C' 穿透表面。由于接触是无摩擦的，该力必须沿接触法线方向作用，因此必为 $\mathbf{n}$ 的标量倍数。为此我们引入一个新的标量变量 λ ，即未知的接触力大小。于是，

$$\mathbf{f}_c = \mathbf{n} \lambda.$$

由于 $\mathbf{f}_c$ 只能起排斥刚体的作用， λ 必须满足约束 $\lambda \geq 0$ 。

现在我们引入第二个标量变量 ζ 来表示接触分离速度（separation velocity），它定义为 C' 的线速度的法向分量，由下式给出

$$\zeta = \mathbf{n} \cdot \mathbf{v}.$$

该式的正确性可通过如下方式验证：放置一个以 C 为原点、 z 轴沿法线方向的坐标系，则 n 的普吕克坐标 (Plücker coordinates) 为 $[0\ 0\ 0\ 0\ 1]^T$ ，于是 $n \cdot v = v_{Cz}$ ，即 C' 线速度的 z 坐标。

可以证明，在当前瞬间 $\zeta = 0$ 。这意味着给定的 n 和 v 必须满足 $n \cdot v = 0$ 。论证过程如下：若刚体只受有限力作用，则其加速度必为有限值；因此，若当前瞬间 $\zeta < 0$ ，则刚体在紧随其后将会穿透表面；若 $\zeta > 0$ ，则刚体在紧接之前已经穿透了表面；但任何时刻都不允许穿透，所以 $\zeta = 0$ 。这一论证的一个推论是： $\zeta \neq 0$ 就意味着存在冲量。

我们还关心接触分离加速度，即 ζ 的导数。该量由下式给出

$$\dot{\zeta} = \frac{d}{dt}(n \cdot v) = n \cdot a + \dot{n} \cdot v,$$

其当前瞬间的值是未知的。项 $\dot{n} \cdot v$ (或向量 $\dot{n}$) 被视为已知量，因为它是位置、速度和

B 以及接触表面形状的函数。既然已确定 $\zeta = 0$ ，可知 $\dot{\zeta}$ 不能为负，因为那将导致穿透，所以它必须满足约束 $\dot{\zeta} \geq 0$ 。

现在可以精确表述接触约束的行为：在当前瞬间，接触要么断开，要么持续。若接触断开，则分离加速度必须严格为正，且接触力必须为零（因为接触正在失去）。若接触持续，则分离加速度必须为零。总而言之，我们有

$$\begin{aligned} \dot{\zeta} &> 0 \text{ and } \lambda = 0 \text{ if the contact breaks, and} \\ \dot{\zeta} &= 0 \text{ and } \lambda \geq 0 \text{ if the contact persists.} \end{aligned}$$

这一行为可由如下约束方程简洁地描述

$$\dot{\zeta} \geq 0, \lambda \geq 0, \dot{\zeta}\lambda = 0.$$

下一步是把 $\dot{\zeta}$ 表示为 λ 的函数。这正是刚体动力学起作用之处。 B 的运动方程为

$$Ia + v \times^* Iv = f + n\lambda,$$

其中 f 包含作用在该刚体上除接触力以外的所有力。把 a 表示为 λ 的函数，有

$$a = I^{-1}(n\lambda + f - v \times^* Iv).$$

将此表达式代入式 11.3，得

$$\begin{aligned} \dot{\zeta} &= n \cdot I^{-1}(n\lambda + f - v \times^* Iv) + \dot{n} \cdot v \\ &= M\lambda + d, \end{aligned}$$

其中

$$M = n \cdot I^{-1}n$$

以及

$$d = n \cdot I^{-1}(f - v \times^* Iv) + \dot{n} \cdot v.$$

现在可以直接观察求解式 11.4 和 11.7 得到 $\dot{\zeta}$ 和 λ ，解为

$$\begin{aligned}\dot{\zeta} &= d \text{ 且 } \lambda = 0 & \text{若 } d \geq 0 \\ \dot{\zeta} &= 0 \text{ 且 } \lambda = -d/M & \text{若 } d < 0.\end{aligned}$$

(注意, 由于 I 是正定的, M 严格为正。) 求出 λ 后, f_c 和 a 的值即可由式 11.1 和 11.6 得出, 问题随之解决。注意, d 是 f 的线性函数, 但 λ 只是 d 的分段线性函数。因此, B 的加速度不是外加力的线性函数, 而只是分段线性函数。在这方面, 接触动力学与带等式约束系统的动力学有着根本性差异。

图 11.2: 一个与固定表面形成多个点接触的刚体

§ 11.2 多点接触

现在我们把 11.1 节的分析推广到单个刚体与固定表面形成多个无摩擦点接触的情形。这样的系统实例如图 11.2 所示。设接触总数为 n_c , 编号为 1 到 n_c ; 并设接触 i 由接触点 C_i 和接触法线向量 $\mathbf{n}_i$ 表征, 二者定义如 11.1 节。我们还定义 C'_i 为在当前瞬间与 C_i 重合的刚体固连点。

固定表面可以通过每个接触对刚体施加约束力。通过接触 i 施加的力为 $n_i \lambda_i$, 其中 λ_i 是未知的接触力大小, 该力用于阻止 C'_i 穿透表面。由于接触力只能排斥 C'_i , 它必为 n_i 的正倍数; 所以 λ_i 必须满足约束 $\lambda_i \geq 0$ 。若 f_c 为作用在刚体上的总约束力, 则

$$f_c = \sum_{i=1}^{n_c} n_i \lambda_i,$$

它取代了式 11.1。

设 ζ_i 表示接触 i 处的分离速度。该量为 C'_i 的线速度在 n_i 方向上的分量, 因此由下式给出

$$\zeta_i = n_i \cdot v.$$

注意, 即使系统中只有一个运动刚体, 每个接触也都有自己的分离速度。采用与 11.1 节相同的论证, 我们可以证明在当前瞬间每个分离速度都必须为零。这意味着刚体的速度必须满足以下方程组:

$$n_i \cdot v = 0 (i = 1, \dots, n_c).$$

若接触法线张成整个 F^6 , 则唯一解将是 $v = 0$ 。然而, 这并不一定意味着刚体不能运动, 因为它

也许能够从一个或多个接触处加速离开 (从而断开这些接触)。

接触 i 处的分离加速度是分离速度的导数, 因此由下式给出

$$\dot{\zeta}_i = \frac{d}{dt}(n_i \cdot v) = n_i \cdot a + \dot{n}_i \cdot v,$$

它取代了式 11.3。既然 $\zeta_i = 0$, $\dot{\zeta}_i$ 的值不能为负, 因为那将导致接触 i 处局部穿透, 所以我们有 $\dot{\zeta}_i \geq 0$ 。

现在可以精确表述这一组接触的行为：在当前瞬间，每个接触要么断开，要么持续。若接触 i 断开，则 $\dot{\zeta}_i > 0$ 且 $\lambda_i = 0$ ，否则 $\dot{\zeta}_i = 0$ 且 $\lambda_i \geq 0$ 。于是我们有

$$\dot{\zeta}_i \geq 0, \lambda_i \geq 0, \dot{\zeta}_i \lambda_i = 0 (i = 1, \dots, n_c),$$

它取代了式 11.4。一开始，我们并不知道哪些接触会断开、哪些会持续。找出这一点本身就是求解过程的一部分。

在这一点上，让我们引入三个新量 N 、 λ 和 ζ ，其定义如下：

$$N = [n_1 n_2 \cdots n_{n_c}], \lambda = \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \vdots \\ \lambda_{n_c} \end{bmatrix} \text{ and } \zeta = \begin{bmatrix} \zeta_1 \\ \zeta_2 \\ \vdots \\ \zeta_{n_c} \end{bmatrix}$$

因此 N 是一个 $6 \times n_c$ 矩阵，另外两个则是含有 n_c 个元素的向量。用这些量来表示，式 11.11、11.13 和 11.14 可写成

$$f_c = N\lambda,$$

$$\dot{\zeta} = N^T a + \dot{N}^T v$$

以及

$$\dot{\zeta} \geq 0, \lambda \geq 0, \dot{\zeta}^T \lambda = 0.$$

形如 $a \geq b$ 的表达式（其中 a 和 b 是向量）意味着对每个下标 i 都有 $a_i \geq b_i$ 。约束 $\dot{\zeta}^T \lambda = 0$ 与式 11.18 中另外两个约束合在一起，意味着对每个 i 都有 $\dot{\zeta}_i \lambda_i = 0$ 。

继续分析，刚体 B 的运动方程为

$$Ia + v \times^* Iv = f + N\lambda,$$

可以重新整理以把 a 表示为 λ 的函数：

$$a = I^{-1}(N\lambda + f - v \times^* Iv).$$

将此表达式代入式 11.17，得

$$\dot{\zeta} = M\lambda + d,$$

图 11.3：一个无解的接触系统示例

其中

$$M = N^T I^{-1} N$$

以及

$$d = N^T I^{-1} (f - v \times^* I v) + \dot{N}^T v.$$

现在可以先求解式 11.18 和 11.20 得到 λ ，再由式 11.19 计算 $\mathbf{a}$ ，从而求出 B 的加速度。然而，与可直接观察求解的式 11.4 和 11.7 不同，式 11.18 和 11.20 的求解需要专门的算法。这一主题将在 11.5 节讨论，但这里值得先作几点说明。

若 $d \geq 0$ ，则一个可能的解是 $\lambda = 0$ 。

若 M 正定，则恰好有一个解；否则，可能有一个解、无解，或有无穷多个解。

若 λ 有多个解，则它们都产生相同的 $\mathbf{a}$ 值。

从式 11.21 可以看出， M 是对称且半正定的；但只有当接触法线向量线性无关时，它才是正定的。因此， λ 可能无解或多于一个解的情况，只可能出现在接触法线线性相关的时候。

例 11.1 式 11.18 和 11.20 可能无解这一事实，意味着存在某些情形，其中任何 λ 值都无法阻止对接触约束的违反。图 11.3 给出了这种现象的一个二维实例。该图展示了一个在槽中滑动的圆盘。在直线 L 的左侧，槽壁是直的，槽宽等于圆盘直径。因此圆盘与槽形成两个点接触，接触法线 $\mathbf{n}_1$ 和 $\mathbf{n}_2$ 如图所示。这两个向量满足 $\mathbf{n}_2 = -\mathbf{n}_1$ ，因此线性相关。在 L 的右侧，槽壁是向内弯曲的圆弧，使槽变得更窄。因此，圆盘的圆心 D 可以位于 L 上或其左侧，但无法越过 L 到右侧，因为槽太窄了。

考虑 D 位于 L 上时出现的情形：圆盘不能再向右移动，但两条接触法线正指向上、下，因而无法影响圆盘的水平运动。若对圆盘施加一个向右的推力，我们就会遇到一种矛盾情形：向右的运动绝不允许发生，但接触力却无法阻止它。若这种情形出现在仿真中，实际可行的解决办法是允许 D 移动到 L 右侧一点，使两条接触法线改变方向，从而能够抵抗进一步向右的运动。

若 D 以图所示的线速度 $\mathbf{v}$ 到达 L ，则需要一个冲量来阻止圆盘的运动。同样，仿真器必须允许 D 在施加该冲量之前略微越过 L 。在这两种情况下，仿真器都必须容忍圆盘与槽之间的少量穿透。

校对标注：给定 $\mathbf{n}_{ad}$ 相关的关系式，可以证明 M 具有如下形式（LaTeX 已损坏，以文本保留）： $M = [[\mathbf{m}, -\mathbf{m}], [-\mathbf{m}, \mathbf{m}]]$ ，其中 $\mathbf{m}$ 为正标量。因此（由式 2.2）

$$\zeta_1 + \zeta_2 = d_1 + d_2.$$

若存在解，则它必须满足 $\zeta_1 + \zeta_2 \geq 0$ ；因此无解的条件是 $d_1 + d_2 < 0$ 。

§ 11.3 带接触的刚体系统

现在让我们考虑一般刚体系统带多个无摩擦点接触的情形。与前一种情形相比，最大的区别在于，一般刚体系统的运动方程是用关节空间（joint space）表示的。¹ 因此，该系统的运动方程将具有如下形式

$$H\ddot{\mathbf{q}} + \mathbf{C} = \boldsymbol{\tau} + \boldsymbol{\tau}_c,$$

其中 τ_c 是表示总接触力的关节空间力，而 $\ddot{q}$ 将受到来自接触的运动约束。因此，必须先把各个接触力和加速度表达式转换到关节空间，然后才能与运动方程合并。

另一个区别是，现在接触可以发生在不同的刚体对之间，因此有必要确定每个接触涉及哪些刚体。为便于这种确定，我们使用前驱体（predecessor）和后继体（successor）这两个术语来指代参与给定接触的两个刚体。接触力被定义为从前驱体传递给后继体的力；分离加速度则被定义为后继体相对于前驱体的加速度。若将这一约定应用于图 11.1 和 11.2，则刚体 B 将被识别为后继体。注意，接触

现在既可以发生在两个运动刚体之间，也可以发生在一个运动刚体与固定基座之间。

设接触总数为 n_c ，编号为 1 到 n_c ；并设 pc 和 sc 是两个整数数组，使得 $pc(i)$ 和 $sc(i)$ 分别是参与接触 i 的前驱体和后继体的刚体编号。每个接触还由以下要素表征：一个接触点 C_i ，位于前驱体表面上；第二个点 C'_i ，在当前瞬间与 C_i 重合，但固定在后继体上；以及一个法线向量 $\mathbf{n}_i$ ，它是沿接触法线方向作用的单位空间力。 $\mathbf{n}_i$ 从前驱体指向后继体，因此 $\mathbf{n}_i$ 的正标量倍数在接触 i 处局地排斥两个刚体。

鉴于接触是无摩擦的，接触 i 处出现的任何接触力都必须是 $\mathbf{n}_i$ 的标量倍数。此外，由于该力在性质上必须是排斥性的，它必须是 $\mathbf{n}_i$ 的正标量倍数。因此我们引入变量 λ_i 来表示接触 i 处未知的接触力大小，并注意它必须满足约束 $\lambda_i \geq 0$ 。

接触 i 处的力从刚体 $pc(i)$ 传递给刚体 $sc(i)$ 。因此，它等价于一个作用在刚体 $sc(i)$ 上的力 $\mathbf{n}_i \lambda_i$ 和一个作用在刚体 $pc(i)$ 上的力 $-\mathbf{n}_i \lambda_i$ 。要把这一对力转换成等价的关节空间力，我们利用一般公式 $\boldsymbol{\tau} = \mathbf{J}_i^T \mathbf{f}$ ，其中 $\mathbf{f}$ 是施加在刚体 i 上的空间力， $\boldsymbol{\tau}$ 是等价的关节空间力， $\mathbf{J}_i$ 是刚体 i 的刚体雅可比矩阵（body Jacobian）。若 $\boldsymbol{\tau}_{ci}$ 表示由接触 i 产生的关节空间力，则该公式给出

$$\boldsymbol{\tau}_{ci} = \left(\mathbf{J}_{sc(i)}^T - \mathbf{J}_{pc(i)}^T \right) \mathbf{n}_i \lambda_i.$$

在这一点上，我们引入一个新量 t_i ，定义如下：

$$t_i = \left(\mathbf{J}_{sc(i)}^T - \mathbf{J}_{pc(i)}^T \right) \mathbf{n}_i.$$

它可以看作是接触 i 的接触法线向量在关节空间中的表示。将式 11.25 代入式 11.24，得

$$\boldsymbol{\tau}_{ci} = t_i \lambda_i.$$

表示总接触力（即所有接触力的合成效果）的关节空间力现在可以写成

$$\boldsymbol{\tau}_c = \sum_{i=1}^{n_c} t_i \lambda_i,$$

它取代了式 11.11。

设 ζ_i 表示接触 i 处的分离速度。由于两个刚体都可能具有非零速度，我们把分离速度定义为 C'_i 相对于 C_i 的相对线速度的法向分量。于是，

$$\zeta_i = \mathbf{n}_i \cdot \left(\mathbf{v}_{sc(i)} - \mathbf{v}_{pc(i)} \right),$$

它是式 11.12 的推广。要把 ζ_i 表示为关节空间速度 $\dot{\mathbf{q}}$ 的函数，我们使用一般公式 $\mathbf{v}_i = \mathbf{J}_i \dot{\mathbf{q}}$ ，其中 $\mathbf{v}_i$ 和 $\mathbf{J}_i$ 分别是刚体 i 的速度和刚体雅可比矩阵。将此公式应用于式 11.27，得

$$\zeta_i = n_i \cdot (J_{sc(i)} - J_{pc(i)}) \dot{q}.$$

然而，将该式与式 11.25 中 t_i 的定义相比较，可以看出

$$\zeta_i = t_i^T \dot{q}.$$

这是式 11.12 的关节空间等价形式。

获得了分离速度的表达式后，分离加速度就是这个表达式的导数，即

$$\dot{\zeta}_i = \frac{d}{dt}(t_i^T \dot{q}) = t_i^T \ddot{q} + \dot{t}_i^T \dot{q}.$$

该式取代了式 11.13。采用与前面各节相同的论证，可以证明在当前瞬间 $\zeta_i = 0$ ，这意味着 $\dot{\zeta}_i \geq 0$ 。项 $\dot{t}_i^T \dot{q}$ 可由下式计算

$$\dot{t}_i^T \dot{q} = n_i \cdot (a_{sc(i)}^{vp} - a_{pc(i)}^{vp}) + \dot{n}_i \cdot (v_{sc(i)} - v_{pc(i)}),$$

其中 a_i^{vp} 是刚体 i 的速度积加速度 (velocity-product acceleration)，它等于 $\dot{f}_i \dot{q}$ 。在这一点上，我们引入式 11.15 所定义的向量 λ 和 ζ ，以及一个新的 $n \times n_c$ 矩阵 T ，其定义如下：

$$T = [t_1 t_2 \cdots t_{n_c}].$$

用这些量来表示，式 11.26 和 11.29 可写成

$$\tau_c = T\lambda$$

以及

$$\dot{\zeta} = T^T \ddot{q} + \dot{T}^T \dot{q},$$

它们取代了式 11.16 和 11.17。然而，对 λ 和 ζ 的约束仍由式 11.18 正确描述。

最后一步是构造一个把 ζ 表示为 λ 的函数的方程。将式 11.32 代入式 11.23，并整理以把 $\ddot{q}$ 移到等号左边，得

$$\ddot{q} = H^{-1}(T\lambda + \tau - C);$$

再将此表达式代入式 11.33，得

$$\dot{\zeta} = M\lambda + d,$$

其中

$$M = T^T H^{-1} T$$

图 11.4: $\phi(q) \geq 0$ 的可能状态

以及

$$d = T^T H^{-1}(\tau - C) + \dot{T}^T \dot{q}.$$

式 11.35 和 11.36 取代了式 11.21 和 11.22，但式 11.34 与式 11.20 本质上是相同的。事实上，这两个方程的唯一区别在于，式 11.20 中 M 的秩由式 11.21 限制为最多 6，而式 11.34 中 M 的秩可能不受限制。求解过程与 11.2 节完全相同：利用 11.5 节所述方法，在满足式 11.18 的前提下求解式 11.34。

§ 11.4 不等式约束

设 B_1 和 B_2 是一般刚体系统中的两个刚体；并设 S_1 和 S_2 是两个表面或表面片，使得 S_1 固连于 B_1 ， S_2 固连于 B_2 。这两个表面之间的接触约束是一条规定它们不得相互穿透的约束。假设我们有一个函数 $\phi(q)$ ，其性质为：在构形 (configuration) q 下，当 S_1 和 S_2 彼此分离、接触或相互穿透时， $\phi(q)$ 分别大于、等于或小于零。例如，当 S_1 和 S_2 彼此分离时， $\phi(q)$ 可以是分离距离的量度；当它们相互穿透时，则可以是穿透距离的负值。给定具有这种性质的函数后，接触约束可用如下方程表示

$$\phi(q) \geq 0.$$

这种形式的约束方程称为不等式约束 (inequality constraint)，或称单侧约束 (unilateral constraint)。后一个名称指的是约束的单侧性：若两个表面相互接触，则允许一个方向的运动，而不允许相反方向的运动。

不等式约束对刚体系统可以产生不同的影响，具体取决于 q 和 $\dot{q}$ 的值。为理解这些影响，我们如图 11.4 所示识别四种状态，并逐一考察。

状态 1 由 $\phi(q) > 0$ 表征，因此适用于所有满足 $\phi(q) > 0$ 的构形 q 。在这种状态下，表面之间的距离大于零；因此没有接触，对系统的速度或加速度没有约束，也没有接触力。处于这种状态的不等式约束对系统的瞬时动力学没有影响，可视为非激活的 (inactive)。

状态 2a 由 $\phi = 0$ 和 $\dot{\phi} < 0$ 表征。（ $\dot{\phi}$ 同时是 q 和 $\dot{q}$ 的函数。）这种状态出现在两个表面相撞的瞬间，它使两个相撞刚体之间产生一个冲量。在该冲量之后紧接的时刻，系统将处于状态 2b 或状态 3，具体取决于碰撞是弹性的（会弹起）还是塑性的（不弹起）。状态 2a 只持续一个瞬间，但它会引起系统速度的阶跃变化，这一变化必须用冲击动力学方程来计算。

状态 2b 由 $\phi = 0$ 和 $\dot{\phi} > 0$ 表征。在这种状态下，表面相互接触，但正以正的分离速度飞离彼此。这种状态只可能出现在冲量之后的紧接时刻，或几何上失去接触的瞬间（见 §11.6）。它只持续一个瞬间，因为约束在当前瞬间之后会立即过渡到状态 1。状态 2b 对系统的瞬时动力学没有影响，可以像状态 1 一样处理。

状态 3 由 $\phi = 0$ 和 $\dot{\phi} = 0$ 表征，是与持续接触相关联的状态。在这种状态下，两个表面处于接触之中，它们的分离速度为零。因此分离加速度被约束为非负，并且会有一个约束力（即接触力）把这一约束施加于系统。更具体地说，在当前瞬间可能发生两种情况之一：两个表面加速离开彼此（ $\ddot{\phi} > 0$ ），或者它们保持接触（ $\ddot{\phi} = 0$ ）。在前一种情况下，不等式约束将在当前瞬间之后

立即过渡到状态 1，且接触力为零。在后一种情况下，约束保持在状态 3，且有一个接触力起作用以防止 $\dot{\phi} < 0$ 。

由上述描述可知，11.1 至 11.3 节都涉及对状态 3 中不等式约束的分析。当我们对 $\phi(\mathbf{q})$ 求导时，还会发现另一层联系：

$$\dot{\phi} = \mathbf{t}^T \dot{\mathbf{q}}$$

以及

$$\ddot{\phi} = \mathbf{t}^T \ddot{\mathbf{q}} + \dot{\mathbf{t}}^T \dot{\mathbf{q}},$$

其中

$$\mathbf{t}^T = \frac{\partial \phi}{\partial \mathbf{q}}.$$

将这些方程与式 11.28 和 11.29 相比较，可以看出，对于适当选取的函数 $\phi_i(\mathbf{q})$ ， ζ_i 和 $\check{\zeta}_i$ 分别对应于 $\dot{\phi}_i$ 和 $\ddot{\phi}_i$ 。同样，式 11.25 中的 $\mathbf{t}_i$ 对应于 $(\partial \phi_i / \partial \mathbf{q})^T$ 。

§ 11.5 求解接触方程

根据 11.3 节，受点接触约束的刚体系统的运动方程为

$$H\ddot{\mathbf{q}} + \mathbf{C} = \boldsymbol{\tau} + \mathbf{T}\boldsymbol{\lambda},$$

其中 $\boldsymbol{\lambda}$ 是如下问题的解

$$\check{\boldsymbol{\zeta}} = \mathbf{M}\boldsymbol{\lambda} + \mathbf{d}, \check{\boldsymbol{\zeta}} \geq 0, \boldsymbol{\lambda} \geq 0, \check{\boldsymbol{\zeta}}^T \boldsymbol{\lambda} = 0.$$

11.1 和 11.2 节推导的方程只是这些方程的特殊情形，其中式 11.41 被替换为单个刚体的运动方程。

类似式 11.41 的方程在前面的章节中已经讨论过，但式 11.42 是新的。事实上，式 11.42 中的方程组和不等式组定义了一个数学中的标准问题，即线性互补问题（linear complementarity problem, LCP）。该问题的数学理论及若干求解算法可见于 Cottle 等人 (1992) 和 Cottle 与 Dantzig (1968)；当 $\mathbf{M}$ 正定这一特殊情形的算法可见于 Featherstone (1987)。

在接触动力学应用中， $\mathbf{M}$ 始终是对称半正定矩阵，并且当 $\mathbf{T}$ 的列（即接触法线向量）线性无关时它是正定的。对称半正定矩阵的 LCP 具有以下特殊性质：

若 $\mathbf{M}$ 正定，则恰好有一个解；否则可能无解、有一个解，或有无穷多个解。

若存在解，则 $\mathbf{T}\boldsymbol{\lambda}$ 和 $\check{\boldsymbol{\zeta}}$ 的值是唯一的（即若存在多个解，则它们仅在 $\boldsymbol{\lambda}$ 的值上不同，且这些差异位于 $\mathbf{T}$ 的零空间中）。

式 11.42 中的 LCP 等价于如下的二次规划（quadratic program）问题：²

$$\begin{aligned} & \text{minimize} \quad \frac{1}{2} \lambda^T M \lambda + \lambda^T d \\ & \text{subject to} \quad \lambda \geq 0. \end{aligned}$$

式 11.42 和式 11.43 具有相同的解，但 11.43 的优点在于，求解二次规划的软件比求解 LCP 的软件更为普及。Löfstedt (1982) 中还提到了另外两种表述。其中一种是

$$\begin{aligned} & \text{minimize} \quad \frac{1}{2} (\ddot{q} - H^{-1}(\tau - C))^T H (\ddot{q} - H^{-1}(\tau - C)) \\ & \text{subject to} \quad T^T \ddot{q} + \dot{T}^T \dot{q} \geq 0. \end{aligned}$$

向前积分，将激活接触视为等式约束，并忽略所有其他接触。

在积分过程中，监视系统发生的两类事件：几何事件（接触的产生与失去）和负的接触力。

若在最近一次积分步中检测到一个或多个此类事件，则将系统插值回到最早事件的时刻。

若该事件是接触的产生（例如碰撞），则按 11.7 节所述应用冲击动力学，并确定新的当前接触集合和新的激活接触集合。

如果事件属于几何接触丢失（geometric contact loss），则从当前接触集合和活动接触集合中移除丢失的接触。

如果事件是负接触力，则建立并求解式 (11.42) 或式 (11.43)，并确定新的当前接触集合和活动接触集合。

转到步骤 1。

表 11.1: 接触动力学仿真的一般流程

这同样是一个二次规划（quadratic program），但对变量 $\ddot{q}$ 而非 λ 求极小值。这种公式化如果存在解，将产生唯一解，并且在 $n_c > n$ 时可能具有计算优势，但从 $\ddot{q}$ 中恢复 λ 并不直接。式 (11.44) 可以被视为高斯最小约束原理（Gauss' principle of least constraint）的推广，使其适用于带有不等式约束的刚体系统。

如果存在许多接触，那么求解式 (11.42) 或式 (11.43) 的计算成本将相对较大，相比之下，使用第 8 章的技术施加一组等价的等式约束的成本要小得多。因此，希望最大限度地减少式 (11.42) 或式 (11.43) 的求解次数。完成这一目标的流程如表 11.1 所示。

按照这一流程，仿真器维护一组当前接触（current contacts）和一组活动接触（active contacts），后者是前者的子集。当前接触集合包含当前处于状态 3（如图 11.4 所定义）的所有接触，并且这些接触对相应的下标 i 满足 $\zeta_i = 0$ （即 $\dot{\phi}_i = 0$ ）。如果发现某个当前接触满足 $\zeta_i > 0$ ，则认为它在当前时刻已经断开，并将其从集合中移除。

活动接触集合是这样的一个集合：其接触法线在 $\text{range}(\mathbf{T})$ 上构成一组基，并且包含所有具有严格正接触力的接触，即所有满足 $\lambda_i > 0$ 的接触。如果我们把活动接触视为等式约束，那么下面的陈述成立：只要接触力变量保持非负，接触的

状态就不会发生变化。因此，只有当在某个活动接触上检测到负的 λ_i ，或者发生碰撞时，才需要求解式 (11.42) 或式 (11.43)。负的 λ_i 总是会引起活动接触集合的变化，但不一定会引起当前接触集合的变化。

大多数求解 LCP 和二次规划的算法都会返回问题的一个基本解（basic solution），即具有尽可能少的非零变量的解。式 (11.42) 或式 (11.43) 的基本解具有如下性质：与正接触力变量相关的接触法线是线性无关的。给定式 (11.42) 或式 (11.43) 的一个基本解，新的当前接触集合将是满足 $\zeta_i = 0$ 的接触所组成的集合，新的活动接触集合将是满足 $\lambda_i > 0$ 的接触所组成的集合。在极少数情况下，这个集合不足以构成一组基，需要补充满足 $\zeta_i = \lambda_i = 0$ 的接触。

11.5.1 对积分时间步进行插值

积分时间步可以被插值这一想法需要一些解释。假设我们希望从 t_0 到 t_1 对微分方程 $\dot{y} = f(y, t)$ 进行积分，其中 $t_1 = t_0 + h$ ， h 是积分步长。许多数值积分方法的工作原理是：在区间 $[t_0, t_1]$ 上用一个多项式拟合 j ，并将该多项式的积分加到 $y(t_0)$ 上。实际上，这些方法都具有如下一般形式

$$\begin{aligned}\dot{y}(t) &= p(t), t_0 \leq t \leq t_1 \\ y_1 &= y_0 + \int_{t_0}^{t_1} p(t) dt\end{aligned}$$

其中 $y_0 = y(t_0)$ 、 $y_1 = y(t_1)$ ， $p(t)$ 是一个多项式。 p 的次数比方法的阶数少 1。因此，我们可以用下面的公式计算任意中间时刻 t 处的 $y(t)$

$$y(t) = y_0 + \int_{t_0}^t p(t) dt.$$

举一个具体的例子，欧拉积分法（Euler's integration method）在积分时间步内用常数来近似 y 。该方法的公式是

$$\begin{aligned}k &= f(y_0, t_0) \\ y_1 &= y_0 + hk,\end{aligned}$$

由此可知 $p(t) = k$ 。因此插值多项式为

$$y(t) = y_0 + (t - t_0)k.$$

设 $\delta = (t - t_0)/h$ 为一个插值变量，当 t 从 t_0 变化到 t_1 时，它从 0 变化到 1。用 δ 表示时，欧拉法的插值多项式为

$$y(\delta) = y_0 + hk\delta.$$

再看一个更复杂的例子，（一种版本的）海因积分法（Heun's integration method）（Gear, 1971）的公式是

$$\begin{aligned}k_1 &= f(y_0, t_0) \\ k_2 &= f(y_0 + \frac{2}{3}hk_1, t_0 + \frac{2}{3}h) \\ y_1 &= y_0 + h(\frac{1}{4}k_1 + \frac{3}{4}k_2).\end{aligned}$$

该方法用线性多项式来近似 y ，该多项式在 t_0 处取值为 k_1 ，在 $t_0 + \frac{2}{3}h$ 处取值为 k_2 。因此该方法的插值多项式将是二次的，其公式为

$$y() = y + h((-3/4)k + 3/4k)$$

诸如此类的公式使动力学仿真器能够在一个已完成的积分时间步内的任意中间时刻计算状态变量的值，因此也能够按任意所需量缩短最近的时间步。

§ 11.6 接触几何

接触动力学仿真器既涉及动力学计算，也涉及几何计算。到目前为止，我们对后者几乎没有提及。在本节中，我们考察接触动力学的一些几何方面：形状的表达、几何事件（接触的建立与丢失），以及边接触和面接触。

11.6.1 形状

最常用的形状计算机表示是多面体（polyhedron）。目前已有大量的技术知识，以及大量用于创建、编辑和显示多面体以及执行高效碰撞检测（collision detection）的软件。遗憾的是，多面体并不十分适合动力学。图 11.5 展示了可能出现的这类问题的一些示例。左侧的图将一个真实球体与一个滚下斜坡的球体的多面体近似（绘制为多边形）进行比较。球体可以平稳地滚动，而多面体则会在每一个

图 11.5：多面体近似的一些问题

新的顶点撞击表面时与斜坡发生一连串的碰撞。这些碰撞应如何处理？如果将它们视为塑性碰撞（plastic collisions），那么每次碰撞都会损失能量；但如果将它们视为弹性碰撞（elastic collisions），那么多面体就会开始弹跳。

这个问题是由多面体的平面和直棱引起的，这些平面和直棱使它在运动过程中能够在点接触、边接触和面接触之间交替。解决办法是对多面体进行变形，使面和棱向外弯曲，从而使得到的形状是严格凸的。理想情况下，这个变形的多面体应该非常接近一个完美的球体。然而，即使它不是完美的球形，它仍然能够滚动而不会引起一连串的碰撞。

另一个示例显示在图 11.5 的右侧图中：一个矩形块沿凹形斜坡滑下。如果用多面体近似替换斜坡，那么每当块的某个顶点到达斜坡中的一条棱时，块就会经历一次碰撞。这一次，问题是由斜坡中尖锐的凹棱引起的，解决办法是将它们磨圆。

这两个示例都是一个潜在问题的表现。假设 p 表示物体表面上的一点， $n(p)$ 是该点的表面法线。在曲面上， n 随 p 连续变化，这种变化通过量 $\dot{n}$ 进入运动方程。在动力学仿真中使用多面体近似曲面的基本困难在于，多面体的 $n(p)$ 是分段常数，因此无法对原始曲面中的 dn/dp 进行建模。

11.6.2 几何事件

我们将几何事件（geometric event）定义为由物体的运动引起的接触状态的阶跃变化。几何事件基本上有两种：接触建立（contact gain）和接触丢失（contact loss）。碰撞是接触建立事件的一个特例，其中参与碰撞的两个物体此前并未接触。这些事件的示例显示在图 11.6 中。一般来说，接触建立事件会引起冲量，而接触丢失事件会引起当前接触集合的缩减（如第 11.5 节所定义）。

假设我们用术语“动态接触丢失（dynamic contact loss）”来指代前几节研究的那种接触丢失。如图 11.6 所示，动态接触丢失与几何接触丢失的区别在于：前者发生在两个表面由于

图 11.6：几何事件

作用于系统的力而彼此加速分离时，而后者发生在分离速度出现阶跃变化时，因为一个顶点滑过了某个面的边缘，或者一条边滑过了另一条边的端点。几何接触丢失事件无法通过求解运动方程来检测，而必须通过几何方法检测。

检测运动物体之间的碰撞和接触建立可能是一种非常昂贵的计算。这是因为真实物理物体的精确几何模型可能包含数百、数千甚至数万个几何图元（geometric primitives）。幸运的是，已经有许多高效的算法被提出用于这些及相关的几何计算，感兴趣的读者可参阅 Bobrow（1989）；Gilbert 等人（1988）；Gottschalk 等人（1996）；Jimenez 等人（2001）；Mirtich（1998）。

11.6.3 线接触和面接触

如果接触沿一条线延伸，或覆盖一个面，那么它在数学上等价于无限多个点接触：线或面上的每个点对应一个。因此，线接触或面接触可以用无限点接触集合来描述。然而，这个集合通常包含的接触数量远多于对原始接触状态建模所需的最小数量。可以通过穷举应用下面的规则来获得最小接触集合：

如果某个点接触的法向量正相关于集合中的其他法线³，那么它对接触约束没有贡献，可以将其移除。

图 11.7 展示了一些线接触和面接触的示例。图（a）显示了一个物体上的直棱与另一个物体上的平面之间的线接触。在这种情况下，线接触可以用一对点接触来建模：一个在线段的每一端。图（b）显示了直棱与直纹面（ruled surface）之间的线接触。后者由方程 $z = x \tan(y)$ 构成，其中 z 轴向上， y 轴位于接触线上。在这个示例中，直纹面的曲率使得每个接触法线都指向不同的方向。此外，点接触集合中的法向量没有一个正相关于其他法线，因此该示例的最小接触集合包含线上的每个点对应的一个接触。

图（c）显示了两个平面之间的面接触，其中接触区域的凸包（convex hull）是一个多边形。（凸包是包含给定形状的最小凸形状。）在这种情况下，凸包是图中以灰色显示的五边形。这样的面接触可以用有限点接触集合来建模：一个在多边形的每个顶点处。图（d）也显示了两个平面之间的面接触，但接触区域是一个圆盘。因此凸包也是一个圆盘。在这个示例中，

图 11.7：线接触和面接触的示例

圆盘内部的每个法向量都正相关于边界上的一对法线，但边界上的任何法线都不正相关于集合中的其他法线。因此，该示例的最小接触集合由边界圆上的每个点组成。

注意，图 11.7 中的示例（a）和（c）涵盖了两个多面体之间所有可能的线接触和面接触；因此两个多面体之间的任何接触状态总是可以用有限点接触集合来表达。在示例（b）和（d）中，精确表达接触约束需要无限多个点接触，但仅用有限数量的点就可以获得任意所需精度的近似。

§ 11.7 — 冲量动力学

冲量（impulse）是力对时间的积分。如果在一个从 t_0 到 t_1 的时段内对刚体施加力 $f(t)$ ，那么该物体收到一个冲量 ι ，由下式给出

$$\iota = \int_{t_0}^{t_1} f(t) dt.$$

这个冲量的作用是如下改变物体的动量：

$$\iota = \int_{t_0}^{t_1} f(t) dt = \int_{t_0}^{t_1} \frac{dh}{dt} dt = h(t_1) - h(t_0),$$

其中 $f = dh/dt$ 是物体的运动方程， h 是它的动量 ($h = I v$)。因此，刚体在给定时间间隔内收到的冲量等于它在该间隔内动量的净变化。

如果两个硬物体碰撞，它们会在短时间内相互施加很大的接触力。如果我们将它们的硬度增加到无穷大，使它们变得刚性，那么接触力发散到无穷大，时间段收敛到零，但冲量保持有限。因此，对于刚体动力学，我们感兴趣的一种特殊冲量，是作用在时长 δt 上的力 $f/\delta t$ 在 $\delta t \rightarrow 0$ 时的极限：

$$\iota = \lim_{\delta t \rightarrow 0} \int_t^{t+\delta t} \frac{f(t)}{\delta t} dt.$$

本节其余部分讨论这种冲量。它们的基本性质如下：它们是力向量（即它们是 $\mathbf{F}^6$ 的元素）；它们产生于刚体之间的碰撞（或接触建立）；它们引起刚体速度的阶跃变化。

现在我们来推导刚体的冲量运动方程。设 ι 是施加在具有惯性 I 和速度 v 的物体上的冲量， Δv 是由 ι 引起的速度阶跃变化。 Δv 与 ι 之间的关系如下推导：

$$\begin{aligned} \Delta v &= \lim_{\delta t \rightarrow 0} v(t + \delta t) - v(t) \\ &= \lim_{\delta t \rightarrow 0} \int_t^{t+\delta t} a(t) dt \\ &= \lim_{\delta t \rightarrow 0} \int_t^{t+\delta t} I^{-1} \left(\frac{f}{\delta t} - v \times^* I v \right) dt \\ &= \lim_{\delta t \rightarrow 0} I^{-1}(t) \int_t^{t+\delta t} \frac{f}{\delta t} dt - \lim_{\delta t \rightarrow 0} \int_t^{t+\delta t} I^{-1} v \times^* I v dt \\ &= I^{-1} \iota, \end{aligned}$$

所以

$$\iota = I \Delta v.$$

在第三行，使用物体的运动方程 ($I a + v \times^* I v = f/\delta t$) 用施加的力 $f/\delta t$ 来表示其加速度。在第四行， I^{-1} 可以移到积分号外，因为 I 只随位置变化，而位置是时间的连续函数，所以当 $\delta t \rightarrow 0$ 时 $I^{-1}(t + \delta t) \rightarrow I^{-1}(t)$ 。第四行中的第二个极限等于零，因为当 $\delta t \rightarrow 0$ 时被积函数保持有限。

利用同样的论证，我们可以证明铰接物体手柄的冲量运动方程为

$$\iota = I^A \Delta v,$$

其中 I^A 是手柄的铰接物体惯性 (articulated-body inertia)。类似地，一般刚体系统的冲量运动方程为

$$u = H \Delta \dot{q},$$

图 11.8: 双物体碰撞

其中 u 是关节空间 (joint-space, 或称广义) 冲量， $\Delta \dot{q}$ 是关节 (或广义) 速度向量的阶跃变化。如果 u 是由物体 i 与系统外部的一个物体之间的碰撞引起的，使得物体 i 从外部物体收到冲量 ι ，导致其速度产生阶跃变化 Δv_i ，那么 u 、 ι 、 $\Delta \dot{q}$ 和 Δv_i 之间的关系由下式给出

$$u = J_i^T \iota$$

以及

$$\Delta v_i = J_i \Delta \dot{q},$$

其中 J_i 是物体 i 的物体雅可比矩阵 (body Jacobian)。

注意：式 (11.56)、式 (11.57) 和式 (11.58) 在数学上是正确的，但它们不一定能为真实多体系统的冲量行为提供准确的预测。这是因为真实的碰撞涉及刚体假设无法建模的物理过程，例如压缩波在固体中传播以及穿过关节轴承。

11.7.1 双物体碰撞

图 11.8 显示了两个刚体 B_1 和 B_2 ，它们在单点 C 处碰撞。碰撞前的速度是 v_1 和 v_2 ，碰撞后的速度是 $v_1 + \Delta v_1$ 和 $v_2 + \Delta v_2$ 。我们最初假设接触处没有摩擦。这意味着从 B_1 传递给 B_2 的冲量可以表示为 $n\lambda$ 的形式，其中 n 是接触法线， $\lambda \geq 0$ 是冲量大小。因此这两个物体的冲量运动方程为

$$\Delta v_1 = -I_1^{-1} n \lambda$$

以及

$$\Delta v_2 = I_2^{-1} n \lambda.$$

设 ζ 和 $\Delta\zeta$ 分别表示碰撞前在 C 处的分离速度 (separation velocity) 以及由冲量引起的分离速度的阶跃变化。它们由以下方程给出

$$\zeta = n \cdot (v_2 - v_1)$$

以及

$$\Delta\zeta = n \cdot (\Delta v_2 - \Delta v_1).$$

这些标量必须满足 $\zeta \leq 0$ 和 $\zeta + \Delta\zeta \geq 0$ 。我们需要再一个方程来求解这个问题。这个方程由恢复系数 (coefficient of restitution) e 提供，它是一个介于 0 和 1 之间的数，表示碰撞前后分离速度的比值。具体而言，

$$e = \frac{\zeta + \Delta\zeta}{-\zeta},$$

由此得出

$$\Delta\zeta = -(1 + e)\zeta.$$

将式 (11.62) 和式 (11.63) 代入式 (11.64) 得到

$$n \cdot (\Delta v_2 - \Delta v_1) = -(1 + e) n \cdot (v_2 - v_1),$$

再将式 (11.60) 和式 (11.61) 代入该方程得到

$$\lambda = \frac{-(1 + e) n \cdot (v_2 - v_1)}{n \cdot (I_1^{-1} + I_2^{-1}) n},$$

这样就解出了这个问题。

11.7.2 摩擦

如果我们在接触处允许库仑摩擦 (Coulomb friction)，那么系统的行为就会变得更加复杂。一般来说，我们必须考虑以下效应：切向摩擦、扭转摩擦、切向恢复 (tangential restitution)、扭转恢复 (torsional restitution)，以及切向和/或扭转滑动运动是否在接触时段结束之前停止。(这些复杂性中的许多都在 Brach (1991) 中进行了考察。) 扭转效应指的是一种抵抗绕接触法线方向角运动的摩擦力偶。它们之所以产生，是因为真实物体并非真正刚性，接触表面的局部变形使点接触变成了面接触。因此，扭转效应在较软的物体上比在较硬的物体上更明显。即便如此，扭转效应通常量级小于切向效应，并且常常可以忽略。切向恢复和扭转恢复分别由接触物体在切向和扭转方向上局部弹性变形的恢复引起。这些效应的一些示例显示在图 11.9 中。

让我们通过考察一个相对简单的带摩擦碰撞示例来结束本节。我们将假设两个物体如图 11.8 所示发生碰撞，并且不存在扭转效应，不存在切向恢复，切向运动被摩擦冲量所止停。这个示例几乎可以用与无摩擦情况相同的过程来求解。

图 11.9: 切向恢复 (a) 和扭转恢复 (b) 的示例

第一步是认识到，允许切向摩擦分量会给碰撞冲量增加两个自由度，因此它现在是三个未知标量的函数，而不再只是一个。与此同时，碰撞后 C 处相对速度的切向分量必须为零这一条件会给系统增加两个约束。

设 ι 是从 B_1 传递给 B_2 的总冲量。那么这两个物体的运动方程为

$$\Delta v_1 = -I_1^{-1} \iota$$

以及

$$\Delta v_2 = I_2^{-1} \iota.$$

我们可以将冲量表示为

$$\iota = N \lambda,$$

其中 $N = [n_x n_y n_z]$ 是一个张成可能冲量空间的 6×3 矩阵， $\lambda = [\lambda_x \lambda_y \lambda_z]^T$ 是未知冲量系数向量。 n_z 是接触法线向量，因此 n_z 和 λ_z 对应于无摩擦情况下的 n 和 λ ，而 n_x 和 n_y 位于接触的切平面内，张成可能的摩擦冲量空间。这三个向量的作用线都经过 C。

设 $\zeta = [\zeta_x \zeta_y \zeta_z]^T$ 是一个线性速度坐标向量，用于度量两个物体在 C 处的相对线性速度；设 $\Delta \zeta = [\Delta \zeta_x \Delta \zeta_y \Delta \zeta_z]^T$ 是由冲量引起的 ζ 的阶跃变化。这些向量由下式给出

$$\zeta = N^T(v_2 - v_1)$$

以及

$$\Delta\zeta = N^T(\Delta v_2 - \Delta v_1).$$

ζ 与 $\Delta\zeta$ 之间的关系一部分由应用于法线方向的恢复系数给出，一部分由切向速度被降至零的条件给出。基本上，我们有以下三个标量方程：

$$\zeta_x + \Delta\zeta_x = 0, \zeta_y + \Delta\zeta_y = 0, \zeta_z + \Delta\zeta_z = -e\zeta_z,$$

这可以表示为一个矩阵方程

$$\Delta\zeta = -(\mathbf{1} + E)\zeta$$

其中

$$E = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & e \end{bmatrix}.$$

为求解这个问题，我们只需将式 (11.69) 和式 (11.70) 代入式 (11.71)，得到

$$N^T(\Delta v_2 - \Delta v_1) = -(\mathbf{1} + E)N^T(v_2 - v_1),$$

然后将式 (11.66)、式 (11.67) 和式 (11.68) 代入该方程，得到

$$\lambda = -(N^T(I_1^{-1} + I_2^{-1})N)^{-1}(\mathbf{1} + E)N^T(v_2 - v_1).$$

§ 11.8 软接触

到目前为止，我们只研究了成对刚体之间的接触现象。然而，许多物理系统包含硬物体和软物体的混合物，或者带有软表面的硬物体，或者总体上够硬可以被视为刚性、但又够软在接触期间会发生显著局部变形的物体。在这些情况下，必须能够对刚性表面与柔性（compliant）表面之间、或两个柔性表面之间的接触动力学进行建模。

对柔性接触感兴趣的另一个原因是，它比刚性接触更容易实现：碰撞时刻没有冲量，因此无需进行冲量动力学计算；接触丢失可以由位置和速度数据确定，因此无需求解线性互补问题（linear complementarity problem）。在柔性接触处实现库仑摩擦也更容易。出于这些原因，在实现工作量成为问题的情况下，用柔性接触代替刚性接触可能更明智，即使在刚性接触是对物理系统更好的模型的场合。

柔性接触的主要缺点是，由于柔性表面中存在刚性弹簧，它会给系统引入高频动力学。如果这些动力学要求积分例程采用更小的步长，那么仿真的速度将会降低。

本节介绍一种对包括库仑摩擦在内的柔性接触进行建模的技术，其中柔性表面被建模为一阶
图 11.10: 柔性接触模型

动力学系统——一个包含弹簧和阻尼器但没有质量的系统。在这样的系统中，施加的力会引起速度，而强加的速度会遭遇反作用力。这项技术的巨大优势在于，它将刚体动力学与接触动力学分离开来：接触力可以由位置和速度变量计算得出，然后作为一组已知的力输入提供给刚体动力学例程。因此，可以将任何标准的正向动力学（forward-dynamics）算法与该接触模型配合使用，例如第 6、7 和 8 章中描述的那些算法。

11.8.1 接触法向力

考虑图 11.10 所示的一维刚体系统。它由一个刚体 B 和一个无质量的表面片 S 组成，后者通过一个弹簧和一个阻尼器连接到固定基座。变量 z 和 p 分别表示 S 和 B 的位置， f_c 是从 S 传递给 B 的接触力。如果 B 和 S 接触，则 $p = z$ 且 $f_c \geq 0$ ；否则， $p > z$ 且 $f_c = 0$ 。弹簧的刚度为 K ，它对 S 施加力 $-Kz$ 。类似地，阻尼器的阻尼系数为 D ，它对 S 施加力 $-D\dot{z}$ 。

现在， S 没有质量，因此作用在它上面的净力必须为零。作用在 S 上的三个力是 $-Kz$ 、 $-D\dot{z}$ 和 $-f_c$ ，所以我们有

$$Kz + D\dot{z} + f_c = 0.$$

重新整理该方程得到

$$\dot{z} = -(Kz + f_c)/D,$$

这就是 S 的运动方程。注意这是一个一阶微分方程：它涉及 z 和 $\dot{z}$ ，但不涉及 $\ddot{z}$ 。还要注意，接触力与 S 的速度成线性关系，而不是与其加速度成线性关系。这个方程中的两个未知量是 $\dot{z}$ 和 f_c ，状态变量是 z 。

现在让我们计算 f_c 。如果 $p > z$ ，则没有接触， $f_c = 0$ 。如果 $p = z$ ，则我们按如下方式进行。如果要使接触持续存在，那么在当前时刻必须有 $\dot{p} = \dot{z}$ 。实现这一点所需的接触力为

$$f_c = -Kz - D\dot{p}.$$

如果这个表达式为零或正数，那么它就是 f_c 的正确值，接触确实持续存在。然而，如果这个表达式结果为负，那么 f_c 的正确值是零，并且我们有 $\dot{p} > \dot{z}$ ，这意味着接触正在断开。将这一论证整合成单个方程，我们得到

$$f_c = \begin{cases} 0 & \text{if } p > z \\ \max(0, -Kz - D\dot{p}) & \text{if } p = z. \end{cases}$$

式 (11.74) 和式 (11.73) 与 B 的运动方程（从 f_c 计算 $\dot{p}$ ）一起，定义了系统的动态行为。

11.8.2 扩展

这个接触模型很容易扩展到更一般的情况。例如，要对两个运动物体之间的柔性接触建模，只需将 p 重新定义为两个物体的相对位置；要对两个柔性表面之间的接触建模，只需对每个表面

使用一个弹簧-阻尼器对和一个状态变量（即一个 z 变量）。如果两个表面的 K/D 比值相同，则可以将两个柔性合并在一起。

可以在不使用显式状态变量的情况下实现这个模型。这个技巧之所以可行，是因为当 $f_c = 0$ 时式 (11.73) 具有简单的解析解。因此，只要存在接触，就有 $z = p$ ，而在其他所有时刻都有 $z = z_0 e^{-K(t-t_0)/D}$ ，其中 t_0 和 z_0 分别是最近一次接触丢失发生的时间和当时 z 的值。这个技巧省去了对式 (11.73) 进行数值积分的需要，这可以提高仿真的效率。

可以将非线性弹簧和阻尼器纳入这个模型。这样做的好处是，非线性弹簧-阻尼器组合能为碰撞固体的物理行为提供更准确的模型（Marhefka 和 Orin, 1999）。另一种可能是使用更一般的黏弹性（visco-elastic）单元，例如与一个弹簧串联的弹簧-阻尼器对。更复杂的模型可能需要额外的状态变量。

要将这个模型应用于两个三维物体之间的点接触，我们需要根据物体的形状和位置计算两个量：接触法线向量 $\mathbf{n}$ 和穿透深度（penetration depth）的负值。后者给出 p 的值。如果变形很小，那么从表面的未变形形状计算 $\mathbf{n}$ 是合理的。 $\dot{p}$ 等价于前几节中的 ζ ，可以由形如 $\dot{p} = \mathbf{n} \cdot (\mathbf{v}_B - \mathbf{v}_S)$ 的方程计算，其中 $\mathbf{v}_B$ 和 $\mathbf{v}_S$ 是接触物体的速度。类似地， f_c 等价于前几节中的 λ ，传递给物体 B 的空间接触力为 $\mathbf{n}f_c$ 。既然接触力大小已知，各个刚体或整个刚体系统的加速度就可以直接由式 (11.5)、式 (11.19) 或式 (11.23) 这样的方程计算得出。

如果接触点在柔性表面上移动，那么弹簧和阻尼器应该跟随该点。如果想要对压缩轨迹在表面上移动所做的功进行建模，那么可以用切平面中的黏性摩擦项来近似这部分功。

图 11.11: 库仑摩擦模型

另一种做法是，在柔性表面中的固定位置嵌入一组弹簧-阻尼器对，并采用基于接触点位置的插值方案。

11.8.3 库仑摩擦

图 11.10 中的表面只在法线方向上是柔性的。我们现在添加一个切向柔性，得到图 11.11 所示的系统。（要将这个模型扩展到三维，我们必须添加第二个切向柔性。）这个额外的柔性可以用于实现一个库仑摩擦模型，其中摩擦力只是位置和速度变量的函数。因此，接触法向力和摩擦力都可以在主刚体动力学计算之前计算出来，并被正向动力学算法视为已知力。

我们引入两个新变量 x 和 v ，它们是 z 和 $\dot{p}$ 的切向对应量。 x 是表面片在其切向上偏离平衡位置的位移；而 v 是 B 中当前与 S 接触的那个点（即第 11.1 节中定义的点 C' ）的切向速度。所以 v 在每个时刻都指 B 中的不同点。 v 用于计算接触处的滑移速度（slip velocity）。

目标是要同时计算接触法向力 f_n 和切向力 f_t （即摩擦力）。第一步是用式 (11.74) 和式 (11.73) 计算 f_n 。这给我们

$$f_n = \begin{cases} 0 & \text{if } p > z \\ \max(0, -K_n z - D_n \dot{p}) & \text{if } p = z \end{cases}$$

以及

$$\dot{z} = -(K_n z + f_n)/D_n,$$

其中 K_n 和 D_n 是法线方向的弹簧和阻尼器系数。下一步是建立 S 在切向方向的运动方程。使用与式 (11.73) 相同的论证，这个方程为

$$\dot{x} = -(K_t x + f_t) / D_t,$$

其中 K_t 和 D_t 是切向方向的弹簧和阻尼器系数。下一步是求出为防止 B 和 S 之间发生任何滑动，切向力必须取什么值。没有滑动的条件是 $\dot{x} = v$ ，产生这个 $\dot{x}$ 值所需的力为

$$f_{stick} = -K_t x - D_t v.$$

计算库仑摩擦力的规则现在可以陈述如下：

$$f_t = \begin{cases} -\mu f_n & \text{if } f_{stick} < -\mu f_n \\ \mu f_n & \text{if } f_{stick} > \mu f_n \\ f_{stick} & \text{otherwise,} \end{cases}$$

其中 μ 是摩擦系数。用语言描述就是， f_t 被限制在摩擦锥（friction cone）内，这意味着它必须位于 $-\mu f_n$ 到 μf_n 的范围内。如果在这个范围内存在一个能够防止 B 和 S 之间滑动的力，那么它就是 f_t 的值；否则， f_t 位于最近的边界上，并且确实会发生一些滑动。

§ 11.9 进一步阅读

接触与碰撞动力学是庞大的主题，本章所提供的不过是一个简短的介绍。从这里出发，合乎逻辑的下一步是阅读 Coutinho (Coutinho, 2001) 和 Brach (Brach, 1991) 的著作。前者同时涵盖接触动力学和碰撞动力学，后者则专门研究碰撞。作为文献入门，可以阅读 Gilardi 和 Sharf (Gilardi and Sharf, 2002) 与 Stewart (Stewart, 2000) 的综述文章。后者更具数学性。另一篇面向数学爱好者的材料是由 Pfeiffer 和 Glocker (Pfeiffer and Glocker, 2000) 编辑的讲义汇编。摩擦建模是另一个本章几乎未曾涉及的庞大主题。关于这一主题的更多内容可见于 Armstrong-Hélouvry 等人 (1994)；Dupont 等人 (2002)；Haessig 和 Friedland (1991)。最后，更多相关材料可见于 Lötstedt (1981, 1982, 1984)；Marhefka 和 Orin (1999)；Nakamura 和 Yamane (2000)；Wittenburg (1977)；Yamane (2004)。

空间向量运算

本附录介绍了实现空间向量运算的两种方法：一种强调简单，另一种强调效率。简单方法同时针对平面向量和空间向量给出，而高效方法只针对空间向量。只要人的工作效率比计算效率更重要，简单方法就适用。它假定用户可以使用通用的矩阵运算工具（如 **Matlab**），¹ 并且只提供少数几个不属于标准矩阵运算的空间运算函数来利用这一工具。高效方法则使用紧凑的数据结构来高效地存储空间量，并利用各种优化和效率技巧来降低执行空间运算的成本。

12.0.1 A.1 简单平面运算

平面向量运算中的几乎所有基本操作都是标准的矩阵运算。因此，如果可以使用通用的矩阵运算工具（如 **Matlab**），那么就只需要少数几个额外的函数，这些函数如表 A.1 所示。

函数 **Xpln** 构造一个平面坐标变换矩阵。参见紧随其下的图示，两个参数 r 和 θ 分别指定坐标系 B 相对于坐标系 A 的位置和姿态；**Xpln** 返回的矩阵是从 A 坐标到 B 坐标的运动向量变换（即 $X = {}^B X_A$ ）。所有其他构造变换的函数都遵循同样的约定：参数描述新坐标系相对于当前坐标系的位置，函数返回从当前坐标系到新坐标系的运动向量坐标变换。要得到力向量变换，可以使用公式 ${}^B X_A^* = {}^B X_A^{-T}$ 。

函数 **crm** 和 **crf** 实现平面叉积算子。在这两种情况下，参数都是一个平面向量 $v = [\omega \ v_{Ox} \ v_{Oy}]^T$ ，表示一个速度。

表 A.1: 平面向量运算函数

这两个函数中的表达式 $v \times$ 和 $v \times^*$ 是输出变量的名称。严格来说，并不真正需要这两个函数，因为 $\text{crf}(v) = -\text{crm}(v)^T$ 。

函数 **mCl** 为平面刚体构造刚体惯性矩阵。参数 m 、 $c = [c_x \ c_y]^T$ 和 I_C 分别指定刚体的质量、质心位置以及绕质心的转动惯量。该函数通常用于初始化系统模型数据结构中的惯性矩阵。

最后，函数 **XtoV** 以如下方式从坐标变换计算运动向量。给定任意两个坐标系 A 和 B ，存在一个速度 u ，其性质是：如果坐标系 A 以该速度运动一个时间单位，那么它将最终与坐标系 B 重合。**XtoV** 的参数是从 A 到 B 坐标的变换，其中包含计算坐标系 B 相对于坐标系 A 位置所需的全部信息；计算得到的向量 v 是 u 的一个近似，随着 A 与 B 之间的夹角趋近于零而收敛到精确值。因此，当 X 是纯平移时， v 精确等于 u 。该函数给出两个版本。第一个较简单，但第二个更精确，且具有两个特殊性质：

(1) v 是 X 的不变量，即 $v = Xv$ ；(2) v 是 u 的标量倍数，两者通过公式 $u = v \sin^{-1}(X_{23})/X_{23}$ 相联系。该函数用于计算运动学约束方程中的位置误差。它在动力学中的主要用途是计算闭环位置误差，用于闭环约束稳定化（见 §8.3）。

12.0.2 A.2 简单空间运算

空间向量运算中的几乎所有操作都是标准的矩阵运算。因此，如果可以使用通用的矩阵运算工具（如 Matlab），那么它几乎可以提供所有必要的操作。只需要少数几个额外的函数，这些函数如表 A.2 所示。

函数 `rotx`、`roty` 和 `rotz` 分别构造坐标系 x 、 y 和 z 轴旋转的 Plücker 变换矩阵。特别地， $\mathbf{X}$ 将运动向量从 A 坐标变换到 B 坐标，其中坐标系 B 相对于坐标系 A 绕它们共同的 x 、 y 或 z 轴旋转角度 θ （视情况而定）。一般来说，力向量的变换矩阵可以通过公式 $\mathbf{X}^* = \mathbf{X}^{-T}$ 获得。然而，对于纯旋转，恰好有 $\mathbf{X}^* = \mathbf{X}$ 。函数 `rx`、`ry` 和 `rz` 使用表顶部的公式计算相应的 3×3 旋转矩阵。

函数 `xlt` 构造坐标系平移量 r 的 Plücker 变换矩阵。参数是一个三维向量，指定坐标系 B 相对于坐标系 A 的位置，返回值是从 A 到 B 坐标的 Plücker 变换。

函数 `crm` 和 `crf` 实现空间叉积算子。在这两种情况下，参数都是一个表示空间速度的运动向量。表达式 $v_{1:3}$ 和 $v_{4:6}$ 分别表示由 v 的前三个和最后三个 Plücker 坐标组成的三维向量；表达式 $v_{1:3} \times$ 和 $v_{4:6} \times$ 是根据表顶部的公式由 $v_{1:3}$ 和 $v_{4:6}$ 计算得到的 3×3 叉积矩阵；但这两个函数中的表达式 $v \times$ 和 $v \times^*$ 是输出变量的名称。

函数 `mcl` 构造刚体惯性矩阵。参数指定刚体的质量、质心位置以及绕质心的转动惯量。 c 是一个三维向量， I_C 是一个 3×3 矩阵。

最后，函数 `XtoV` 以与平面版本相同的方式从坐标变换计算运动向量。给定任意两个坐标系 A 和 B ，存在一个速度 u ，其性质是：如果坐标系 A 以该速度运动一个时间单位，那么它将最终与坐标系 B 重合。该函数的参数是从 A 到 B 坐标的变换，其中包含计算坐标系 B 相对于坐标系 A 位置所需的全部信息；计算得到的向量 v 是 u 的一个近似，随着 A 与 B 之间的夹角趋近于零而收敛到精确值。因此，当 $\mathbf{X}$ 是纯平移时， v 精确等于 u 。 v 也是 $\mathbf{X}$ 的不变量，所以 $v = \mathbf{X}v = \mathbf{X}^{-1}v$ ，但它不是 u 的标量倍数。该

表 A.2: 空间向量运算函数及辅助的三维公式

表 A.3: 空间量紧凑表示的数据结构

函数用于计算运动学约束方程中的位置误差。它在动力学中的主要用途是计算闭环位置误差，用于闭环约束稳定化（见 §8.3）。

12.0.3 A.3 紧凑表示

空间运算的纯矩阵实现中，大部分低效来源于使用一般的 6×6 矩阵来表示 Plücker 变换、刚体惯性和连杆刚体惯性。举一个极端的例子：计算 $\mathbf{X}^T \mathbf{I} \mathbf{X}$ ，其中 $\mathbf{X}$ 是一个 Plücker 变换， $\mathbf{I}$ 是一个刚体惯性，如果 $\mathbf{X}$ 和 $\mathbf{I}$ 被当作一般的 6×6 矩阵处理，则需要 432 次乘法和 360 次加法（两次 $6 \times 6 \times 6$ 矩阵乘法）；但如果充分利用这两个矩阵的特殊结构和性质，则只需 52 次乘法和 56 次加法即可完成该运算。这种低效来源可以通过定义特殊的数据结构来存储这些量，以及实现空间运算优化版本的特殊函数来消除。

表 A.3 说明了如何做到这一点。像 $\text{mv}(\omega, v)$ 这样的表达式是规定数据对象的类型和值的简写方式。在 $\text{mv}(\omega, v)$ 的情况下，类型是“运动向量”，值是一对三维向量。将该表达式与它所代表的数学量比较，可以看出 ω 包含空间运动向量的前三个 Plücker 坐标， v 包含其余的坐标。在惯性的数据结构中，表达式 $\text{lt}()$ 表示“下三角”。因此，数据结构 $\text{rbi}(m, h, \text{lt}(\mathbf{I}))$ 包含标量 m 、三维向量 h 和下三角

矩阵 $\mathbf{I}$ 的下三角，共 $1 + 3 + 6 = 10$ 个数。

严格来说，数据类型 **mv** 和 **fv** 并不比它们所代表的六维向量更紧凑，这就引出一个问题：它们为何值得存在？答案是类型安全。通过强制程序员将每个向量变量声明为运动向量或力向量，编译器就有可能检测出各种不正确的表达式，如 $m + f$ 、 $m \cdot m$ 、 $f \times f$ 、 $f = m$ 、 If 和 X^*m 。在某些编程语言中，甚至可以更进一步，让编译器根据操作数的类型自动选择合适的运算版本。下面的代码片段给出了一个例子（使用一种虚构的编程语言）。

```
variable X : PluXfm; variable f1, f2 : SpFvec; variable I1, I2 : SpRBI; X := (plx(.); f1 := fv(.); I1 := rbi(.); f2 := X.apply(f1); I2 := X.apply(I1);
```

在这个例子中，前六行声明了五个变量并初始化其中三个；随后两行将 **X** 所描述的 **Plücker** 变换分别作用于 **f1** 所描述的力向量和 **I1** 所描述的刚体惯性。这里的思想是：**apply** 函数在软件库中以多个不同版本存在——每种可被变换的对象对应一个版本——编译器根据参数的类型选择正确的版本。如果参数是力向量，编译器就选择计算 X^*f 的版本；如果参数是刚体惯性，编译器就选择计算 X^*IX^{-1} 的版本。通过以这种方式设计软件，可以使程序员不必记住每种对象的坐标变换规则。

任何设计良好的空间运算软件包还会包含一个 **invApply** 函数，它用于施加 **X** 所描述变换的逆变换。因此，如果我们在上面的例子中加上如下几行：

```
variable f3 : SpFvec; f3 := X.invApply(f2);
```

那么 **f3** 除了舍入误差外将与 **f1** 完全相同。这比显式求 **X** 的逆并施加该逆变换更高效。同样，编译器会根据待变换对象的类型选择合适的 **invApply** 版本。

表 A.4 列出了高效空间运算软件包应提供的大部分算术运算，以及使用紧凑数据结构实现这些运算的公式和它们的计算成本。

表 A.4: 空间运算公式及成本数据

这些数据。成本列中的符号 **m** 和 **a** 分别指浮点乘法和加法，减法按加法计。为避免名称冲突，符号 $\hat{v}$ 、 $\hat{f}$ 和 $\hat{I}$ 分别表示空间运动（速度）、力和刚体惯性。这里引用的成本数据是针对计算的优化实现。例如，像 $r \times M$ 这样的计算被视为一个斜对称矩阵与一个对称矩阵的乘积，只需 $15m + 9a$ 。同样，像 EME^T 这样的计算假定使用第 A.5 节中的相应效率技巧来实现，只需 $28m + 29a$ 。**X.apply(...)** 和 **X.invApply(...)** 将使用的坐标变换公式如下：

请记住，此表中的成本数据并非效率的最终结论。特别是，如第 A.4 节所述，使用轴向螺旋变换代替一般的 **Plücker** 变换可以获得显著的进一步改进。

为了了解表 A.4 中的公式和成本数据是如何得到的，让我们详细考察运算 $X^*\hat{I}X^{-1}$ 。用 6×6 矩阵表示，我们有

$$\begin{aligned}
\mathbf{X}^* \hat{\mathbf{I}} \mathbf{X}^{-1} &= \begin{bmatrix} E & 0 \\ 0 & E \end{bmatrix} \begin{bmatrix} 1 & -r \times \\ 0 & 1 \end{bmatrix} \begin{bmatrix} I & h \times \\ -h \times & m\mathbf{1} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ r \times & 1 \end{bmatrix} \begin{bmatrix} E^T & 0 \\ 0 & E^T \end{bmatrix} \\
&= \begin{bmatrix} E & 0 \\ 0 & E \end{bmatrix} \begin{bmatrix} I + r \times h \times & h \times -mr \times \\ -h \times & m\mathbf{1} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ r \times & 1 \end{bmatrix} \begin{bmatrix} E^T & 0 \\ 0 & E^T \end{bmatrix} \\
&= \begin{bmatrix} E & 0 \\ 0 & E \end{bmatrix} \begin{bmatrix} I + r \times h \times + y \times r \times & y \times \\ -h \times & m\mathbf{1} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ r \times & 1 \end{bmatrix} \begin{bmatrix} E^T & 0 \\ 0 & E^T \end{bmatrix} \\
&= \begin{bmatrix} E & 0 \\ 0 & E \end{bmatrix} \begin{bmatrix} I + r \times h \times + y \times r \times & y \times \\ -h \times + mr \times & m\mathbf{1} \end{bmatrix} \begin{bmatrix} E^T & 0 \\ 0 & E^T \end{bmatrix} \\
&= \begin{bmatrix} E & 0 \\ 0 & E \end{bmatrix} \begin{bmatrix} Z & y \times \\ -y \times & m\mathbf{1} \end{bmatrix} \begin{bmatrix} E^T & 0 \\ 0 & E^T \end{bmatrix} \\
&= \begin{bmatrix} E & 0 \\ 0 & E \end{bmatrix} \begin{bmatrix} EZE^T & Ey \times E^T \\ -Ey \times E^T & m\mathbf{1} \end{bmatrix} \\
&= \begin{bmatrix} EZE^T & (Ey) \times \\ -(Ey) \times & m\mathbf{1} \end{bmatrix},
\end{aligned}$$

其中

$$y = h - mr \text{ and } Z = I + r \times h \times + y \times r \times.$$

这与表中的公式完全一致。要得到成本数据，我们按如下方式进行：
（关于如何计算 EZE^T ，见第 A.5 节。）

12.0.4 A.4 轴向螺旋变换

轴向螺旋变换是绕单一坐标轴的旋转与沿该轴的平移的组合。我们将用表达式 $\text{scrx}(\theta, d)$ 、 $\text{scry}(\theta, d)$ 和 $\text{scrz}(\theta, d)$ 分别表示绕 x 、 y 和 z 轴的轴向螺旋变换，其中旋转角度为 θ ，平移量为 d 。因此，

$$\text{scrx}(\theta, d) = \text{rotx}(\theta) \text{xlt}([d \ 0 \ 0]^T) = \text{xlt}([d \ 0 \ 0]^T) \text{rotx}(\theta)$$

（因为旋转与平移可交换），以此类推。描述轴向螺旋变换的紧凑数据结构将包含三个基本量 $c = \cos(\theta)$ 、 $s = \sin(\theta)$ 和 d ，以及派生量 s^2 、 cs 、 $2cs$ 和 $1 - 2s^2$ ，如第 A.5 节所述，这些派生量用于降低旋转矩阵的重复成本。

轴向螺旋变换之所以有用，是因为它们可以高效地执行。表 A.5 显示了将轴向螺旋变换应用于各种空间量的成本。量 I^a 之所以被包括进来，是因为它与连杆刚体算法相关。它将在本节稍后讨论。总成本被划分为重复成本（即施加变换的成本）和单次成本（即计算上述量 s^2 、 cs 、 $2cs$ 和 $1 - 2s^2$ 的成本）。如果角度是常数，那么这一

表 A.5: 轴向螺旋变换的计算成本

计算可以在离线时进行，或在程序执行开始时进行；否则，每次旋转角度变化时都必须重新计算。

比较表 A.5 与表 A.4 中的成本可以看出，轴向螺旋变换的成本约为一般 Plücker 变换成本的

12.0.5 向量

将 Xm (其中 $X = \text{scrz}(\theta, d)$) 的表达式完整写出, 即为

$$\begin{bmatrix} c & s & 0 & 0 & 0 & 0 \\ -s & c & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ -sd & cd & 0 & c & s & 0 \\ -cd & -sd & 0 & -s & c & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} m_x \\ m_y \\ m_z \\ m_{Ox} \\ m_{Oy} \\ m_{Oz} \end{bmatrix} = \begin{bmatrix} c(m_{Ox} + m_y d) + s(m_{Oy} - m_x d) \\ c(m_{Oy} + m_y d) + s(m_{Oy} - m_x d) \\ c(m_{Oy} - m_x d) - s(m_{Ox} + m_y d) \\ m_{Oz} \end{bmatrix}$$

该运算的成本显然是 $10m + 6a$ 。容易证明, 计算 X^*f 的成本也是 $10m + 6a$, 且绕另外两个轴的螺旋变换成本相同。

12.0.6 刚体惯性

轴向螺旋变换使用表 A.4 中的公式施加于刚体惯性, 但由于 E 和 r 的特殊形式, 计算成本有所降低。运算 X^*IX^{-1} 的成本分解如下:

EZE^T 的计算使用第 A.5 节中相应的效率技巧完成。 Z 的计算过程如下: 如果 $r = [0 \ 0 \ d]^T$, 则

$$\text{lt}(r \times h \times + y \times r \times) = \begin{bmatrix} -(h_z + y_z)d & : \\ 0 & -(h_z + y_z)d \\ h_x d & h_y d \end{bmatrix}.$$

计算该表达式的成本是 $3m + 1a$, 将其加到 $\text{lt}(I)$ 上的成本是 $4a$ 。如果 $r = [d \ 0 \ 0]^T$ 或 $[0 \ d \ 0]^T$, 则细节将不同, 但成本相同。

12.0.7 连杆刚体惯性

轴向螺旋变换使用表 A.4 中的公式施加于连杆刚体惯性, 但由于 E 和 r 的特殊形式, 计算成本有所降低。运算 $X^*I^A X^{-1}$ 的成本分解如下:

($X^T I^A X$ 的成本分解和总成本相同。) 三次旋转使用第 A.5 节中相应的效率技巧执行。计算 EYE^T 的 $2m$ 单次成本已包含在其他两次旋转的 $2m + 3a$ 单次成本中。计算 Y 和 Z 的成本数据可以这样理解。如果 $r = [0 \ 0 \ d]^T$, 则

$$r \times M = \begin{bmatrix} 0 & -d & 0 \\ d & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \\ M_{31} & M_{32} & M_{33} \end{bmatrix} = \begin{bmatrix} -M_{21}d & -M_{22}d & -M_{23}d \\ M_{11}d & M_{12}d & M_{13}d \\ 0 & 0 & 0 \end{bmatrix}$$

以及

$$\text{lt}(-r \times H^T + Yr \times) = \begin{bmatrix} (H_{12} + Y_{12})d & \cdot & \cdot \\ (Y_{22} - H_{11})d & -(H_{21} + Y_{21})d & \cdot \\ H_{32}d & -H_{31}d & 0 \end{bmatrix}.$$

计算 $r \times M$ 的成本是 $5m$ (因为 $M_{12} = M_{21}$)；将其从 H 中减去的成本是 $6a$ ；计算 $\text{lt}(-r \times H^T + Yr \times)$ 的成本是 $5m + 3a$ ；将其加到 $\text{lt}(I)$ 上的成本是 $5a$ 。如果 $r = [d \ 0 \ 0]^T$ 或 $[0 \ d \ 0]^T$ ，则细节将不同，但成本相同。

12.0.8 零值行列的连杆刚体惯性

连杆刚体算法中单项最昂贵的运算是将连杆刚体惯性从一个坐标系变换到另一个坐标系。然而，被变换的量并不是一般的连杆刚体惯性，而是如下形式：

$$I^a - I^A - I^A S (S^T I^A S)^{-1} S^T I^A.$$

该量具有如下特殊性质：如果 S 是一个具有 $6 - d$ 个零值行的 $6 \times d$ 矩阵，那么 I^a 中与 S 的非零行对应的所有行和列都将为零。例如，如果 $S = [0 \ 0 \ 1 \ 0 \ 0 \ 0]^T$ (转动关节)，则 I^a 的第 3 行和第 3 列将为零；如果 $S = [0_{3 \times 2} \ 1_{3 \times 3} \ 0_{3 \times 1}]^T$ (平面关节)，则 I^a 的第 3、4 和 5 行以及第 3、4 和 5 列都将为零。

利用 I^a 的这一特殊性质可以显著提高连杆刚体算法的速度。要创建定制的坐标变换过程，可以从一般连杆刚体惯性的过程出发，然后利用 I^a 的特殊形式进行各种简化。表 A.5 中给出了两个这种定制变换的成本数据，每个变换的成本分解见下文。在这两种情况下， I^a 的第 3 行和第 3 列均为零。注意， scrz 保持该行和该列中的零不变，而 scrx 和 scry 则不能。在典型应用中， I^a 从一个 Denavit-Hartenberg 坐标系变换到另一个坐标系时，首先施加 scrz ，然后对结果施加 scrx 。该过程的总成本 (仅包括 scrz 的单次成本，因为 x 旋转角度是常数) 为 $62m + 65a$ 。这不到表 A.4 中一般变换成本的一半，并且比 McMillan 和 Orin (1995) 报告的数据大约好 10

12.0.9 A.5 一些效率技巧

12.0.10 旋转矩阵的乘积

一个 3×3 旋转矩阵的任意一行或一列都是其余两行或两列的向量积。因此，如果 F 和 G 是两个旋转矩阵，且 $E = FG$ ，则 E 可以按如下方式计算：

$$\begin{bmatrix} E_{11} & E_{12} \\ E_{21} & E_{22} \\ E_{31} & E_{32} \end{bmatrix} = \begin{bmatrix} F_{11} & F_{12} & F_{13} \\ F_{21} & F_{22} & F_{23} \\ F_{31} & F_{32} & F_{33} \end{bmatrix} \begin{bmatrix} G_{11} & G_{12} \\ G_{21} & G_{22} \\ G_{31} & G_{32} \end{bmatrix},$$

$$\begin{bmatrix} E_{13} \\ E_{23} \\ E_{33} \end{bmatrix} = \begin{bmatrix} E_{11} \\ E_{21} \\ E_{31} \end{bmatrix} \times \begin{bmatrix} E_{12} \\ E_{22} \\ E_{32} \end{bmatrix}.$$

该计算的总成本是 $24m + 15a$ ，相对于标准矩阵乘法节省了 $3m + 3a$ 。更一般地，计算 $E = F_1 F_2 \cdots F_n G$ 的成本是 $(18m + 12a)n + 6m + 3a$ 。

12.0.11 一般矩阵的旋转

考虑计算 $B = EAE^T$ ，其中 E 是旋转矩阵， A 是一般 3×3 矩阵。该计算可以作为两次 $3 \times 3 \times 3$ 乘法完成，总成本为 $54m + 36a$ 。然而，可以将这一成本降低到 $40m + 39a$ ，方法如下。

首先, 将 A 分解为三个分量:

$$A = L + D + v,$$

其中

$$L = \begin{bmatrix} A_{11} - A_{33} & A_{12} & 0 \\ A_{21} & A_{22} - A_{33} & 0 \\ A_{31} + A_{13} & A_{32} + A_{23} & 0 \end{bmatrix}, D = \begin{bmatrix} A_{33} & 0 & 0 \\ 0 & A_{33} & 0 \\ 0 & 0 & A_{33} \end{bmatrix}, v = \begin{bmatrix} -A_{23} \\ A_{13} \\ 0 \end{bmatrix}.$$

现在可以将 B 表示为

$$\begin{aligned} B &= E(L + D + v \times) E^T \\ &= ELE^T + D + (Ev) \times. \end{aligned}$$

定义 $Y = EL$ 和 $Z = YE^T$, 我们计算

$$\begin{bmatrix} Y_{11} & Y_{12} \\ Y_{21} & Y_{22} \\ Y_{31} & Y_{32} \end{bmatrix} = \begin{bmatrix} E_{11} & E_{12} & E_{13} \\ E_{21} & E_{22} & E_{23} \\ E_{31} & E_{32} & E_{33} \end{bmatrix} \begin{bmatrix} L_{11} & L_{12} \\ L_{21} & L_{22} \\ L_{31} & L_{32} \end{bmatrix},$$

$$\begin{bmatrix} \cdot & Z_{12} & Z_{13} \\ Z_{21} & Z_{22} & Z_{23} \\ Z_{31} & Z_{32} & Z_{33} \end{bmatrix} = \begin{bmatrix} Y_{11} & Y_{12} \\ Y_{21} & Y_{22} \\ Y_{31} & Y_{32} \end{bmatrix} \begin{bmatrix} E_{11} & E_{21} & E_{31} \\ E_{12} & E_{22} & E_{32} \end{bmatrix}$$

以及

$$Z_{11} = L_{11} + L_{22} - Z_{22} - Z_{33}.$$

(最后一个方程利用了迹在旋转下的不变性。或者, Z_{11} 也可以与 Z 的其余部分一起计算, 成本为 $2m + 1a$ 。)最后, Ev 通过 E 最左边两列与 v 前两个元素之间的 $3 \times 2 \times 1$ 矩阵乘法计算, 并将量 Z 、 D 和 $(Ev) \times$ 相加。该过程的总成本计算如下:

这略高于绕坐标轴连续三次旋转的重复成本。

12.0.12 对称矩阵的旋转

现在考虑计算 $B = EAE^T$, 其中 E 是一般 3×3 旋转矩阵, A 是对称矩阵。该计算可以使用与一般矩阵 A 完全相同的过程, 但对称性允许一些简化。具体来说, 我们只需要计算 B 的下三角。因此, 无需计算 Z_{12} 、 Z_{13} 或 Z_{23} ; 进而也无需计算 Y_{11} 或 Y_{12} 。于是, Y 和 Z 的计算简化为

$$\begin{bmatrix} Y_{21} & Y_{22} \\ Y_{31} & Y_{32} \end{bmatrix} = \begin{bmatrix} E_{21} & E_{22} & E_{23} \\ E_{31} & E_{32} & E_{33} \end{bmatrix} \begin{bmatrix} L_{11} & L_{12} \\ L_{21} & L_{22} \\ L_{31} & L_{32} \end{bmatrix}$$

以及

$$\begin{bmatrix} Z_{21} & Z_{22} & \cdot \\ Z_{31} & Z_{32} & Z_{33} \end{bmatrix} = \begin{bmatrix} Y_{21} & Y_{22} \\ Y_{31} & Y_{32} \end{bmatrix} \begin{bmatrix} E_{11} & E_{21} & E_{31} \\ E_{12} & E_{22} & E_{32} \end{bmatrix},$$

其中 Z_{11} 的计算方法同前。该简化过程的总成本为：

与两次 $3 \times 3 \times 3$ 矩阵乘法的成本相比，这是一个显著的改进，但不如绕坐标轴连续三次旋转那么快。

12.0.13 绕坐标轴旋转一般矩阵

假设我们希望计算 $B = EAE^T$ ，其中 E 是 $\text{rx}(\theta)$ 、 $\text{ry}(\theta)$ 或 $\text{rz}(\theta)$ ， A 是一般 3×3 矩阵。这三个旋转的公式可以写成如下形式。²

$$\begin{aligned} \text{rx}(\theta) \mathbf{A} \text{rx}(\theta)^T &= \begin{bmatrix} A_{11} & cA_{12} + sA_{13} & cA_{13} - sA_{12} \\ cA_{21} + sA_{31} & A_{22} + \alpha_x & A_{23} + \beta_x \\ cA_{31} - sA_{21} & A_{32} + \beta_x & A_{33} - \alpha_x \end{bmatrix} \\ \text{ry}(\theta) \mathbf{A} \text{ry}(\theta)^T &= \begin{bmatrix} A_{11} - \alpha_y & cA_{12} - sA_{32} & A_{13} + \beta_y \\ cA_{21} - sA_{23} & A_{22} & cA_{23} + sA_{21} \\ A_{31} + \beta_y & cA_{32} + sA_{12} & A_{33} + \alpha_y \end{bmatrix} \\ \text{rz}(\theta) \mathbf{A} \text{rz}(\theta)^T &= \begin{bmatrix} A_{11} + \alpha_z & A_{12} + \beta_z & cA_{13} + sA_{23} \\ A_{21} + \beta_z & A_{22} - \alpha_z & cA_{23} - sA_{13} \\ cA_{31} + sA_{32} & cA_{32} - sA_{31} & A_{33} \end{bmatrix} \end{aligned}$$

其中

$$\begin{aligned} \alpha_x &= cs(A_{23} + A_{32}) + s^2(A_{33} - A_{22}) \\ \alpha_y &= cs(A_{31} + A_{13}) + s^2(A_{11} - A_{33}) \\ \alpha_z &= cs(A_{12} + A_{21}) + s^2(A_{22} - A_{11}) \\ \beta_x &= cs(A_{33} - A_{22}) - s^2(A_{23} + A_{32}) \\ \beta_y &= cs(A_{11} - A_{33}) - s^2(A_{31} + A_{13}) \\ \beta_z &= cs(A_{22} - A_{11}) - s^2(A_{12} + A_{21}) \\ s &= \sin(\theta) \text{ and } c = \cos(\theta). \end{aligned}$$

在所有情况下，成本相同，计算方式如下：

请注意，总成本已被划分为重复成本和单次成本。这里的思想是定义一个轴向旋转数据结构，其中包含值 c 、 s 、 cs 和 s^2 ，在执行旋转时这四个量均可获得。因此，单次成本是在数据结构初始化或更新时产生的成本，³ 而重复成本是每次执行旋转时产生的成本。这种策略使我们利用这样一个事实：动力学计算中的许多旋转角度是已知常数，而且即使旋转角度变化，数据结构在两次变化之间也常常会被使用不止一次。

12.0.14 绕坐标轴旋转对称矩阵

现在考虑计算 $B = EAE^T$ ，其中 E 是 $\text{rx}(\theta)$ 、 $\text{ry}(\theta)$ 或 $\text{rz}(\theta)$ ， A 是对称 3×3 矩阵。这些旋转的公式是旋转一般矩阵公式的简化版本，可以写成如下形式：

$$\text{lt}(\text{rx}(\theta) \mathbf{A} \text{rx}(\theta)^T) = \begin{bmatrix} A_{11} & \cdot & \cdot \\ cA_{21} + sA_{31} & A_{22} + \alpha_x & \cdot \\ cA_{31} - sA_{21} & \beta_x & A_{33} - \alpha_x \end{bmatrix}$$

$$\text{lt}(\text{ry}(\theta) \mathbf{A} \text{ry}(\theta)^T) = \begin{bmatrix} A_{11} - \alpha_y & \cdot & \cdot \\ cA_{21} - sA_{23} & A_{22} & \cdot \\ \beta_y & cA_{32} + sA_{12} & A_{33} + \alpha_y \end{bmatrix} \quad \text{lt}(\text{rz}(\theta) \mathbf{A} \text{rz}(\theta)^T) = \begin{bmatrix} A_{11} + \alpha_z & \cdot & \cdot \\ \beta_z & A_{22} - \alpha_z & \cdot \\ cA_{31} + sA_{32} & cA_{32} - sA_{31} & A_{33} \end{bmatrix}$$

其中

$$\begin{aligned} \alpha_x &= 2csA_{32} + s^2(A_{33} - A_{22}) \\ \alpha_y &= 2csA_{31} + s^2(A_{11} - A_{33}) \\ \alpha_z &= 2csA_{21} + s^2(A_{22} - A_{11}) \\ \beta_x &= cs(A_{33} - A_{22}) + (1 - 2s^2)A_{32} \\ \beta_y &= cs(A_{11} - A_{33}) + (1 - 2s^2)A_{31} \\ \beta_z &= cs(A_{22} - A_{11}) + (1 - 2s^2)A_{21} \end{aligned}$$

在所有情况下，成本相同，计算方式如下：

再次强调，总成本已被分解为重复成本和单次成本。假设轴向旋转数据结构中包含 c 、 s 、 cs 、 s^2 、 $2cs$ 以及 $1 - 2s^2$ 这些值，而这些值在执行旋转时即可获取。因此，单次成本在数据结构初始化或更新时产生，而重复成本则在每次执行旋转时产生。

数量	计算成本
重复使用	单次
cs, s^2	$2m$
$2cs, (1 - 2s^2)$	$3a$
,	$4m + 3a$
B 部分其余部分	$4m + 4a$
总计：	$8m + 7a$

12.0.15 参考文献

- Amirouche, F. M. L. 2006. 多体动力学基础：理论与应用。波士顿：Birkhäuser 出版社。
- Angeles, J. 2003. 机器人机械系统基础（第 2 版）纽约：Springer-Verlag 出版社。
- Armstrong, W. W. 1979. 用于 n 个连杆机械臂运动方程的递归求解方法。第五届世界机器与机构理论大会论文集（蒙特利尔），第 1343-1346 页，7 月。
- Armstrong-Hélouvry, B.、Dupont, P. 和 Canudas de Wit, C. 1994. 用于控制带摩擦机械的模型、分析工具及补偿方法综述。Automatica，第 30 卷，第 7 期，第 1083-1138 页。
- Ascher, U. M.、Pai, D. K. 和 Cloutier, B. P. 1997. 机器人仿真中的正向动力学、消元方法及刚度建模。国际机器人研究期刊，第 16 卷，第 6 期，第 749-758 页。

Baker, E. J. 和 Wohlhart, K. 1996. 电机计算：一种全新的力学理论装置。奥地利格拉茨：格拉茨工业大学力学研究所。（译自 von Mises 1924a 和 1924b。）

Balafoutis, C. A.、Patel, R. V. 和 Misra, P. 1988. 利用正交笛卡尔张量高效建模并计算机械臂动力学。IEEE 机器人与自动化杂志，第 4 卷，第 6 期，第 665-676 页。

Balafoutis, C. A. 和 Patel, R. V. 1989. 机械臂惯性矩阵的高效计算及直接动力学问题的解决。IEEE 系统、人机与控制论汇刊，第 19 卷，第 5 期，第 1313-1321 页。

Balafoutis, C. A. 和 Patel, R. V. 1991. 机器人机械臂的动力学分析：基于笛卡尔张量的方法。波士顿：Kluwer 学术出版社。

Ball, R. S. 1900. 螺旋理论论著。伦敦：剑桥大学出版社。1998 年再版。

Baraff, D. 1996. 使用拉格朗日乘数实现线性时间动力学。ACM SIGGRAPH '96 论文集，新奥尔良，8 月 4 日至 9 日，第 137-146 页。

鲍姆加特，J. 1972. 动力系统中约束与运动积分的稳定化。《计算机方法在应用力学与工程学》期刊，第 1 卷，第 1-16 页。

鲍布罗夫，J. E. 1989. 用于求解凸多面体之间距离的直接最小化方法。《国际机器人研究杂志》，第 8 卷，第 3 期，第 65-76 页。

布拉赫，R. M. 1991. 《机械冲击动力学》。纽约：Wiley 出版社。

布雷迪，M.、霍勒巴赫，J. M.、约翰逊，T. L.、洛萨诺-佩雷斯，T.、梅森，M. T.（编者）1982. 《机器人运动：规划与控制》。马萨诸塞州剑桥市：麻省理工学院出版社。

布兰德，L. 1953. 《向量与张量分析》（第 4 版）。纽约/伦敦：Wiley/查普曼与霍尔出版社。

布兰德尔，H.、约翰尼，R.、奥特尔，M. 1988. 一种非常高效的算法，用于在不求逆质量矩阵的情况下模拟机器人及类似多体系统。载于《机器人理论》，P. 科帕切克、I. 特罗赫和 K. 德索耶（编者），牛津：佩加蒙出版社，第 95-100 页。

科特，R. W.、丹齐格，G. B. 1968. 数学规划的互补 pivot 理论。《线性代数及其应用》期刊，第 1 卷，第 103-125 页。

科特，R. W.、庞，J.-S.、斯通，R. E. 1992. 《线性互补问题》。波士顿：学术出版社。

库蒂尼奥，M. G. 2001. 多体系统的动态仿真。纽约：Springer 出版社。

克雷格，J. J. 2005. 《机器人学导论：力学与控制》（第 3 版）。新泽西州上萨德尔河：培生普林斯顿出版社。

德纳维特，J.、哈滕贝格，R. S. 1955. 基于矩阵的低副机构运动学符号法。《美国机械工程师学会期刊·应用力学》第 22 卷，第 215-221 页。

杜博夫斯基，S.、帕帕多普洛斯，E. 1993. 自由飞行与自由漂浮空间机器人系统的运动学、动力学与控制。《IEEE 机器人与自动化技术汇刊》，第 9 卷，第 5 期，第 531-543 页。

达夫，I.S.、埃里斯曼，A. M.、里德，J. K. 1986. 稀疏矩阵的直接方法。牛津：克拉伦登出版社。

达菲，J. 1990. 基于“扭转空间”与“力矩空间”“正交补”的现代混合控制理论的谬误。《机器人系统杂志》，第 7 卷，第 2 期，第 139-144 页。

杜邦，P.、海沃德，V.、阿姆斯特朗，B.、阿尔特彼得，F. 2002. 单状态弹塑性摩擦模型。《IEEE 自动控制汇刊》，第 47 卷，第 5 期，第 787-792 页。

方，A. C.、波拉德，N. S. 2003. 高效合成符合物理规律的人类运动。ACM 图形学汇刊（SIGGRAPH 2003），第 22 卷，第 3 期，第 417-426 页。

费瑟斯通, R. 1983. 利用连杆体惯量计算机器人动力学。《国际机器人研究杂志》, 第 2 卷, 第 1 期, 第 13-30 页。

费瑟斯通, R. 1984. 机器人动力学算法。爱丁堡大学博士论文。

费瑟斯通, R. 1987. 《机器人动力学算法》。波士顿: 克鲁威尔学术出版社。

费瑟斯通, R. 1999a. 一种分治法的连杆体算法, 用于并行计算刚体动力学的 $O(\log(n))$ 时间复杂度。第一部分: 基本算法。《国际机器人研究杂志》, 第 18 卷, 第 9 期, 第 867-875 页。

费瑟斯通, R. 1999b. 一种分治法的连杆体算法, 用于并行计算刚体动力学的 $O(\log(n))$ 时间复杂度。第二部分: 树、循环与精度。《国际机器人研究杂志》, 第 18 卷, 第 9 期, 第 876-892 页。

费瑟斯通, R. 2004. 关节空间惯性矩阵的实证研究。《国际机器人研究杂志》, 第 23 卷, 第 9 期, 第 859-871 页。

费瑟斯通, R. 2005. 用于分支运动学树的关节空间惯性矩阵的高效分解。《国际机器人研究杂志》, 第 24 卷, 第 6 期, 第 487-500 页。

戈蒂耶, M., 和哈立勒, W. 1990. 直接计算串联机器人的最小惯性参数集。IEEE 机器人与自动化技术汇刊, 第 6 卷, 第 3 期, 第 368-373 页。

吉尔, C. W. 1971. 常微分方程中的数值初值问题。新泽西州恩格尔伍德悬崖: 普伦蒂斯-霍尔出版社。

乔治, A., 和刘, J. W. H. 1981. 大型稀疏正定系统计算机求解。新泽西州恩格尔伍德悬崖: 普伦蒂斯-霍尔出版社。

吉拉尔迪, G., 和沙夫, I. 2002. 接触动力学建模的文献综述。《机构与机器理论》, 第 37 卷, 第 10 期, 第 1213-1239 页。

吉尔伯特, E. G., 约翰逊, D. W., 和基尔蒂, S. S. 1988. 在三维空间中快速计算复杂物体之间距离的算法。IEEE 机器人与自动化技术期刊, 第 4 卷, 第 2 期, 第 193-203 页。

用于快速干扰检测的结构。ACM SIGGRAPH '96 大会论文集, 新奥尔良, 8 月 4 日至 9 日, 第 171-180 页。

格林伍德, D. T. 1988. 动力学原理。新泽西州恩格尔伍德悬崖: 普伦蒂斯-霍尔出版社。

哈西格, D. A., 和弗里德兰德, B. 1991. 关于摩擦建模与仿真。ASME 动态系统、测量与控制杂志, 第 113 卷, 第 3 期, 第 354-362 页。

何, X., 和戈尔登伯格, A. A. 1989. 用于高效计算机器人机械臂动力学的算法。第四届先进机器人国际会议(俄亥俄州哥伦布市)论文集, 第 175-188 页, 6 月。

霍勒巴赫, J. M. 1980. 机械臂动力学的递归拉格朗日形式及其动力学形式复杂度的比较研究。IEEE 系统、人机与控制技术汇刊, 第 SMC-10 卷, 第 11 期, 第 730-736 页。

胡克, W. W., 和玛古利斯, G. 1965. n 体卫星的动力学姿态方程。《航天科学杂志》, 第 12 卷, 第 4 期, 第 123-128 页。

胡克, W. W. 1970 年。针对具有 r 个旋转自由度的任意 n 体卫星, 一套动力学姿态方程。AIAA 期刊, 第 8 卷(2 期), 第 7 号, 第 1205-1207 页。

胡, W., 马赫夫卡, D. W. 和奥林, D. E. 2005 年。跑步机器的混合运动学与动力学仿真。IEEE 机器人技术汇刊, 第 21 卷, 第 3 期, 第 490-497 页。

亨特, K. H. 1978 年。机构的运动学几何。牛津: 牛津大学出版社。

哈斯顿, R. L. 1990 年。多体动力学。波士顿: 巴特沃斯出版社。

贾因, A. 1991 年。串行刚性多体系统的动力学统一公式。《导航、控制与动力学杂志》, 第 14 卷, 第 3 期, 第 531-542 页。

贾因, A 和罗德里格斯, G. 1993 年。欠驱动机械手的运动学与动力学分析。IEEE 机器人与自动化技术汇刊, 第 9 卷, 第 4 期, 第 411-422 页。

希梅内斯, P、托马斯, F. 和托拉斯, C. 2001 年。三维碰撞检测: 综述。《计算机与图形——英国》, 第 25 卷, 第 2 期, 第 269-285 页。

卡恩, M·E. 和罗斯, B. 1971 年。开放回路关节运动学链的近最小时间控制。《动态系统、测量与控制杂志》, 第 93 卷, 第 164-172 页。

哈利勒, W. 和克莱因芬格, J·F. 1987 年。树形机器人动力学模型的最小运算量与最小参数。IEEE 机器人与自动化技术杂志, 第 3 卷, 第 6 期, 第 517-526 页。

哈利勒, W. 和东布雷, E. 2002 年。机器人的建模、辨识与控制。纽约州, 纽约: 泰勒与弗朗西斯出版社。

哈提卜, O. 1987 年。机器人机械手运动与力控制的统一方法: 操作空间公式。IEEE 机器人与自动化技术杂志, 第 3 卷, 第 1 期, 第 43-53 页。

哈提卜, O. 1995 年。机器人操作中的惯性特性: 基于对象级的框架。国际机器人研究杂志, 第 14 卷, 第 1 期, 第 19-36 页。

拉思罗普, R·H. 1985 年。机械手动力学中的并联性。国际机器人研究杂志, 第 4 卷, 第 2 期, 第 80-102 页。

莉莉, K·W. 和奥林, D·E. 1991 年。机械手惯性矩阵的替代公式。国际机器人研究杂志, 第 10 卷, 第 1 期, 第 64-74 页。

莉莉, K·W. 1993 年。机器人机构的高效动力学仿真。波士顿: 克鲁尔学术出版社。

洛特施泰特, P. 1981 年。二维刚体系统中的库仑摩擦。应用数学与力学杂志, 第 61 卷, 第 12 期, 第 605-615 页。

洛特施泰特, P. 1982 年。受单侧约束的刚体机械系统。美国工业与应用数学学会期刊, 第 42 卷, 第 2 期, 第 281-296 页。

勒特施泰特, P. 1984。刚体力学中时变接触与摩擦问题的数值模拟。SIAM 期刊——科学与统计计算, 第 5 卷, 第 2 期, 第 370-393 页。

朗曼, R. W.、林德伯格, R. E.、泽德, M. F. 1987。卫星搭载机器人机械手——新型运动学与反作用力矩补偿。国际机器人研究杂志, 第 6 卷, 第 3 期, 第 87-103 页。

卢, J. Y.S.、沃克, M. W.、保罗, R. P. C. 1980a。机械机械手在线计算方案。美国机械工程师学会期刊——动态系统、测量与控制, 第 102 卷, 第 2 期, 第 69-76 页。

卢, J. Y.S.、沃克, M. W.、保罗, R. P. C. 1980b。机械机械手的解算加速度控制。IEEE 自动控制汇刊, 第 25 卷, 第 3 期, 第 468-474 页。

麦克米伦, S. 1994。陆地与水下机器人系统的计算动力学。博士论文, 俄亥俄州立大学电气工程系。

利用连续轴向螺钉实现惯性。IEEE 机器人与自动化汇刊, 第 11 卷, 第 4 期, 第 606-611 页。

麦克米伦, S.、奥林, D. E.、麦基, R. B. 1995。具有机器人机械手的水下航行器高效动力学仿真。IEEE 系统、人机与控制汇刊, 第 25 卷, 第 8 期, 第 1194-1206 页。

麦克米伦, S.、奥林, D. E. 1998。采用复合刚体方法进行多腿车辆的前向动力学分析。IEEE 国际机器人与自动化大会论文集 (比利时鲁汶), 第 464-470 页, 5 月。

马尔赫夫卡, D. W.、奥林, D. E. 1999. 用于机器人系统仿真的非线性阻尼顺应性接触模型。IEEE 系统、人机与控制汇刊——A 部分, 第 29 卷, 第 6 期, 第 566-572 页。

米尔蒂奇, B. 1998. V-Clip: 快速且稳健的多面体碰撞检测。ACM 图形学汇刊, 第 17 卷, 第 3 期, 第 177-208 页。

冯·米塞斯, R. 1924a. 《电机计算: 力学的新工具》[Motor Calculus: a new Theoretical Device for Mechanics]。应用数学与力学杂志, 第 4 卷, 第 2 期, 第 155-181 页。(英文译本: 贝克和沃尔哈特, 1996 年。)

冯·米塞斯, R. 1924b. 《电机计算的应用》[Applications of Motor Calculus]。应用数学与力学杂志, 第 4 卷, 第 3 期, 第 193-213 页。(英文译本: 贝克和沃尔哈特, 1996 年。)

穆恩, F. C. 1998. 应用动力学。纽约: 威利出版社。

默里, J. J.、纽曼, C. P. 1984. ARM: 一种代数式机器人动力学建模程序。IEEE 国际机器人与自动化大会论文集, 佐治亚州亚特兰大, 第 103-114 页, 3 月 13-15 日。

默里, R. M.、李, Z.、萨斯特里, S. S. 1994. 机器人操作的数学入门。佛罗里达州博卡拉顿: CRC 出版社。

中村, Y.、山根, K. 2000. 结构变化运动链的动力学计算及其在人体姿态中的应用。IEEE 机器人与自动化技术汇刊, 第 16 卷, 第 2 期, 第 124-134 页。

纽曼, C. P.、默里, J. J. 1987. 定制化计算机机器人动力学。J. 机器人系统, 第 4 卷, 第 4 期, 第 503-526 页。

奥林, D. E.、麦基, R. B.、武科布拉托维奇, M.、哈托赫, G. 1979. 利用牛顿-欧拉方法对开放链连杆机构进行运动学与动力学分析。数学生物科学, 第 43 卷, 第 107-130 页, 2 月。

参考文献

奥兰迪亚, N.、切斯, M. A.、卡拉汉, D. A. 1977. 一种以稀疏性为导向的机械系统动力学分析与设计方法——第一部分。ASME 工程工业杂志, 第 99 卷, 第 3 期, 第 773-779 页。

朴, F. C.、鲍布罗, J. E.、普伦, S. R. 1995. 机器人动力学的李群形式化。国际机器人研究期刊, 第 14 卷, 第 6 期, 第 609-618 页。

保罗, B. 1975. 机构的解析动力学——以计算机为导向的概述。机构与机器理论, 第 10 卷, 第 6 期, 第 481-507 页。

普费弗, F.、格洛克, C. (编者) 2000. 带单侧接触的多体动力学。维也纳: 施普林格出版社。

普吕克尔, J. 1866. 关于力学的基本观点。哲学事务, 第 156 卷, 第 361-380 页。

罗伯森, R. E.、施韦尔塔塞克, R. 1988. 多体系统动力学, 柏林/海德堡: 施普林格出版社。

罗德里格斯, G. 1987. 卡尔曼滤波、平滑以及递归机器人臂正逆动力学。IEEE 机器人与自动化杂志, 第 RA-3 卷, 第 6 期, 第 624-639 页。

罗德里格斯, G.、贾因, A.、克鲁茨-德尔加多, K. 1991. 用于机械手建模与控制的空间算子代数。国际机器人研究期刊, 第 10 卷, 第 4 期, 第 371-381 页。

鲁尼, J. 1977. 关于围绕固定点进行空间旋转表示的综述。环境与规划 B, 第 4 卷, 第 2 期, 第 185-210 页。

罗斯恩塔尔, D. E. 1990. 机器人系统的 n 阶形式化。航天科学杂志, 第 38 卷, 第 4 期, 第 511-529 页。

萨哈, S. K. 1997. 机械手惯量矩阵的分解。IEEE 机器人与自动化技术汇刊, 第 13 卷, 第 2 期, 第 301-304 页。

- 塞利格, J. M. 1996. 机器人学中的几何方法。纽约: 施普林格出版社。
- 沙巴纳, A. A. 2001. 计算动力学 (第 2 版)。纽约: 威利出版社。
- 西西利亚诺, B., 卡蒂布, O. (编者) 2008. 施普林格机器人学手册。柏林: 施普林格出版社。
- 施泰伊斯卡尔, V., 瓦拉谢克, M. 1996. 机械的运动学与动力学。纽约: 马塞尔·德克尔出版社。
- Stepanenko, Y. 和 Vukobratovic, M. 1976. 铰接式开放链主动机构的动力学。数学生物科学, 第 28 卷, 第 137-170 页。
- Stewart, D. E. 2000. 含摩擦与碰撞的刚体动力学。SIAM 评论, 第 42 卷, 第 1 期, 第 3-39 页。
- Uicker, J. J. 1965. 利用 4×4 矩阵进行空间连杆的动力学分析。西北大学博士论文。
- Uicker, J. J. 1967. 空间连杆的动力学分析。ASME 期刊应用力学专刊, 第 34 卷, 第 418-424 页。
- Umetani, Y. 和 Yoshida, K. 1989. 基于广义雅可比矩阵的空间机械臂的解耦运动速率控制。IEEE 机器人与自动化技术汇刊, 第 5 卷, 第 3 期, 第 303-314 页。
- Vafa, Z. 和 Dubowsky, S. 1990a. 空间机械臂的运动学与动力学: 虚拟机械臂方法。国际机器人研究杂志, 第 9 卷, 第 4 期, 第 3-21 页。
- Vafa, Z. 和 Dubowsky, S. 1990b. 基于虚拟机械臂的动力学分析, 及其在路径规划中的应用。宇航科学杂志, 第 38 卷, 第 4 期, 第 441-472 页。
- Vereshchagin, A. F. 1974. 机器人机械臂复杂机构动力学的计算机模拟。工程控制论, 第 6 期, 第 65-70 页。
- Walker, M. W. 和 Orin, D. E. 1982. 机器人机构高效动力学计算机模拟。ASME 期刊, 动态系统、测量与控制专刊, 第 104 卷, 第 205-211 页。
- Wittenburg, J. J. 1977. 刚体系统动力学。斯图加特: B. G. Teubner 出版社。
- Wittenburg, J. 和 Wolz, U. 1985. 用于机器人动力学模拟的 Mesa Verde 程序。在“机器人研究: 第三届国际研讨会”中, O. D. Faugeras 和 G. Giralt (编者) 著, 马萨诸塞州剑桥市: 麻省理工学院出版社, 第 197-204 页。
- Woo, L. S. 和 Freudentein, F. 1970. 将直线几何应用于理论运动学以及机械系统的运动学分析。机械学杂志, 第 5 卷, 第 3 期, 第 417-460 页。
- Yamane, K. 2004. 模拟并生成人体动作。柏林: 施普林格出版社。

12.0.16 符号

以下列表展示了本书中符号与其含义之间的基本对应关系。该列表并不全面; 下标和上标则对基本含义进行了进一步细化。例如, v_j 表示关节处的速度; a_g 表示重力加速度; I_i 表示物体 i 的惯性, 等等。前导上标表示坐标系。许多符号具有多种含义, 而少数含义则有多个符号可供使用。

$\times^*$ 、 $\mathbf{X}^*$ 是双线性运算符 (即力矢量版本的叉积运算符 $\times$ 与坐标变换 $\mathbf{X}$)

速度和加速度变量的矢量 $\mathbf{a}$ 和 $\dot{\mathbf{a}}$ ——尤其是作为 $\dot{\mathbf{q}}$ 和 $\ddot{\mathbf{q}}$ 的替代方案。

显式运动约束函数 γ ($\mathbf{q} = \gamma(\mathbf{y})$)。

δ 回路闭合位置误差; 一个元素非零的矢量。

ζ 、 $\dot{\zeta}$ 为接触-分离速度和加速度变量的矢量。

l 空间冲量矢量。

$\lambda(i)$ 、 $\mu(i)$ 、 $\kappa(i)$ 、 $\nu(i)$ 是 (λ) 的父节点， (μ) 的子节点集合，支持 (κ) 的关节集合，以及以 (ν) 为起点的子树中的所有体（第 4.1.4 节）。

约束力变量的矢量 λ ；接触法向力的矢量。

σ 在例如基于时间显式依赖关系的柔顺约束中所使用的空间偏置速度。

τ 为关节空间或广义力变量的矢量；用于某个关节的主动力变量（ $\tau = S^T f_J$ ）。

ϕ 显式运动约束函数（ $\phi(q) = 0$ ）。

Φ 空间逆惯性。

ω 三维角速度矢量。

a 空间加速度矢量。

b 空间偏置加速度（如 $a = \Phi f + b$ ）。

c 与速度相关的空间加速度项；三维矢量，用于定位质心。

C 关节空间或广义偏置力（如 $\tau = H\dot{q} + C$ ）。

d 、 e （普吕克）基矢量，分别位于运动空间（ d ）和力空间（ e ）。

E 三维旋转矩阵。

F 三维线性力；空间力（有时记作 $\hat{f}$ ）； F^6 中的一般元素。

G 、 g 是显式运动约束方程的系数（ $\dot{q} = G\dot{y}$ ， $\ddot{q} = G\dot{y} + g$ ，其中 $G = \partial\gamma/\partial y$ ，若 γ 已定义； $g = \ddot{G}\dot{y}$ ）。

H 空间动量矢量；质量的第一矩（ $h = mc$ ）。

H 关节空间或广义惯性。

$\bar{I}$ 三维旋转惯性。

空间惯性矩阵（ I^A 、 I^a ：关节体， I^c ：复合刚体）

J ， J_L 体（ J ）与环路（ J_L ）雅可比矩阵： $v = J\dot{q}$ 和 $v_J = J_L\dot{q}$ ，其中 v 和 v_J 分别为体和环路关节的速度

K ， k 显式运动约束方程的系数（ $K\dot{q} = 0$ ， $K\ddot{q} = k$ ； $K = \partial\phi/\partial q$ ， $k = -\dot{K}\dot{q}$ ）

m 一般的空间运动矢量（ M^6 中的任意元素）

M^n ， F^n 空间中的广义（ $n = n$ ）、空间（ $n = 6$ ）以及平面（ $n = 3$ ）运动矢量（ M ）与力矢量（ F ）

N ， n 事物数量（ N ）与变量数量（ n ）；例如， N_B 、 N_J 、 N_L ， n ：体、关节、运动学环路及运动自由度的数量

n 力矩或力偶；空间接触法向量

N contact 法向量集合： $N = [n_1 n_2 \dots]$

O 、 P 是空间中的点；坐标系的原点；具有这些原点的坐标系名称

$p(i)$ 、 $s(i)$ 是关节 i 的前驱体（ p ）和后继体（ s ）

p 空间偏置力（如 $f = Ia + p$ 或 $f = I^A a + p^A$ ）

P 系统范围内，用于将体相关矢量与关节相关矢量进行映射的矩阵（ $6N_B \times 6N_J$ ）

q 、 $\dot{q}$ 、 $\ddot{q}$ 是关节空间或广义位置、速度与加速度变量的矢量；此类变量适用于单个关节

r 是一个点的三维位置矢量（例如，原点或体内的某一点）

S 、 T 是通用的矢量子空间；运动自由度（ S ）以及约束与主动力（ T 和 T_a ）子空间

S 、 T 矩阵，分别表示 S 和 T ；（仅 T ）是接触法向量的集合：

$T = [t_1 t_2 \dots]$

$\mathbf{t}$ 是关节空间或广义接触法向量

$\mathbf{u}$ 是通用矢量；对应于 $\dot{\mathbf{y}}$ 的广义力； $\boldsymbol{\tau}$ 的被驱动子集；广义冲量

$\mathbf{v}$ 是通用矢量；三维线性速度；空间速度（有时也用 $\hat{\mathbf{v}}$ 表示）

$\mathbf{X}$ 坐标变换矩阵（ ${}^I\mathbf{X}_i$ ：用于将 i 坐标转换为 j 坐标；

$\mathbf{X}_J$ ：联合变换； $\mathbf{X}_T$ 、 $\mathbf{X}_P$ 和 $\mathbf{X}_S$ ：参见第 4.2 节）

$\mathbf{y}$ 、 $\dot{\mathbf{y}}$ 、 $\ddot{\mathbf{y}}$ 是显式约束方程中的独立变量；在应用约束后， $\mathbf{q}$ 、 $\dot{\mathbf{q}}$ 和 $\ddot{\mathbf{q}}$ 是变量的独立子集。

12.0.17 索引

绝对坐标，74、196

加速度

偏置 ($\mathbf{b}$)，36、58

偏置 ($\mathbf{b}^A$)，122、124、136、138

经典形式，30、97、99

闭合形式表达式，92

接触分离，214、217、221

广义形式 ($\dot{\mathbf{a}}$)，41

广义形式 ($\ddot{\mathbf{q}}$)，40、43-45

广义形式 ($\ddot{\mathbf{y}}$)，44-46

重力 ($\mathbf{a}_g$)，94、181、182、185

关节 ($\mathbf{a}_j$)，30、55、61

关节 ($\mathbf{c}_j$)，55、83、85、95

关节 ($\ddot{\mathbf{q}}$)，55、58

关节空间，31

在系统中作为实体的，30

相对的，184

空间的，19、28-31

求和，30

时变形式 ($\dot{\sigma}$)，55

速度乘积 ($\mathbf{a}^{vp}$)，144、153、182

速度乘积 ($\mathbf{c}$)，126、128、131

算法复杂度，89-90

铰接体，120

作为刚体系统，123

箱-圆柱形示例，122

手柄，120、139

多个手柄，122、136-139

两两组装，125、137

铰接体算法，128-131，

251

替代版本，131

分治法, 136

公式与伪代码, 132

浮动基座, 181

伪代码, 182

混合动力学, 176

伪代码, 178

公式表, 177

articulated 体惯性, 119-128; 算术运算, 247、251; 基本性质, 120; 装配方法的计算, 125-128; 装配, 多手柄, 136-139; 投影法, 123-125; 系统化方法, 131-136; 公式表, 134-135; I^a 和 p^a , 127、176; 逆变换, 121、124、137; 多手柄, 122、137; 逆变换的性质, 124; 求和, 121; 增广体, 192; 轴向螺纹变换, 204、249-252; 成本表, 249。

基准坐标系, 74; 基底, 参见坐标系; 双基底, 8、17; 在子空间上, 47; 正交规范, 9、11、14; 普吕克尔, 11、14、24; 平面结构, 37; 互易性, 9、17; 鲍姆加特稳定化, 145-147; K 和 G 的块对角结构, 158; 体 关节映射 (P), 61、167; 体坐标系 (F_i), 73; 体坐标, 74、95、106、131; 体雅可比矩阵, 参见雅可比矩阵; 体位置, 74、75; 分支诱导的稀疏性, 110-112、151; 175、206。

笛卡尔坐标系, 11、14、20、73; 子集 (μ), 72。

闭环系统, 69、141。

配置模糊度, 160。

运动自由度, 159。

过约束, 161。

性质, 159-161。

恢复系数, 233。

碰撞, 223、228、232。

柔性接触, 235-239。

复合刚体算法,

104-108。

替代推导, 108-110。

浮动基座, 182。

伪代码, 183。

混合动力学, 173-176。

伪代码, 174。

稀疏性, 175。

运算次数分析, 204-208。

伪代码, 107。

计算成本, 参见效率。

条件数, 198、201。

配置空间, 40。

连通性, 66-72。

连通性图, 66-72。

扩展, 114。

用于混合动力学, 175。

表示方式, 71-72。
覆盖树, 67。
约束, 参见运动、力、不等式。
接触约束方程, 215、217、223。
接触动力学, 213-222、236。
无解, 218。
模拟流程, 225。
接触法向量, 213、216、220。
关节空间, 220。
坐标系
笛卡尔坐标系, 11、14。
双基底, 9、17、40。
广义坐标系, 40、43、59。
齐次坐标系, 22。
普吕克尔, 11、15-16、20、43。
坐标变换, 9、18。
应用, 246、248。
.invApply(), 246、248。
齐次坐标系, 22。
如何描述, 23, 241
普吕克, 20-23
平面, 38, 241
旋转, 21, 243
力对, 参见“力”
叉积, 23-25, 243
作为微分算子, 23
对偶 ($\times^*$), 23, 25
算子形式 ($\mathbf{a} \times$), 9, 21, 25
算子形式 ($\mathbf{a} \times^*$), 25
平面, 37, 241
性质表, 22, 25
向量的分解, 47, 50
德纳维特-哈滕贝格 (DH) 参数, 75-77
德纳维特-哈滕贝格坐标系, 75, 204, 207, 250
导数
表观的, 27, 30, 55
逐分量的, 26
关于坐标变换的, 28
关于普吕克基矢量的, 23
关于向量的, 25
关于惯性, 33

微分, 25-27
点积符号 ($\cdot$), 26, 27
在运动坐标中, 26
环形符号 ($\circ$), 27
公式表, 26
直和 ($\oplus$), 48
二元组, 10, 33
二元组式, 10, 33-34
性质表, 34
动态等价, 189
动力学算法, 1
基于模型的, 3, 65, 209
 E'' , 7, 48
效率, 201-204
分支树, 206-208
算法之间的比较, 201-204
度量, 201
优化, 204-206
符号化简化, 209-212
效率技巧, 252-256
运动方程
连杆体, 120-122, 124, 176
封闭形式, 92
闭环逆动力学, 165
闭环系统, 142, 143, 150, 152
集合, 43
约束刚体, 57-60, 215, 217
构建, 42-46
接触动力学, 215, 217, 221
解法, 224-227
浮动底座, 180, 181
通用, 2, 40-42, 44, 59, 102, 219
脉冲式的, 231
在中的不一致, 150
多体系统, 60
投影的, 46, 124, 152
刚体, 3, 19, 35-36
覆盖树, 142
状态空间表述, 42
双体系统, 190
带有显式约束的系统, 45

带有齿轮约束的系统, 188
 带有隐式约束的系统, 45、59、63、143、150、167
 欧拉角, 84
 欧拉参数, 86
 欧拉方程, 36
 过量机动性, 161
 F^3 , 37、38
 F^6 , 14、17、18、23、38、43
 F^n , 7、40、43、49
 分解方法, 参见稀疏矩阵
 FD (……), 2、41
 FD_x , 42
 一阶动力学系统, 235
 固定基座, 66、74
 浮动基座, 67、179
 浮动基座坐标, 156、179-182
 浮动基座前向动力学, 181-183
 浮动基座反向动力学, 183-185
 公式与伪代码, 185
 浮动基座系统, 156、179-181
 浮点运算, 197
 浮点运算次数, 201
 针对已发表的算法, 202
 图示, 203、209
 力
 作用于体 i , 93
 作用与反作用, 18
 执行器, 165-166
 角动量 (偶力), 14
 偏置 (C), 40、59、103、152-154
 偏置 (p), 35、57
 偏置 (p^A), 120、124、127
 偏置 (p^c), 180、184
 约束 (f_c), 57
 约束 (λ), 44、54、59、63、150-151、167
 约束 (τ_c), 44
 接触 (f_c), 214、216
 接触 (λ), 217
 接触 (τ_c), 220
 科里奥利力与离心力, 40、94
 外部 (f^x), 93、153、167

广义 (τ), 40、54、59
 广义 (u), 46
 引力, 35、40、57
 混合偏置 (C'), 173
 关节 (f_j), 53、62
 关节 (τ), 54、63
 线性, 13
 循环关节, 142、148-149
 主动 (τ^a 、 f^a), 148、152-154
 约束 (τ^c 、 λ), 143、149
 以点为中心的力矩, 14
 软接触, 236
 空间, 13-15
 求和, 18
 力子空间矩阵
 主动, 54、56、63、153
 约束, 54、56、58、62、143、155、167
 前向动力学, 2、41、101、117、119
 前向动力学关节集 (fd), 172
 摩擦, 233、238
 系数, 239
 高斯最小约束原理, 225
 齿轮, 55、81、186-189
 几何事件, 228
 几何
 接触, 227-230
 系统模型, 73-74、77
 重力, 参见力与加速度
 重力补偿, 94
 手柄, 参见铰接体
 混合动力学, 171-176
 $ID(\cdots)$, 2、41、89、103、165、173
 $ID_\delta(\cdots)$, 103、173
 伪代码, 104
 碰撞, 213、232
 冲量, 214、230
 脉冲动力学, 230-235
 不等式约束, 222-223
 可能的状态, 222
 惯性
 铰接体, 参见

铰接体惯性
复合刚体, 105、184
广义的, 40、59
逆向, 36-37、50、58
关节空间, 102-103、105
用于齿轮电机, 189
平面的, 38、242
特性的, 34
刚体, 32、35、243、250
旋转的, 31、33
空间的, 33-35
惯性参数, 189-193
修改 (I_{Δ}), 190
数值表, 191
简化, 192
逆向动力学, 2、41、89、99、164-166、183-185
雅可比矩阵
身体, 20、31、73、124、144、220、232
浮动底座, 186
回路, 145、148
jcalc($\dots$), 参见关节计算函数
关节, 39、53、65
6 自由度, 67、78、179
圆柱形的, 52、78、147、148
螺旋形 (螺纹) 的, 77、78
平面的, 78、81
棱柱形的, 20、52、77、78、148、191
齿条与齿轮, 81
回转式的, 20、55、77、78、148、191
球形在圆柱体内, 161
球形的, 78、84、86、148、191
零自由度的, 147、148
关节轴矢量, 20、30、91
关节轴承, 56、94
关节计算函数, 80、83、146、154
关约束, 参见运动约束
关节位置框架 ($F_{i,j}$), 74
关节模型, 78-80
公式表, 78、148
关节模型库, 78
关节参数, 80、82

关节极性, 69
反向运动, 83
关节对称性, 70、83
关节变换 ($\mathbf{X}_j$), 74、82、147
关节类型描述符, 78、83
关节变量, 20、30、53、77、81、84、86
关节空间惯性矩阵, 参见惯性
朱尔丹原理, 44
 $jtype(i)$, 参见关节类型描述符
运动链, 19、30、69、75、91
运动约束, 参见运动约束
运动回路, 68、141
闭式解法, 164
独立的, 68
数量, 68
运动树, 67
浮动的, 125
未分支的, 参见运动链
动能, 32、35、40、46、104、105、186
拉格朗日乘子, 44
李代数, 38
线接触, 229
线性互补问题 (LCP), 224
连接件, 67
连接件坐标, 参见体坐标
回路闭合函数, 参见运动约束
回路约束, 参见运动约束
回路关节, 69、142、144、146、147
被动的, 142
通用的, 147
回路位置误差 (δ), 146–148、155
 $\mathbf{M}^3$, 37、38
 $\mathbf{M}^6$, 11、17、18、23、38、43
 $\mathbf{M}^n$, 7、40、43、49
质量, 31
表观的, 121、123
中心的, 121、123
中心的, 31、32、36
最小回路簇, 158
建模误差, 199
动量

角动量, 31
线性动量, 31
以点为中心的力矩, 31
浮动基座系统的, 185
空间的, 19、31-32
运动约束, 44-46、50-53
分类层次结构, 51
显式方法, 44-46、161
非完整约束, 50
隐式方法, 44、45、55、62、143
关节, 53-55
回路闭合, 143-145、154-156
伪代码, 155
舍入误差问题, 156
循环闭合函数, 161-164
示例, 162
非诺莫动, 50
莱诺莫动, 51
斯科罗诺莫动, 51
稳定化, 145-147
三体问题, 52
运动子空间矩阵, 50、53、56、57、83、85、179
运动代数, 38
运动体, 66
牛顿方程, 36
数值积分, 198、226
 $O(\cdots)$, 90
 $O(N_B) = O(n)$, 90
 $O(nd^2)$ vs. $O(n^3)$, 102、116、206
正交补, 48-49
性质表, 49
过约束, 161
父数组 (λ), 71
展开式, 114
用于混合动力学, 175
平面向量, 37-38、196、241
算术函数, 242
点接触, 213、216
多面体, 227
幂, 17、19
功率平衡方程, 54、59

前驱, 53、60、69、143、148、219
 前驱数组 (p), 71
 前驱数组 (pc), 220
 前驱框架, 78、146、154; 前驱变换数组 ($\mathbf{X}_p$), 74; 规定运动, 172; 棱柱形系统, 见关节投影法, 46、123-125; 适当驱动的系统, 166; 适当约束的系统, 159、161; 剪枝约束方程, 150。
 qdfn (……), 41; 二次规划, 224; 四元数, 84
 $\mathbb{R}^n$, 7
 转动半径, 196
 互易条件, 见基底
 递归关系, 90-92
 递归公式, 90
 递归牛顿-欧拉算法,
 92-97
 浮动基座, 182
 伪代码, 183
 修改后的循环结构, 153
 伪代码, 154
 原始版本, 97-99
 伪代码, 96
 冗余驱动系统, 166
 参考坐标, 74、180
 常规编号, 69
 消除高频动力学, 172
 恢复力
 系数, 233
 切向力, 233、234
 扭转力, 233、234
 回转运动, 见关节
 刚体系统, 39、65
 曲柄滑块机构, 66
 梯形机构, 168
 卫星, 67
 $\text{rot}(\mathbf{E})$, 23
 旋转, 84-86、253-256
 旋转矩阵 ($\mathbf{E}$), 21、85、86、252
 旋转矩阵 (θ), 23、244
 旋转矩阵 (θ), 23、244
 旋转矩阵 (θ), 23、244
 舍入误差, 196-198
 $\text{rx}(\theta)$, 23、244
 $\text{ry}(\theta)$, 23、244

$\text{rz}(\theta)$, 23、244

$\text{rz}(\theta)$, 23、244

标量积, δ , $1t - 1g$ 无零性, 8 个算子形式 ($\mathbf{a} \cdot$), 9 个螺旋轴, 16 个螺旋理论, 16、38 敏感度, 199-201 软接触, 参见“顺应性接触求解 $K\ddot{\mathbf{q}} = \mathbf{k}$, 以得到 $\ddot{\mathbf{q}} = \mathbf{G}\ddot{\mathbf{y}} + \mathbf{g}$ ”, 156-157 拓扑树, 43、141-142 最小深度, 201 稀疏矩阵算法, 112-116 反向代换, 115 Cholesky, 111 计算成本, 116 LDL^T , 111 L^TDL , 112 L^TL , 112、151 乘法, 115 稀疏矩阵方法, 166-168 $\mathbf{K}$ 和 $\mathbf{G}$ 中的稀疏性, 158-159 空间向量, 3、7、40 力, 8、18 大小, 17 运动, 8、18 单位, 17 使用方式, 18-19 空间向量运算, 243-252 紧凑型数据结构, 245 简单函数, 244 运算表, 247 类型安全, 246 状态空间, 42 子树集合 (ν), 72、174 后继节点, 53、60、69、143、148、219 后继数组 (s), 71 后继数组 (sc), 220 后继框架, 78、146、154 后继变换数组 ($\mathbf{X}_S$), 74 支持, 72 支持集 (κ), 72 表面接触, 229 符号化简化, 参见“效率系统模型”, 2、41、65、87

张量, 参见二元张量、惯性拓扑树, 67 拓扑, 参见连通性树关节, 69

向前方向, 71; 向后方向, 71、83 拓扑变换数组 ($\mathbf{X}_T$), 74、77 截断误差, 198 扭转, 16

欠驱动系统, 166 单边约束, 222 单位 (测量), 4

向量, 73D, 3、8 抽象, 8 坐标, 8 欧几里得, 8、48 自由, 16 直线, 16 平面, 参见平面向量空间, 参见空间向量向量场, 12 向量空间, 7 对偶空间, 8、17 向量量子空间, 46-50 主动力, 54、56 约束力, 49、54、57 矩阵表示, 47 运动, 46、49、53、57

速度角, 10 偏差 (σ), 53-55、143 接触分离, 214、216、220 广义 (α), 41 广义 ($\dot{q}$), 40 广义 ($\dot{y}$), 44 关节 (α), 81 关节 ($\dot{q}$), 53 关节 (v_j), 19、53、61、80、82、95、143 关节空间, 20 线性, 10 体在系统中的, 20、72、80 体中点的, 10 相对的, 18 空间, 10-13

力矩, 16

$\text{xlt}(\mathbf{r})$, 23, 244XtoV, 242, 243

零维矩阵, 53